

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-02-10

Inhaltsverzeichnis

Methodenbaustein Sensordatenanalyse	6
Voraussetzungen	7
Lernziele	8
1 Das Prinzip von Messungen	9
1.1 Messung	11
1.1.1 Direkte und indirekte Messung	11
1.1.2 Genauigkeit und Präzision	13
1.2 Messreihen	13
1.3 Varianz	13
1.4 Standardabweichung	14
1.4.1 Experiment Verteilungskenngrößen	15
1.5 Ergebnisse	16
1.6 grafische Darstellung	16
1.7 Code	17
1.7.1 Aufgabe Verteilungskenngrößen	19
1.7.2 Varianz und Standardabweichung mit NumPy und Pandas	21
2 Die Normalverteilung	23
2.1 ein Würfel	23
2.2 zwei Würfel	24
2.3 drei Würfel	26
2.4 Grafik	31
2.5 Code	31
2.6 absolute Häufigkeit	35
2.7 relative Häufigkeit	36
2.8 Häufigkeitsdichte	38
2.9 3 Klassen	38
2.10 5 Klassen	39
2.11 7 Klassen	40
2.12 Normalverteilung anpassen	42
2.13 Das Paket SciPy	44
2.14 Aufgaben Normalverteilung	46
2.15 Konfidenzintervalle	47

2.16	Standardnormalverteilung	48
2.17	Einzelner Messwert	49
2.18	Stichprobe $N = 12$	49
2.19	Code	50
2.20	Die t-Verteilung	52
2.21	Grafik	53
2.22	Code	54
2.22.1	Beispiel Gewicht weiblicher Pinguine	55
2.23	Grafik	56
2.24	Code	56
2.25	Aufgabe Konfidenzintervalle	58
3	Lineare Parameterschätzung	62
3.1	Messreihe Hooke'sches Gesetz	62
3.1.1	Deskriptive Statistik	64
3.1.2	Explorative Statistik	70
3.2	Federkonstante bestimmen	73
3.2.1	Lineare Ausgleichsrechnung	74
3.3	Grafik	75
3.4	Code	75
3.5	partielle Ableitung nach dem y-Achsenschnittpunkt	77
3.6	partielle Ableitung nach dem Anstieg	78
3.6.1	Messabweichung quantifizieren	85
3.6.2	Das Modul SciPy	87
3.6.3	Ergebnis Federkonstante	88
4	Fehlerrechnung	90
4.1	Bevor es losgeht	90
4.1.1	Der Begriff des Fehlers	90
4.1.2	Konfidenzniveau	91
4.1.3	Notation	91
4.2	Fehlerarten	91
4.3	Grundbegriffe nach DIN 1319	93
4.3.1	Messabweichung	94
4.3.2	Messunsicherheit	94
4.3.3	Vollständiges Messergebnis	96
4.3.4	Übersicht	97
4.4	Absolute und relative Fehler	98
4.5	Größtfehler	99
4.6	Signifikante Stellen	100
4.7	Methoden der Fehlerrechnung	101
4.7.1	1. Fehlerabschätzung	102
4.7.2	2. Fehlerstatistik	108

4.7.3	3. Fehlerfortpflanzung	111
4.8	Vollständiges Messergebnis	119
4.9	95-%-Vertrauensintervall	119
4.10	Lineare Fehlerfortpflanzung	122
4.11	Gauß'sche Fehlerfortpflanzung	123
5	Übung Hooke'sches Gesetz	125
5.1	Dateien einlesen	125
5.2	Aufgabe Dateien einlesen	127
5.3	Musterlösung Daten aufbereiten	135
5.3.1	Auf Ausreißer prüfen	136
5.4	Ausreißer bestimmen	136
5.5	Gemeinsame Ausgabe der Messreihen	137
5.5.1	Umwandlung der Rechengrößen	140
5.5.2	Grafische Darstellung	143
5.6	Grafische Darstellung	145
5.7	Mittelwerte und Standardfehler berechnen	146
5.8	Code	146
5.9	Auswertung	147
5.10	Musterlösung Federkonstanten bestimmen	148
5.11	$\alpha = 0.10$	148
5.12	$\alpha = 0.05$	149
5.13	Code	149
5.14	Auswertung	150
6	Kalibrierung	152
6.1	Kalibrieren und Justieren	152
6.2	Kalibriermethoden nach DIN 1319	153
6.2.1	Korrektionstabelle	153
6.2.2	Ermittlung von Kalibrierfaktoren	154
6.2.3	Ermittlung einer (empirischen) Kalibrierfunktion	154
6.3	Additive Fehler: Der Nullpunktfehler	154
6.3.1	Nullpunktfehler quantifizieren	155
6.3.2	Nullpunktfehler korrigieren	158
6.3.3	Übung Nullpunktfehler	158
6.4	Multiplikative Fehler: Der Empfindlichkeitsfehler	167
6.4.1	Empfindlichkeitsfehler quantifizieren	168
6.4.2	Empfindlichkeitsfehler korrigieren	170
6.4.3	Übung Empfindlichkeitsfehler	171
6.5	Thermoelement 4	174
6.6	Thermoelement 15	176
6.7	Thermoelement 4	178
6.8	Thermoelement 15	178

6.9	Nullpunkt- und Empfindlichkeitsfehler	179
6.10	Grafik	180
6.11	Code	180
6.11.1	Nullpunkt- und Empfindlichkeitsfehler quantifizieren	181
6.11.2	Nullpunkt- und Empfindlichkeitsfehler korrigieren	182
6.11.3	Übung Nullpunkt- und Empfindlichkeitsfehler	184
6.12	Nichtlinearität: Linearitätsfehler	184
6.12.1	Beispiel Pt100	185
6.12.2	Aufgabe Pt100-Tabelle einlesen	186
6.12.3	Nichtlinearität darstellen	192
6.12.4	Nichtlinearität quantifizieren	196
6.13	Die Zweipunktkalibrierung	199
6.14	Eis kochen	201
6.14.1	Arnoldformel	233
6.15	Weitere Fehlerarten	235

Methodenbaustein Sensordatenanalyse



Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch und Sebastian Seipel. Methodenbaustein Sensordatenanalyse von Maik Poetzsch ist lizenziert unter [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Das Werk ist abrufbar auf [GitHub](https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse). Ausgenommen von der Lizenz sind alle Logos Dritter und anders gekennzeichneten Inhalte. 2025

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, und Sebastian Seipel. 2025. “Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse.” <https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse>.

BibTeX-Vorlage

```
@misc{BCD-m-sensordatenanalyse-2025,  
  title={Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse},  
  author={Arnold, Lukas and Arnold, Simone and Bagemihl, Florian and Baitsch, Matthias and Fehr, Marc and Hollmann, Franca and Poetzsch, Maik and Seipel, Sebastian},  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse}}
```

Voraussetzungen

Die Bearbeitungszeit dieses Bausteins beträgt circa **Platzhalter**. Für die Bearbeitung dieses Bausteins werden folgende Bausteine vorausgesetzt:

- Werkzeugbaustein Python
- Werkzeugbaustein NumPy
- Werkzeugbaustein Pandas
- Werkzeugbaustein matplotlib

In diesem Baustein werden die folgenden Module und Pakete verwendet:

- numpy
 - numpy.polynomial
- pandas
- openpyxl
- matplotlib
- glob
- scipy Version 1.14.1

Im Baustein werden folgende Daten verwendet:

- Zahnwachstum bei Meerschweinchen [CSV-Datei](#)
- Vermessung von Pinguinen an der Palmer Station [GitHub](#)
- Elektrische Widerstandswerte eines Pt100-Thermometers aus der DIN 60751 und von [hier \(PDF\)](#)
- Abstandsmessungen mit einem Ultraschallsensor (Messungen an der FH Dortmund)

Querverweis auf:

- Methodenbaustein Datenfitting und Datenoptimierung

Lernziele

In diesen Baustein lernen Sie ...

- Statistische Grundbegriffe
- Werkzeuge zur Auswertung und grafischen Darstellung von Sensordaten kennen
- Methoden zur Verarbeitung von mehreren Dateien anzuwenden
- Fehlerrechnung und -analyse
- Sensorkennlinien
- Kennlinienfehler und deren Korrektur

1 Das Prinzip von Messungen

“In der Physik existiert nur das, was gemessen worden ist” (Merz 1968, 14).
Merz, Ludwig. 1968. “Grundkurs der Messtechnik. Teil I: Das Messen elektrischer Größen.” 2. Auflage. München; Wien. R. Oldenbourg Verlag.

In diesem Baustein werden die folgenden Module verwendet:

```
import numpy as np
import numpy.polynomial.polynomial as poly
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import glob
```

Physikalische Größen werden mit der Hilfe von Messgeräten bestimmt. Diese ordnen der tatsächlichen Merkmalsausprägung eine numerische Entsprechung relativ zu einem Bezugssystem zu.

Ein Beispiel: “Johanna ist am Messbrett 173 Zentimeter groß.”

- Die tatsächliche Merkmalsausprägung ist Johannas Größe.
- Das Messgerät ist das Messbrett.
- Die numerische Entsprechung ist 173.
- Das Bezugssystem ist das metrische System.

Messwerte können aus verschiedenen Gründen immer nur eine Annäherungen an den wahren Wert der zugrundeliegenden physikalischen Größe sein. Zum einen variiert die [Größe eines Menschen im Tagesverlauf](#). Zum anderen ist das Messergebnis auch ein Ergebnis der verwendeten Skala. Wäre die Messung im imperialen Messsystem erfolgt, wäre Johannas Größe mit 68 Zoll bestimmt worden, was 172,72 Zentimetern entspricht.

Das Messergebnis ist also keine exakte Entsprechung der tatsächlichen Merkmalsausprägung. Ein bekanntes Beispiel für die mit dem Messvorgang verbundene Unsicherheit ist das Küstenlinienparadox: Das Ergebnis der Vermessung unregelmäßiger Küstenlinien wird umso größer, je kleiner die Messabschnitte gewählt werden.



- (a) Gerade Messabschnitte von 200 km Länge, Gesamtlänge ungefähr 2350 km
 (b) Gerade Messabschnitte von 100 km Länge, Gesamtlänge ungefähr 2775 km
 (c) Gerade Messabschnitte von 50 km Länge, Gesamtlänge ungefähr 3425 km

Abbildung 1.1: Küstenlinienparadox

Britain-fractal-coastline-200km, Britain-fractal-coastline-100km und Britain-fractal-coastline-50km von Maksim stehen unter der Lizenz [CC BY-SA 3.0](#) und sind abrufbar auf Wikipedia ([200km](#), [100km](#), [50km](#)). 2006

1.1 Messung

! Wichtig 1: Messung

“Eine Messung ist der experimentelle Vorgang, durch den ein spezieller Wert einer physikalischen Größe als Vielfaches einer Einheit oder eines Bezugswertes ermittelt wird. Die Messung ergibt zunächst einen Messwert. Dieser stimmt aber aufgrund störender Einflüsse mit dem wahren Wert der Messgröße praktisch nie überein, sondern weist eine gewisse Messabweichung auf. Zum *vollständigen Messergebnis* wird der Messwert, wenn er mit quantitativen Aussagen über die zu erwartende Größe der Messabweichung ergänzt wird. Dies wird in der Messtechnik als Teil der Messaufgabe und damit der Messung verstanden.”

Messung. von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia](#). 2025

- Die ideale Messung ist eine direkte Messung oder der gesuchte Wert hängt linear vom gemessenen Wert ab.
- Die ideale Messung ist *genau* und *präzise*.

1.1.1 Direkte und indirekte Messung

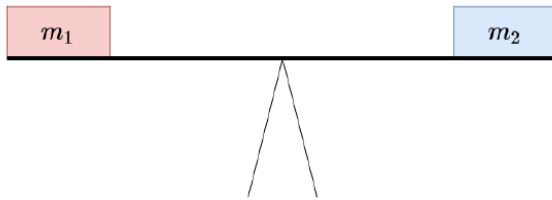
Bei einer direkten Messung wird die Messgröße durch den unmittelbaren Vergleich mit einem Normal oder einem genormten Bezugssystem gewonnen.

Gliedermaßstäbe von Fst76 ist lizenziert unter [CC-BY-SA 3.0](#) und ist abrufbar auf [Wikimedia](#). 2014

Bei einer indirekten Messung wird die Messgröße auf eine andere physikalische Größe zurückgeführt.

Spring scale von Amada44 steht unter der Lizenz [CC-BY-SA-3.0 unported](#) und ist abrufbar auf [Wikimedia](#). 2016

Observe the Moon wurde von der NASA veröffentlicht und ist abrufbar unter [nasa.gov](#). 2010



(a) Balkenwaage



(b) Zollstock

Abbildung 1.2: Direkte Messung



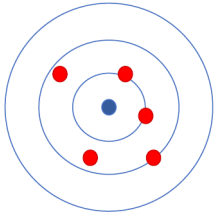
(a) Federwaage



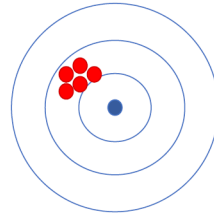
(b) Laserentfernungsmessung

Abbildung 1.3: Indirekte Messung

1.1.2 Genauigkeit und Präzision



(a) Genauigkeit



(b) Präzision

Die Genauigkeit einer Messung ist ein Maß für die Abweichung der Messwerte vom realen Wert. Die Genauigkeit ist nur bestimmbar, wenn anerkannte Referenzwerte vorhanden sind.

Die Präzision einer Messung beschreibt, wie gut die einzelnen Messwerte miteinander übereinstimmen. Die Präzision einer Messung wird über die Standardabweichung der Stichprobe bestimmt.

Abbildung 1.4: Genauigkeit und Präzision

1.2 Messreihen

Um die Unsicherheit einer Messung zu verringern, kann man einen Messwert in Form einer Messreihe wiederholt aufnehmen. Die beste Schätzung der Messgröße bietet der arithmetische Mittelwert der Messreihe.

Der arithmetische Mittelwert einer Messreihe $\bar{x}$ ist die Summe aller Einzelmesswerte x_i dividiert durch die Anzahl der Messwerte N .

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Mit Hilfe des arithmetischen Mittelwerts kann eine Aussage über die Streuung der Messwerte und die Präzision der Messung getroffen werden. Dazu werden die Varianz und die Standardabweichung der Messreihe berechnet.

1.3 Varianz

Die Varianz ist der Mittelwert der quadrierten Abweichungen vom Mittelwert.

$$\text{Var}(x_i) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

1.4 Standardabweichung

Die Quadratwurzel der Varianz wird als Standardabweichung bezeichnet. Diese hat den Vorteil, dass sie in der Einheit der Messwerte vorliegt und dadurch leichter zu interpretieren ist. Die Standardabweichung s wird so berechnet:

$$s_N = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Für Stichproben wird die Stichprobenvarianz verwendet. Für die Standardabweichung einer Stichprobe gilt:

$$s_{N-1} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Da die Varianz das Quadrat der Standardabweichung s ist, wird diese häufig mit s^2 gekennzeichnet.

Warning 1: Standardabweichung und Varianz in der Grundgesamtheit

In der Stochastik werden Formeln häufig auch mit griechischen Buchstaben geschrieben, wenn Sie sich statt auf eine Stichprobe auf die Grundgesamtheit beziehen.

Der Mittelwert in der Grundgesamtheit wird auch Erwartungswert genannt und mit dem griechischen Buchstaben μ (My) dargestellt. Die Standardabweichung des Erwartungswerts wird mit σ (Sigma) gekennzeichnet.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

Mit Hilfe der Standardabweichung kann der *Standardfehler des Mittelwerts* bestimmt werden. Der Standardfehler des Mittelwerts ist ein Maß dafür, wie genau sich der arithmetische Mittelwert der Stichprobe an den tatsächlichen Mittelwert der Grundgesamtheit, den Erwartungswert, annähert (dazu gleich mehr) und wird auch *Stichprobenfehler* genannt. Der Standardfehler des Mittelwerts wird aus der Standardabweichung einer Messung und der Wurzel der Stichprobengröße berechnet.

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}}$$

Da die Standardabweichung in der Grundgesamtheit in der Regel unbekannt ist, wird der Standardfehler des Mittelwerts mit der Stichprobenstandardabweichung geschätzt.

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}}$$

Manchmal wird der Standardfehler zur längeren Schreibweise umgeformt.

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Der Standardfehler wird umso kleiner (die Messung umso präziser), je kleiner die Varianz in der Grundgesamtheit und je größer der Stichprobenumfang ist.

Dies lässt sich mit einem simulierten Würfelexperiment verdeutlichen. Bei einem idealen, fairen Würfel kommt jede Augenzahl gleich oft vor. Der Erwartungswert eines sechsseitigen Würfels ist:

$$\frac{1}{6} \sum_{i=1}^{i=6} (x_i) = 3,5$$

Die Standardabweichung eines fairen, sechsseitigen Würfels beträgt:

$$\sqrt{\frac{1}{6} \sum_{i=1}^{i=6} (x_i - 3,5)^2} \approx 1,71$$

Da die Varianz in der Grundgesamtheit bekannt ist, hängt der Standardfehler des Mittelwerts eines fairen Würfels allein von der Stichprobengröße ab.

1.4.1 Experiment Verteilungskenngrößen

In einem simulierten Experiment würfeln 100 Personen jeweils 3, 10 und 50 Mal und bilden den Mittelwert der Augen. Weil ein fairer Würfel simuliert wird, kann der Standardfehler mit der Standardabweichung der Grundgesamtheit berechnet werden.

1.5 Ergebnisse

Würfe pro Person: 3
kleinster Mittelwert: 1.00
Stichprobenmittelwert: 3.57

Stichprobengröße: 300
größter Mittelwert: 6.00
Standardfehler: 0.10

Würfe pro Person: 10
kleinster Mittelwert: 1.70
Stichprobenmittelwert: 3.49

Stichprobengröße: 1000
größter Mittelwert: 4.60
Standardfehler: 0.05

Würfe pro Person: 50
kleinster Mittelwert: 2.98
Stichprobenmittelwert: 3.50

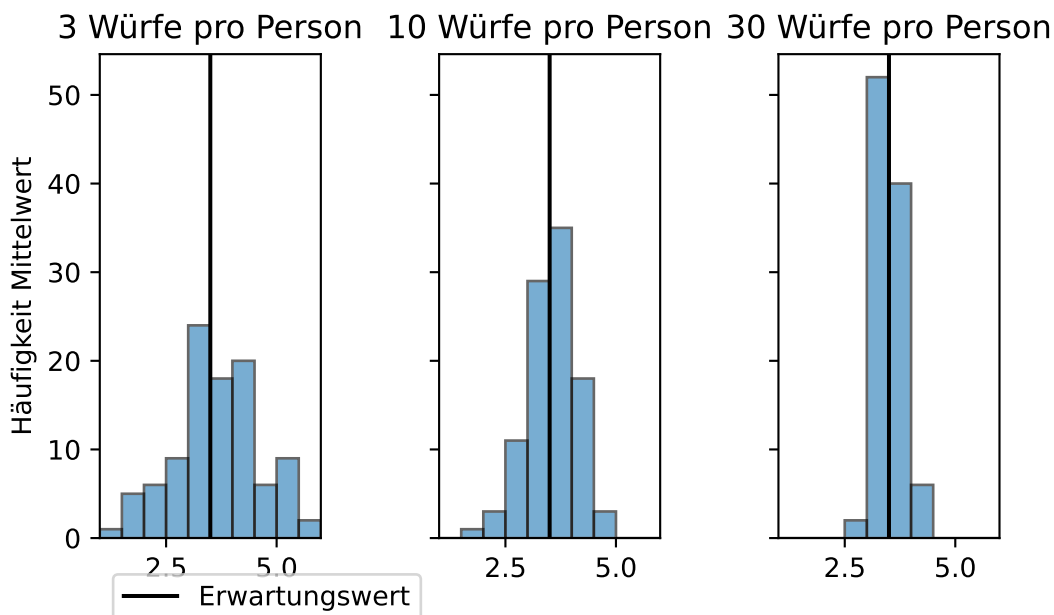
Stichprobengröße: 5000
größter Mittelwert: 4.36
Standardfehler: 0.02

Mit zunehmender Anzahl an Würfeln nähern sich Minimum und Maximum der individuellen Durchschnittswerte sowie der Stichprobenmittelwert dem Erwartungswert an.

Hinweis: Da das Skript dynamisch generiert wird, wurden die Zufallszahlen mit einem festgelegten Startwert erzeugt.

1.6 grafische Darstellung

Die Häufigkeit der individuellen Mittelwerte ist in den folgenden Histogrammen dargestellt.



1.7 Code

Berechnung

```
personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen3.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen3.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen3.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / (augen3.size ** (1/2)):.2f}\n",
      sep = "")

# 10 Würfe
würfe = 10

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen10.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen10.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen10.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / (augen10.size ** (1/2)):.2f}\n",
      sep = "")

# 50 Würfe
würfe = 50

## Personen stehen in den Zeilen (axis = 1), Würfe in den Spalten (axis = 1)
```

```

augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = (personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen50.mean(axis = 1).min():.2f}\t\t",
      f"größter Mittelwert: {augen50.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen50.mean():.2f}\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen50.size ** (1/2) ):.2f}\n",
      sep = "")

```

Darstellung

```

personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = (personen, würfe))

# 10 Würfe
würfe = 10
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = (personen, würfe))

# 50 Würfe
würfe = 50
augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = (personen, würfe))

# plotten
bins = 10

# 3 Würfe
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey = True)

ax1.hist(augen3.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax1.set_xlim(1, 6)
ax1.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black', label = 'Erwartungswert')
ax1.set_ylabel('mittleres Würfelergebnis')
ax1.set_ylabel('Häufigkeit Mittelwert')
ax1.set_title("3 Würfe pro Person")
ax1.legend(loc = 'lower left', bbox_to_anchor = (0, -0.2))

```

```

# 10 Würfe
ax2.hist(augen10.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax2.set_xlim(1, 6)
ax2.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax2.set_ylabel('mittleres Würfelergebnis')
ax2.set_title("10 Würfe pro Person")

# 30 Würfe
ax3.hist(augen50.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax3.set_xlim(1, 6)
ax3.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax3.set_ylabel('mittleres Würfelergebnis')
ax3.set_title("30 Würfe pro Person")

plt.tight_layout()
plt.show()

```

1.7.1 Aufgabe Verteilungskenngrößen

Im Datensatz ToothGrowth.csv ist eine Messreihe zur Länge zahnbildender Zellen bei Meerschweinchen gespeichert. Die Tiere erhielten Vitamin C direkt (VC) oder in Form von Orangensaft (OJ) in unterschiedlichen Dosen.

Code-Block 1.1

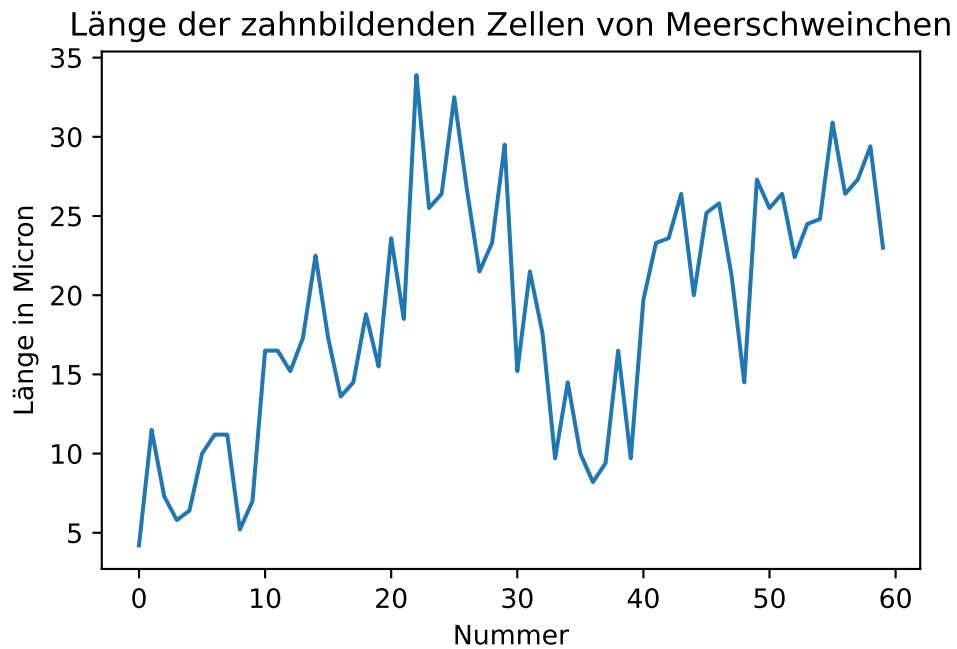
```

dateipfad = "01-daten/ToothGrowth.csv"
meerschweinchen = pd.read_csv(filepath_or_buffer = dateipfad, sep = ',', header = 0, \
    names = ['ID', 'len', 'supp', 'dose'], dtype = {'ID': 'int', 'len': 'float', 'dose': 'float'})

```

Crampton, E. W. 1947. „THE GROWTH OF THE ODONTOBLASTS OF THE INCISOR TOOTH AS A CRITERION OF THE VITAMIN C INTAKE OF THE GUINEA PIG“. The Journal of Nutrition 33 (5): 491–504. <https://doi.org/10.1093/jn/33.5.491>

Der Datensatz kann in R mit dem Befehl “ToothGrowth” aufgerufen werden.



Berechnen Sie den arithmetischen Mittelwert, die Varianz, die Standardabweichung und den Stichprobenfehler der Messreihe zur Zahnlänge (len). Verwenden Sie dazu die vorgestellten Formeln.

Das Ergebnis könnte so aussehen:

N: 60

arithmetisches Mittel: 18.81

Stichprobenfehler: 0.99

Stichprobenvarianz: 58.51

Stichprobenstandardabweichung: 7.65

💡 Tipp 1: Musterlösung Verteilungskenngrößen

```
def verteilungskennwerte(x, output = True):

    # Anzahl Messwerte bestimmen
    N = len(x)

    # arithmetisches Mittel bestimmen
    stichprobenmittelwert = sum(x) / N

    # Stichprobenvarianz bestimmen
    stichprobenvarianz = sum((x - stichprobenmittelwert) ** 2) / (N - 1)

    # Standardabweichung bestimmen
    standardabweichung = stichprobenvarianz ** (1/2)

    # Stichprobenfehler bestimmen
    stichprobenfehler = standardabweichung / (N ** (1/2))

    # Ausgabe
    if output: # output = True
        print(f"N: {N}\n",
              f"arithmetisches Mittel: {stichprobenmittelwert:.2f}\n",
              f"Stichprobenfehler: {stichprobenfehler:.2f}\n",
              f"Stichprobenvarianz: {stichprobenvarianz:.2f}\n",
              f"Standardabweichung: {standardabweichung:.2f}",
              sep = '')

    else: # output = False
        return N, stichprobenmittelwert, stichprobenfehler, stichprobenvarianz, standardabweichung

verteilungskennwerte(meerschweinchen['len'])
```

1.7.2 Varianz und Standardabweichung mit NumPy und Pandas

Die Module NumPy und Pandas verfügen über eigene Funktionen zur Berechnung der Varianz und der Standardabweichung. Die Varianz und Standardabweichung werden mit den Funktionen `np.var()` und `np.std()` bzw. den Methoden `pd.var()` und `pd.std()` berechnet. Der Parameter `ddof` (delta degrees of freedom) steuert, welcher Nenner zur Berechnung der Varianz verwendet wird in der Form $N - \text{ddof}$. Während der Standardwert in NumPy `ddof=0` ist, berechnet Pandas mit dem Standardwert `ddof=1` die Stichprobenvarianz.

```
print("Varianz:")
print(f"NumPy:\t{np.var(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].var():.2f}")

print("\nStandardabweichung:")
print(f"NumPy:\t{np.std(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].std():.2f}")
```

Varianz:

NumPy: 57.54

Pandas: 58.51

Standardabweichung:

NumPy: 7.59

Pandas: 7.65

2 Die Normalverteilung

Mit zunehmender Stichprobengröße wird eine immer bessere Schätzung des Erwartungswerts erreicht. Mathematisch liegt dieser Beobachtung der [zentrale Grenzwertsatz](#) zugrunde. So werden beim Würfeln mit mehreren Würfeln weit vom Erwartungswert entfernte Wurfresultate immer unwahrscheinlicher. Dies lässt sich bereits mit wenigen Würfeln zeigen (siehe Beispiel).

i Hinweis 1: Häufigkeitsverteilung von Würfelresultaten

Für einen Würfel gibt es 6 mögliche Resultate, für 2 Würfel $6 * 6$ mögliche Kombinationen, für 3 Würfel $6 * 6 * 6$ Kombinationen und so weiter. Weil viele Kombinationen wertgleich sind, kommen Wurfresultate in der Nähe des Erwartungswerts häufiger vor als beispielsweise ein Einserpasch.

2.1 ein Würfel

```
ein_würfel = []

for i in range(1, 7):
    ein_würfel.append(i)

ein_würfel = pd.Series(ein_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(ein_würfel.value_counts(), "\n")
print(f"Durchschnitt: {ein_würfel.mean():.1f}")

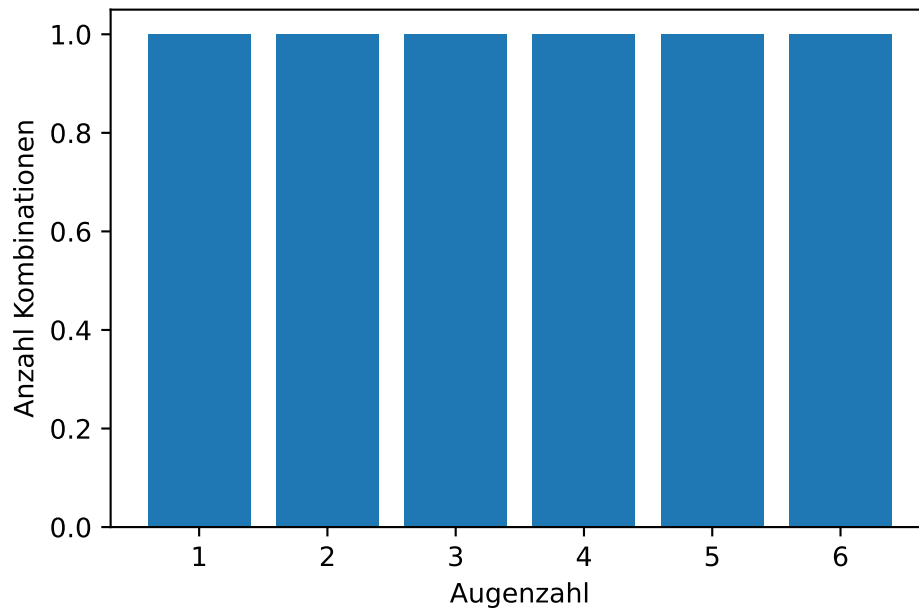
plt.bar(ein_würfel.unique(), ein_würfel.value_counts())
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.show()
```

Häufigkeitsverteilung der Augensumme:

1	1
2	1
3	1

```
4    1
5    1
6    1
Name: count, dtype: int64
```

Durchschnitt: 3.5



2.2 zwei Würfel

```

zwei_würfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j
        zwei_würfel.append(würfel_1 + würfel_2)

zwei_würfel = pd.Series(zwei_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(zwei_würfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {zwei_würfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {zwei_würfel.mean() / 2:.1f}")

plt.bar(zwei_würfel.unique(), zwei_würfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.grid()
plt.show()

```

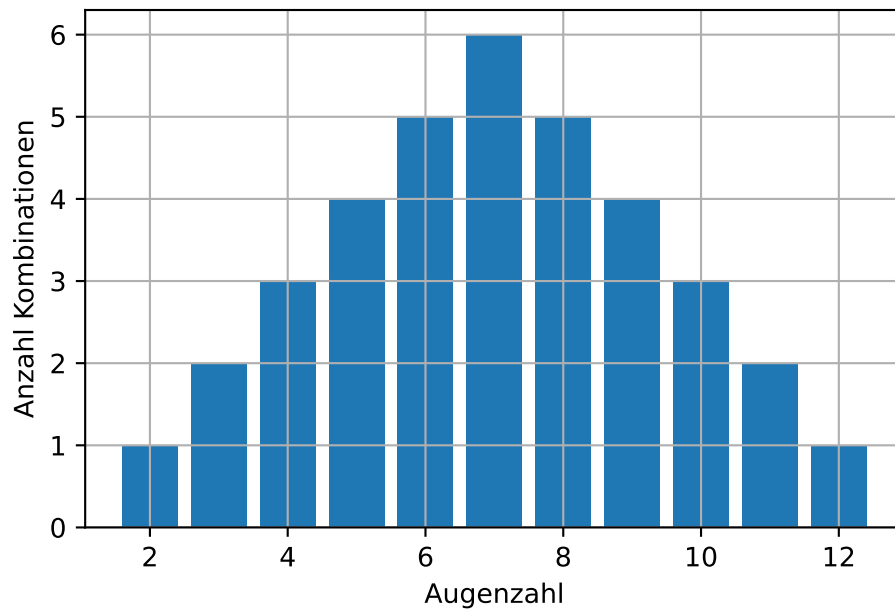
Häufigkeitsverteilung der Augensumme:

2	1
3	2
4	3
5	4
6	5
7	6
8	5
9	4
10	3
11	2
12	1

Name: count, dtype: int64

Durchschnitt: 7.0

Durchschnitt pro Würfel: 3.5



2.3 drei Würfel

```

dreiwürfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j

        for k in range (1, 7):
            würfel_3 = k
            dreiwürfel.append(würfel_1 + würfel_2 + würfel_3)

dreiwürfel = pd.Series(dreiwürfel)

print("Häufigkeitsverteilung der Augensumme:")
print(dreiwürfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {dreiwürfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {dreiwürfel.mean() / 3:.1f}")

plt.bar(dreiwürfel.unique(), dreiwürfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel ('Anzahl Kombinationen')
plt.grid()
plt.show()

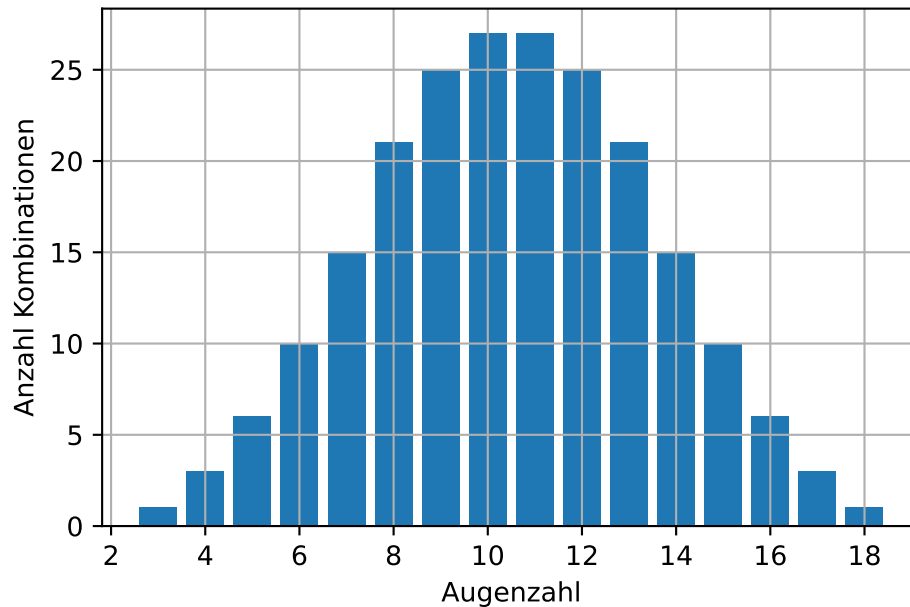
```

Häufigkeitsverteilung der Augensumme:

3	1
4	3
5	6
6	10
7	15
8	21
9	25
10	27
11	27
12	25
13	21
14	15
15	10
16	6
17	3

```
18      1
Name: count, dtype: int64

Durchschnitt: 10.5
Durchschnitt pro Würfel: 3.5
```



Die mit steigender Stichprobengröße zu beobachtende Annäherung von Messwerten an einen in der Grundgesamtheit geltenden Erwartungswert gilt auch, wenn der Erwartungswert und die Varianz in der Grundgesamtheit unbekannt sind. Mit zunehmender Stichprobengröße nähern sich die Messwerte der [Normalverteilung](#) an, die nach ihrem Entdecker Carl Friedrich Gauß auch als Gaußsche Glockenkurve bekannt ist.

Die für größere Stichproben zu beobachtende Annäherung der Verteilung von Messwerten an die Normalverteilung kann anhand des Gewichts von Pinguinen aus dem Datensatz `palmerpenguins` gezeigt werden.

palmerpenguins

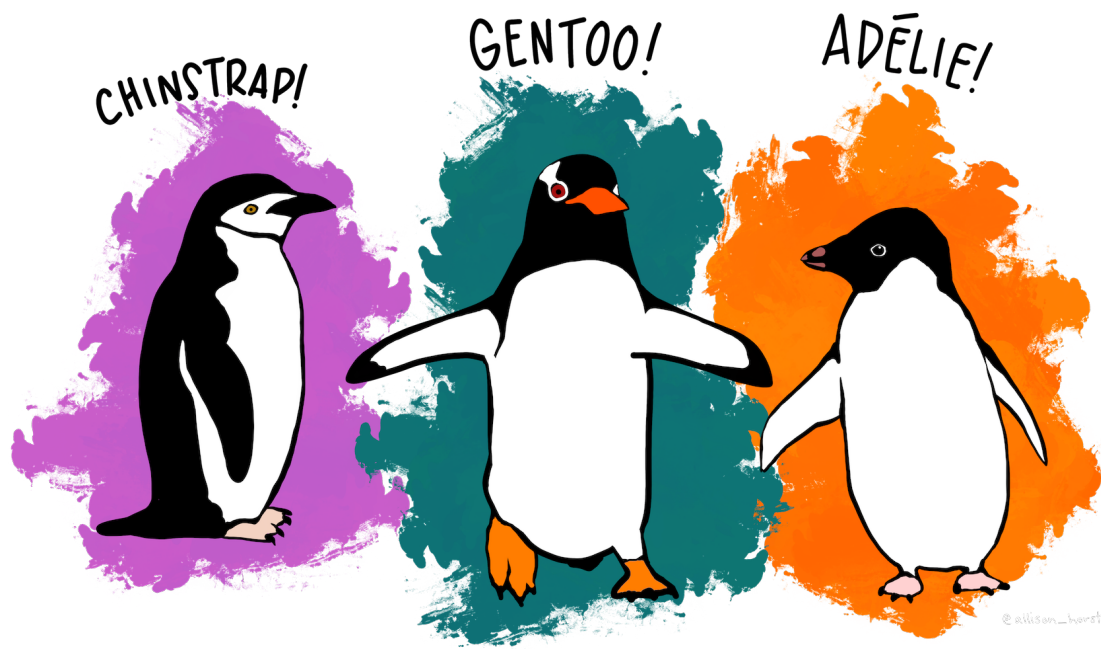


Abbildung 2.1: Pinguine des Palmer-Station-Datensatzes

Meet the Palmer penguins von @allison_horst steht unter der Lizenz [CC0-1.0](#) und ist auf [GitHub](#) abrufbar. 2020

Der Datensatz steht unter der Lizenz [CCO](#) und ist in R sowie auf [GitHub](#) verfügbar. 2020

```
# R Befehle, um den Datensatz zu laden
install.packages("palmerpenguins")
library(palmerpenguins)
```

Horst AM, Hill AP und Gorman KB. 2020. palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. <https://allisonhorst.github.io/palmerpenguins/>. doi: 10.5281/zenodo.3960218.

```
penguins = pd.read_csv(filepath_or_buffer = "01-daten/penguins.csv")

# Tiere mit unvollständigen Einträgen entfernen
penguins.drop(np.where(penguins.apply(pd.isna).any(axis = 1))[0], inplace = True)

print(penguins.info(), "\n");
```

```

<class 'pandas.DataFrame'>
Index: 333 entries, 0 to 343
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                333 non-null   str
1   island                 333 non-null   str
2   bill_length_mm         333 non-null   float64
3   bill_depth_mm          333 non-null   float64
4   flipper_length_mm      333 non-null   float64
5   body_mass_g            333 non-null   float64
6   sex                    333 non-null   str
7   year                   333 non-null   int64
dtypes: float64(4), int64(1), str(3)
memory usage: 23.4 KB
None

```

Der Datensatz enthält Daten für drei Pinguinarten.

```
print(penguins.groupby(by = penguins['species']).size())
```

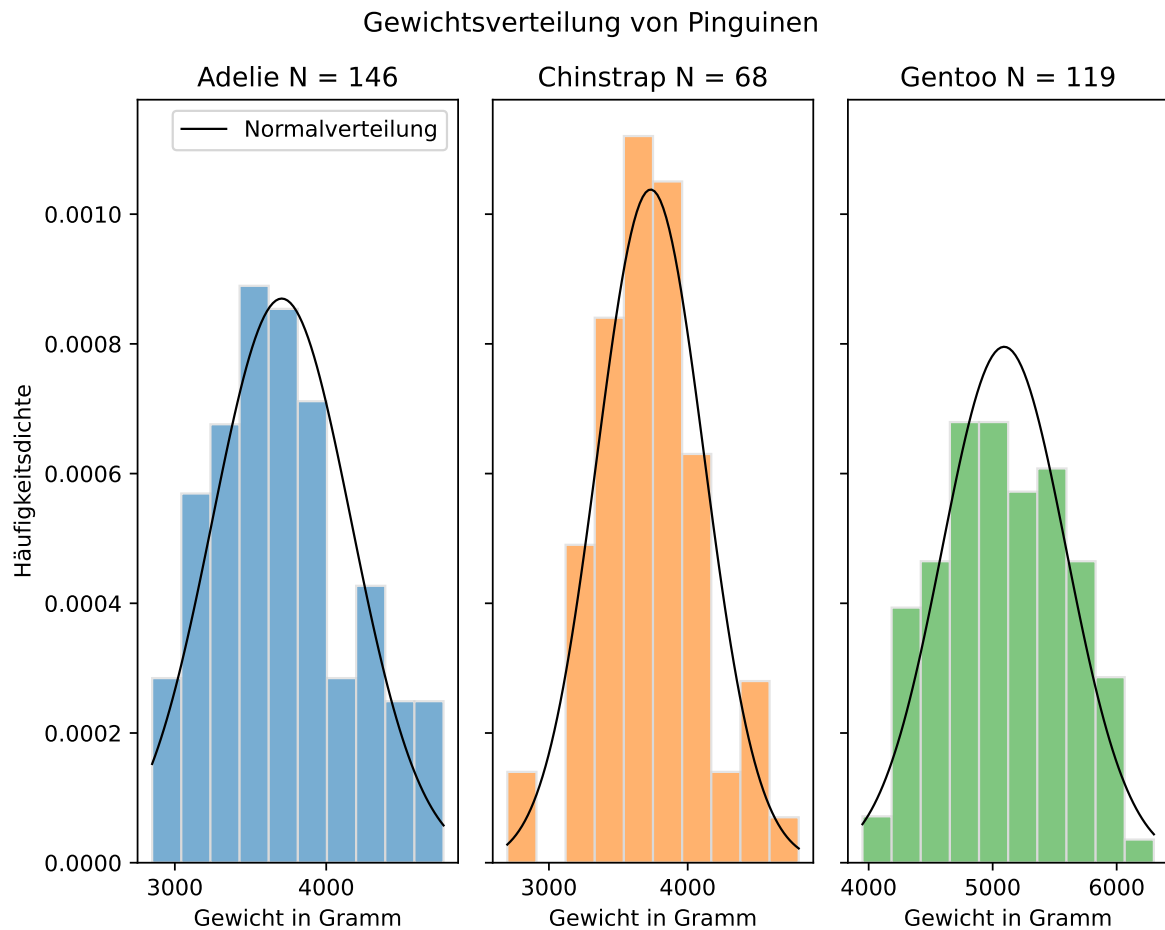
```

species
Adelie      146
Chinstrap   68
Gentoo     119
dtype: int64

```

Unter anderen wurde das Körpergewicht in Gramm gemessen, das in der Spalte 'body_mass_g' eingetragen ist. Die Gewichtsverteilung der drei Spezies wird jeweils mit einem Histogramm dargestellt. Außerdem werden für jede Spezies der Stichprobenmittelwert und die Stichprobenstandardabweichung bestimmt. Mit diesen Werten kann eine Normalverteilungskurve berechnet und in das Histogramm eingezeichnet werden (wie das geht, wird in Beispiel 2 gezeigt). So kann optisch geprüft werden, ob die empirische Verteilung der Werte in der Stichprobe einer Normalverteilung mit den selben Werten für Mittelwert und Standardabweichung entspricht. (Bei aus Messungen gewonnenen Daten ist dies häufig nicht so eindeutig, wie bei der Häufigkeitsverteilung von Würfelergebnissen.)

2.4 Grafik



2.5 Code

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von Pinguinen')

# Adelie
species = 'Adelie'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
```

```

ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(-(x_values - stichprobenmittelwert)**2 / (2 * stichprobenstandardabweichung**2))

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 'Normalverteilung')
ax1.legend()

# Chinstrap
species = 'Chinstrap'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(-(x_values - stichprobenmittelwert)**2 / (2 * stichprobenstandardabweichung**2))

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve

```

```

stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(-(x_values - stichprobenmittelwert)**2 / (2 * stichprobenstandardabweichung**2))

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

Die Normalverteilung ist eine Dichtekurve, an die sich der Verlauf eines Histogramms mit einer gegen unendlich gehenden Anzahl von Messwerten und einer gegen Null gehenden Klassenbreite annähert.

! Wichtig 2: Histogramm

Das Histogramm ist eine grafische Darstellung der Häufigkeitsverteilung kardinal skalierten Merkmale (d. h. mit numerischen, geordneten Merkmalsausprägungen). Die Daten werden in Klassen, die eine konstante oder variable Breite haben können, eingeteilt. Es werden direkt nebeneinanderliegende Rechtecke von der Breite der jeweiligen Klasse gezeichnet, deren Flächeninhalte die (relativen oder absoluten) Klassenhäufigkeiten darstellen. Die Höhe jedes Rechtecks stellt dann die (relative oder absolute) Häufigkeitsdichte dar, also die (relative oder absolute) Häufigkeit dividiert durch die Breite der entsprechenden Klasse.

Beispiel 2.2: Histogramm berechnen und visualisieren

Als Beispiel wird die Länge der zahnbildenden Zellen der Meerschweinchen verwendet, die eine Vitamin-C-Dosis von 2 erhielten.

```

dose2 = meerschweinchen.loc[meerschweinchen['dose'] == 2, 'len']
print(*list(dose2)) # * = Ausgabe ohne Kommata
print("N", len(dose2), "Minimum:", dose2.min(), "Maximum:", dose2.max(), "Spannweite",

```

```

23.6 18.5 33.9 25.5 26.4 32.5 26.7 21.5 23.3 29.5 25.5 26.4 22.4 24.5 24.8 30.9 26.4 2
N 20 Minimum: 18.5 Maximum: 33.9 Spannweite 15.399999999999999

```

Mit der Funktion `np.histogram(a, bins = 10, range = None, density = None)` kann ein Histogramm berechnet werden.

- `a` sind die zu berechnenden Daten
- `bins` spezifiziert die Anzahl an Klassen, standardmäßig werden 10 gewählt.

- `range = (float, float)` erlaubt es, die untere und obere Grenze der Klassen festzulegen.
- `density = True` erlaubt es statt der absoluten Häufigkeiten, den Wert der Häufigkeitsdichtefunktion darzustellen. Dies berechnet sich wie folgt:
 - relative Häufigkeit = Anzahl Werte je Klasse / Anzahl aller Werte
 - Häufigkeitsdichte = Anzahl Werte je Klasse / (Anzahl aller Werte * Klassenbreite)
 - Klassenbreite = Maximum(Werte) - Minimum(Werte) / Anzahl Klassen

Für die überschaubare Anzahl an Werten wird ein Histogramm mit 5 Klassen berechnet. Zum Vergleich wird auch die Häufigkeitsdichte ausgegeben.

```
print(np.histogram(dose2, bins = 5))
print("Häufigkeitsdichte:", np.histogram(dose2, bins = 5, density = True)[0])
```

```
(array([2, 5, 8, 2, 3]), array([18.5 , 21.58, 24.66, 27.74, 30.82, 33.9 ]))
Häufigkeitsdichte: [0.03246753 0.08116883 0.12987013 0.03246753 0.0487013 ]
```

Die Funktion `np.histogram()` gibt an erster Stelle ein array mit den absoluten Häufigkeiten bzw. der Häufigkeitsdichte jeder Klasse zurück. An zweiter Stelle wird ein array mit den x-Positionen der Klassenrechtecke zurückgegeben - dabei wird für jede Klasse die Position der linken Seite sowie für die letzte Klasse zusätzlich die Position der rechten Seite des Rechtecks ausgegeben. Für 5 Klassen werden also 6 Positionswerte ausgegeben.

Die Klassenbreite kann zum Beispiel mit der Methode `np.diff()` ausgegeben werden.

```
hist_abs, bin_edges = np.histogram(dose2, bins = 5)
klassenbreite = np.diff(bin_edges)
print(klassenbreite)
```

```
[3.08 3.08 3.08 3.08 3.08]
```

Durch Multiplikation der Häufigkeitsdichte mit der Klassenbreite können die relativen Häufigkeiten berechnet werden.

```
hist_dichte = np.histogram(dose2, bins = 5, density = True)[0]
hist_relativ = hist_dichte * klassenbreite
print(hist_relativ)
```

```
[0.1  0.25 0.4  0.1  0.15]
```

Die Summe der relativen Häufigkeiten ist 1.

Ein Histogramm kann mit der Funktion `plt.hist(x, bins = None, *, range = None, density = False)` aufgerufen werden, welche intern `np.histogram()` für die Berechnungen aufruft. Die Parameter der Funktion entsprechen denen der NumPy-Funktion, wobei mit dem Argument `x` die darzustellenden Daten übergeben werden. Zusätzlich können verschiedene Grafikparameter übergeben werden.

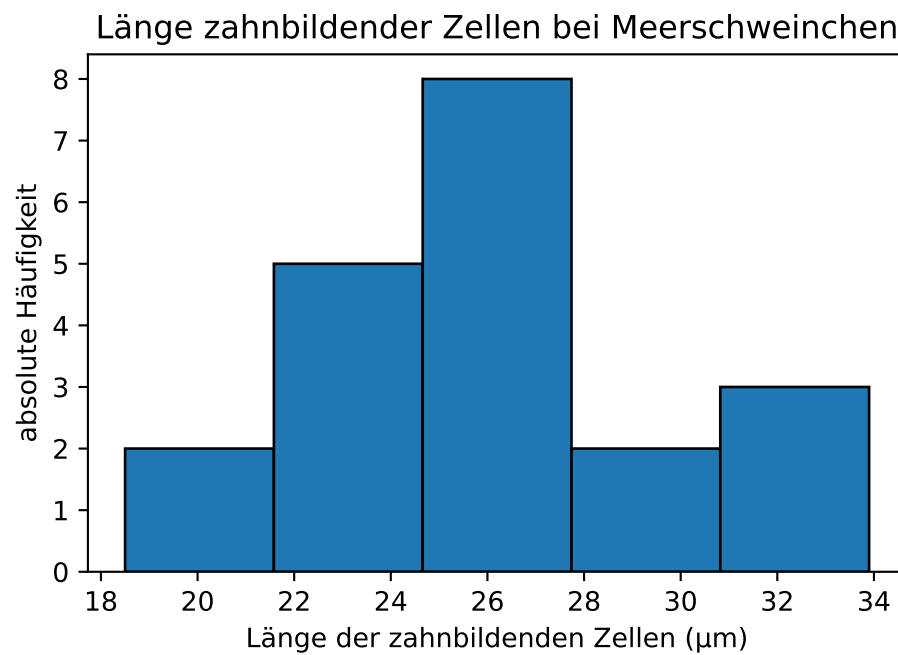
Die Funktion hat 3 Rückgabewerte: die absolute Häufigkeit der Klassen (bzw. wenn `density = True` die Häufigkeitsdichte), die x-Position der Rechtecke und die Objekte für die Grafikerstellung (letztere werden im folgenden Code im Objekt `ignore` gespeichert und nicht weiter verwendet.).

2.6 absolute Häufigkeit

```
plt.hist(dose2, bins = 5, edgecolor = 'black')
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('absolute Häufigkeit')

plt.show()
```



2.7 relative Häufigkeit

Eine Darstellung der relativen Häufigkeiten ist nicht direkt möglich.


```

hist_dichte, bins, ignore = plt.hist(dose2, bins = 5, density = True, edgecolor = 'black')
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

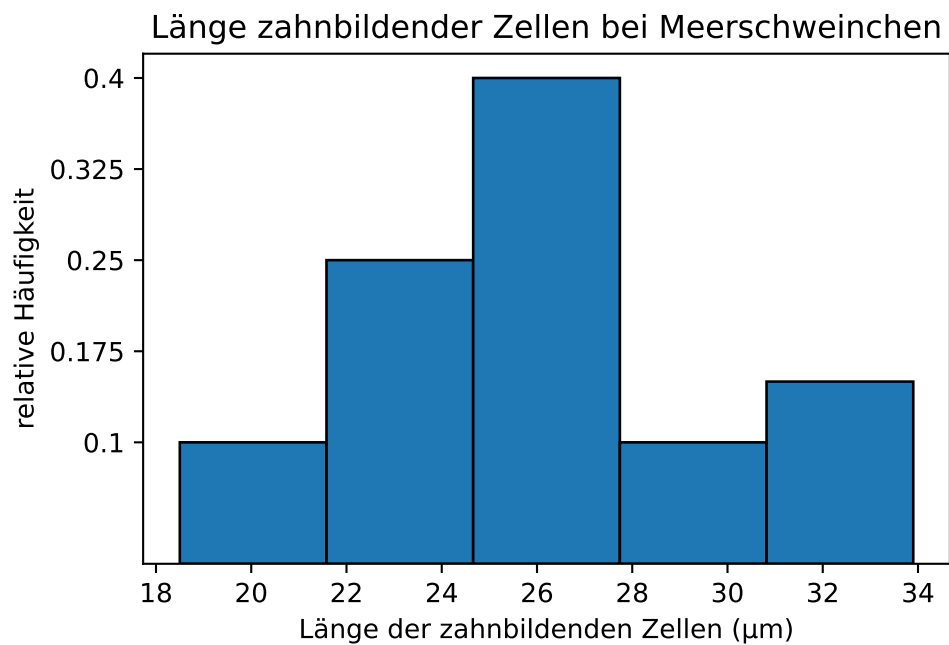
# relative Häufigkeit berechnen
klassenbreite = np.diff(bins)[0]
hist_relativ = hist_dichte * klassenbreite

# yticks erzeugen an der Position von min(hist_dichte) bis max(hist_dichte)
# aber mit Werten von hist_relativ
plt.yticks(ticks = np.linspace(min(hist_dichte), max(hist_dichte), len(hist_relativ)),
labels = np.linspace(hist_relativ.round(2).min(), hist_relativ.round(2).max(), len(hist_relativ)))

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (µm)')
plt.ylabel('relative Häufigkeit')

plt.show()

```

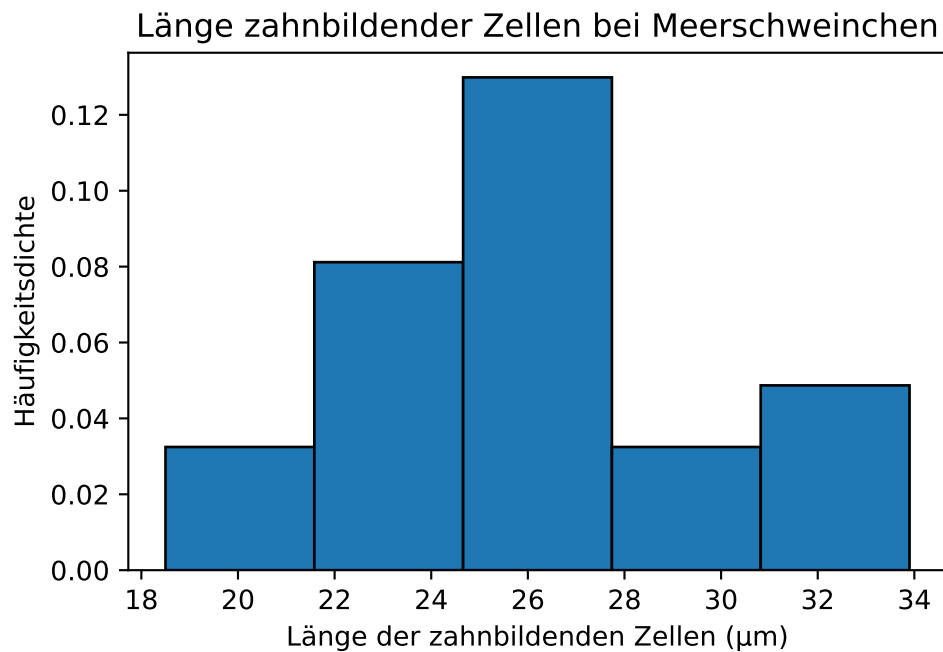


2.8 Häufigkeitsdichte

```
plt.hist(dose2, bins = 5, density = True, edgecolor = 'black')
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

plt.show()
```



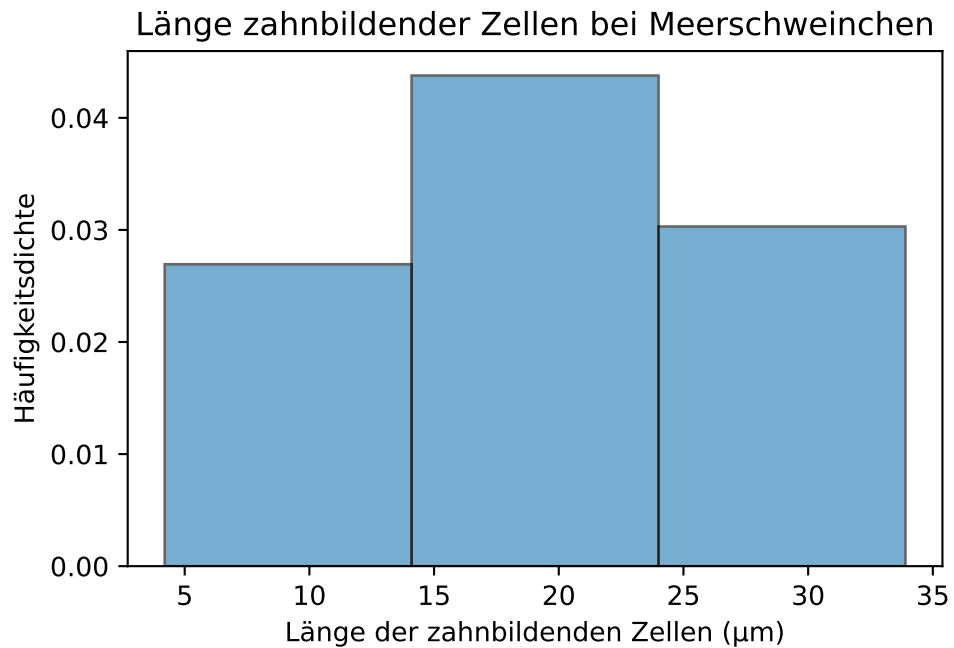
Histogramme sind nicht immer gut geeignet, um die Verteilung einer Stichprobe zu charakterisieren. Der visuelle Eindruck hängt von der gewählten Klassenzahl ab - ein Beispiel:

2.9 3 Klassen

```
plt.hist(meerschweinchen['len'], bins = 3, density = True, edgecolor = 'black', alpha
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

plt.show()
```

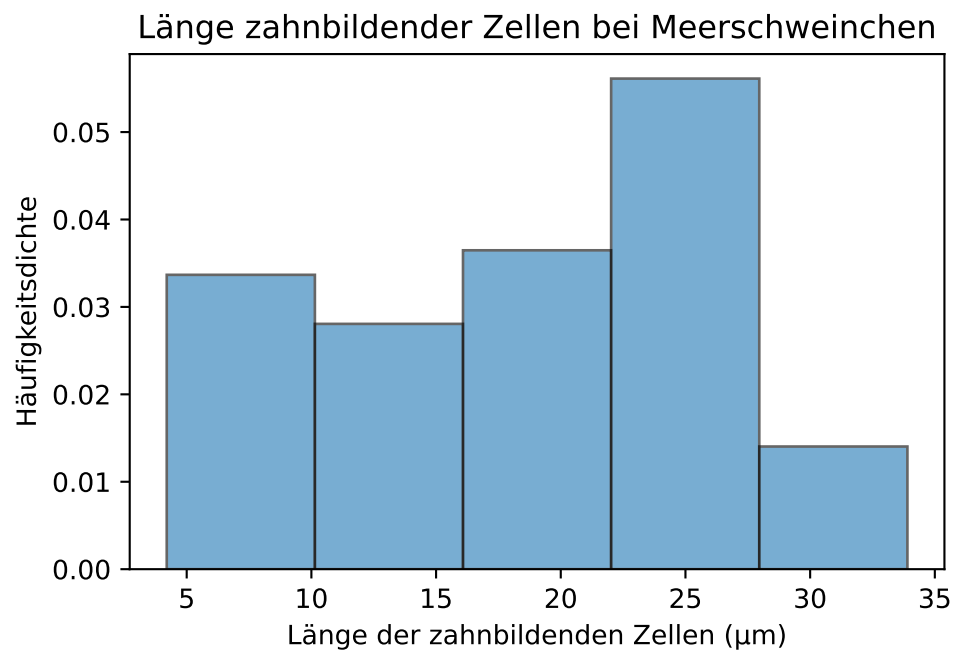


2.10 5 Klassen

```
plt.hist(meerschweinchen['len'], bins = 5, density = True, edgecolor = 'black', alpha
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

plt.show()
```

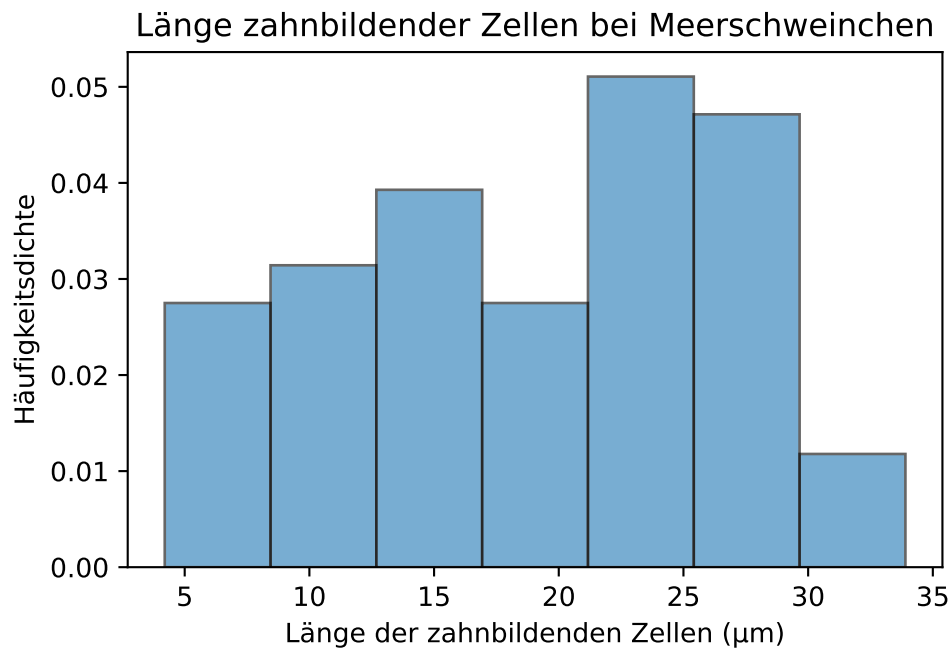


2.11 7 Klassen

```
plt.hist(meerschweinchen['len'], bins = 7, density = True, edgecolor = 'black', alpha=0.5)
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

plt.show()
```



Die Dichtefunktion der Normalverteilung beschreibt, welcher Anteil der Werte innerhalb eines bestimmten Wertebereichs liegt. Bei der Berechnung der relativen Häufigkeiten in Beispiel 2 haben wir gesehen, dass die Summe der relativen Häufigkeiten 1 ist. Dies entspricht der Fläche unterhalb der Dichtekurve.

Die Dichtefunktion der Normalverteilung ist definiert als:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Die Form der Normalverteilung ergibt sich aus dem Faktor $e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$ der Funktionsgleichung. Das Maximum der Funktion liegt am Punkt $x = \mu$. Von dort fällt sie symmetrisch ab und nähert sich der x-Achse an. Der Abfall der Funktion erfolgt umso schneller, je kleiner σ ist. Die Wendepunkte der Kurve liegen jeweils eine Standardabweichung vom Mittelwert entfernt.

Eine Normalverteilung mit dem Mittelwert $\mu = 0$ und einer Standardabweichung $\sigma = 1$ heißt Standardnormalverteilung.

(Baitsch, Matthias 2019, S. 51–54)

2.12 Normalverteilung anpassen

Um die Verteilung in einem Datensatz durch eine Normalverteilung anzunähern, werden dessen Mittelwert und Standardabweichung in die Funktionsgleichung der Normalverteilung eingesetzt. Mit Python können die Berechnungen direkt vorgenommen werden. In der Handhabung einfacher sind die vom Paket SciPy bereitgestellten Funktionen, die im nächsten Abschnitt ausführlicher vorgestellt werden. Das folgende Beispiel zeigt die Berechnung und Visualisierung mit Python und mit SciPy.

i Hinweis 2: Dichtekurven berechnen und darstellen

Betrachten wir die Verteilungskennwerte der Gruppe der Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten.

```
print(verteilungskennwerte(dose2), "\n");

dose2_mean = verteilungskennwerte(dose2, output = False)[1]
dose2_std = verteilungskennwerte(dose2, output = False)[4]

print("Exakter Mittelwert:", dose2_mean)
print("Exakte Standardabweichung:", dose2_std)
```

```
N: 20
arithmetisches Mittel: 26.10
Stichprobenfehler: 0.84
Stichprobenvarianz: 14.24
Stichprobenstandardabweichung: 3.77
None
```

```
Exakter Mittelwert: 26.1
Exakte Standardabweichung: 3.7741503052098744
```

Wenn wir die Standardabweichung und das arithmetische Mittel in die Normalverteilungsfunktion einsetzen, erhalten wir:

$$f(x) = \frac{1}{3.7742\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

$$f(x) = 0.1057 \cdot e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

In Python können die Berechnungen umgesetzt und grafisch dargestellt werden:

```

# Histogram der Häufigkeitsdichte zeichnen
plt.hist(dose2, bins = 7, density = True, edgecolor = 'black', alpha = 0.6);
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

# Normalverteilung berechnen.
hist, bin_edges = np.histogram(dose2, bins = 7)

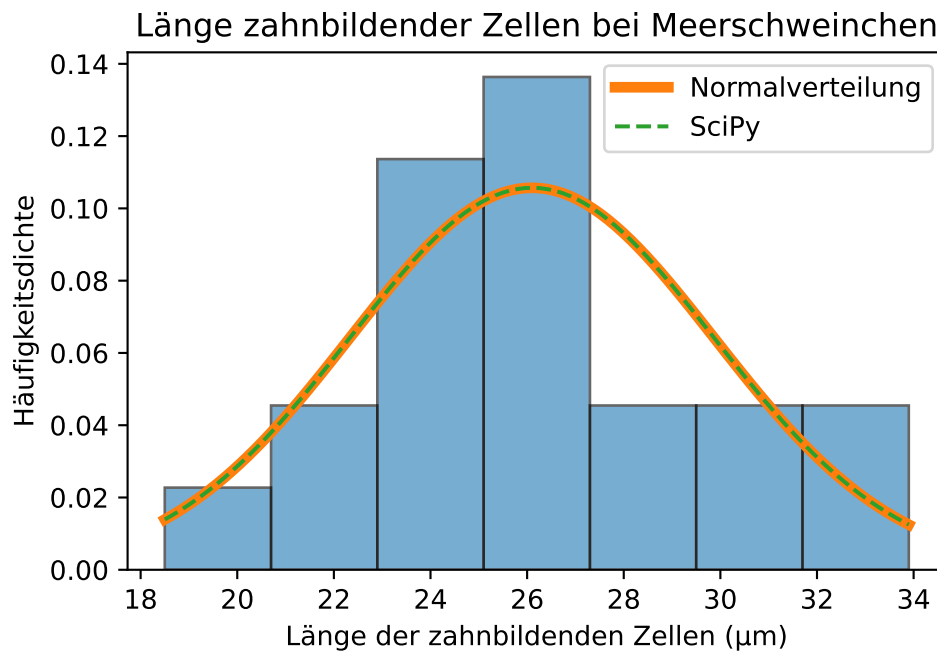
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)

## Normalverteilungsfunktion mit Python berechnen
y_values = 1 / (dose2_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - dose2_mean) ** 2 /
plt.plot(x_values, y_values, label = 'Normalverteilung', lw = 4)

## scipy
y_values_scipy = scipy.stats.norm.pdf(x_values, loc = dose2_mean, scale = dose2_std)
plt.plot(x_values, y_values_scipy, label = 'SciPy', linestyle = 'dashed')

plt.legend()
plt.show()

```



Die Verteilung der Länge zahnbildender Zellen bei Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten, könnte einer Normalverteilung entsprechen. Aufgrund der geringen Stichprobengröße ist dies aber schwer zu beurteilen.

2.13 Das Paket SciPy

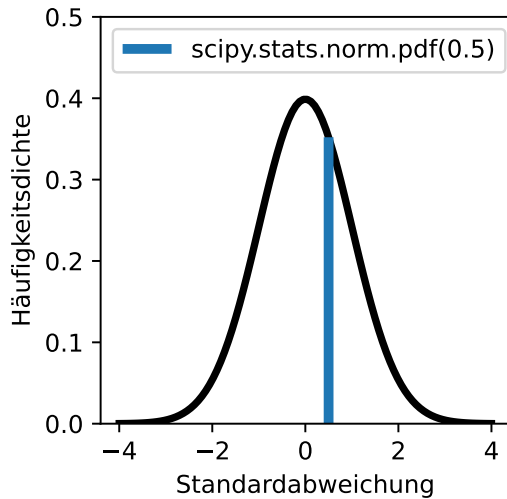
Funktionen zur Berechnung von Dichtekurven können über Paket SciPy importiert werden. Das Modul stats (statistical functions) umfasst zahlreiche Funktionen zum Testen von Hypothesen. Funktionen für die Normalverteilung werden wie folgt aufgerufen:

```
import scipy
print("Häufigkeitsdichte der Normalverteilung bei x = 0:", scipy.stats.norm.pdf(0), "\n")
```

Häufigkeitsdichte der Normalverteilung bei x = 0: 0.3989422804014327

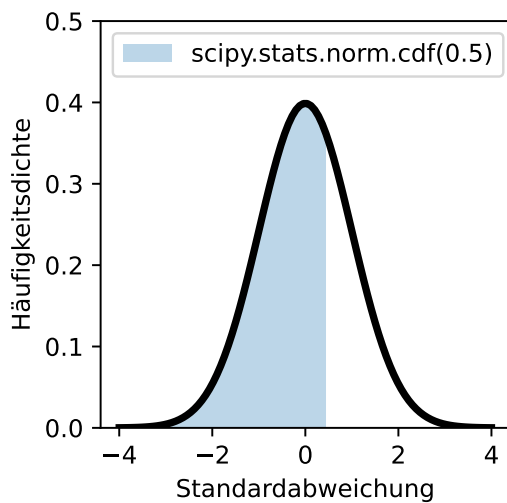
Für die Normalverteilung sind vier Funktionen relevant:

Mit den Parametern `loc = mittelwert` und `scale = standardabweichung` kann die Form der Normalverteilung angepasst werden. Standardmäßig wird die Standardnormalverteilung



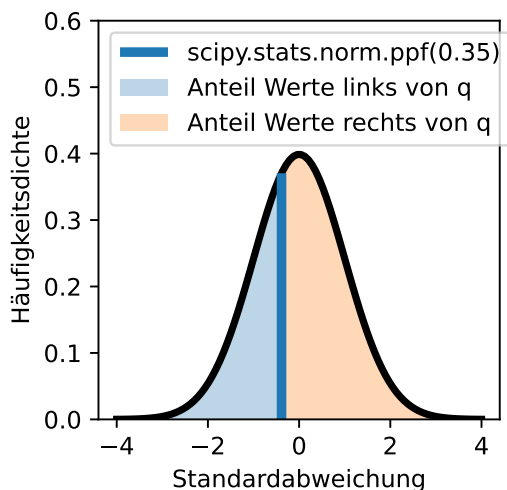
Beschreibung

Die Funktion `scipy.stats.norm.pdf(x)` berechnet die Dichte der Normalverteilung am Punkt x (pdf = probability density function). x kann auch ein array sein - so wurde die linksstehende Kurve mit dem Befehl `scipy.stats.norm.pdf(np.linspace(-4, 4, 100))` berechnet.



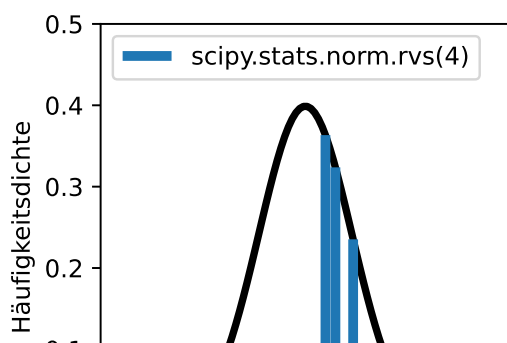
Beschreibung

Die Funktion `scipy.stats.norm.cdf(x)` berechnet den Anteil der Werte links von x (cdf = cumulative density function).



Beschreibung

Die Funktion `scipy.stats.norm.ppf(q)` ist die Quantilfunktion der Normalverteilung und die Umkehrfunktion der kumulativen Häufigkeitsdichtefunktion (cdf). Die Funktion berechnet für $0 \leq q \leq 1$ den Wert x , links von dem der Anteil q aller Werte liegt und rechts von dem der Anteil $1-q$ liegt (ppf = percentile point function).



45

Beschreibung

Die Funktion `scipy.stats.norm.rvs(size)` zieht `size` Zufallszahlen aus der Normalverteilung.

Hinweis: Die Zufallszahlen werden im Skript dynamisch gezogen.

mit `loc = 0` und `scale = 1` berechnet. Die Parameter der Funktionen können Einzelwerte (Skalare) oder auch Arrays bzw. Listen sein.

2.14 Aufgaben Normalverteilung

Möglicherweise haben Sie schon einmal von Mensa International gehört, einer Vereinigung für Hochbegabte. Wer Mitglied in dieser Vereinigung werden möchte, soll einen höheren Intelligenzquotienten (IQ) haben als 98 % der Bevölkerung seines/ihrer Herkunftslandes ([Wikipedia](#)).

1. Wenn der durchschnittliche IQ 100 und die Standardabweichung 15 beträgt, welchen IQ müssten Sie haben, um bei Mensa International aufgenommen zu werden?
2. Mensa International ist nicht die einzige Organisation ihrer Art. Andere Organisationen haben sogar noch strengere Kriterien. Welcher IQ wird benötigt, um hier Mitglied zu werden?
 - Intertel (Kriterium: IQ aus dem höchsten 1 %)
 - Triple Nine Society (Kriterium: IQ aus dem höchsten 0,1 %)
 - Prometheus Society (Kriterium: IQ aus dem höchsten 0,003 %)
3. Der IQ ist nicht mit angeborener Intelligenz gleichzusetzen und auch abhängig davon, wie viel Gelegenheit man zum Gehirntraining hatte, etwa durch den Schulbesuch. Der niedrigste durchschnittliche IQ wurde mit 71 [im Land Niger](#) gemessen. Angenommen Sie hätten einen IQ von 100. Würden Sie in Niger das Kriterium der Mensa International erfüllen?

Musterlösung Normalverteilung

Aufgabe 1: Einen IQ von mehr als ...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.98))
```

130.80623365947733

Aufgabe 2: Sie benötigen einen IQ von mindestens...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.99))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.999))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 1 - (0.003 / 100)))
```

134.8952181106126

146.3534845925172

160.19216216677682

Aufgabe 3: Nicht ganz.

```
print(scipy.stats.norm.cdf(loc = 71, scale = 15, x = 100))
```

0.9734024259789904

Übrigens: Wie [der Spiegel berichtet](#), schneiden Studierende mit mittelmäßigem Intelligenzquotienten ebenso erfolgreich ab wie Hochbegabte, vorausgesetzt sie sind neugierig genug und arbeiten gewissenhaft.

2.15 Konfidenzintervalle

Die schließende Statistik beruht auf dem Prinzip, von Stichprobenwerten auf den tatsächlichen Wert in der Grundgesamtheit zu schließen. Die Überlegung ist wie folgt:

1. Wenn eine Stichprobe aus einer Grundgesamtheit gezogen wird, dann streuen die Stichprobenwerte normalverteilt um den Mittelwert der Grundgesamtheit. Bei einer Normalverteilung liegen
 - 68,27 % aller Werte im Intervall $\pm 1 s$,
 - 95,45 % aller Werte im Intervall $\pm 2 s$ und
 - 99,73 % aller Werte im Intervall $\pm 3 s$.
2. Mit der gleichen Wahrscheinlichkeitsverteilung liegt der unbekannte Mittelwert der Grundgesamtheit um einen zufälligen Wert aus der Stichprobe.
3. Der Erwartungswert kann mit einer gewissen Wahrscheinlichkeit aus dem Standardfehler des Mittelwerts einer Stichprobe geschätzt werden. Man wählt dazu ein Konfidenzniveau, also eine Vertrauenswahrscheinlichkeit, dass der Erwartungswert tatsächlich im Bereich der Schätzung liegt. Der umgekehrte Fall, dass der Erwartungswert nicht im Bereich der Schätzung liegt, wird Signifikanz- oder Alphaniveau genannt und mit dem griechischen Buchstaben α (alpha) gekennzeichnet. α liegt im Bereich 0 - 1, das Konfidenzniveau ist $1 - \alpha$ (siehe: [Fehler 1. und 2. Art](#)).
 - der Erwartungswert liegt in 68,27 % aller Fälle im Intervall $\pm 1 \frac{s}{\sqrt{n}}$,
 - der Erwartungswert liegt in 95,45 % aller Fälle im Intervall $\pm 2 \frac{s}{\sqrt{n}}$ und
 - der Erwartungswert liegt in 99,73 % aller Fälle im Intervall $\pm 3 \frac{s}{\sqrt{n}}$.

Häufig wird das Alphaniveau $\alpha = 0.05$ bzw. das Konfidenzintervall 95 % gewählt, was $\pm 1.96 \frac{s}{\sqrt{n}}$ entspricht. Dies gilt aber nur für große Stichproben. Für kleine Stichprobengrößen folgen die Stichprobenmittelwerte der t-Verteilung, die im nächsten Abschnitt vorgestellt wird.

to do Maik: hier könnte / müsste man noch einseitige und zweiseitige Hypothesentests und den Begriff “Alpha-Halbe” einführen. Das ließe sich auch gut grafisch mit nur nach rechts gehenden und beidseitigen Pfeilen darstellen.

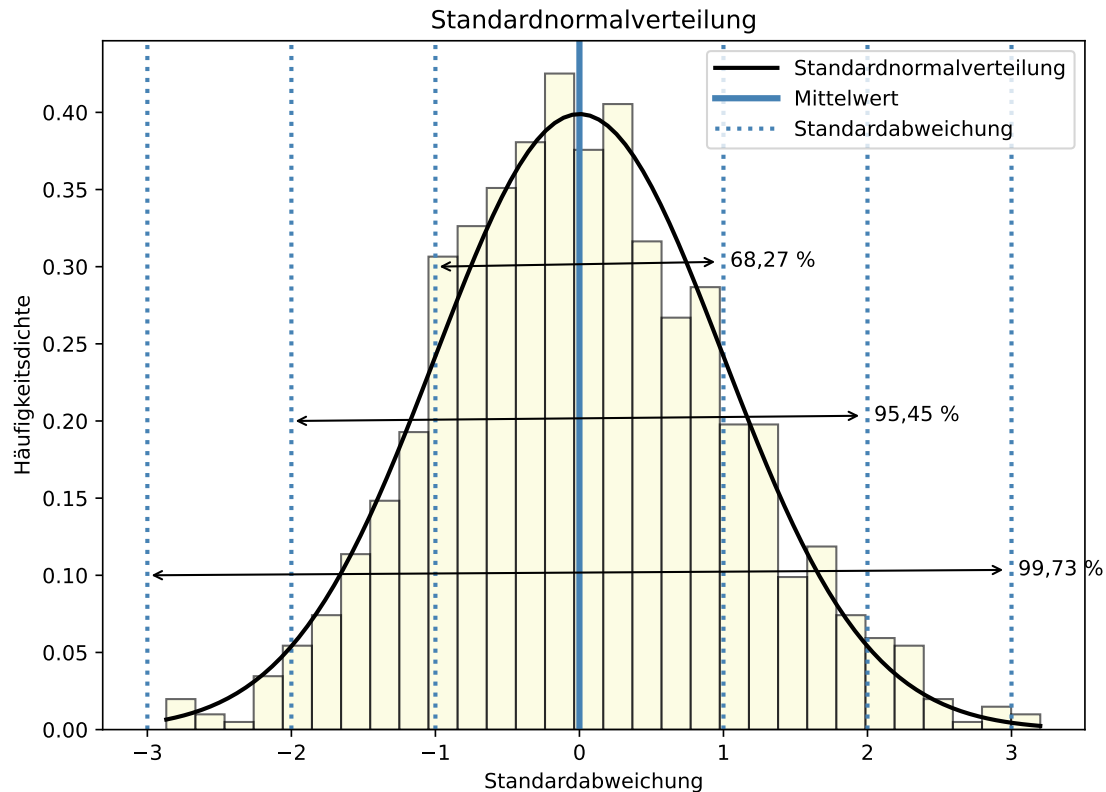
Im folgenden Beispiel wird die Idee, dass mit einer gewissen Wahrscheinlichkeit vom Stichprobenmittelwert auf den Mittelwert der Grundgesamtheit (Erwartungswert) geschlossen werden kann, noch einmal grafisch dargestellt.

i Hinweis 3: Prinzip der schließenden Statistik

2.16 Standardnormalverteilung

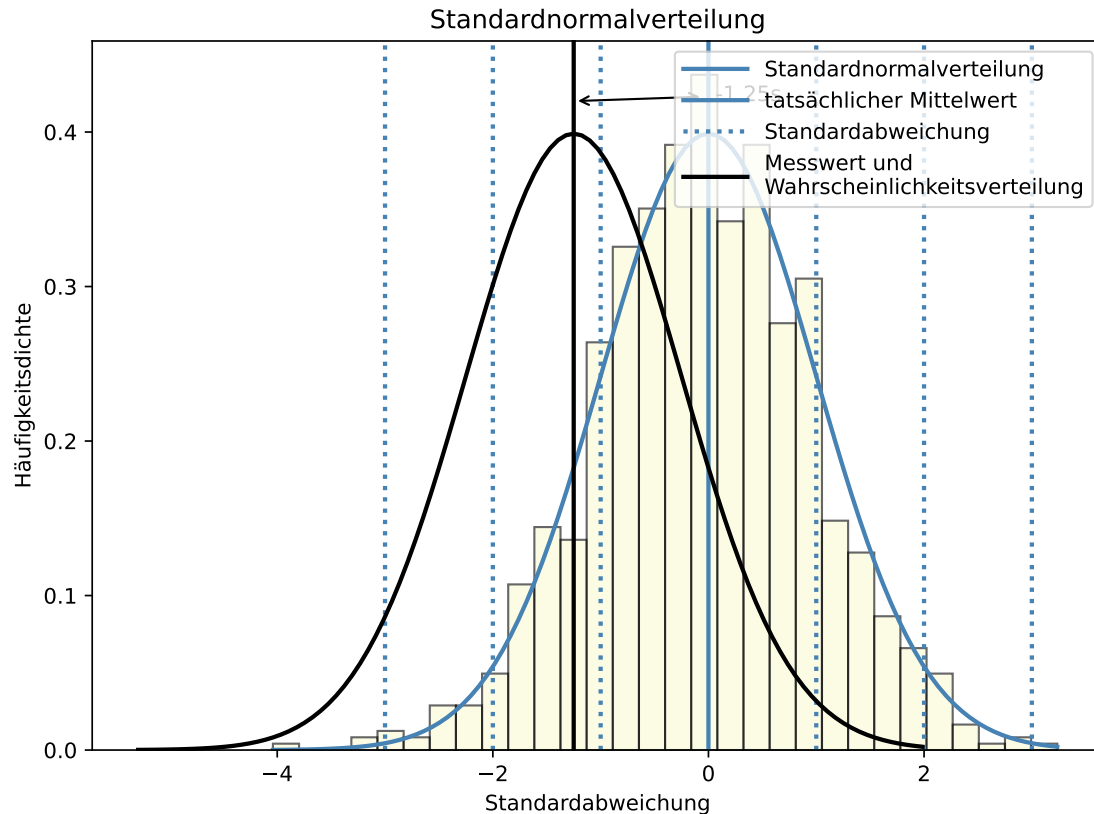
In einer Normalverteilung kommen Werte in der Nähe des Erwartungswerts häufiger vor als weit vom Erwartungswert entfernt liegende Werte. Wie häufig oder selten ein Wert relativ zum Mittelwert der Verteilung vorkommt, kann mit der Standardabweichung ausgedrückt werden.

In der Grafik sehen Sie zufällig gezogene Werte und eine Normalverteilungskurve.



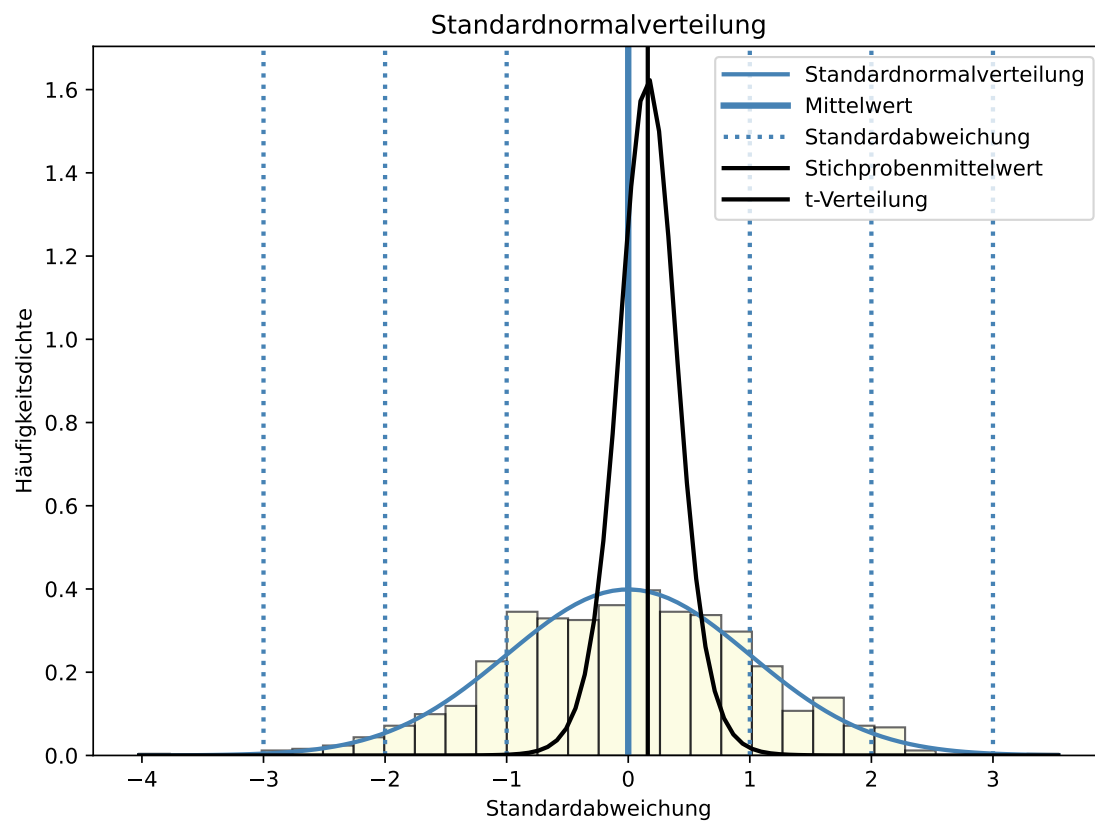
2.17 Einzelner Messwert

Ein einzelner Messwert aus der Verteilung entspricht so gut wie nie dem Erwartungswert. Man weiß aber, dass ein Wert nahe des Erwartungswerts häufiger vorkommt, als ein weit entfernter.



2.18 Stichprobe $N = 12$

Der Mittelwert einer Stichprobe entspricht ebenfalls so gut wie nie dem Erwartungswert. Der zu erwartende, in Einheiten des Standardfehlers des Mittelwerts ausgedrückte Streuungsbereich ist jedoch erheblich schmäler als der eines einzelnen Messwerts. Dies ist auch grafisch gut zu sehen. Die Dichtekurve ist erheblich schmäler und höher als die Normalverteilungskurve eines einzelnen Messwerts. Dies liegt an der geringen Standardabweichung in der Stichprobe von ~ 0.2 , was die Kurve staucht.



2.19 Code

Code für das Panel Stichprobe $N = 12$

```

import numpy as np
import matplotlib.pyplot as plt
import scipy

# Parameter der Standardnormalverteilung
mu, sigma = 0, 1 # Mittelwert und Standardabweichung

# Daten generieren
seed = 4
np.random.seed(seed = seed)
data = np.random.default_rng().normal(mu, sigma, 1000)

# Grafik
plt.figure(figsize = (8.5, 6))

# Histogramm plotten
array, bins, patches = plt.hist(data, bins = 30, density = True, alpha = 0.6, color = 'lightblue')

# Mittelwert einzeichnen
mean_line = plt.axvline(mu, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# positive und negative Standardabweichungen einzeichnen
pos_std_lines = [plt.axvline(mu + i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]
neg_std_lines = [plt.axvline(mu - i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]

# Normalverteilungskurve
x_values = np.linspace(min(bins), max(bins), 100)
y_values = 1 / (sigma * np.sqrt(2 * np.pi)) * np.exp(- (x_values - mu) ** 2 / (2 * sigma ** 2))
normal_dist_curve = plt.plot(x_values, y_values, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# Stichprobe
N = 12
np.random.seed(seed = 4)
stichprobe = np.random.default_rng().normal(mu, sigma, N)

stichprobenstandardabweichung = stichprobe.std(ddof = 1)
stichprobenmittelwert = stichprobe.mean()
standardfehler = stichprobenstandardabweichung / np.sqrt(len(stichprobe))

# Histogramm berechnen
# hist, bins = np.histogram(stichprobe, bins = 30, density = True)

# Standardfehlerkurve Stichprobe
# x_values = np.linspace(min(bins), max(bins), 100)
x = np.linspace(stichprobenmittelwert - 4 * stichprobenstandardabweichung, stichprobenmittelwert + 4 * stichprobenstandardabweichung, 100)
y_values = scipy.stats.t.pdf(x = x_values, df = N - 1, loc = stichprobenmittelwert, scale = standardfehler)

# Stichprobenmittelwert einzeichnen
mean_stichprobe = plt.axvline(stichprobenmittelwert, color = 'black', linestyle = 'solid', linewidth = 3)

# Verteilungskurve einzeichnen
stichprobe_dist_curve = plt.plot(x_values, y_values, color = 'black', linestyle = 'solid', linewidth = 3)

```

2.20 Die t-Verteilung

Die t-Verteilung wurde von William Sealy Gosset entdeckt (wenngleich nicht als erstem) und popularisiert. Die Verteilung ist auch als Student'sche Verteilung bekannt: Da Gossets Arbeitgeber, die Guinness-Brauerei, die Veröffentlichung der Entdeckung nicht gestattete, publizierte Gosset unter dem Synonym Student. ([Wikipedia](#))

Die t-Verteilung beschreibt die Verteilung von Stichprobenmittelwerten mit unbekannter Varianz in der Grundgesamtheit, deren Standardfehler mit der Stichprobenstandardabweichung geschätzt wird. Die t-Verteilung hat gegenüber der Normalverteilung die [Anzahl der Freiheitsgrade](#) als zusätzlichen Parameter

! Wichtig 3: Anzahl Freiheitsgrade

“Die Anzahl unabhängiger Information, die in die Schätzung eines Parameters einfließen, wird als Anzahl der Freiheitsgrade bezeichnet. Im Allgemeinen sind die Freiheitsgrade einer Schätzung eines Parameters gleich der Anzahl unabhängiger Einzelinformationen, die in die Schätzung einfließen, abzüglich der Anzahl der zu schätzenden Parameter, die als Zwischenschritte bei der Schätzung des Parameters selbst verwendet werden. Beispielsweise fließen n Werte in die Berechnung der Stichprobenvarianz ein. Dennoch lautet die Anzahl der Freiheitsgrade $n - 1$, da als Zwischenschritt der Mittelwert geschätzt wird und somit ein Freiheitsgrad verloren geht.”

Anzahl der Freiheitsgrade (Statistik). von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia](#). 2025

Die allgemeine Häufigkeitsdichtefunktion der t-Verteilung lautet:

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

- ν (ny) ist die Anzahl der Freiheitsgrade.
- Γ ist die [Gammafunktion](#), die für ganzzahlige Argumente n den Wert $\Gamma(n) = (n-1)!$ hat.

Da für die Berechnung des Stichprobenmittelwerts die Anzahl der Freiheitsgrade $n - 1$ ist, kann auch geschrieben werden:

$$f(x) = \frac{\Gamma\left(\frac{n}{2}\right)}{\sqrt{(n-1)\pi} \Gamma\left(\frac{n-1}{2}\right)} \left(1 + \frac{x^2}{n-1}\right)^{-\frac{n}{2}}$$

- n ist die Stichprobengröße.

Das Modul `scipy.stats` stellt Funktionen zur Berechnung der t-Verteilung bereit.

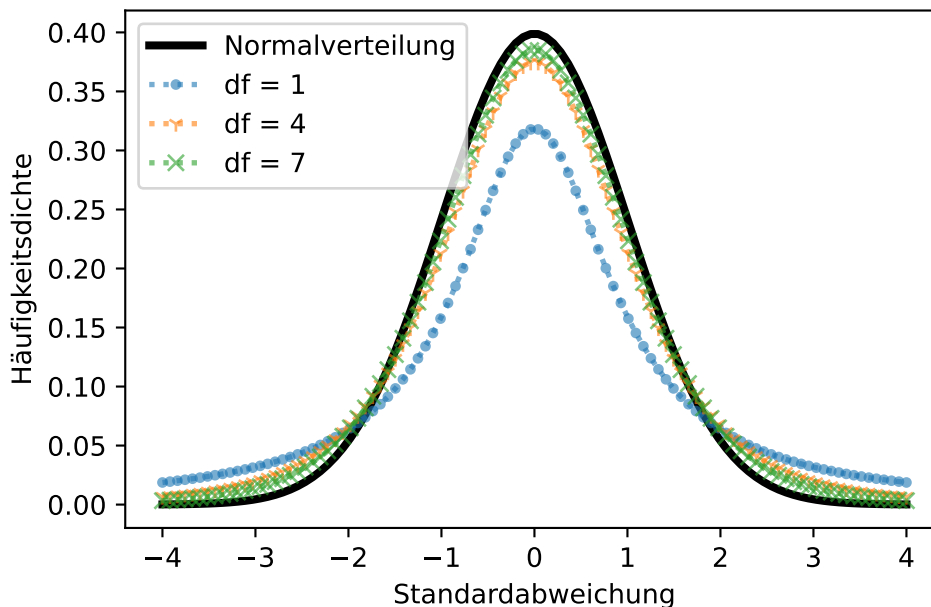
- `scipy.stats.t.pdf(x, df, loc=0, scale=1)` berechnet die Häufigkeitsdichte für die Werte x für eine t-Verteilung mit df Freiheitsgraden, Mittelwert `loc` und Standardabweichung `scale` (pdf = probability density function).
- `scipy.stats.t.cdf(x, df, loc=0, scale=1)` berechnet den Anteil der Werte links von x (cdf = cumulative density function).
- `scipy.stats.t.ppf(q, df, loc=0, scale=1)` ist die Quantilfunktion der t-Verteilung (ppf = percentile point function).
- `scipy.stats.t.rvs(df, loc=0, scale=1, size=1)` zieht `size` Zufallzahlen aus der t-Verteilung.

Die Parameter der Funktionen können Einzelwerte (Skalare) oder auch Arrays bzw. Listen sein.

Mit zunehmender Stichprobengröße nähert sich die t-Verteilung der Normalverteilung an. Als Faustformel gilt $n > 30$. Untenstehende Grafik zeigt die Annäherung der t-Verteilung an die Normalverteilung.

2.21 Grafik

Das Argument `df` der t-Verteilung



2.22 Code

```
x_values = np.linspace(-4, 4, 100)

# Normalverteilung
y_values = scipy.stats.norm.pdf(x_values)
plt.plot(x_values, y_values, color = 'black', lw = 3, label = 'Normalverteilung')
# plt.ylim(bottom = 0, top = 0.5)

# t-Verteilungen
marker = [".", "1", "x"]

[plt.plot(x_values, scipy.stats.t.pdf(x_values, df = (i + (i - 1) * 2)), linestyle = 'dotted',
          marker = marker[i], label = f't-Verteilung (df={i+1})') for i in range(4)]

plt.suptitle('Das Argument df der t-Verteilung')
plt.xlabel('Standardabweichung')
plt.ylabel('Häufigkeitsdichte')
plt.legend(loc = 'upper left')
plt.show()
```

Das Maximum der t-Verteilung ist weniger dicht, dafür sind die Ränder der Verteilung dichter als die Normalverteilung.

Somit gilt für die t-Verteilung von Stichprobenmittelwerten:

$$\bar{x} \pm t_{n-1} \cdot \frac{s}{\sqrt{n}}$$

- t ist der Rückgabewert der Funktion `scipy.stats.t.ppf(q, df = n - 1, loc = 0, scale = 1)`
- q ist das gewählte Alphaniveau bzw. für einen zweiseitigen Hypothesentest $\frac{\alpha}{2}$ und $1 - \frac{\alpha}{2}$.
- Das Ergebnis ist der Rückgabewert der Funktionen:
 - `scipy.stats.t.ppf(q = alpha/2, df = n - 1, loc = stichprobenmittelwert, scale = stichprobenstandardfehler)`
 - `scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 1, loc = stichprobenmittelwert, scale = stichprobenstandardfehler)`

💡 Tipp 2: t-Verteilung oder Normalverteilung?

Am Computer ist die t-Verteilung genauso leicht (oder schwer) zu berechnen wie die Normalverteilung. Da sich die t-Verteilung für größere Stichprobengrößen ohnehin der Normalverteilung annähert, empfiehlt es sich, stets die t-Verteilung zu verwenden.

2.22.1 Beispiel Gewicht weiblicher Pinguine

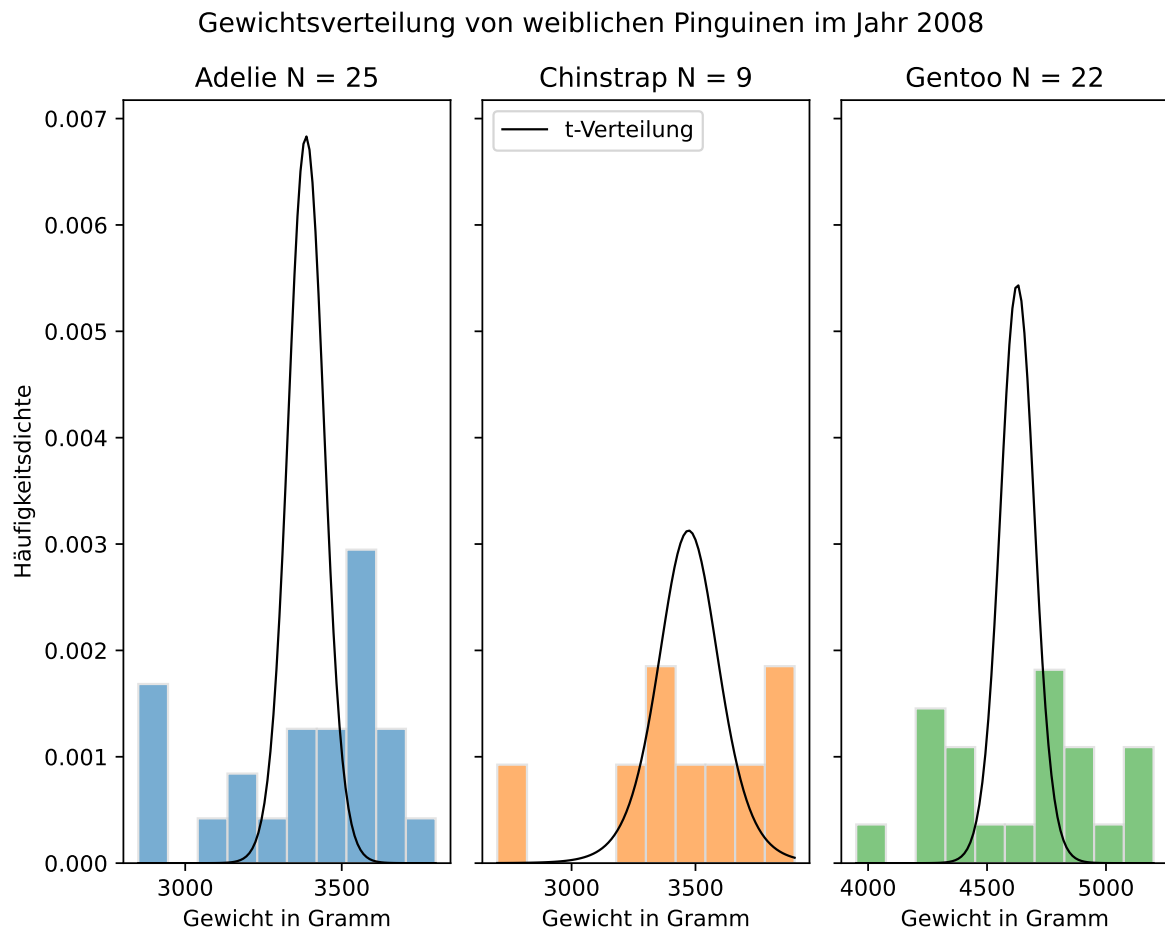
Die t-Verteilung des geschätzten Stichprobenmittelwerts für kleine Stichproben wird für im Jahr 2008 beobachtete weibliche Pinguine dargestellt.

```
print(penguins.groupby(by = [penguins['species'], penguins['sex'], penguins['year']]).size())
```

species	sex	year	
Adelie	female	2007	22
		2008	25
		2009	26
	male	2007	22
		2008	25
		2009	26
Chinstrap	female	2007	13
		2008	9
		2009	12
	male	2007	13
		2008	9
		2009	12
Gentoo	female	2007	16
		2008	22
		2009	20
	male	2007	17
		2008	23
		2009	21

dtype: int64

2.23 Grafik



2.24 Code

```
year = 2008
sex = 'female'
species = 'Adelie'

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von weiblichen Pinguinen im Jahr 2008')

# Adelie
data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
```

```

stichprobengröße = data.size

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 't-Verteilung')
ax1.legend(loc = 'upper left')

# Chinstrap
species = 'Chinstrap'

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'

```

```

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

2.25 Aufgabe Konfidenzintervalle

1. Schätzen Sie das Gewicht für im Jahr 2008 beobachtete weibliche Pinguine der Spezies Adelie, Chinstrap und Gentoo.
2. Welches Konfidenzintervall können Sie für die Mittelwerte angeben, wenn eine Vertrauenswahrscheinlichkeit von 90 % gelten soll?

💡 Tipp 3: Tipp und Musterlösung

Folgende Schritte helfen Ihnen bei der Lösung:

1. Bestimmen Sie den Stichprobenmittelwert $\bar{x}$.
2. Bestimmen Sie die Stichprobenstandardabweichung s , die Stichprobengröße N und den Standardfehler $\frac{s}{\sqrt{N}}$.
3. Bestimmen Sie die z- oder t-Werte der Normal- bzw. t-Verteilung für das gewählte Konfidenzniveau - für einen zweiseitigen Hypothesentest $\frac{\alpha}{2}$ und $1 - \frac{\alpha}{2}$
4. Berechnen Sie das Konfidenzintervall $\bar{x} \pm t_{\alpha/2} \frac{s}{\sqrt{n}}$.

Musterlösung

Alphaniveau definieren und Pinguine auswählen

```
alpha = 1 - 0.9
data = penguins[(penguins['sex'] == sex) & (penguins['year'] == year)]
```

1. Stichprobenmittelwerte bestimmen.

```
penguin_means = data['body_mass_g'].groupby(by = data['species']).mean()
print(penguin_means)
```

```
species
Adelie      3386.000000
Chinstrap   3472.222222
Gentoo      4627.272727
Name: body_mass_g, dtype: float64
```

2. Stichprobenstandardabweichung, Stichprobengröße und Standardfehler bestimmen.

```
penguin_stds = data['body_mass_g'].groupby(by = data['species']).std(ddof = 1)
penguin_sizes = data['body_mass_g'].groupby(by = data['species']).size()
penguin_stderrors = penguin_stds / np.sqrt(penguin_sizes)

print("Stichprobenstandardabweichungen:\n", penguin_stds)
print("\nStichprobengrößen:\n", penguin_sizes)
print("\nStandardfehler:\n", penguin_stderrors)
```

```
Stichprobenstandardabweichungen:
species
Adelie      288.862712
Chinstrap   370.903551
Gentoo      339.722321
Name: body_mass_g, dtype: float64
```

```
Stichprobengrößen:
species
Adelie      25
Chinstrap    9
Gentoo      22
Name: body_mass_g, dtype: int64
```

```
Standardfehler:
```

```

species
Adelie      57.772542
Chinstrap  123.634517
Gentoo      72.429042
Name: body_mass_g, dtype: float64

```

3. t-Werte bestimmen

```

t_unten = scipy.stats.t.ppf(alpha / 2, loc = 0, scale = 1, df = penguin_sizes - 1)
print("t-Wert untere Intervallgrenze:", t_unten)

t_oben = scipy.stats.t.ppf(1 - alpha / 2, loc = 0, scale = 1, df = penguin_sizes - 1)
print("t-Wert obere Intervallgrenze:", t_oben)

```

```

t-Wert untere Intervallgrenze: [-1.71088208 -1.85954804 -1.7207429 ]
t-Wert obere Intervallgrenze: [1.71088208 1.85954804 1.7207429 ]

```

4. Konfidenzintervall bestimmen

```

# mit scipy.stats.t.ppf
untere_intervalle = scipy.stats.t.ppf(alpha / 2, loc = penguin_means, scale = penguin_
print("untere Intervallgrenzen:", untere_intervalle)

obere_intervalle = scipy.stats.t.ppf(1 - alpha / 2, loc = penguin_means, scale = pengu
print("obere Intervallgrenzen:", obere_intervalle)

print("\n'manuelle' Berechnung:\n")
# 'manuell'
print(penguin_means + t_unten * penguin_stderrors)
print(penguin_means + t_oben * penguin_stderrors)

```

```

untere Intervallgrenzen: [3287.15799233 3242.31789851 4502.64096694]
obere Intervallgrenzen: [3484.84200767 3702.12654593 4751.90448761]

```

'manuelle' Berechnung:

```

species
Adelie      3287.157992
Chinstrap    3242.317899
Gentoo       4502.640967
Name: body_mass_g, dtype: float64

```



```
species
Adelie      3484.842008
Chinstrap   3702.126546
Gentoo      4751.904488
Name: body_mass_g, dtype: float64
```

3 Lineare Parameterschätzung

Viele physikalische Größen werden indirekt gemessen. Häufig liegt dabei ein linearer Zusammenhang zwischen der gemessenen und der gesuchten Größe vor. In der Praxis sind die Daten einer Messreihe nie exakt und mit mehr oder weniger großen Abweichungen vom wahren Wert der gesuchten Größe behaftet. Die lineare Parameterschätzung ist ein Verfahren, um aus einer Menge von Daten denjenigen Wert zu bestimmen, der die Abweichungen der einzelnen Messwerte minimiert.

Mögliche Quellen:

- <https://messtechnik-und-sensorik.org/2-kennlinien-und-messgenauigkeit/>
- <https://www.stssensors.com/de/characteristic-curve-hysteresis-measurement-error-terminology-in-pressure-measurement-technology/>

3.1 Messreihe Hooke'sches Gesetz

Das [Hooke'sche Gesetz](#), benannt nach dem englischen Wissenschaftler Robert Hooke, beschreibt die Beziehung zwischen der Kraft F und der Längenänderung Δx einer Feder durch die Gleichung $F = k \cdot \Delta x$, wobei k die Federkonstante ist.

Die Federkonstante ist eine grundlegende Eigenschaft elastischer Materialien und gibt an, wie viel Kraft erforderlich ist, um eine Feder um eine bestimmte Länge zu dehnen oder zu komprimieren. Das Hooke'sche Gesetz besagt, dass die Deformation eines elastischen Körpers proportional zur aufgebrachten Kraft ist, solange die Feder nicht über den elastischen Bereich hinaus gedehnt oder gestaucht wird.

In einem Experiment wurde das Hooke'sche Gesetz überprüft. An einer an einer Halterung hängenden Metallfeder ist ein (variables) Gewicht angebracht. Darunter befindet sich in einem Abstand ein Ultraschallsensor zur Abstandsmessung. Der Abstand zwischen der Unterseite des an der Feder befestigten Gewichts und dem Ultraschallsensor ist der gemessene Abstand.

Die Gewichte konnten mit einer Genauigkeit von $\epsilon_m = 0,5g$ mit einer Küchenwaage bestimmt werden.

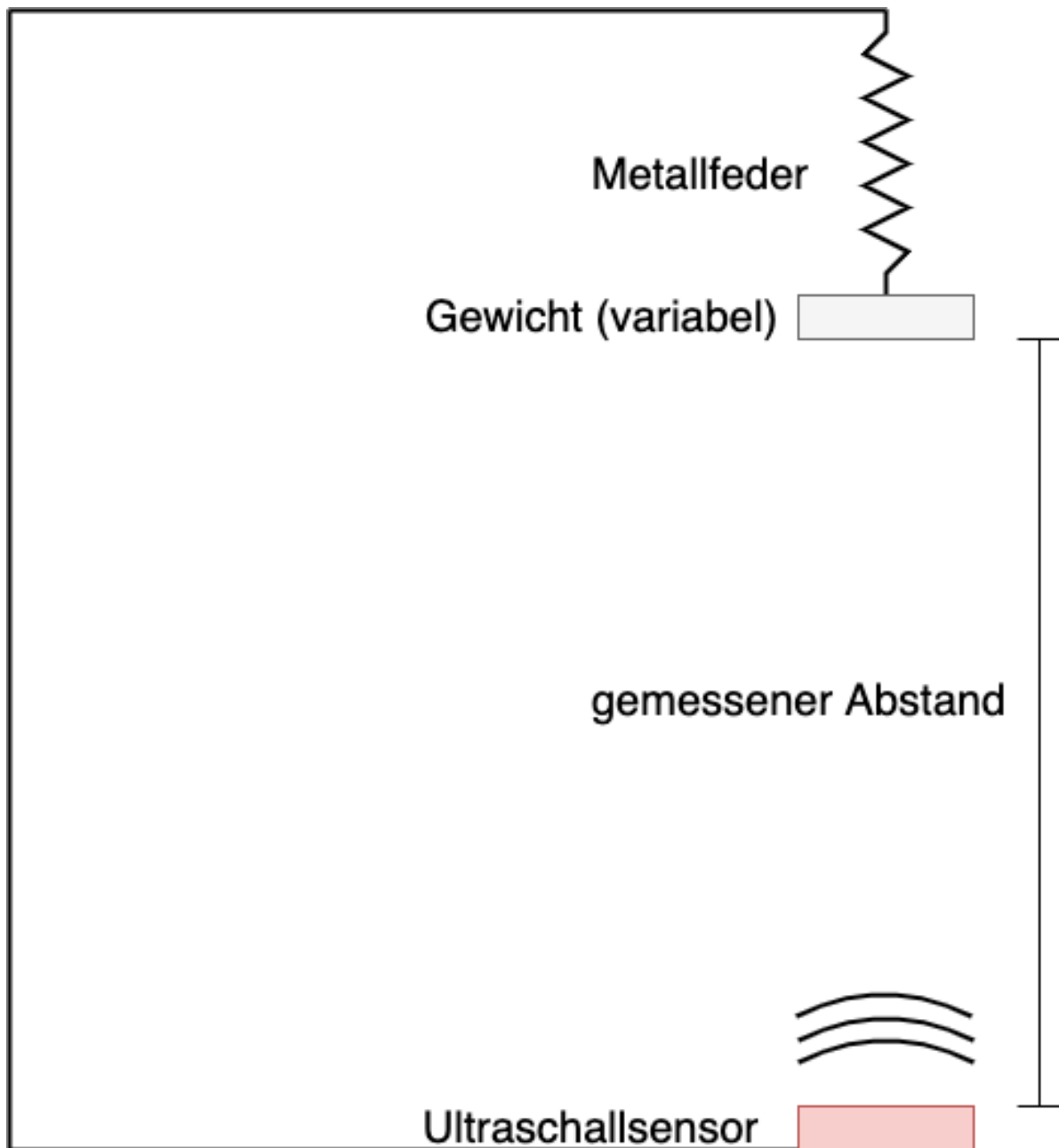


Abbildung 3.1: Versuchsaufbau

Die Messreihe liegt in Form einer CSV-Datei unter dem Pfad '01-daten/hooke_data.csv' vor. Die Datei wird mit Pandas eingelesen.

```
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
```

3.1.1 Deskriptive Statistik

Nach dem Einlesen sollte man sich einen Überblick über die Daten verschaffen. Mit den Methoden `pd.DataFrame.head()` und `pd.DataFrame.tail()` kann ein Ausschnitt vom Beginn und vom Ende der Daten betrachtet werden.

```
print(hooke.head(), "\n")
print(hooke.tail())
```

```

      no  mass  distance
0     0   705    153.29
1     1   705    152.74
2     2   705    153.27
3     3   705    152.81
4     4   705    152.77
```

```

      no  mass  distance
109  109     0    173.70
110  110     0    173.44
111  111     0    173.75
112  112     0    173.30
113  113     0    200.00
```

Die Methode `pd.DataFrame.describe()` erstellt die deskriptive Statistik für den Datensatz. Diese ist in diesem Fall jedoch noch nicht sonderlich nützlich. Die Spalte 'no' enthält lediglich eine laufende Versuchsnummer, die Spalte 'mass' enthält verschiedene Gewichte.

```
hooke.describe()
```

	no	mass	distance
count	114.000000	114.000000	114.000000
mean	56.561404	394.921053	162.301754
std	33.131552	226.237605	7.483767
min	0.000000	0.000000	152.740000
25%	28.250000	201.000000	156.622500
50%	56.500000	452.000000	160.720000

	no	mass	distance
75%	84.750000	605.000000	167.767500
max	113.000000	705.000000	200.000000

Sinnvoller ist eine nach dem verwendeten Gewicht aufgeteilte beschreibende Statistik der gemessenen Ausdehnung. Dafür kann die Pandas-Methode `pd.DataFrame.groupby()` verwendet werden. So kann für jedes der gemessenen Gewichte der arithmetische Mittelwert und die Standardabweichung abgelesen werden.

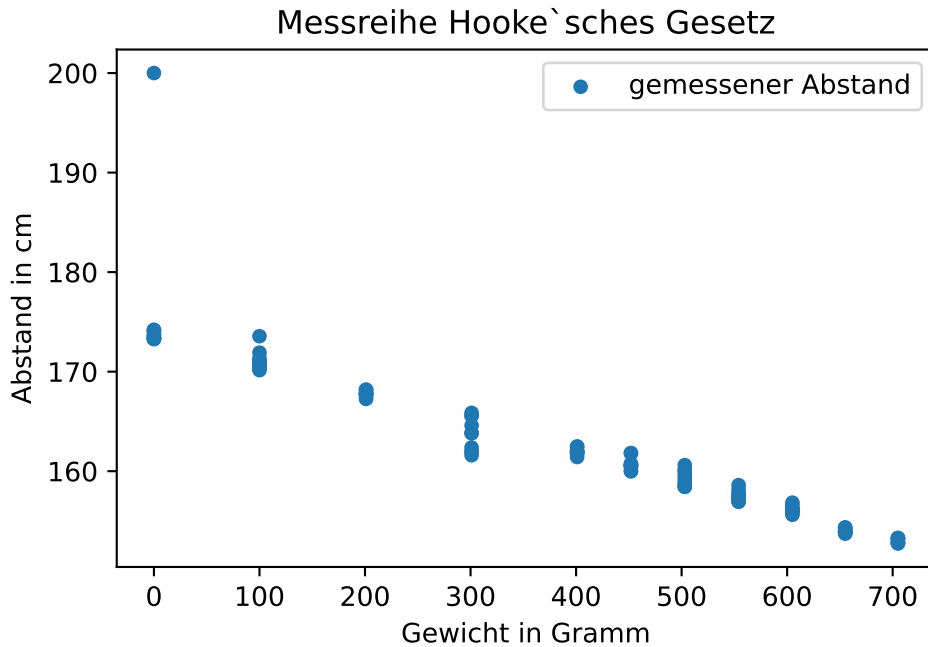
```
hooke.groupby(by = 'mass')['distance'].describe()
```

	count	mean	std	min	25%	50%	75%	max
mass								
0	12.0	175.828333	7.620157	173.27	173.3150	173.570	174.1125	200.00
100	11.0	171.044545	0.985833	170.15	170.3650	170.800	171.2400	173.56
201	11.0	167.791818	0.296305	167.26	167.7200	167.780	167.9750	168.19
301	10.0	163.710000	1.660977	161.60	162.0575	163.825	165.3250	165.86
401	10.0	161.967000	0.313229	161.42	161.8450	161.915	162.0250	162.48
452	10.0	160.713000	0.627854	159.98	160.4575	160.555	160.7400	161.83
503	10.0	159.314000	0.781099	158.43	158.6400	159.220	159.9650	160.61
554	10.0	157.547000	0.523791	156.92	157.2075	157.435	157.7100	158.60
605	10.0	156.142000	0.354206	155.62	156.0700	156.080	156.2075	156.84
655	11.0	154.022727	0.224414	153.72	153.8800	153.920	154.2400	154.35
705	9.0	153.008889	0.241425	152.74	152.8100	152.910	153.2700	153.29

Bereits an dieser Stelle könnte die hohe Standardabweichung in der Messreihe mit 0 Gramm auffallen. Leichter ist es jedoch in der grafischen Betrachtung.

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = "Messreihe Hooke'sches Gesetz",
plt.legend()

plt.show()
```



Grafisch fällt der Messwert von 200 cm für das Gewicht 0 Gramm als stark von den übrigen Messwerten abweichend auf.

Die Messwerte für das Gewicht 0 Gramm sollen näher betrachtet werden. Dafür werden die Messwerte sowohl absolut, als auch [standardisiert in Einheiten der Standardabweichung \(z-Werten\)](#) ausgedrückt ausgegeben.

Eine Variable wird standardisiert, indem von jedem Wert der Erwartungswert abgezogen und das Ergebnis durch die Standardabweichung geteilt wird.

$$Z = \frac{x - \mu}{\sigma}$$

Da in der Regel der Erwartungswert und die Standardabweichung unbekannt sind, werden der Stichprobenmittelwert und die Stichprobenstandardabweichung verwendet. Dies nennt man *studentisieren*, nach dem Pseudonym des bereits im vorherigen Kapitel erwähnten William Sealy Gosset.

$$z_i = \frac{x_i - \bar{x}}{s}$$

```

gewicht = 0

# z-Transformation manuell berechnen
mittelwert_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].mean()
standardabweichung_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].std(ddof=1)

z_values = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].apply(lambda x: (x - mittelwert_ausdehnung) / standardabweichung_ausdehnung)
z_values.name = 'z-values'

# z-Transformation mit scipy
## scipy gibt ein np.array zurück
## zur besseren Darstellung wird das np.array in eine pd.Series umgewandelt, die ein name Attribut hat
scipy_z_values = pd.Series(scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance']))
scipy_z_values.name = 'scipy z-values'

# gemeinsame Ausgabe der Daten
print(pd.concat([hooke[hooke['mass'] == gewicht], z_values, scipy_z_values], axis = 1))

```

	no	mass	distance	z-values	scipy z-values
102	102.0	0.0	173.32	-0.329171	NaN
103	103.0	0.0	174.11	-0.225498	NaN
104	104.0	0.0	173.42	-0.316048	NaN
105	105.0	0.0	174.12	-0.224186	NaN
106	106.0	0.0	173.30	-0.331795	NaN
107	107.0	0.0	174.21	-0.212375	NaN
108	108.0	0.0	173.27	-0.335732	NaN
109	109.0	0.0	173.70	-0.279303	NaN
110	110.0	0.0	173.44	-0.313423	NaN
111	111.0	0.0	173.75	-0.272742	NaN
112	112.0	0.0	173.30	-0.331795	NaN
113	113.0	0.0	200.00	3.172069	NaN
0	NaN	NaN	NaN	NaN	-0.329171
1	NaN	NaN	NaN	NaN	-0.225498
2	NaN	NaN	NaN	NaN	-0.316048
3	NaN	NaN	NaN	NaN	-0.224186
4	NaN	NaN	NaN	NaN	-0.331795
5	NaN	NaN	NaN	NaN	-0.212375
6	NaN	NaN	NaN	NaN	-0.335732
7	NaN	NaN	NaN	NaN	-0.279303
8	NaN	NaN	NaN	NaN	-0.313423
9	NaN	NaN	NaN	NaN	-0.272742
10	NaN	NaN	NaN	NaN	-0.331795

Der Wert 200 cm in Zeile 113 scheint fehlerhaft zu sein. Eine Eigendehnung der Feder um zusätzliche 16 Zentimeter ist nicht plausibel. Auch der z-Wert > 3 kennzeichnet den Messwert als [Ausreißer](#). Die Zeile wird deshalb aus dem Datensatz entfernt.

! Wichtig 4: Ausreißer

In der Statistik wird ein Messwert als Ausreißer bezeichnet, wenn dieser stark von der übrigen Messreihe abweicht. In einer Messreihe können auch mehrere Ausreißer auftreten. Diese Werte können zur Verbesserung der Schätzung aus der Messreihe entfernt werden, wenn anzunehmen ist, dass diese durch Messfehler und andere Störgrößen verursacht sind. Eine Möglichkeit, Ausreißer zu identifizieren, ist die z-Transformation. Dabei muss ein Schwellenwert gewählt werden, ab dem ein Messwert als Ausreißer klassifiziert werden soll, bspw. 2,5 oder 3 Einheiten der Standardabweichung. In der Statistik wurde eine ganze Reihe von Ausreißertests entwickelt (siehe [Ausreißertests](#))

Die Einstufung eines Messwerts als Ausreißer kann aber nicht allein auf der Grundlage statistischer Verfahren erfolgen, sondern ist immer eine Ermessensentscheidung auf der Grundlage Ihres Fachwissens. Denn nicht alle abweichenden Werte sind automatisch ungültig, sondern treten mit einer gewissen statistischen Wahrscheinlichkeit auf (siehe Kapitel Normalverteilung). Man spricht dann von gültigen Extremwerten.

Ausreißer von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) und ist abrufbar auf [Wikipedia](#)

```
hooke.drop(index = 113, inplace = True)

hooke.groupby(by = 'mass')['distance'].describe()
```

	count	mean	std	min	25%	50%	75%	max
mass								
0	11.0	173.630909	0.367409	173.27	173.3100	173.440	173.9300	174.21
100	11.0	171.044545	0.985833	170.15	170.3650	170.800	171.2400	173.56
201	11.0	167.791818	0.296305	167.26	167.7200	167.780	167.9750	168.19
301	10.0	163.710000	1.660977	161.60	162.0575	163.825	165.3250	165.86
401	10.0	161.967000	0.313229	161.42	161.8450	161.915	162.0250	162.48
452	10.0	160.713000	0.627854	159.98	160.4575	160.555	160.7400	161.83
503	10.0	159.314000	0.781099	158.43	158.6400	159.220	159.9650	160.61
554	10.0	157.547000	0.523791	156.92	157.2075	157.435	157.7100	158.60
605	10.0	156.142000	0.354206	155.62	156.0700	156.080	156.2075	156.84
655	11.0	154.022727	0.224414	153.72	153.8800	153.920	154.2400	154.35
705	9.0	153.008889	0.241425	152.74	152.8100	152.910	153.2700	153.29

	count	mean	std	min	25%	50%	75%	max
mass								

Hiernach ist die höchste Standardabweichung für die Messreihe mit 301 Gramm zu verzeichnen. Die gemessenen Werte sind jedoch unauffällig.

```
gewicht = 301

## scipy gibt ein np.array zurück
## zur besseren Darstellung wird das np.array in eine pd.Series umgewandelt, die ein name At
z_values = pd.Series(scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance'],
z_values.name = 'z-values'

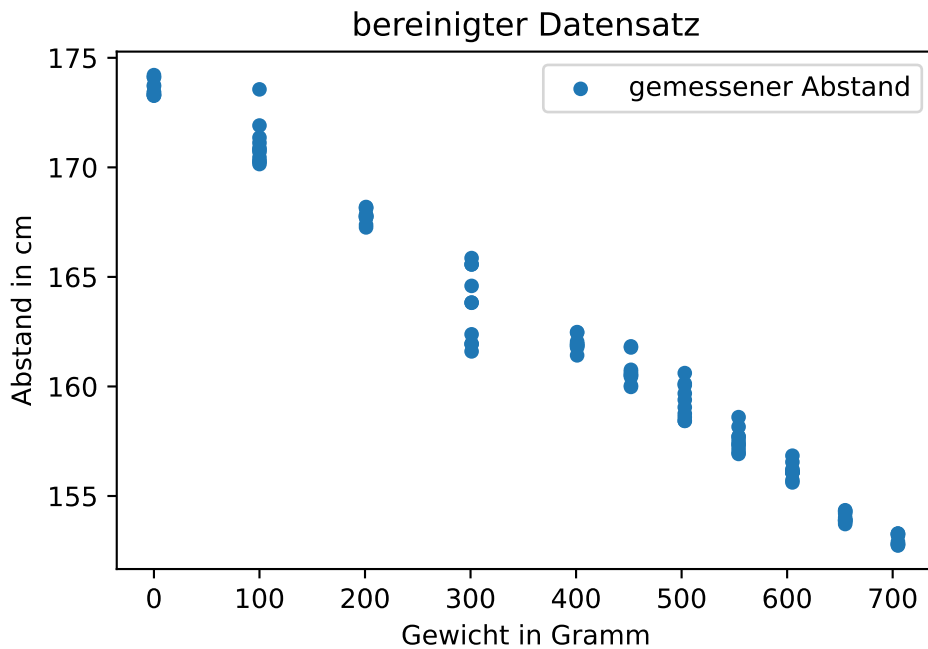
print(pd.concat([hooke[hooke['mass'] == gewicht], z_values], axis = 1))
```

	no	mass	distance	z-values
70	70.0	301.0	162.38	NaN
71	71.0	301.0	161.93	NaN
72	72.0	301.0	161.95	NaN
73	73.0	301.0	161.60	NaN
74	74.0	301.0	164.59	NaN
75	75.0	301.0	165.86	NaN
76	76.0	301.0	163.82	NaN
77	77.0	301.0	163.83	NaN
78	78.0	301.0	165.57	NaN
79	79.0	301.0	165.57	NaN
0	NaN	NaN	NaN	-0.800734
1	NaN	NaN	NaN	-1.071658
2	NaN	NaN	NaN	-1.059617
3	NaN	NaN	NaN	-1.270337
4	NaN	NaN	NaN	0.529809
5	NaN	NaN	NaN	1.294419
6	NaN	NaN	NaN	0.066226
7	NaN	NaN	NaN	0.072247
8	NaN	NaN	NaN	1.119823
9	NaN	NaN	NaN	1.119823

3.1.2 Explorative Statistik

Die Grafik des bereinigten Datensatzes legt einen linearen Zusammenhang nahe. Darüber hinaus sticht der mit zunehmendem Gewicht abfallende Trend der Datenpunkte ins Auge.

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter Datensatz', ylab = 'Abstand in cm', xlab = 'Gewicht in Gramm', legend = True, style = 'r', markersize = 10, color = 'blue', alpha = 0.5)
plt.legend()
plt.show()
```



Entsprechend des Versuchsaufbaus nimmt mit zunehmender Dehnung der Feder der Abstand zum Abstandssensor ab. Da die Federausdehnung gemessen werden soll, bietet es sich an, die Daten entsprechend zu transformieren. Dazu wird der gemessene Abstand bei 0 Gramm Gewicht als Nullpunkt aufgefasst, von dem aus die Federdehnung gemessen wird. Das bedeutet, dass von allen Datenpunkten das arithmetische Mittel der für 0 Gramm Gewicht gemessenen Ausdehnung abgezogen und das Ergebnis mit -1 multipliziert wird.

```
nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
print(f"Nullpunkt: {nullpunkt:.2f} cm")

hooke['distance'] = hooke['distance'].sub(nullpunkt).mul(-1)
```

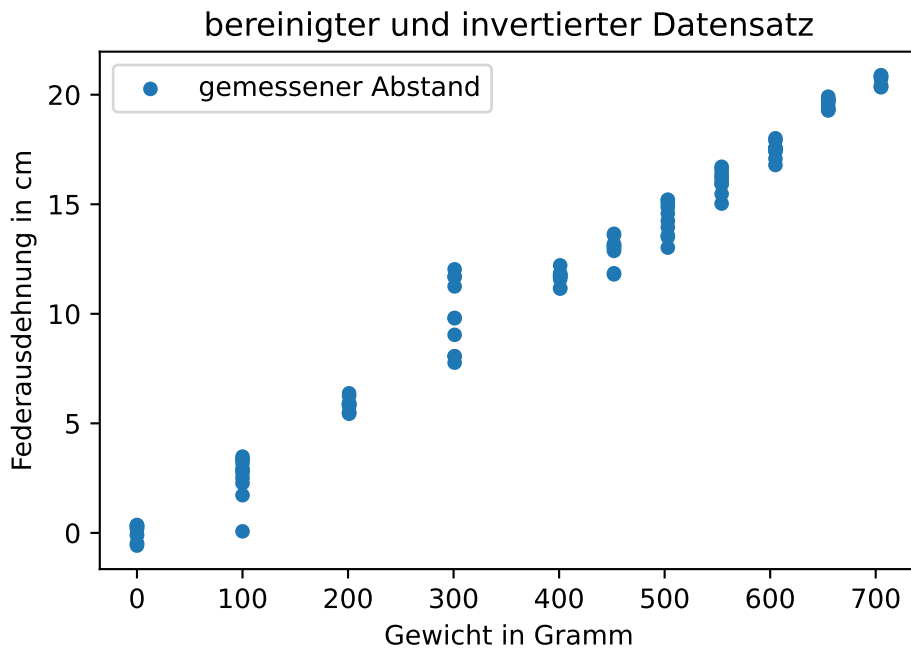
```

hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter und invertierter Datensatz')
plt.legend()

plt.show()

```

Nullpunkt: 173.63 cm



Mit der Funktion `plt.errorbars()` können die Mittelwerte und Standardfehler für jedes Gewicht grafisch dargestellt werden. Da die Standardfehler eher klein sind, werden mit dem Parameter `capsize` horizontale Linien am Ende des Fehlerbalkens eingezeichnet.

```

# Mittelwerte nach Gewicht
distance_means_by_weight = hooke['distance'].groupby(by = hooke['mass']).mean()
distance_means_by_weight.name = 'Federausdehnung'

# Standardfehler nach Gewicht
distance_stderrors_by_weight = hooke['distance'].groupby(by = hooke['mass']).std(ddof = 1).d
distance_stderrors_by_weight.name = 'Standardfehler'

datenpunkte = hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter u

```

```

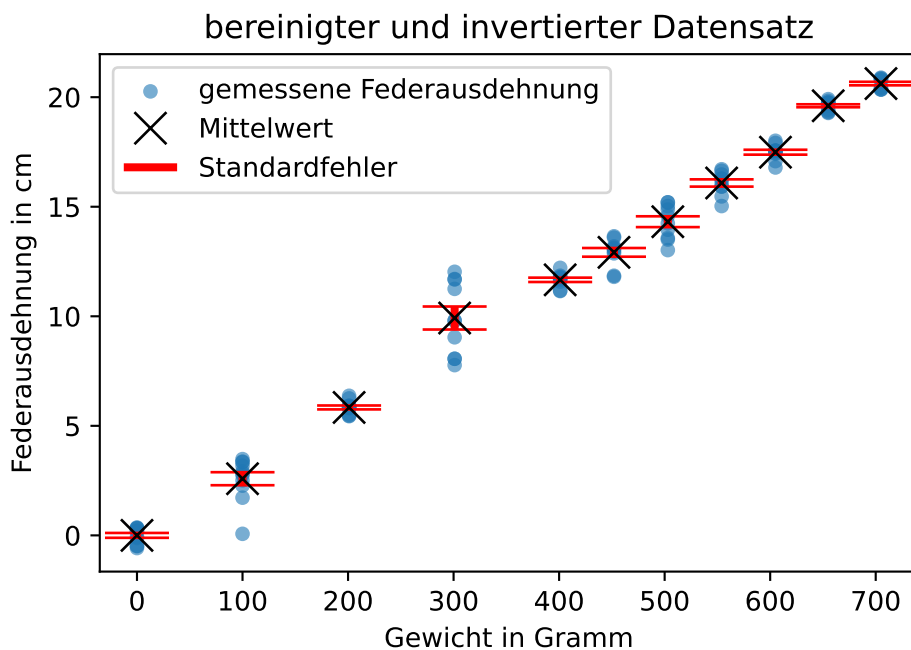
# Legendeneinträge abgreifen
# siehe: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.get_legend_handles_labels.html
datenpunkte_handle, datenpunkte_label = datenpunkte.get_legend_handles_labels()

errorbar_container = plt.errorbar(
    x = distance_means_by_weight.index, y = distance_means_by_weight, yerr = distance_stderrors_by_weight,
    linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor = 'red')

# Legende manuell erstellen
# siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer
plt.legend([datenpunkte_handle[0], errorbar_container.lines[0], errorbar_container.lines[2], errorbar_container.lines[3]],
           [datenpunkte_label[0], 'Mittelwert', 'Standardfehler'],
           loc = 'upper left')
plt.show()

print("\n", pd.concat([distance_means_by_weight, distance_stderrors_by_weight], axis = 1))

```



	Federausdehnung	Standardfehler
mass		
0	-7.751375e-15	0.110778

100	2.586364e+00	0.297240
201	5.839091e+00	0.089339
301	9.920909e+00	0.525247
401	1.166391e+01	0.099052
452	1.291791e+01	0.198545
503	1.431691e+01	0.247005
554	1.608391e+01	0.165637
605	1.748891e+01	0.112010
655	1.960818e+01	0.067663
705	2.062202e+01	0.080475

3.2 Federkonstante bestimmen

Die Beziehung zwischen der Kraft F und der Längenänderung Δx einer Feder mit Federkonstante k wird durch die Gleichung $F = k \cdot \Delta x$ beschrieben. Dabei entspricht die Kraft F dem mit der Fallbeschleunigung g multiplizierten Gewicht in Kilogramm m . Die Fallbeschleunigung beträgt auf der Erde $9,81 \frac{m}{s^2}$.

Deshalb wird im Datensatz das in der Spalte 'mass' eingetragene Gewicht in Gramm in die wirkende Kraft umgerechnet. Ebenso wird die gemessene Abstandsänderung in der Spalte 'distance' von Zentimeter in Meter umgerechnet.

```
hooke['mass'] = hooke['mass'].div(1000).mul(9.81)
hooke.rename(columns = {'mass': 'force'}, inplace = True)

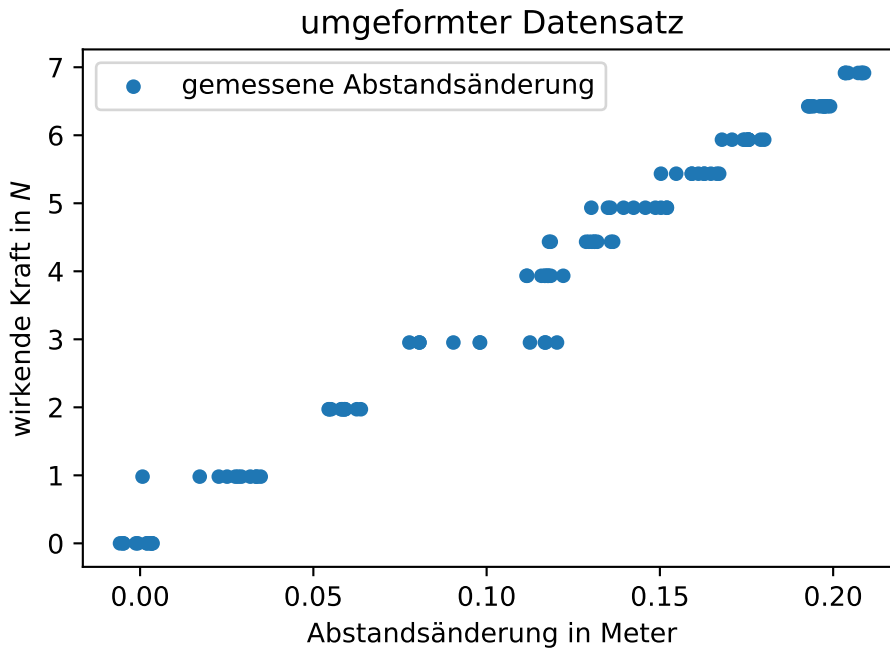
hooke['distance'] = hooke['distance'].div(100)

print(hooke.head())
```

	no	force	distance
0	0	6.91605	0.203409
1	1	6.91605	0.208909
2	2	6.91605	0.203609
3	3	6.91605	0.208209
4	4	6.91605	0.208609

Für die grafische Darstellung des Zusammenhangs $F = k \cdot \Delta x$ ist es zweckmäßiger, die Abstandsänderung auf der x-Achse und die wirkende Kraft auf der y-Achse darzustellen.

```
hooke.plot(x = 'distance', y = 'force', kind = 'scatter', title = 'umgeformter Datensatz', y.  
plt.legend()  
  
plt.show()
```



3.2.1 Lineare Ausgleichsrechnung

Die Ausgleichsrechnung (oder auch Parameterschätzung) ist eine Methode, um für eine Messreihe die unbekannten Parameter des zugrundeliegenden physikalischen Modells zu schätzen. Das Ziel besteht darin, eine (in diesem Fall lineare) Funktion zu bestimmen, die bestmöglich an die Messdaten angepasst ist. ([Wikipedia](#))

Eine lineare Funktion wird durch die Konstante β_0 , den Schnittpunkt mit der y-Achse, und den Steigungskoeffizienten β_1 bestimmt.

$$y = \beta_0 + \beta_1 \cdot x$$

In der Regel liegt kein deterministischer Zusammenhang vor, sondern es treten zufällige Abweichungen auf, die mit dem additiven Fehlerterm ausgedrückt und aus dem Englischen error mit e_i notiert werden. Diese Fehler werden Residuen genannt.

$$y = \beta_0 + \beta_1 \cdot x + e_i$$

Zur Bestimmung der Parameter einer linearen Funktion wird die Methode der **kleinsten Quadrate** verwendet.

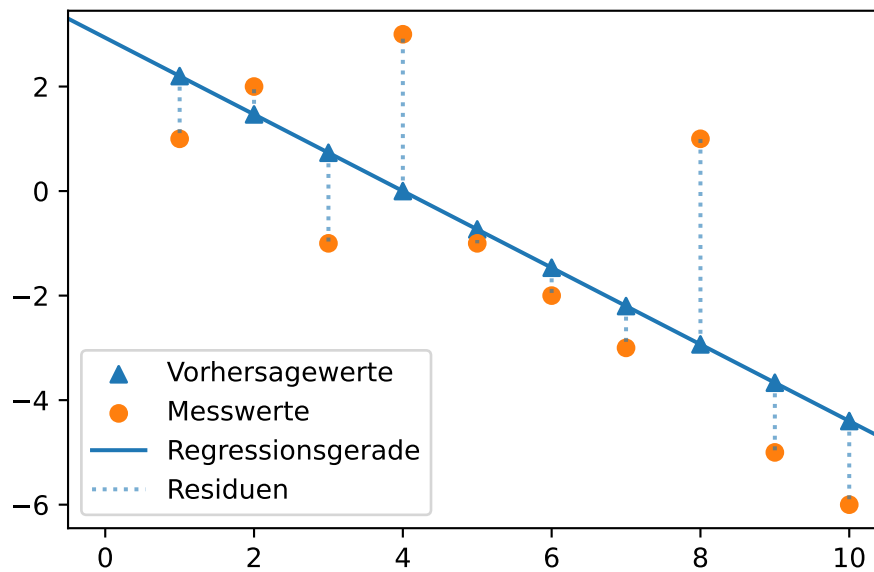
i Hinweis 4: Methode der kleinsten Quadrate

Mit der Methode der kleinsten Quadrate soll diejenige Gerade $\hat{y} = \beta_0 + \beta_1 \cdot x$ gefunden werden, die die quadrierten Abstände der Vorhersagewerte $\hat{y}$ von den tatsächlich gemessenen Werten y minimiert. Die Werte $y_i - \hat{y}_i$ sind die Residuen e_i . Es gilt also:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N e_i^2 = \min$$

Grafisch kann man sich die Minimierung der quadrierten Abstände so vorstellen.

3.3 Grafik



Regressionskoeffizienten: [2.93333333 -0.73333333]

3.4 Code

```

x = np.arange(1, 11)
y = - x.copy() + 4
y[0] -= 2
y[2] -= 2
y[3] += 3
y[-3] += 5

lm = poly.polyfit(x, y, 1)
vorhersagewerte = poly.polyval(x, lm)

plt.scatter(x, vorhersagewerte, label = 'Vorhersagewerte', marker = "^", color = "tab:blue")
plt.scatter(x, y, label = 'Messwerte', marker = 'o', color = "tab:orange")
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = "Regressionsgerade", color = "tab:blue")
dotted = plt.vlines(x, ymin = vorhersagewerte, ymax = y, alpha = 0.6, ls = 'dotted', label

plt.legend()
plt.show()

print("Regressionskoeffizienten:", lm)

```

Die eingezeichnete Gerade entspricht der linearen Funktion $\hat{y} = \beta_0 + \beta_1 \cdot x + e_i$. Die Dreiecksmarker sind die Vorhersagewerte $\hat{y}_i$ des linearen Modells für die Werte $x_i = np.arange(1, 11)$. Die tatsächlichen Messwerte y sind mit Kreismarkern markiert. Die Länge der gestrichelten Linien entspricht der Größe der Abweichung zwischen den Mess- und Vorhersagewerten $y_i - \hat{y}_i$, also den Residuen e_i .

Gesucht wird diejenige Gerade, die die Summe der quadrierten Residuen minimiert. Die gesuchten Werte β_0 und β_1 sind die Kleinst-Quadrate-Schätzer.

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Der Vollständigkeit halber leiten wir die Kleinst-Quadrate-Schätzer her. Gesucht werden Werte für β_0 und β_1 , damit die Summe der Residuenquadrate $\sum_{i=1}^n e_i^2$ möglichst klein wird. Die Residuenquadratsumme ist die Summe der quadrierten Differenzen aus beobachteten Werten y_i und der durch die lineare Funktion vorhergesagten Werte.

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

Wir untersuchen also eine Funktion, die von zwei Variablen abhängig ist.

$$f(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

Das Summenzeichen ist die Kurzschreibweise für eine Summe.

$$f(\beta_0, \beta_1) = (y_1 - (\beta_0 + \beta_1 \cdot x_1))^2 + (y_2 - (\beta_0 + \beta_1 \cdot x_2))^2 + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n))^2$$

Im Minimum der Funktion müssen die beiden partiellen Ableitungen gleich Null sein (Warum das so ist, wird [hier](#) leicht verständlich erklärt.)

Beispiel 3.1: Partielle Ableitung

Die partielle Ableitung ist die Ableitung einer Funktion mit mehreren Variablen nach einer Variablen, wobei die übrigen Variablen als Konstanten behandelt werden.

Für eine Funktion $f(x, y) = 2x + y^2$ wird die partielle Ableitung nach x so ausgedrückt:

$$\frac{\partial f(x, y)}{\partial x}$$

- Das Symbol ∂ ist die kursive Darstellung des kyrillischen Kleinbuchstaben (d) und wird als “del” gelesen. Es zeigt an, dass eine partielle Ableitung durchgeführt wird.
- Im Zähler steht die Funktion, die abgeleitet werden soll. Im Nenner steht die Variable nach der abgeleitet wird. Der Term wird gelesen als “del f von x und y nach del x”.

Die partielle Ableitung $\frac{\partial f(x, y)}{\partial x} = 2$. y^2 wird als Konstante behandelt (z. B. 5^2) und ist abgeleitet Null.

Die partielle Ableitung $\frac{\partial f(x, y)}{\partial y} = 2y$. $2x$ wird als Konstante behandelt (z. B. $2 \cdot 3$) und ist abgeleitet Null.

In beiden partiellen Ableitungen sind x_i und y_i konstant. In der partiellen Ableitung nach β_0 ist außerdem β_1 konstant, in der partiellen Ableitung nach β_1 ist entsprechend β_0 konstant.

3.5 partielle Ableitung nach dem y-Achsenschnittpunkt

Für die partielle Ableitung nach β_0 gilt also nach der Kettenregel für die äußere Funktion (oben) und die innere Funktion (Mitte):

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (1 + 0 \cdot 0))$$

Für die partielle Ableitung nach β_0 gilt also:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = -2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

Diese kann vereinfacht werden, indem der Vorfaktor -2 entfällt (weil $-2 \cdot 0 = 0$ gelten muss) und die Vorzeichen aufgelöst werden. Sodass:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 \cdot x_i) = 0$$

Man kann auch schreiben:

$$\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_1 \cdot x_i = 0$$

β_0 und β_1 sind Konstanten, sodass gilt $\sum_{i=1}^n \beta_0 = \beta_0 \cdot \sum_{i=1}^n 1 = \beta_0 \cdot n$ und $\sum_{i=1}^n \beta_1 \cdot x_i = \beta_1 \cdot \sum_{i=1}^n 1 \cdot x_i$. So gilt:

$$\sum_{i=1}^n y_i - n \cdot \beta_0 - \beta_1 \cdot \sum_{i=1}^n x_i = 0$$

Jetzt kann man durch n teilen. Dabei entspricht $\frac{\sum_{i=1}^n y_i}{n}$ dem arithmetischen Mittelwert von y und $\frac{\sum_{i=1}^n x_i}{n}$ dem arithmetischen Mittelwert von x . Somit steht:

$$\bar{y} - \beta_0 - \beta_1 \cdot \bar{x} = 0$$

Umgestellt:

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

3.6 partielle Ableitung nach dem Anstieg

Für die partielle Ableitung nach β_1 ist ebenfalls die Kettenregel anzuwenden, sodass die äußere Funktion (oben) identisch abgeleitet wird:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (0 + 1 \cdot x_i))$$

Für die partielle Ableitung nach β_1 gilt also:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = -2 \sum_{i=1}^n x_i \cdot (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

Auch diese kann vereinfacht werden, indem der Vorfaktor -2 entfällt (weil $-2 \cdot 0 = 0$ gelten muss) und die Vorzeichen aufgelöst werden. Außerdem kann ausmultipliziert werden:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 \cdot x_i - \sum_{i=1}^n \beta_1 \cdot x_i x_i = 0$$

Wieder können die Konstanten herausgezogen werden:

$$\sum_{i=1}^n x_i y_i - \beta_0 \cdot \sum_{i=1}^n x_i - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Jetzt kann man $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$ und $\sum_{i=1}^n x_i = n \cdot \bar{x}$ einsetzen:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \cdot \bar{x}) \cdot n \cdot \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Der mittlere Term wird ausmultipliziert und $x_i x_i$ im letzten Term als x_i^2 geschrieben:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot n \bar{x} \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Die letzten beiden Terme werden unter Anwendung des Distributivgesetzes $a - b = -(b - a)$ zusammengefasst.

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = 0$$

Jetzt kann nach β_1 umgestellt werden. Erst:

$$\beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = \sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}$$

Dann:

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}$$

Nun kann zuerst mit $\sum_{i=1}^n x_i^2 - n \bar{x}^2 = \sum_{i=1}^n (x_i - \bar{x})^2$ umgeformt werden.

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Dann - und das wird gleich gezeigt - mit $\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$. Sodass steht:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Der letzte Schritt wird ausgehend vom Ergebnis gezeigt und beginnt mit dem Ausmultiplizieren:

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})$$

Man kann auch schreiben:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \bar{y} - \sum_{i=1}^n \bar{x} y_i + \sum_{i=1}^n \bar{x} \bar{y}$$

$\bar{x}$ und $\bar{y}$ sind Konstanten, sodass $\bar{x} \cdot \sum_{i=1}^n y_i$ und $\bar{y} \cdot \sum_{i=1}^n x_i$ geschrieben werden kann. $\sum_{i=1}^n x_i$ ist gleich $n \cdot \bar{x}$ (analog für y). So ergibt sich:

$$\sum_{i=1}^n x_i y_i - \bar{y} \cdot n \cdot \bar{x} - \bar{x} \cdot n \cdot \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Sortieren:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - n \bar{x} \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Der letzte Term $\sum_{i=1}^n \bar{x} \bar{y}$ kann auch $n \cdot \bar{x} \bar{y}$ geschrieben werden, sodass sich ergibt:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - n \bar{x} \bar{y} + n \bar{x} \bar{y}$$

Die letzten beiden Terme entfallen somit und es bleibt:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}$$

(Baitsch, Matthias 2019, S. 73–74)

Die Funktionen dafür stellen sowohl das Paket `numpy.polynomial` bzw. für Polynomfunktionen dessen Modul `numpy.polynomial.polynomial` als auch das Modul `scipy.stats.linregress` bereit. Im Folgenden wird die Berechnung mit NumPy gezeigt und anschließend die Funktionen aus dem Modul SciPy vorgestellt. Die Funktionsweise beider Module ist ähnlich.

NumPy polyfit und polyeval

```
import numpy.polynomial.polynomial as poly
```

Zur Schätzung von Funktionsparametern nach der Methode der kleinsten Quadrate wird die Funktion `poly.polyfit(x, y, deg)` verwendet. `x` sind die Werte der unabhängigen Variablen, `y` die Werte der abhängigen Variablen und `deg` spezifiziert den Grad der gesuchten Polynomfunktion. `deg = 1` spezifiziert eine lineare Funktion.

i Hinweis 5: polyfit und polyeval erklärt

```
# Beispieldaten erzeugen
x = np.array(list(range(0, 100)))
y = x ** 2

print(poly.polyfit(x, y, 1))
```

```
[-1617.    99.]
```

Die Funktion gibt die geschätzten Regressionsparameter als NumPy-Array zurück. Die Terme sind aufsteigend angeordnet, d. h. der Achsabschnitt steht an Indexposition 0, der Steigungskoeffizient an Indexposition 1. Die Ausgabe für ein Polynom zweiten Grades würde beispielsweise so aussehen:

```
print(poly.polyfit(x, y, 2).round(2))
```

```
[ 0. -0.  1.]
```

Mit den Regressionskoeffizienten können die Vorhersagewerte der linearen Funktion berechnet werden. Dafür wird die Funktion `poly.polyeval(x, c)` verwendet. Diese berechnet die Funktionswerte für in `x` übergebene Werte mit den Funktionsparametern `c`. Aus der Differenz der gemessenen Werte und der Vorhersagewerte können die Residuen bestimmt werden.

```
# 'manuelle' Berechnung
regressions_koeffizienten = poly.polyfit(x, y, 1)
vorhersagewerte = regressions_koeffizienten[0] + x * regressions_koeffizienten[1]
residuen = y - vorhersagewerte

# Berechnung mit polyeval
lm = poly.polyfit(x, y, 1)
vorhersagewerte_polyval = poly.polyval(x, lm)

print("Die Ergebnisse stimmen überein:", np.equal(vorhersagewerte, vorhersagewerte_polyval))
print("\nAusschnitt der Vorhersagewerte:", vorhersagewerte[:10])
```

Die Ergebnisse stimmen überein: True

Ausschnitt der Vorhersagewerte: [-1617. -1518. -1419. -1320. -1221. -
1122. -1023. -924. -825. -726.]

Das **Bestimmtheitsmaß** R^2 gibt an, wie gut die Schätzfunktion an die Daten angepasst ist. Der Wertebereich reicht von 0 bis 1. Ein Wert von 1 bedeutet eine vollständige Anpassung. Für eine einfache lineare Regression mit nur einer erklärenden Variable kann das Bestimmtheitsmaß als Quadrat des **Bravais-Pearson-Korrelationskoeffizienten** r berechnet werden. Dieser wird mit der Funktion `np.corrcoef(x, y)` ermittelt (die eine Matrix der Korrelationskoeffizienten ausgibt).

```
print(f"r = {np.corrcoef(x, y)[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(x, y)[0, 1] ** 2:.2f}")
```

```
r = 0.97
R2 = 0.94
```

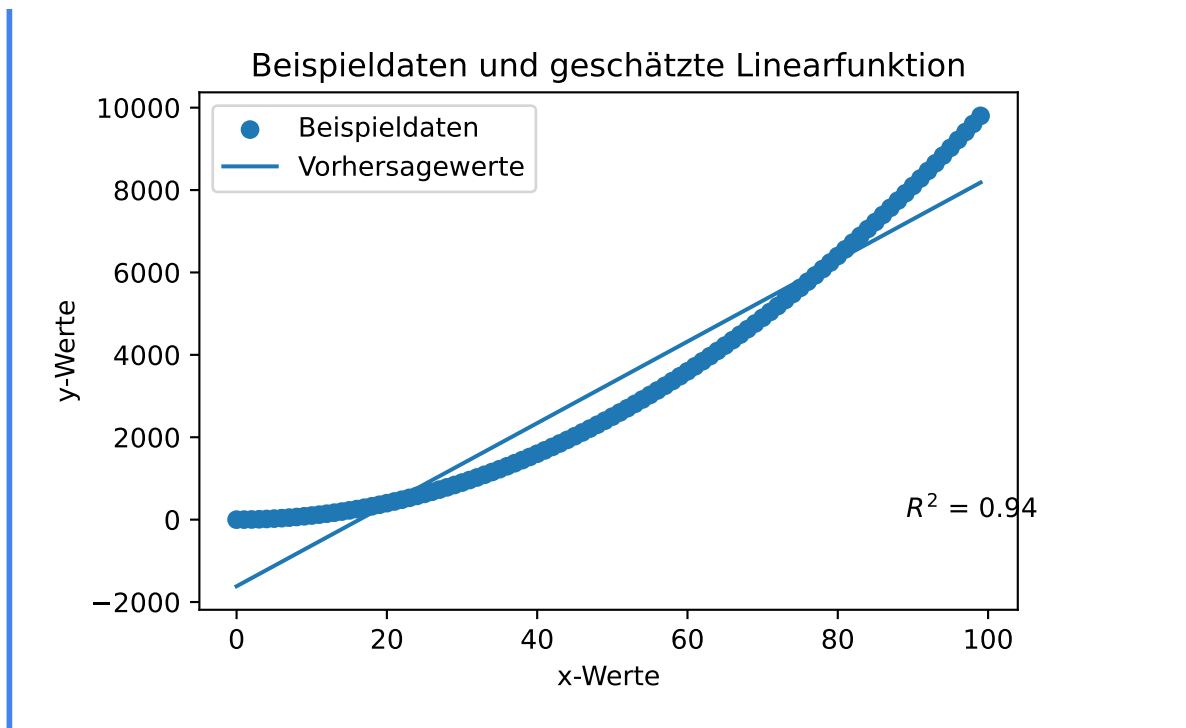
Die Daten und die geschätzte Gerade können grafisch dargestellt werden.

```
import matplotlib.pyplot as plt

plt.scatter(x, y, label = 'Beispieldaten')
plt.plot(x, vorhersagewerte, label = 'Vorhersagewerte')
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Beispieldaten und geschätzte Linearfunktion')
plt.xlabel('x-Werte')
plt.ylabel('y-Werte')
plt.legend()

plt.show()
```



NumPy umfasst außerdem die inzwischen veralteten Funktionen `np.polyfit(x, y, deg)` und `np.polyval(p, x)`.

i Hinweis 6: `np.polyfit` & `np.polyval`

Die Funktionen `np.polyfit(x, y, deg)` und `np.polyval(p, x)` funktionieren wie die vorgestellten Funktionen aus dem Modul `numpy.polynomial.polynomial`. Ein wichtiger Unterschied besteht jedoch darin, dass **die Parameter der Funktion `polyfit` in umgekehrter Reihenfolge** ausgegeben werden.

```
print(poly.polyfit(x, y, deg = 1))
print(np.polyfit(x, y, deg = 1))
```

```
[-1617.    99.]
[   99. -1617.]
```

Note

This forms part of the old polynomial API. Since version 1.4, the new polynomial API defined in `numpy.polynomial` is preferred. A summary of the differences can be found in the [transition guide](#).

[NumPy-Dokumentation](#)

Die Parameter der an die Messwerte angepassten linearen Funktion und das Bestimmtheitsmaß lauten:

```
print(poly.polyfit(hooke['distance'], hooke['force'], 1))

print(f"r = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1] ** 2:.2f}")
```

```
[ 0.05753159 33.01899551]
r = 0.99
R2 = 0.99
```

Mit den Regressionskoeffizienten können die Vorhersagewerte der linearen Funktion berechnet werden.

```
# Berechnung mit polyeval
lm = poly.polyfit(hooke['distance'], hooke['force'], 1)
vorhersagewerte_hooke = poly.polyval(hooke['distance'], lm)
```

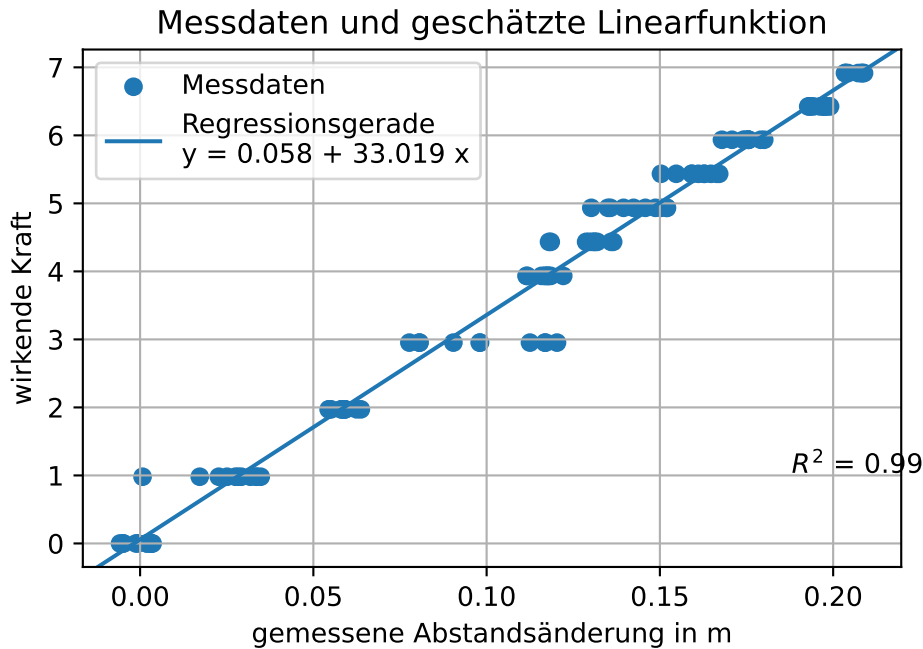
Die Messreihe und die darauf angepasste lineare Funktion können grafisch dargestellt werden.

```
# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\nty = ' + "{beta_0:.3f} + {beta_1:.3f}x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Messdaten und geschätzte Linearfunktion')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()
```

3.6.1 Messabweichung quantifizieren

Für den geschätzten Regressionskoeffizienten kann für die lineare Regression mit einer erklärenden Variable der Standardfehler des Regressionskoeffizienten $SE = \hat{\sigma}_{\hat{\beta}_1}$ ermittelt werden (siehe [Wikipedia](#)).

$$SE = \sqrt{\frac{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

- Im Zähler steht die mittlere [Residuenquadratsumme](#) (Summe der quadrierten Residuen / Anzahl der Freiheitsgrade).
- Im Nenner steht die [Summe der Abweichungsquadrate von \$x\$](#) .

Für ein Signifikanzniveau α kann ein Konfidenzniveau $1 - \alpha$ angegeben werden als:

$$\hat{\beta}_1 \pm SE \cdot t_{1-\alpha/2} (n-2)$$

- $t_{1-\alpha/2} (n-2)$ ist der Wert der t-Verteilung mit $n - 2$ Freiheitsgraden bzw. der Rückgabewert der Funktion:
 – `scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)` für die obere Intervallgrenze.

```

print(f"Regressionskoeffizient: {lm[1]:.4f}")

# 'manuell' Standardfehler des Regressionskoeffizienten berechnen
standardfehler_beta_1 = np.sqrt( (1 / (len(x) - 2) * sum((y - vorhersagewerte_hooke) ** 2)) )

print(f"Standardfehler des Regressionskoeffizienten: {standardfehler_beta_1:.4f}")

# Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n = len(x)

t_wert = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)
print(f"t-Wert 95%-Intervall (zweiseitig): {t_wert:.4f}")
print(f"Konfidenzintervall 95 %: {lm[1]:.4f} ± {t_wert:.4f} * {standardfehler_beta_1:.4f}")
print(f"untere 95%-Intervallgrenze: {lm[1] - t_wert * standardfehler_beta_1:.4f}")
print(f"obere 95%-Intervallgrenze: {lm[1] + t_wert * standardfehler_beta_1:.4f}")

```

```

Regressionskoeffizient: 33.0190
Standardfehler des Regressionskoeffizienten: 0.3784
t-Wert 95%-Intervall (zweiseitig): 1.9816
Konfidenzintervall 95 %: 33.0190 ± 1.9816 * 0.3784
untere 95%-Intervallgrenze: 32.2692
obere 95%-Intervallgrenze: 33.7688

```

Das Konfidenzintervall kann auch grafisch dargestellt werden.

```

# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\
ny = ' + "{beta_0:.3f} + {beta_1:.3f} * x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

# 95%-Konfidenzintervall einzeichnen
## poly.polyval(hooke['distance'], [lm[0]])
beta1_lower_boundary = lm[1] - (t_wert * standardfehler_beta_1)
beta1_upper_boundary = lm[1] + (t_wert * standardfehler_beta_1)

y_lower_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_lower_boundary])

```

```

y_upper_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_upper_boundary])

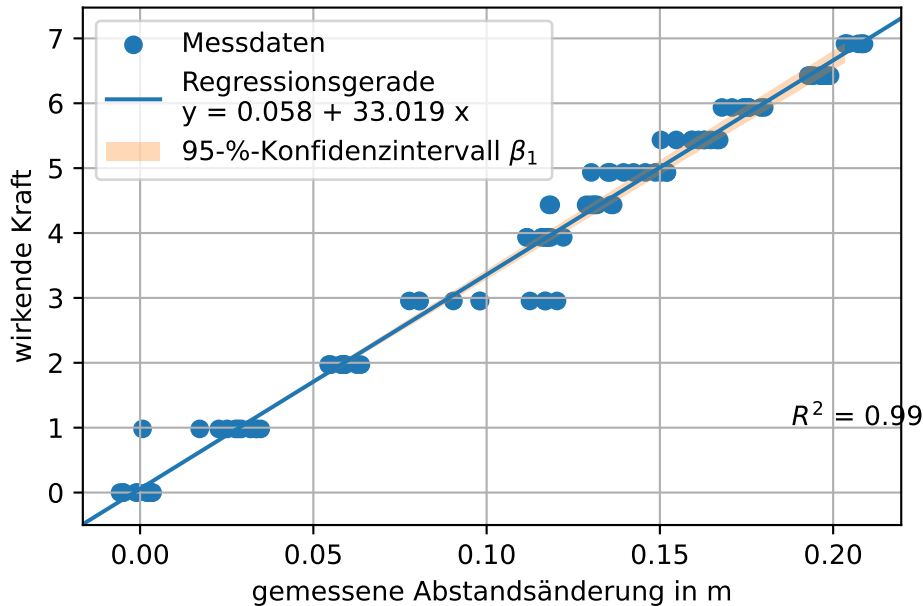
plt.fill_between(x = x, y1 = y_lower_boundary , y2 = y_upper_boundary, alpha = 0.3, label =

plt.title(label = 'Messdaten und geschätzte Linearfunktion im 95%-Intervall')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()

```

Messdaten und geschätzte Linearfunktion im 95%-Intervall



3.6.2 Das Modul SciPy

Die Funktion `scipy.stats.linregress(x, y)` liefert mit einem Funktionsaufruf zahlreiche Rückgabewerte:

- Steigung der Regressionsgerade,
- y-Achsen Schnittpunkt der Regressionsgerade,
- Bravais-Pearson-Korrelationskoeffizient r ,
- p-Wert der Nullhypothese, dass die Steigung der Regressionsgerade Null ist,

- Standardfehler der Steigung und
- Standardfehler des y-Achsenschnittpunkts.

Der Standardfehler des y-Achsenschnittpunkts ist nur verfügbar, wenn die Rückgabewerte in einem Objekt gespeichert werden. Die Rückgabewerte können dann als Attribute abgerufen werden.

```
# Zuweisung mehrerer Objekte
slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

# Zuweisung eines Objekts
lm = scipy.stats.linregress(x, y)

print("\n", lm, sep = '')
print(f"y-Achsenschnittpunkt: {lm.intercept:.4f}\nStandardfehler des y-Achsenschnittpunkts: {lm.stderr:.4f}")
```

```
y = 0.0575 + 33.0190 * x
r = 0.9928 R2 = 0.9856 p = 0.0000
Standardfehler des Anstiegs: 0.3784
```

```
LinregressResult(slope=np.float64(33.01899550918018), intercept=np.float64(0.057531589079701104), stderr=np.float64(0.37837320019327897), intercept_stderr=np.float64(0.05067079726769251),
y-Achsenschnittpunkt: 0.0575
Standardfehler des y-Achsenschnittpunkts:0.0507
```

So kann mit dem entsprechenden t-Wert das Konfidenzintervall berechnet werden.

```
alpha = 0.05
n = len(x)

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}      {slope + scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}")
```

```
32.269      33.019      33.769
```

3.6.3 Ergebnis Federkonstante

Die Federkonstante des Versuchsaufbaus liegt mit 95 prozentiger Sicherheit im Intervall zwischen 32,27 und 33,77. Die Punktschätzung für die Federkonstante beträgt 33,02.

Aufgabe könnte sein, das Konfidenzintervall 99-Prozent zu berechnen.
-> Dann muss man aber nur eine Zahl ändern

4 Fehlerrechnung

“Das Experimentieren ist ein dornenvoller Weg und so ganz sicher ist man sich nie.”
Wagner, Paul. 2015. “Einführung in die Physik I. PH I - 04 - Die Messgenauigkeit / Fehlerrechnung Teil 1.” Universität Wien Physik, [YouTube](#), Zeitstempel 04:05

Kein Messgerät misst exakt und nicht alle Umwelteinflüsse können kontrolliert werden. Das Ziel der Fehlerrechnung ist es, die sich daraus ergebende Abweichung der Messwerte um den wahren Wert einer Größe zu quantifizieren.

4.1 Bevor es losgeht

Das Thema ist jedoch mit einigen Schwierigkeiten verbunden, die den Einstieg und die eigenständige Vertiefung des Themas erschweren.

4.1.1 Der Begriff des Fehlers

Das Wort Fehler steckt zwar schon in der Überschrift dieses Kapitels, doch handelt es sich seit 1995 nicht mehr um einen genormten Begriff. Die DIN 1319 normt Grundbegriffe und Verfahren der Messtechnik. Die Norm ersetzt den Begriff des Fehlers durch die Begriffe **Messabweichung** und **Messunsicherheit**, umfasst aber darüber hinaus eine Vielzahl aufeinander bezogener Begriffsdefinitionen, Anmerkungen und Bemerkungen.

In diesem Kapitel werden die in der Fehlerrechnung gebräuchlichen Begriffe aufgegriffen, um die Verbindung zu den genormten Begriffen herzustellen. Im Allgemeinen werden jedoch die genormten Begriffe verwendet. Um den Einstieg in das Thema zu erleichtern, beschränkt sich dieses Kapitel auf Messungen mit:

- unkorrelierten Eingangsgrößen und
- vernachlässigbarer Unsicherheit über die systematische Messabweichung.

4.1.2 Konfidenzniveau

Wiederholte Messungen enthalten eine zufällige Komponente statistischer Unsicherheit. Die statistische Unsicherheit kann immer nur mit einer Vertrauenswahrscheinlichkeit angegeben werden. Die Berechnung von Konfidenzintervallen haben wir in Kapitel 2 und Kapitel 3 behandelt.

Zur statistischen Analyse gehört als erster Schritt, das Konfidenzniveau, das gelten soll, festzulegen. Hieraus leiten sich die zu verwendenden Vielfachen der Standardabweichung s bzw. des Korrekturfaktors t ab. Hierbei gibt es je nach Disziplin unterschiedliche Konventionen.

“In der Physik und der Vermessungstechnik begnügt man sich mit einer statistischen Sicherheit von 68,3% und setzt deshalb $t = 1$ [...] (In der Industrie wird $t = 2$, in der Biologie $t = 3$ bevorzugt.)” (Hochschule Aalen 2016, S. 4)

In der ingenieurwissenschaftlichen Literatur wird dieser Schritt nicht immer explizit gemacht (weil es üblich ist, $1s$ bzw. $1t$ zu verwenden). Dagegen heißt es in der DIN 1319: “Es ist üblich, das Vertrauensniveau $(1 - \alpha)$ in Prozent anzugeben, wobei meist $\alpha = 0,05$ (Vertrauensniveau: 95 %) benutzt wird.” (Deutsches Institut für Normung 1995, S. 17)

4.1.3 Notation

Die Notation ist komplex: Neben dem lateinischen und griechischen Alphabet wird auch das kyrillische Alphabet verwendet. Trotzdem lassen sich Dopplungen von Symbolen für Formelzeichen und Einheiten nicht immer vermeiden (und die Federkonstante ist in dieser Hinsicht der denkbar ungünstigste Fall).

Die Notation wird außerdem uneinheitlich gehandhabt. Während in der DIN 1319 Messabweichungen mit e notiert werden, sind in der Literatur ϵ (epsilon) und Δ (Delta) gebräuchlich, wobei letzteres auch für den Größtfehler verwendet wird.

Übrigens: Das in der DIN verwendete e steht für error, also Fehler (ϵ).

4.2 Fehlerarten

Bei Messungen können unterschiedliche Arten von Abweichungen des Messergebnisses vom tatsächlichen Wert der gemessenen Größe auftreten. Am gebräuchlichsten ist die Unterteilung in grobe, systematische und zufällige *Fehler*. Nach DIN 1319 sollte besser von systematischen und zufälligen Messabweichungen gesprochen werden. Grobe Fehler werden in der Norm jedoch nicht behandelt und hier ist der Fehlerbegriff durchaus angemessen.

- Systematische Messabweichungen: unter identischen Bedingungen konstante, d. h. nach Betrag und Vorzeichen gleiche, Verzerrung der Messergebnisse durch Abweichungen der Messgeräte. Beispielsweise:
 - in Abhängigkeit von der Temperatur
 - fehlerhafte Kalibrierung
 - Alterung der Bauteile
 - die Art der Messung, z. B. Ablese- und Skalenfehler, Messintervall bei digitalen Messgeräten ([Quantifizierungsfehler](#))
- Zufällige Messabweichungen: statistische Streuung der Messwerte um ihren Erwartungswert. Beispielsweise:
 - Stichprobenfehler
 - Ablesefehler
- Grobe Fehler: Verfälschen der Messergebnisse. Beispielsweise durch:
 - falschen Versuchsaufbau
 - ungeeignete Messgeräte
 - falsches Ablesen, z. B. Missachtung der [Einschwingzeit](#) oder Unachtsamkeit, z. B. Mitwiegen des Gefäßes

(Hempel [2016](#), S. 1–2; Eden und Gebhard [2024](#), S. 17–18)

Bestimmte Tätigkeiten, wie das Ablesen, können mehreren Fehlerarten zugehörig sein:

- Unaufmerksames Ablesen und Flüchtigkeitsfehler sind grobe Fehler,
- winkel- und positionsabhängige Fehler wie der [Parallaxenfehler](#) sind systematische Abweichungen,
- Fehler infolge des begrenzten Auflösungsvermögens des menschlichen Auges führen zu zufälligen Abweichungen.

Tipp 4: Umgang mit verschiedenen Arten von Messabweichungen

1. Grobe Fehler: Betroffene Werte streichen und Messung wiederholen.
2. Systematische Messabweichungen: Systematische Messabweichungen werden, wenn sie bekannt sind, korrigiert.
3. Zufällige Messabweichungen: Die statistische Streuung von Messwerten um den Erwartungswert wird quantifiziert.

Einige systematische und zufällige Messabweichungen können nur mit hohem Aufwand reduziert werden (genauere Messgeräte, größere Anzahl von Messvorgängen). Der Aufwand muss gegen den Gewinn an Genauigkeit abgewogen werden: Ist das Messergebnis mit der

angegebenen Unsicherheit akzeptabel?
(Hempel (2016), S. 2)

4.3 Grundbegriffe nach DIN 1319

Die DIN 1319 normt Grundbegriffe und Verfahren der Messtechnik.

- Teil 1 DIN 1319-1 und Teil 2 DIN 1319-2 definieren Grundbegriffe der Messtechnik und für Messmittel.
- Teil 3 DIN 1319-3 befasst sich mit der Auswertung von Messungen einer *einzelnen* Messgröße und deren Messunsicherheit. DIN 1319-3 wird auch angewendet, wenn die Messgröße mittels einer gegebenen Funktion aus anderen Größen berechnet wird (beispielsweise die Geschwindigkeit v als Quotient der Strecke s und der Zeit t).
- Teil 4 DIN 1319-4 befasst sich mit der Angabe von Messergebnissen und deren Messunsicherheit, wenn die Messgrößen aus anderen Größen berechnet werden, z. B. durch eine Ausgleichsrechnung.

DIN 1319-4 befasst sich mit der Berechnung einer Ergebnisgröße Y durch eine Funktion wie z. B. die lineare Ausgleichsrechnung. Der Anwendungsfall des vorherigen Kapitels, die Federkonstante über den Regressionskoeffizienten β_1 zu bestimmen, wird in der Norm nicht behandelt. Dieser Teil der Norm wird hier deshalb nicht vorgestellt.

Das Lernziel dieses Kapitels besteht darin, dass Sie die folgende Grafik aus der DIN 1319 erklären können.

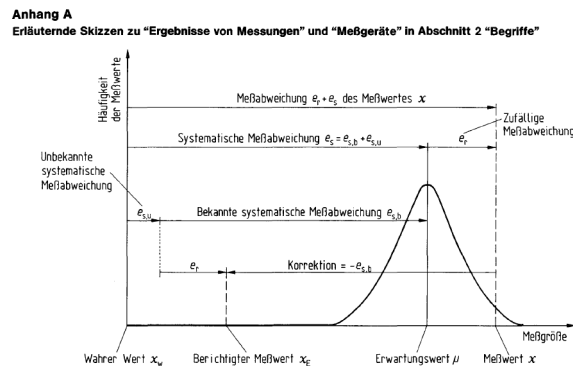


Abbildung 4.1: Messergebnis nach DIN 1319

(Deutsches Institut für Normung 1995, S. 30)

Die drei wichtigsten Grundbegriffe sind:

1. Messabweichung,
2. Messunsicherheit und
3. vollständiges Messergebnis.

4.3.1 Messabweichung

Der in der Fehlerrechnung gebräuchliche Begriff des (Mess-)Fehlers meint im engeren Sinn die Messabweichung nach DIN 1319, also die *tatsächliche Abweichung* eines einzelnen Messwerts oder des arithmetischen Mittelwerts einer Messreihe (dem unberichtigten Messergebnis) vom wahren Wert der gemessenen Größe.

! Wichtig 5: Messabweichung nach DIN 1319

Die Messabweichung ist die: “Abweichung eines aus Messungen gewonnenen und der Meßgröße [...] zugeordneten Wertes vom wahren Wert [...]. Ist m der der Meßgröße zugeordnete Wert und x_w der wahre Wert, so ist die Meßabweichung des zugeordneten Wertes $m - x_w$ [...]” (Deutsches Institut für Normung 1995, S. 12)

Die Messabweichung des **unberichtigten Messergebnisses** setzt sich additiv aus der zufälligen Messabweichung e_r und der systematischen Messabweichung e_s zusammen.

- Die systematische Messabweichung ist die Differenz aus dem Erwartungswert der Messung μ und dem wahren Wert x_w . Es gilt: $e_s = \mu - x_w$
- Die systematische Messabweichung e_s setzt sich aus einem bekannten $e_{s,b}$ und einem unbekannten Anteil $e_{s,u}$ zusammen.
- Die bekannte, systematische Messabweichung $e_{s,b}$ wird mit entgegengesetzten Vorzeichen $-e_{s,b}$ *Korrektur* genannt.

Das (**berichtigte**) **Messergebnis** $\bar{x}_E$ ist das um die bekannte, systematische Messabweichung $e_{s,b}$ berichtigte arithmetische Mittel einer Messreihe $\bar{x}$, das als Schätzer des wahren Werts x_w verwendet wird. $\bar{x}_E = \bar{x} - e_{s,b}$

(Deutsches Institut für Normung 1995, S. 11–13)

4.3.2 Messunsicherheit

Der wahre Wert einer gemessenen Größe ist in der Regel unbekannt, sodass die tatsächliche Messabweichung ebenfalls nicht bekannt ist. Deshalb kann nur eine *aus den Messgrößen gewonnene Schätzung* der Messabweichung - die Messunsicherheit - angegeben werden. Der

in der Fehlerrechnung gebräuchliche Begriff des (Mess-)Fehlers meint im weiteren Sinn die Messunsicherheit nach DIN 1319.

! Wichtig 6: Messunsicherheit nach DIN 1319

“Kennwert, der aus Messungen [...] gewonnen wird und zusammen mit dem Meßergebnis [...] zur Kennzeichnung eines **Wertebereiches** für den wahren Wert der Meßgröße [...] dient.” (Deutsches Institut für Normung 1995, 14, eigene Hervorhebung)

Während die Messabweichung ein Einzelwert ist, kennzeichnet die Messunsicherheit einen Wertebereich: “Die Meßunsicherheit ist von der Meßabweichung [...] deutlich zu unterscheiden. Letztere ist nur die Differenz zwischen einem der Meßgröße zuzuordnenden Wert [...] und dem wahren Wert. Die Meßabweichung kann gleich Null sein, ohne daß dies bekannt ist. Diese Unkenntnis drückt sich in einer Meßunsicherheit größer als Null aus.” (Deutsches Institut für Normung 1996, S. 3)

Die Messunsicherheit einer Messgröße Y besteht einerseits aus der zufälligen Messabweichung e_r und andererseits aus der Unsicherheit über die systematische Messabweichung $u(e_s)$.

zufällige Messabweichung

Wird eine Messgröße $Y = y(x)$ durch mehrere, direkte Messungen bestimmt, so streuen die Messwerte x_i zufällig um ihren Erwartungswert μ mit der Standardabweichung σ . Der Erwartungswert μ und die Standardabweichung σ sind in der Regel nicht bekannt und werden aus den Messwerten durch den arithmetischen Mittelwert $\bar{x}$ und die empirische Standardabweichung s_{n-1} geschätzt. (siehe Kapitel 1)

Als Unsicherheit der zufälligen Streuung wird die empirische Standardabweichung des Mittelwerts verwendet.

$$u(\bar{x}) = \frac{s_{n-1}}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

(Deutsches Institut für Normung 1996, Gleichung (4), S. 5, Notation angepasst)

systematische Messabweichung

Die systematische Messabweichung e_s setzt sich aus einem bekannten Anteil $e_{s,b}$ und einem unbekannten Anteil $e_{s,u}$ zusammen. Da der unbekannte Anteil nicht bestimmt werden kann, wird die gesamte systematische Messabweichung über die bekannte systematische Messabweichung $e_{s,b}$ geschätzt. Daraus ergibt sich eine Unsicherheit über die tatsächliche gesamte systematische Messabweichung $u(e_s)$. Diese Unsicherheit besteht auch, wenn die bekannte systematische Messabweichung $e_{s,b}$ Null ist, weil der unbekannte Anteil $e_{s,u}$ ungleich Null sein kann.

In der DIN 1319 werden zwei Fälle unterschieden, je nachdem, ob die Unsicherheit $u(e_s)$ als vernachlässigbar eingeschätzt wird oder nicht. In diesem Kapitel gehen wir von einer vernachlässigbaren Unsicherheit $u(e_s)$ aus (Für den zweiten Fall siehe Beispiel).

i Hinweis 7: $u(e_s)$ nicht vernachlässigbar

Für den Fall, dass die Unsicherheit der systematischen Messabweichung nicht als vernachlässigbar angesehen wird, verweist die DIN 1319-3 auf Berechnungsregeln in Abschnitt 6.2.5 auf die wir hier nicht eingehen. (Deutsches Institut für Normung (1996), S. 5-6)
Die Messunsicherheit des berichtigten Messergebnisses berechnet sich aus der quadratischen Kombination der Unsicherheiten der zufälligen und der systematischen Messabweichung.

$$u(y) = \sqrt{u^2(\bar{x}) + u^2(e_s)}$$

(Deutsches Institut für Normung 1996, Gleichung (8), S. 6, Notation angepasst)

Hinweis 4.1: Notation in der DIN 1319

In der DIN wird die zufällige Messabweichung statt mit $u^2(\bar{x})$ mit $u^2(x_1)$ und die Unsicherheit der systematischen Messabweichung statt mit $u^2(e_s)$ mit $u^2(x_2)$ notiert.

4.3.3 Vollständiges Messergebnis

Das **berichtigte Messergebnis** $\bar{x}_E = y$ (in der DIN 1319 wechselt die Notation von $\bar{x}_E$ zu y) berechnet sich durch die Korrektur der bekannten systematischen Messabweichung:

$$y = \bar{x} - e_{s,b}$$

(Deutsches Institut für Normung 1996, Gleichung (7), S. 6, Notation angepasst)

Das **vollständige Messergebnis** Y besteht aus dem berichtigten Messergebnis und einer Angabe über die Unsicherheit der Messung z. B. in der Form:

$$Y = y \pm u(y)$$

(Deutsches Institut für Normung 1996, Gleichung (9), S. 6)

Zusätzlich zum vollständigen Messergebnis kann ein Vertrauensbereich (auch: Vertrauensintervall oder Konfidenzintervall) angegeben werden. Der Berechnung von Konfidenzintervallen haben wir im Kapitel Kapitel 2 vorgegriffen.

$$y - t \cdot u(y) \leq Y \leq y + t \cdot u(y)$$

(Deutsches Institut für Normung 1996, Gleichung (14), S. 7)

Die Wahl eines Vertrauensbereichs ist Ermessenssache. Die DIN 1319 formuliert dazu: “Es ist üblich, das Vertrauensniveau $(1 - \alpha)$ in Prozent anzugeben, wobei meist $\alpha = 0,05$ (Vertrauensniveau: 95 %) benutzt wird.” (Deutsches Institut für Normung 1995, S. 17) “Dieses Vertrauensniveau von 95 % sollte verwendet werden, aber kein höheres.” (Deutsches Institut für Normung 1996, S. 7)

So ergibt sich beispielsweise:

$$y - t_{\alpha/2} \cdot \frac{s_{n-1}}{\sqrt{n}} \leq Y \leq y + t_{\alpha/2} \cdot \frac{s_{n-1}}{\sqrt{n}}$$

Warning 2: Notation in the DIN 1319

In der DIN wird der arithmetische Mittelwert einer Messreihe statt mit $\bar{x}$ mit $\bar{v}$ notiert.

4.3.4 Übersicht

In der DIN 1319 werden eine Reihe weiterer Begriffe definiert, die im folgenden Kasten zusammen mit den wichtigsten Grundbegriffen zusammengestellt wurden.

Wichtig 7: Grundbegriffe nach DIN 1319

Messabweichung: “Abweichung eines aus Messungen gewonnenen und der Meßgröße [...] zugeordneten Wertes vom wahren Wert [...]. Ist m der der Meßgröße zugeordnete Wert und x_w der wahre Wert, so ist die Meßabweichung des zugeordneten Wertes $m - x_w$ [...]” (Deutsches Institut für Normung 1995, S. 12)

- Die Messabweichung des **unberichtigten Messergebnisses** setzt sich additiv aus der zufälligen Messabweichung e_r und der systematischen Messabweichung e_s zusammen.
- Die systematische Messabweichung e_s setzt sich aus einem bekannten $e_{s,b}$ und einem unbekannten Anteil $e_{s,u}$ zusammen.
- Die systematische Messabweichung ist die Differenz aus Erwartungswert μ und wahren Wert x_w . $e_s = \mu - x_w$
- Die bekannte, systematische Messabweichung $e_{s,b}$ wird mit entgegengesetzten Vorzeichen $-e_{s,b}$ *Korrektur* genannt.

Das **Messergebnis** $\bar{x}_E$ ist das um die bekannte, systematische Messabweichung $e_{s,b}$ berichtigte arithmetische Mittel einer Messreihe $\bar{x}$, das als Schätzer des Erwartungswerts μ verwendet wird.

$$\bar{x}_E = \bar{x} - e_{s,b}$$

(Deutsches Institut für Normung 1995, S. 11–13)

Messunsicherheit u oder $u(y)$: “Kennwert, der aus Messungen [...] gewonnen wird und zusammen mit dem Meßergebnis [...] zur Kennzeichnung eines Wertebereichs für den wahren Wert der Meßgröße [...] dient.” (Deutsches Institut für Normung 1995, S. 14)

- **Standardmessunsicherheit** oder kurz Standardunsicherheit wird verwendet, wenn die Messunsicherheit durch eine Standardabweichung ausgedrückt wird. (Deutsches Institut für Normung 1996, S. 3)
- **erweiterte Messunsicherheit** $U(y) = k \cdot u(y)$: Wertebereich, der den wahren Wert der Messgröße wahrscheinlich enthält. Der Erweiterungsfaktor k sollte zwischen 2 und 3 festgelegt werden, vorzugsweise wird $k = 2$ verwendet. Der Erweiterungsfaktor k ist mitzuteilen, damit die Standardmessunsicherheit $u(y) = U(y)/k$ ermittelt werden kann. (Deutsches Institut für Normung 1996, S. 7)

Hinweis: Die erweiterte Messunsicherheit unterscheidet sich vom Konfidenzintervall dadurch, dass keine Normalverteilung unterstellt wird - es ist schlicht ein Faktor, ‘um auf Nummer sicher zu gehen’, dass der wahre Wert vom angegebenen Bereich eingeschlossen wird.

Das **vollständige Messergebnis** ist das Messergebnis mit quantitativen Angaben zur Messunsicherheit u . $x = \bar{x}_E \pm u$. (Deutsches Institut für Normung 1995, S. 16)

- Der **Vertrauensbereich** $\bar{x} - t \cdot u(y) \leq Y \leq y + t \cdot u(y)$ kann zusätzlich zum vollständigen Messergebnis angegeben werden.
 - Ist die systematische Messabweichung bekannt und korrigiert worden, dann liegt der wahre Wert der Messgröße Y innerhalb des Vertrauensbereichs.
 - Wenn die systematische Messabweichung nicht bekannt ist, liegt der Erwartungswert μ der Verteilung im Vertrauensbereich. (Deutsches Institut für Normung 1996, S. 7)

Hinweis: In der DIN wird der arithmetische Mittelwert einer Messreihe statt mit $\bar{x}$ mit $\bar{v}$ geschrieben.

4.4 Absolute und relative Fehler

In der Fehlerrechnung werden absolute und relative *Fehler* unterschieden. Hier sollte besser von absoluter und relativer Messunsicherheit gesprochen werden.

- Die **absolute Messunsicherheit** u wird in der Einheit der gemessenen Größe angegeben: $u(m) = 0,2\text{kg}$. Das vollständige Messergebnis lautet: $m = 5,0 \pm 0,2\text{kg}$ oder $m = 5,0\text{kg} \pm 0,2\text{kg}$.

- Die **relative Messunsicherheit** $u_{rel}(m) = \frac{u(m)}{|m|}$ wird entweder als Produkt in der Form $m \cdot (1 \pm \frac{u(m)}{|m|})$ angegeben. Bspw.:

$$m = 5,0\text{kg} \cdot (1 \pm \frac{0,2\text{kg}}{5,0\text{kg}}) = 5,0\text{kg} \cdot (1 \pm 0,04)$$

Alternativ wird die relative Messunsicherheit in Prozent $u_{rel}(m) = \frac{u(m)}{|m|} \cdot 100\%$ angegeben. Bspw.:

$$m = 5,0\text{kg} \pm 4\%$$

(Deutsches Institut für Normung [1995](#), S. 9–10, 16; Deutsches Institut für Normung [1996](#), S. 10)

Warnung 3: (Alternative) Notationen

In der DIN 1319 werden Messabweichungen mit e , die Messunsicherheit mit u notiert. In der Literatur zur Fehlerrechnung wird zumeist der Begriff des (Mess-)Fehlers verwendet. Die Notation ist allerdings uneinheitlich.

- Der absolute Fehler wird häufig mit ϵx (epsilon x) notiert. Üblich ist aber auch die Notation mit Δx (Delta), (beispielsweise (Hochschule Aalen [2016](#); Dinter, Ralf [2011](#); Schrüfer, Reindl und Zagar [2022](#)). Δ wird aber ebenfalls für den Größtfehler verwendet.
- Der relative Fehler wird auch mit δx (delta x) notiert, bspw: $\delta x = 4\%$.

4.5 Größtfehler

Systematische Messabweichungen $e(x)_s$ bewirken eine einseitige Verzerrung des Messergebnisses, sodass der Messwert immer zu hoch oder zu niedrig ausfällt. Zufällige Messabweichungen $e(x)_r$ führen unkontrolliert mal zum Über-, mal zum Unterschätzen des Messwerts.

In der Regel werden sich die verschiedenen Messabweichungen teilweise ausgleichen. Als **Größtfehler** wird der ungünstigste Fall bezeichnet, bei dem sich alle Messabweichungen jeweils um ihren maximalen (oder minimalen) Wert gegenseitig verstärken, sodass der Messwert die maximal mögliche Abweichung vom (unbekannten) tatsächlichen Wert aufweist. Der **Größtfehler** wird als Δx (Delta x) notiert.

Der Größtfehler einer Messreihe für eine Größe wird ermittelt, indem man die Beträge der einzelnen Messabweichungen addiert.

$$\Delta x = |e(x)_s| + |e(x)_r|$$

Da Messungen bei groben Fehlern als ungültig anzusehen sind, entfällt der Anteil grober Fehler $|e(x)_{\text{grob}}|$.

(Eden und Gebhard (2024), S. 35-36)

Werden mehrere Größen betrachtet, addieren sich die Größtfehler der beteiligten Größen zum Größtfehler der Ergebnisgröße. So gilt für eine Größe $Y = f(x_1, x_2)$:

$$\Delta y = \Delta x_1 + \Delta x_2$$

Tipp 5: Größtfehler

Der Größtfehler ist keine genormte Angabe nach DIN 1319, sondern eine vereinfachte Form der Fehlerabschätzung und überschätzt die wahrscheinliche Messunsicherheit. “Für viele Betrachtungen, speziell in den Grundlagenpraktika der ersten Semester in technischen, naturwissenschaftlichen und ingenieurmäßigen Studiengängen, reicht es aus, den maximalen Fehler (*absoluter Größtfehler*) zu bestimmen.” (Eden und Gebhard (2024), S. 35)

4.6 Signifikante Stellen

Bei der Angabe des vollständigen Messergebnisses ist die Anzahl der signifikanten oder zuverlässig bekannten Stellen zu beachten.

Wichtig 8: signifikante Stellen

“Jede zuverlässig bekannte Stelle mit Ausnahme der Nullen, die die Position des Dezimalkommata angeben, wird signifikante Stelle genannt.” (Eden und Gebhard 2024, S. 24)
“Gerundete numerische Unsicherheitswerte sind **mit zwei (oder bei Bedarf mit drei) signifikanten Ziffern** anzugeben. **Sie sind aufzurunden.** Das Meßergebnis ist an derselben Stelle wie die zugehörige Unsicherheit zu runden, z.B. $y = 245,5716\text{mm}$ auf $y = 245,57\text{mm}$, wenn $u(y) = 0,4528\text{mm}$ auf $u(y) = 0,46\text{mm}$ aufgerundet wird.” (Deutsches Institut für Normung 1996, 6, eigene Hervorhebung)

Beispielsweise liegen die im vorherigen Kapitel verwendeten Abstandsmessungen im Format 153,29 vor, die Messwerte weisen also 5 signifikante Stellen auf. Für den arithmetischen Mittelwert einer Messreihe 153,008889 gelten deshalb ebenfalls 5 signifikante Stellen. Es wird an der ersten nicht mehr signifikanten Stelle gerundet, sodass der arithmetische Mittelwert mit 153,01 angegeben wird.

Die Anzahl signifikanter Stellen ist allerdings nicht immer zuverlässig zu bestimmen. Es gilt:

1. Alle Ziffern ungleich Null sind immer signifikant.

2. Führende Nullen sind niemals signifikante Stellen. Beispielsweise hat 0012 km 2 signifikante Stellen. 0,001 mm hat eine signifikante Stelle.
3. Nullen am Ende einer Zahl sind signifikant, wenn sie sich rechts vom Dezimaltrennzeichen befinden, bspw: 12,100 km hat 5 signifikante Stellen.
4. Nullen, die sich am Ende einer Zahl ohne Komma befinden, können signifikant sein. 1200 m kann zwei oder vier signifikante Stellen haben.

💡 signifikante Stellen und wissenschaftliches Runden

Angaben zur Messunsicherheit werden zwar aufgerundet. Trotzdem an dieser Stelle: Kennen Sie den Unterschied zwischen kaufmännischem und wissenschaftlichem Runden?

<https://www.youtube.com/watch?v=s0gPyypigOs>

Runden und signifikante Stellen (Vorkurs Mathematik) von Edmund Weitz ist abrufbar auf [YouTube](#). 2019

Python rundet wissenschaftlich:

```
print(round(2.5), round(3.5))
```

2 4

4.7 Methoden der Fehlerrechnung

Das Ziel der Fehlerrechnung ist es, Unsicherheitsintervalle für die gemessene Größe zu ermitteln. Dafür stehen verschiedene Methoden zur Verfügung:

1. Fehlerabschätzung,
2. Fehlerstatistik und
3. Fehlerfortpflanzung.

Bei der Auswahl der Methode besteht ein Ermessensspielraum. (Hempel (2016), S. 2) Für Gebrauchsmessungen reicht es aus, die Unsicherheit im Prozentbereich anzugeben. Im Eichwesen oder beim Vergleich mit einem [Normal](#) werden genauere Angaben benötigt. (Schrüfer, Reindl und Zagar 2022, S. 33)

Die Auswahl der geeigneten Methode hängt auch von den zur Verfügung stehenden Informationen ab. Beispielsweise hängt die im vorherigen Kapitel ermittelte Federkonstante von drei Größen ab:

1. der Masse m , die mit einer Küchenwaage ermittelt wurde, deren Ablesegenauigkeit mit $e_r(m) = 0,5g = 0,0005kg$ geschätzt wird.

2. dem Abstand zum Gewicht bzw. nach der Aufbereitung der Daten der Ausdehnung der Feder Δx , die mit einem elektronischen Abstandssensor und einem Arduino ermittelt wurden (Ultrasonic Ranging Module HC - SR04).
 - Die Ablesegenauigkeit des Messsystems beträgt eine Einheit der Messauflösung, also $e_r = 0,01\text{cm} = 0,0001\text{m}$.
 - Die Messabweichung des Messsystems beträgt laut Datenblatt bis zu 3 mm, die symmetrisch zu verstehen ist, also $e_r = 3\text{mm} = 0,003\text{m}$.
 - Die zufällige Komponente der Messabweichung beträgt also $e_r(x) = 0,0001\text{m} + 0,003\text{m} = 0,0031\text{m}$
3. der statistischen Schätzung der Federkonstante $k = \frac{m \cdot g}{\Delta x} = 33,019 \pm 0,378$ bzw. im festgelegten Vertrauensintervall $k_{95\%} = 32,269 \leq 33,019 \leq 33,769$.

Warning 4: Formelzeichen und Einheiten

Im Folgenden werden identische Zeichen für Sekunde und Strecke s , Masse und Meter m , Gramm und die Konstante g sowie Δ für die Veränderung einer Größe und den Größtfehler verwendet. Die Formeln sollten also mit großer Aufmerksamkeit gelesen werden.

4.7.1 1. Fehlerabschätzung

Die Fehlerabschätzung wird verwendet, wenn

- **eine Größe**
- **nur einmal**

gemessen wird.

Da der wahre Wert einer Messgröße unbekannt ist, bezweckt die Fehlerabschätzung, die maximal mögliche Abweichung des Messwerts vom tatsächlichen Wert, also den **Größtfehler**, zu bestimmen. Bei einer einmaligen Messung gibt es keine statistische Komponente. In die Fehlerabschätzung eines einzelnen Messwerts gehen deshalb ein:

- die Garantiefehlergrenze des Messgerätes, die vom Hersteller auf dem Gerät oder in der Bedienungsanleitung angegeben ist (siehe Beispiel), also die bekannte systematische Messabweichung $e_{s,b}$, und
- ein abgeschätzter Anteil, der sich nach den konkreten Bedingungen des Experiments, etwa der Skalenteilung des Messgerätes oder den Ablesebedingungen richtet. Dieser Anteil kann also aus einem systematischen Anteil (bspw. eine analoge Skala konnte nur aus einem Winkel abgelesen werden) und einem zufälligen Anteil (bspw. Ablesegenauigkeit des Messgeräts) bestehen.

Der Größtfehler Δx ergibt sich aus der Addition der Beträge der bekannten systematischen Messabweichung und der geschätzten zufälligen Messabweichung: $\Delta x = |e(x)_{s,b}| + |e(x)_r|$.

(Hempel (2016), S. 3)

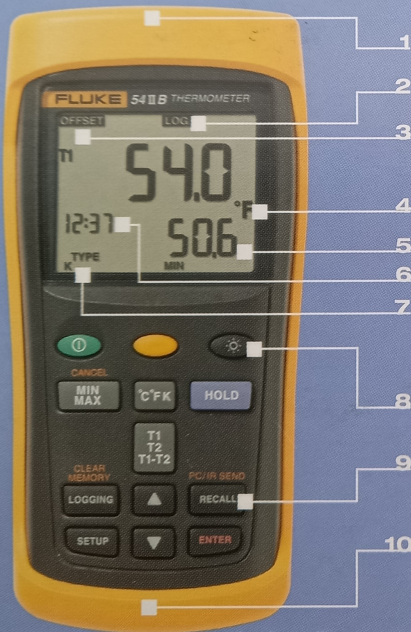
Aufgabe Temperaturmessung

Mit einem digitalen Thermometer und einem thermoelektrischen Element vom Typ K wurde die Temperatur $T = 112,1\text{ }^{\circ}\text{C}$ gemessen. Welche systematischen und zufälligen Messabweichungen sind bekannt und können angegeben werden (siehe Beispiel)?

FLUKE®

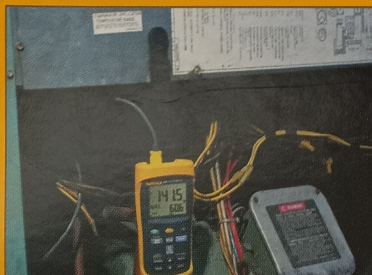
Thermocouple Thermometer

53/54 II B



Key features

- 1 Accepts magnetic hanger (optional TPAK accessory) for hands-free operation
- 2 Datalog up to 500 points to internal memory
- 3 Electronic offset compensates for thermocouple error
- 4 Selectable °C, °F, or Kelvin readout
- 5 Large dual display shows any combination of T1, T2 (54 II only), T1-T2 (54 II only), plus MIN, MAX, or AVG
- 6 Real time clock captures exact time of day when major events occur
- 7 Thermometer accepts seven different thermocouple types
- 8 Bright backlight turns on for two minutes with a touch of a button
- 9 Recall logged data on-screen or download via IR port to optional PC software
- 10 Rugged design easily withstands a 1 m (3 ft) drop



Specifications

Accuracy	
Above -100 °C	[0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types
Below -100 °C	[0.05 % + 0.4 °C (0.7 °F)] for R and S types
	[0.2 % + 0.3 °C] for J, K, E, and N types
	[0.5 % + 0.3 °C] for T type
Resolution	0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000
Battery Life	1000 hours typical
Warranty	Three-years

Abbildung 4.2: Thermometer

Specifications	
Accuracy	
Above -100 °C	[0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types
Below -100 °C	[0.05 % + 0.4 °C (0.7 °F)] for R and S types [0.2 % + 0.3 °C] for J, K, E, and N types [0.5 % + 0.3 °C] for T type
Resolution	0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000
Battery Life	1000 hours typical
Warranty	Three-years

Abbildung 4.3: Spezifikationen

💡 Tipp 6: Halber oder ganzer Skalenstrich?

Messgeräte mit digitaler Anzeige machen es leicht: Die Ablesegenauigkeit beträgt eine Skaleneinheit. Bei Messgeräten mit analoger Anzeige ist es Ermessenssache, welche Messgenauigkeit Sie angeben können bzw. möchten.

Sind Sie sich absolut sicher, dass Sie zwischen “genau auf dem Strich” und “zwischen zwei Strichen” unterscheiden können, beträgt Ihr Ablesefehler einen halben Skalenstrich. Sind Sie sich nicht sicher, beträgt Ihr Ablesefehler einen ganzen Skalenstrich.

(Fehlerrechnung leicht gemacht, FSU Jena)

💡 Tipp 7: Musterlösung Temperaturmessung

systematische Messabweichung

Die absolute Messabweichung beträgt $u(T) = 0,3 \text{ °C}$. Die relative Messabweichung beträgt 0,05 %. Das bedeutet:

$$\frac{u(T)}{112,1 \text{ °C}} \cdot 100 \% = 0,05 \%$$

$$\frac{u(T)}{112,1 \text{ °C}} \cdot 100 = 0,05$$

$$u(T) = 0,05 \cdot \frac{112,1 \text{ °C}}{100}$$

$$u(T) = 0,05605 \text{ °C}$$

Die bekannte systematische Messabweichung beträgt also:

$$e(T)_{s,b} = 0,3 \text{ °C} + 0,05605 \text{ °C} = 0,35605 \text{ °C}$$

zufällige Messabweichung

Die zufällige Messabweichung beträgt eine Skaleneinheit, also:

$$e(T)_r = 0,1 \text{ °C}$$

Größtfehler

Der Größtfehler beträgt:

$$\Delta T = |e(T)_{s,b}| + |e(T)_r|$$

Die Messunsicherheit ist auf zwei Stellen aufzurunden:

$$\Delta T = |0,35605^\circ\text{C}| + |0,1^\circ\text{C}| = 0,45605^\circ\text{C} \approx 0,46^\circ\text{C}$$

Das vollständige Messergebnis lautet also $T = 112,1 \pm 0,46^\circ\text{C}$.

Beispiel Messbedingungen

Was ist mit den konkreten Bedingungen eines Messvorgangs gemeint? Das soll mit zwei einfachen Messvorgängen demonstriert werden.

Hier können Sie mitmachen!

Sie benötigen:

- ein geeignetes Messinstrument, z. B. ein Lineal, und
- eine Münze (idealerweise 1-Euro-Münze).

Im ersten Messvorgang soll der Durchmesser der 1-Euro-Münze bestimmt werden.

i Hinweis 9: Erster Messvorgang Münze

Die Messung wird mit einem **Messschieber mit Nonius** 0,05 mm nach DIN 862 durchgeführt. Die DIN 862 normiert symmetrische Fehlergrenzen von Messschiebern. Die Fehlergrenze ist abhängig von der Skalengenauigkeit des Messschiebers sowie von der gemessenen Länge und ist aus einer Tabelle abzulesen. (DIN 862 wurde 2011 durch DIN EN ISO 13385-1 ersetzt, in der der Begriff der Abweichung verwendet wird. Die DIN 862 ist aber auf dem Messschieber aufgedruckt.)

Messwert: $x = 23,25\text{mm}$

- Die Skaleneinteilung ist 0,05 mm, die zufällige Ableseabweichung e_r beträgt also $\pm 0,025\text{mm}$.
- Die DIN 862 normiert Fehlergrenzen, die abhängig von der gemessenen Länge zunehmen, mindestens aber $50\text{ }\mu\text{m}$ oder 0,05 mm betragen. Für den Messwert 23,25 mm gilt somit als bekannte systematische Messabweichung $e_{s,b} = \pm 0,05\text{mm}$.

Aus der Summe der Beträge ergibt sich der Größtfehler: $\Delta x = |0,05|\text{mm} + |0,025|\text{mm} = 0,075\text{mm}$.

Das vollständige Messergebnis, bestehend aus dem Messwert und der Messunsicherheit lautet also: $23,25 \pm 0,075\text{mm}$. Der relative Größtfehler beträgt: $23,25\text{mm} \pm 0,32\%$.

Im zweiten Messvorgang soll die Länge des Daumens bestimmt werden. (**Der Daumen besteht anatomisch aus nur 2 Fingergliedknochen**)

i Hinweis 10: Zweiter Messvorgang Daumen

Die Messung wird erneut mit dem Messschieber durchgeführt.

Messwert: $x = 64,70\text{mm}$

- Die Skaleneinteilung ist $0,05\text{ mm}$, die zufällige Ableseabweichung beträgt also $\pm 0,025\text{mm}$.
- Die von der DIN 862 für gemessene Längen zwischen 50 und 300 mm normierte Fehlergrenze beträgt $50\text{ }\mu\text{m}$ oder $0,05\text{ mm}$. Für den Messwert $64,7\text{ mm}$ gilt somit die Fehlergrenze $\pm 0,05\text{mm}$.
- zusätzlich wird eine symmetrische Messabweichung von 3 mm veranschlagt, da
 - das Messgerät nicht gerade angesetzt,
 - die Daumenwurzel nicht exakt lokalisiert und
 - die elastische Fingerkuppe nicht fixiert werden konnte.

Aus der Summe der Beträge ergibt sich der Größtfehler $\Delta x = |0,025|\text{mm} + |0,05|\text{mm} + |3|\text{mm} = 3,075\text{mm} \approx 3,08\text{mm}$.

Das Messergebnis, bestehend aus dem Messwert und dem Fehlerintervall lautet also: $64,70 \pm 3,08\text{mm}$. Der relative Größtfehler beträgt: $64,70\text{mm} \pm 4,76\%$.

4.7.2 2. Fehlerstatistik

Die Fehlerstatistik wird verwendet, wenn

- **eine Größe**
- **mehrfach**

gemessen wird.

Die Fehlerstatistik ist nur dann als zuverlässig anzusehen, wenn die Messabweichung zufällig ist, die Messwerte also betragsmäßig und mit wechselndem Vorzeichen streuen. Dann ist die Messabweichung in der ermittelten Standardabweichung enthalten. Liegt jedoch eine bekannte, systematische Messabweichung vor, muss das Messergebnis entsprechend korrigiert werden $\bar{x}_E = \bar{x} - e_{s,b}$.

Der Fehlerstatistik haben wir in vorausgehenden Kapiteln mit der statistischen Beschreibung der empirischen Streuung des Mittelwerts vorgegriffen. Zur Wiederholung:

1. Legen Sie ein Signifikanzniveau α bzw. ein zweiseitiges Konfidenzniveau $1 - \frac{\alpha}{2}$ fest.
2. Bestimmen Sie den Stichprobenmittelwert $\bar{x}$.
3. Bestimmen Sie die Stichprobenstandardabweichung s , die Stichprobengröße N und den empirischen Standardfehler $\frac{s_{n-1}}{\sqrt{N}}$.

4. Bestimmen Sie die Werte der Normal- bzw. der t-Verteilung für das gewählte Konfidenzniveau - für einen zweiseitigen Hypothesentest $\frac{\alpha}{2}$ und $1 - \frac{\alpha}{2}$
5. Berechnen Sie das Konfidenzintervall $\bar{x} \pm t_{\alpha/2} \frac{s_{n-1}}{\sqrt{n}}$.

Dieses Konfidenzintervall sagt für eine gewählte Wahrscheinlichkeit aus, in welchem Bereich der Mittelwert einer erneut durchgeführten Messreihe oder, wenn die unbekannte systematische Messabweichung Null (oder vernachlässigbar) ist, der *wahre Wert* liegen wird.

Dagegen gibt die Standardabweichung s Auskunft darüber, in welchem Bereich ein einzelner Messwert liegen wird. Die Sicherheit dieser Aussage kann durch Ausweitung des Konfidenzintervalls erhöht werden (bspw. $2 \cdot s$).

(Hempel 2016, S. 3–7)

Im folgenden Beispiel wird das Vorgehen gezeigt. Dabei wird auch die Pandas-Methode `.sem()` zur einfachen Berechnung des Standardfehlers vorgestellt.

Hinweis 11: Fehlerstatistik

Aus dem im vorherigen Kapitel behandelten Experiment wird die Messreihe für das Gewicht 503g ausgewählt.

```
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
hooke.drop(index = 113, inplace = True) # Fehlwert, siehe vorheriges Kapitel

hooke_503g = hooke.loc[hooke['mass'] == 503, :]
print(hooke_503g.head(), hooke_503g.describe(), sep = "\n\n")
```

	no	mass	distance
40	40	503	158.43
41	41	503	158.43
42	42	503	158.60
43	43	503	158.76
44	44	503	159.05

	no	mass	distance
count	10.00000	10.0	10.000000
mean	44.50000	503.0	159.314000
std	3.02765	0.0	0.781099
min	40.00000	503.0	158.430000
25%	42.25000	503.0	158.640000
50%	44.50000	503.0	159.220000
75%	46.75000	503.0	159.965000
max	49.00000	503.0	160.610000

Betrachten wir für diese Messreihe noch einmal das arithmetische Mittel (das unbereinigte Messergebnis) und den Standardfehler des Mittelwerts (das statistische Streuungsmaß des Mittelwerts). Die Methode `.sem()` berechnet den empirischen Standardfehler des Mittelwerts (standardmäßig ist der Parameter `ddof = 1` gesetzt.)

Ausgabe auf 3 Dezimalstellen genau.

```
print(f"Gewicht in Gramm: {hooke_503g['mass'].unique()}",
      f"Mittelwert der Messwerte: {hooke_503g['distance'].mean():.3f} cm",
      f"empirischer Standardfehler: {hooke_503g['distance'].sem():.3f} cm",
      sep = "\n")
```

Gewicht in Gramm: [503]

Mittelwert der Messwerte: 159.314 cm

empirischer Standardfehler: 0.247 cm

Die Messunsicherheit wird auf 2 Dezimalstellen genau aufgerundet. Dafür kann die NumPy-Funktion `np.ceil()` verwendet werden. Diese rundet allerdings zur nächsten Ganzzahl, ein Parameter, um auf eine bestimmte Dezimalstelle zu runden, ist nicht verfügbar. Deshalb muss die Dezimalstelle um die gewünschte Anzahl Stellen durch Multiplikation mit der entsprechenden Zehnerpotenz verschoben werden. Für 2 Dezimalstellen entsprechend um 10^2 : `np.ceil(wert * 100) / 100`.

```
wert = pd.Series([0.247])
print(np.ceil(wert * 100) / 100)

# mit Pandas-Methoden
print(wert.mul(100).apply(np.ceil).div(100))
```

```
0    0.25
dtype: float64
0    0.25
dtype: float64
```

Für die Verkettung mit weiteren Methoden in Pandas muss die Rückgabe der Funktion, ein NumPy-Skalar oder -Array, wieder in eine `pd.Series` umgewandelt werden.

```
print(f"Gewicht in Gramm: {hooke_503g['mass'].unique()}",
      f"Mittelwert der Messwerte: {hooke_503g['distance'].mean():.3f} cm",
      f"empirischer Standardfehler: { ( sem_hooke_503g:= pd.Series(hooke_503g['distance'].sem()) ):.3f} cm",
      f"vollständiges Messergebnis: {hooke_503g['distance'].mean().round(2):.2f} cm ± {sem_hooke_503g:.2f} cm",
      sep = "\n")
```

Gewicht in Gramm: [503]
Mittelwert der Messwerte: 159.314 cm
empirischer Standardfehler: 0.250 cm
vollständiges Messergebnis: 159.31 cm ± 0.25 cm

Zusätzliche Angabe des 95-%-Konfidenzintervalls.

```
# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n = hooke_503g.shape[0] # Anzahl Zeilen
t_wert = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 1)

print(f"95-%-Konfidenzintervall : {hooke_503g['distance'].mean().round(2)} cm ± {t_wert:.4} cm ± {(t_wert * sem).round(2)} cm")
print(f"95-%-Konfidenzintervall : {hooke_503g['distance'].mean().round(2)} cm ± {(t_wert * sem).round(2)} cm")
print(f"untere 95-%-Intervallgrenze: {hooke_503g['distance'].mean().round(2) - round(t_wert * sem, 2)} cm")
print(f"obere 95-%-Intervallgrenze: {hooke_503g['distance'].mean().round(2) + round(t_wert * sem, 2)} cm")
sep = "\n")
```

```
95-%-Konfidenzintervall : 159.31 cm ± 2.2622 * 0.25 cm
95-%-Konfidenzintervall : 159.31 cm ± 0.57 cm
untere 95-%-Intervallgrenze: 158.74 cm
obere 95-%-Intervallgrenze: 159.88 cm
```

4.7.3 3. Fehlerfortpflanzung

Die Fehlerfortpflanzung wird verwendet, wenn

- mehrere Eingangsgrößen,
- die durch eine Funktion die Ergebnisgröße ergeben,

gemessen werden.

Um die Messunsicherheit einer Größe Y , die durch eine Funktion $y = f(x_1, x_2, \dots, x_m)$ geschätzt wird, zu ermitteln, gibt es zwei Verfahren:

- die **lineare Fehlerfortpflanzung** wird benutzt, wenn die Unsicherheit der beteiligten Größen durch Fehlerabschätzung einer **Einzelmessung** ermittelt wurden.
- die **Gauß'sche Fehlerfortpflanzung** wird benutzt, wenn die Unsicherheit aus einer **Mehrfachmessung statistisch ermittelt** und mit Konfidenzintervallen angegeben werden.

(Hempel 2016, S. 8–9)

Lineare Fehlerfortpflanzung

Die lineare Fehlerfortpflanzung betrachtet, wie sich die **Größtfehler** mehrerer gemessener Größen im ungünstigsten Fall auswirken. Es wird also der **Größtfehler** der gesuchten Größe Y berechnet. (vgl. Deutsches Institut für Normung (1996), S. 9)

Für eine Funktion mit zwei Eingangsgrößen $f = f(x_1, x_2)$ wird der Größtfehler Δf aus den Größtfehlern Δx_1 und Δx_2 ermittelt.

$$f + \Delta f = f(x_1 + \Delta x_1, x_2 + \Delta x_2)$$

Unter der Voraussetzung gegenüber den Messwerten kleiner Fehler kann der Fehler Δf durch die Beträge der partiellen Ableitung des ersten Glieds der Taylorreihe berechnet werden (siehe Beispiel).

$$f + \Delta f = f(x_1, x_2) + \left| \frac{\partial f(x_1, x_2)}{\partial x_1} \right| \cdot \Delta x_1 + \left| \frac{\partial f(x_1, x_2)}{\partial x_2} \right| \cdot \Delta x_2$$

Wann sind die Fehler gegenüber den Messwerten klein? Dafür gibt es leider keine absolute Grenze. Es kommt hierbei darauf an, dass bei der Taylor-Entwicklung die Polynome höheren Grades deutlich kleiner als der lineare Term sind und somit vernachlässigt werden können, wenn das x klein ist. Die Entscheidung hängt letztlich von der benötigten Genauigkeit ab.

Hinweis 12: Taylorreihe

<https://www.youtube.com/watch?v=F2lKRuUguBM>

Die Taylorreihe - einfach erklärt von Denkbar ist abrufbar auf [YouTube](#). 2015

Hinweis 13: Partielle Ableitung

Die partielle Ableitung ist die Ableitung einer Funktion mit mehreren Variablen nach einer Variablen, wobei die übrigen Variablen als Konstanten behandelt werden.

Für eine Funktion $f(x, y) = 2x + y^2$ wird die partielle Ableitung nach x so ausgedrückt:

$$\frac{\partial f(x, y)}{\partial x}$$

- Das Symbol ∂ ist die kursive Darstellung des kyrillischen Kleinbuchstaben (d) und wird als “del” gelesen. Es zeigt an, dass eine partielle Ableitung durchgeführt wird.
- Im Zähler steht die Funktion, die abgeleitet werden soll. Im Nenner steht die Variable nach der abgeleitet wird. Der Term wird gelesen als “del f von x und y nach del x”.

Die partielle Ableitung $\frac{\partial f(x, y)}{\partial x} = 2$. y^2 wird als Konstante behandelt (z. B. 5^2) und ist abgeleitet Null.

Die partielle Ableitung $\frac{\partial f(x,y)}{\partial y} = 2y \cdot 2x$ wird als Konstante behandelt (z. B. $2 \cdot 3$) und ist abgeleitet Null.

Der Größtfehler ergibt sich aus:

$$\Delta f = \left| \frac{\partial f(x_1, x_2)}{\partial x_1} \right| \cdot \Delta x_1 + \left| \frac{\partial f(x_1, x_2)}{\partial x_2} \right| \cdot \Delta x_2$$

(Hempel 2016, S. 9–10)

Bzw. in Kurzschreibweise für $i = N$ Eingangsgrößen:

$$\Delta f(x_i) = \pm \sum_{i=1}^N \left| \frac{\partial f}{\partial x_i} \right| \cdot \Delta x_i$$

(nach Eden und Gebhard 2024, 35, Notation angepasst)

i Hinweis 14: Beispiel Federkonstante

Beispielhaft betrachten wir die erste Messung aus der Messreihe mit dem Gewicht 503 Gramm.

```
hooke_503g_einzelwert = hooke_503g.iloc[0, :].copy()
print(hooke_503g_einzelwert)
```

```
no          40.00
mass        503.00
distance    158.43
Name: 40, dtype: float64
```

Da mehrere Größen betrachtet werden, werden die Messwerte in eine SI-Einheit umgerechnet. Als Nullpunkt wird die erste Messung der Messreihe mit dem Gewicht 0 Gramm aufgefasst.

```
hooke_503g_einzelwert['mass'] = hooke_503g_einzelwert['mass'] / 1000
hooke_503g_einzelwert['distance'] = hooke_503g_einzelwert['distance'] / 100

nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].head(n = 1) / 100
print(f"Nullpunkt in Metern: {nullpunkt.item():.3f}\n")
```

```
hooke_503g_einzelwert['ausdehnung'] = (hooke_503g_einzelwert['distance'].item() - nullpunkt)
print(hooke_503g_einzelwert)
```

Nullpunkt in Metern: 1.733

```
no          40.0000
mass        0.5030
distance    1.5843
ausdehnung  0.1489
Name: 40, dtype: float64
```

Für die Federkonstante gilt (die Abstandsänderung Δx wird mit s notiert):

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

Einsetzen der Werte (s steht nun für Sekunde, m für Meter):

$$k = \frac{0,503kg \cdot 9,81m}{0,1489m \cdot s^2} = \frac{0,503 \cdot 9,81}{0,1489} \frac{N}{m} = 33.139 \frac{N}{m}$$

So gilt für den Größtfehler der Funktion:

$$\Delta k(m, s) = \left| \frac{\partial f(m, s)}{\partial m} \right| \cdot \Delta m + \left| \frac{\partial f(m, s)}{\partial s} \right| \cdot \Delta s$$

Im ersten Schritt werden die partiellen Ableitungen der Funktion $k = \frac{m \cdot g}{s}$ gebildet. Die partielle Ableitung nach m lautet mit Faktorregel:

$$\frac{\partial f(m, s)}{\partial m} = \frac{\partial}{\partial m} \frac{m \cdot g}{s} = \frac{\partial}{\partial m} m \cdot \frac{g}{s} = 1 \cdot \frac{g}{s} = \frac{g}{s}$$

Die partielle Ableitung nach s lautet:

$$\frac{\partial f(m, s)}{\partial s} = \frac{\partial}{\partial s} \frac{m \cdot g}{s} = -\frac{m \cdot g}{s^2}$$

Der Größtfehler ergibt sich also durch:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| (-) \frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

Im zweiten Schritt wird der Größtfehler der beteiligten Größen ermittelt.

- Die Masse m der verwendeten Gewichte wurde mit einer Küchenwaage ermittelt, deren Ablesabweichung auf $e_r(m) = 0,5g = 0,0005kg$ geschätzt wird. Eine systematische Messabweichung ist nicht bekannt.
Der Größtfehler der Masse beträgt also $\Delta m = 0,0005kg$.
- Der Größtfehler der Abstandsmessung beträgt $e_r(s) = 0,01cm + 0,3cm = 0,0031m$.

Die Messwerte, die Konstante g und die Größtfehler werden eingesetzt.

$$\Delta k = \left| \frac{9,81m}{s^2 \cdot 0,1489m} \right| \cdot 0,0005kg + \left| (-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1489m)^2} \right| \cdot 0,0031m$$

Trennen von Werten und Einheiten:

$$\Delta k = \left| \frac{9,81 \cdot 0,0005}{0,1489} \frac{m \cdot kg}{s^2 \cdot m} \right| + \left| (-) \frac{0,503 \cdot 9,81 \cdot 0,0031}{0,02217121} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right|$$

Berechnen und Newton $\frac{kg \cdot m}{s^2}$ einsetzen:

$$\Delta k = \left| 0,0329 \frac{N}{m} \right| + \left| (-) 0,6899 \frac{N \cdot m}{m^2} \right|$$

Meter im zweiten Term kürzen:

$$\Delta k = \left| 0,0329 \frac{N}{m} \right| + \left| (-) 0,6899 \frac{N}{m} \right|$$

Die Messunsicherheit wird auf zwei Dezimalstellen aufgerundet.

$$\Delta k = 0,7228 \frac{N}{m} \approx 0,73 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 33,14 \frac{N}{m}; \Delta k = 0,73 \frac{N}{m}$$

Gauß'sche Fehlerfortpflanzung

Bei der Gauß'schen Fehlerfortpflanzung wird unterstellt, dass sich die Messunsicherheiten der beteiligten Größen teilweise kompensieren können. Dadurch wird eine realistischere Abschätzung der Messunsicherheit erreicht.

Die Ermittlung der Messunsicherheit ist in DIN 1319-3 geregelt. Für eine Funktion $Y = f(x_1, x_2, \dots, x_m)$ ermittelt sich die Messunsicherheit aus der Messunsicherheit der beteiligten Größen. Dazu werden die Varianzen der Mittelwerte verwendet (Deutsches Institut für Normung (1996), Gleichung (29), S. 11).

$$u^2(x_i) = s^2(\bar{x}_i) = s_i^2/n_i; \quad s_i^2 = \frac{1}{n_i - 1} \sum_{j=1}^{n_i} (x_{ij} - \bar{x}_i)^2$$

Hinweis: In der Regel wird man die empirische Standardabweichung s_{n-1} verwenden.

Hinweis: In der DIN wird der arithmetische Mittelwert einer Messreihe statt mit $\bar{x}$ mit $\bar{v}$

geschrieben.

Die Messunsicherheit der beteiligten Größen wird also über den Standardfehler des Mittelwerts der beteiligten Größen ermittelt:

$$u(x_i) = s_i / \sqrt{n_i}$$

Hinweis: In der Regel wird man die empirische Standardabweichung s_{n-1} verwenden.

(Deutsches Institut für Normung 1996, S. 11)

Die Unsicherheit der Funktion $y = f(x_1, x_2, \dots, x_m)$ berechnet sich wie folgt (DIN 1319-3, Gleichung (43)):

$$u(y) = \sqrt{\sum_{i=1}^m \left(\frac{\partial f}{\partial x_i} \right)^2 \cdot u^2(x_i)}$$

Die quadrierten Terme können auch zusammengefasst werden:

$$u(y) = \sqrt{\sum_{i=1}^m \left(\frac{\partial f}{\partial x_i} \cdot u(x_i) \right)^2}$$

Für eine Funktion mit zwei Eingangsgrößen $y = f(x_1, x_2)$ berechnet sich die Unsicherheit der Ergebnisgröße beispielsweise wie folgt:

$$u(y) = \sqrt{\left(\frac{\partial f}{\partial x_1} \cdot \sigma_{x_1} \right)^2 + \left(\frac{\partial f}{\partial x_2} \cdot \sigma_{x_2} \right)^2}$$

Hinweis: In der Regel wird man die empirische Standardabweichung $s_{n-1} / \sqrt{N}$ verwenden.

i Hinweis 15: Beispiel Messreihe 503 Gramm

Betrachten wir noch einmal die Messreihe für das Gewicht 503 Gramm. Es soll das vereinbarte Konfidenzniveau 95 % gelten.


```

# Messdaten aus dem Kapitel hooke.qmd wiederherstellen
## Datei einlesen und Fehler entfernen
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
hooke.drop(index = 113, inplace = True)

## Abstandsmessung auf Nullpunkt normieren
nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
hooke['ausdehnung'] = hooke['distance'].sub(nullpunkt).mul(-1)

# Messreihe 503 Gramm auswählen
hooke_503g = hooke.loc[hooke['mass'] == 503, :].copy()

## Masse in kg, Abstand und Ausdehnung in m umrechnen
hooke_503g['mass'] = hooke_503g['mass'].div(1000)
hooke_503g['distance'] = hooke_503g['distance'].div(100)
hooke_503g['ausdehnung'] = hooke_503g['ausdehnung'].div(100)

# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n_ausdehnung = hooke_503g.shape[0] # Anzahl Zeilen
t_wert_ausdehnung = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_ausdehnung - 1)

# Ausgabe
print(f"Gewicht in Kilogramm: {hooke_503g['mass'].unique()}",
      f"Mittlere Ausdehnung: {hooke_503g['ausdehnung'].mean():.4f} m",
      f"empirischer Standardfehler: { (sem_hooke_503g:= pd.Series(hooke_503g['ausdehnung']).sem):.4f} m",
      f"t-Wert der Ausdehnung: {t_wert_ausdehnung:.4f}",
      f"95%-Intervallgrenzen: {(hooke_503g['ausdehnung'].mean() - t_wert_ausdehnung * sem_hooke_503g, hooke_503g['ausdehnung'].mean() + t_wert_ausdehnung * sem_hooke_503g):.4f} m",
      sep = "\n")

```

```

Gewicht in Kilogramm: [0.503]
Mittlere Ausdehnung: 0.1432 m
empirischer Standardfehler: 0.002470 m
t-Wert der Ausdehnung: 2.2622
95%-Intervallgrenzen: 0.1376 m  0.1432 m  0.1488 m

```

Für die Federkonstante gilt (die Abstandsänderung Δx wird mit s notiert):

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

$$k = \frac{0,503 \text{ kg} \cdot 9,81 \text{ m}}{0,1432 \text{ m} \cdot \text{s}^2} = \frac{0,503 \cdot 9,81}{0,1432} \frac{\text{N}}{\text{m}} = 34,458 \frac{\text{N}}{\text{m}}$$

Um die Unsicherheit der Ergebnisgröße k nach der Gauß'schen Fehlerfortpflanzung zu bestimmen, müsste die Masse des verwendeten Gewichts ebenfalls durch eine Messreihe bestimmt worden sein. (Wie Messreihen und Einzelmessungen kombiniert werden können, wird im nächsten Abschnitt gezeigt.) Deshalb wird angenommen, dass mehrere Messungen zur Ermittlung des Gewichts durchgeführt wurden. Die Messreihe sowie ihr Mittelwert und Standardfehler könnten wie folgt aussehen:

```
messreihe_503g = pd.Series([503 , 503, 504, 502, 503, 502, 503, 504])
messreihe_503g /= 1000

# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n_masse = messreihe_503g.shape[0] # Anzahl Elemente
t_wert_masse = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_masse - 1)

# Ausgabe
print(f"arithmetisches Mittel der Masse: {messreihe_503g.mean()} kg",
      f"empirischer Standardfehler: { (sem_messreihe_503g:= pd.Series(messreihe_503g.sem()))",
      f"t-Wert der Masse: {t_wert_masse:.4f}",
      f"95%-Intervallgrenzen: {(messreihe_503g.mean() - t_wert_masse * sem_messreihe_503g",
      sep = "\n")
```

```
arithmetisches Mittel der Masse: 0.503 kg
empirischer Standardfehler: 0.000267 kg
t-Wert der Masse: 2.3646
95%-Intervallgrenzen: 0.5024 kg - 0.5036 kg
```

Die Gauß'sche Fehlerfortpflanzung wird ähnlich zur linearen Fehlerfortpflanzung berechnet.

$$u(k(m, s)) = \sqrt{\left(\frac{\partial f(m, s)}{\partial m} \cdot \sigma_m\right)^2 + \left(\frac{\partial f(m, s)}{\partial s} \cdot \sigma_s\right)^2}$$

Die partiellen Ableitungen werden eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2}$$

4.8 Vollständiges Messergebnis

Die Messwerte, die Konstante g und die Standardfehler der Mittelwerte werden eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,000267kg\right)^2 + \left((-)\frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 0,00247m\right)^2}$$

Trennen von Werten und Einheiten:

$$u(k(m, s)) = \sqrt{\left(\frac{9,81 \cdot 0,000267}{0,1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 + \left((-)\frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2}$$

Berechnen, Newton $\frac{kg \cdot m}{s^2}$ einsetzen und Meter im 2. Term kürzen:

$$u(k(m, s)) = \sqrt{\left(\frac{9,81 \cdot 0,000267}{0,1432} \frac{N}{m}\right)^2 + \left((-)\frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{N}{m}\right)^2}$$

Zusammenfassen:

$$u(k(m, s)) = \sqrt{\left(0,0183 \frac{N}{m}\right)^2 + \left((-)0,594 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 0,595 \frac{N}{m} \approx 0,60 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 34,46 \frac{N}{m} \pm 0,60 \frac{N}{m}$$

4.9 95%-Vertrauensintervall

Zusätzlich zum vollständigen Messergebnis kann ein Vertrauensintervall angegeben werden. Entsprechend der vereinbarten Vertrauenswahrscheinlichkeit, sind die entsprechenden t-Werte als Korrekturfaktoren einzusetzen.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

Dies sind für die Masse $t = 2,3646$ und für die Ausdehnung $t = 2,2622$.

Die Messwerte, die Konstante g , der Standardfehler des Mittelwerts und der t-Wert werden in den ersten Term eingesetzt.

$$\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 2,3646 \cdot 0,000267kg\right)^2$$

Trennen von Werten und Einheiten, Newton $\frac{kg \cdot m}{s^2}$ einsetzen:

$$\left(\frac{9,81 \cdot 2,3646 \cdot 0,000267}{0,1432} \frac{m \cdot kg}{s^2 \cdot m} \right)^2 = \left(0,0433 \frac{N}{m} \right)^2$$

Die Messwerte, die Konstante g , der Standardfehler des Mittelwerts und der t-Wert werden in den zweiten Term eingesetzt.

$$\left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s \right)^2 = \left((-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 2,2622 \cdot 0,00247m \right)^2$$

Trennen von Werten und Einheiten, Newton $\frac{kg \cdot m}{s^2}$ einsetzen und m kürzen:

$$\left((-) \frac{0,503 \cdot 9,81 \cdot 2,2622 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right)^2 = \left((-) 1,344 \frac{N}{m} \right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left(0,0433 \frac{N}{m} \right)^2 + \left((-) 1,344 \frac{N}{m} \right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 1,345 \frac{N}{m} \approx 1,35 \frac{N}{m}$$

Die Intervallgrenzen lauten somit:

$$k = 34,46 \frac{N}{m} \pm 1,35 \frac{N}{m}$$

Also:

$$k_{95\%} = 33,11 \frac{N}{m} \leq 34,46 \frac{N}{m} \leq 35,81 \frac{N}{m}$$

Übersicht Fehlerfortpflanzung für Grundrechenarten und Potenzprodukte

Einige Funktionen sind besonders leicht zu berechnen. In der folgenden Tabelle sind Rechenwege zur Ermittlung der Messunsicherheit nach der Gauß'schen Fehlerfortpflanzung (Standardfehler) und nach der linearen Fehlerfortpflanzung (Größtfehler) aufgelistet.

Tabelle 3.1: Berechnung der Größtfehler für Potenzprodukte und Grundrechenarten

$y = f(x_1, x_2)$	Standardfehler	Größtfehler
$y = x_1 \pm x_2$	$\delta y = \pm \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$	$\delta y = \delta x_1 + \delta x_2$ absolute Fehler werden addiert
$y = x_1 \cdot x_2$	$\delta y = \pm x_2 \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$ $\delta y = \pm x_1 \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$	$\delta y = \delta x_1 + \delta x_2 = \frac{\delta x_1}{x_1} + \frac{\delta x_2}{x_2}$
$y = \frac{x_1}{x_2}$	$\delta y = \pm x_2 \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$ $\delta y = \pm x_1 \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$	$\delta y = \delta x_1 + \delta x_2$
$y = x_1^n \cdot x_2^m$	$\delta y = \pm \sqrt{(n \delta x_1)^2 + (m \delta x_2)^2}$	$\delta y = n \delta x_1 + m \delta x_2$

Abbildung 4.4: Fehlerfortpflanzung für Grundrechenarten und Potenzprodukte

(Eden und Gebhard [2024](#), S. 36)

Spezialfall: Fehlerfortpflanzung mit Einzelwerten

Ein Sonderfall ist die Ermittlung der Messunsicherheit, wenn diese sich aus statistischen Vertrauensbereichen und abgeschätzten Fehleranteilen aus einer Einzelmessung zusammensetzt.

Für die Berechnung eines Gesamtfehlers muss ein Kompromiss eingegangen werden.

1. Die lineare Fehlerfortpflanzung wird mit dem abgeschätzten Fehler und dem statistischen Vertrauensbereich durchgeführt.
2. Die Gauß'sche Fehlerfortpflanzung wird mit dem statistischen Vertrauensbereich durchgeführt und ein Fehleranteil addiert, der aus der linearen Fehlerfortpflanzung der abgeschätzten Fehler ermittelt wird.

“Beide Verfahren sind wohl gleich gut (oder schlecht) als Kompromiss in der Laborpraxis geeignet.” (Hempel ([2016](#)), S. 9)

(Hempel ([2016](#)), S. 8 - 9)

Die DIN 1319 behandelt nur die Gauß'sche Fehlerfortpflanzung. Liegt für eine Eingangsgröße nur ein einzelner Wert vor, wird dieser als Schätzwert für diese Größe verwendet. Die Unsicherheit ist aus den verfügbaren Informationen oder nach der Erfahrung, zum Beispiel aus früheren Messungen, anzusetzen. (Deutsches Institut für Normung [1996](#), S. 11)

Als Beispiel verwenden wir wieder die Messreihe für das Gewicht 503 Gramm.

Hinweis 16: Beispiel Federkonstante

```
# Ausgabe
print(f"Gewicht in Kilogramm: {hooke_503g['mass'].unique()}",
      f"Mittlere Ausdehnung: {hooke_503g['ausdehnung'].mean():.4f} m",
      f"empirischer Standardfehler: { (sem_hooke_503g:= pd.Series(hooke_503g['ausdehnung']).sem()):.4f} m",
      f"t-Wert der Ausdehnung: {t_wert_ausdehnung:.4f}",
      f"95%-Intervallgrenzen: {(hooke_503g['ausdehnung'].mean() - t_wert_ausdehnung * sem_hooke_503g,
                               hooke_503g['ausdehnung'].mean() + t_wert_ausdehnung * sem_hooke_503g)}",
      sep = "\n")
```

```
Gewicht in Kilogramm: [0.503]
Mittlere Ausdehnung: 0.1432 m
empirischer Standardfehler: 0.002470 m
t-Wert der Ausdehnung: 2.2622
95%-Intervallgrenzen: 0.1376 m   0.1432 m   0.1488 m
```

4.10 Lineare Fehlerfortpflanzung

Analog zur Fehlerabschätzung ergibt sich der Größtfehler der Federkonstante durch:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| (-) \frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

- Die Ablesabweichung der Küchenwaage wird auf $e_r(m) = 0,5g = 0,0005kg$ geschätzt. Eine systematische Messabweichung ist nicht bekannt. Der Größtfehler der Masse beträgt also $\Delta m = 0,0005kg$
- Der empirische Standardfehler der Ausdehnung beträgt $\sigma_x = 0.00247m$ bzw. zum vereinbarten Vertrauensniveau von $\sigma_{x95\%} = 0.00247m \cdot 2.2622 = 0,00559m$

Die Messwerte, die Konstante g und die Größtfehler werden eingesetzt.

$$\Delta k = \left| \frac{9,81m}{s^2 \cdot 0,1432m} \right| \cdot 0,0005kg + \left| (-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \right| \cdot 0,00247m$$

Trennen von Werten und Einheiten:

$$\Delta k = \left| \frac{9,81 \cdot 0,0005}{0,1432} \frac{m \cdot kg}{s^2 \cdot m} \right| + \left| (-) \frac{0,503 \cdot 9,81 \cdot 0,00247}{0,020506} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right|$$

Berechnen und Newton $\frac{kg \cdot m}{s^2}$ einsetzen:

$$\Delta k = \left| 0,0342 \frac{N}{m} \right| + \left| (-) 0,594 \frac{N \cdot m}{m^2} \right|$$

Meter im zweiten Term kürzen:

$$\Delta k = |0,0342 \frac{N}{m}| + |(-)0,594 \frac{N}{m}|$$

Die Messunsicherheit wird auf zwei Dezimalstellen aufgerundet.

$$\Delta k = 0,628 \frac{N}{m} \approx 0,63 \frac{N}{m}$$

Das Messergebnis und der Größtfehler lauten somit:

$$k = 34,46 \frac{N}{m}; \Delta k = 0,63 \frac{N}{m}$$

4.11 Gauß'sche Fehlerfortpflanzung

Die Unsicherheit der Federkonstante berechnet sich durch:

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2}$$

Die Messwerte, die Konstante g und der abgeschätzte Fehleranteil (Größtfehler) der Masse werden in den ersten Term eingesetzt.

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,0005kg\right)^2$$

Trennen von Werten und Einheiten, Newton $\frac{kg \cdot m}{s^2}$ einsetzen:

$$\left(\frac{9,81 \cdot 0,0005}{0,1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 = \left(0,0342 \frac{N}{m}\right)^2$$

Die Masse des Gewichts, die Konstante g und der Standardfehler des Mittelwerts der Ausdehnung werden in den zweiten Term eingesetzt.

$$\left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2 = \left((-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 0,00247m\right)^2$$

Trennen von Werten und Einheiten, Newton $\frac{kg \cdot m}{s^2}$ einsetzen und m kürzen:

$$\left((-) \frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)0,594 \frac{N}{m}\right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left(0,0342 \frac{N}{m}\right)^2 + \left((-)0,594 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 0,595 \frac{N}{m} \approx 0,60 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 34,46 \frac{N}{m}; \Delta k = 0,60 \frac{N}{m}$$

Für das 95%-Konfidenzintervall wird der t-Wert der Ausdehnung $t = 2,2622$ eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

Der erste Term lautet unverändert:

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,0005kg\right)^2 = \left(0,0342 \frac{N}{m}\right)^2$$

Die Masse, die Konstante g , der Standardfehler des Mittelwerts der Ausdehnung und der t-Wert der Ausdehnung $t = 2,2622$ werden in den zweiten Term eingesetzt.

$$\left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2 = \left((-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 2,2622 \cdot 0,00247m\right)^2$$

Trennen von Werten und Einheiten, Newton $\frac{kg \cdot m}{s^2}$ einsetzen und m kürzen:

$$\left((-) \frac{0,503 \cdot 9,81 \cdot 2,2622 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)1,344 \frac{N}{m}\right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left(0,0342 \frac{N}{m}\right)^2 + \left((-)1,344 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 1,344 \frac{N}{m} \approx 1,35 \frac{N}{m}$$

Die Intervallgrenzen lauten somit:

$$k_{95\%} = 33,11 \frac{N}{m} \leq 34,46 \frac{N}{m} \leq 35,81 \frac{N}{m}$$

5 Übung Hooke'sches Gesetz

Häufig liegen Sensordaten in mehreren Dateien vor. Mögliche Gründe dafür können sein, dass die Messung

- von unterschiedlichen Personen,
- an unterschiedlichen Standorten,
- zu unterschiedlichen Zeiten,
- mit verschiedenen Geräten oder
- für unterschiedliche Messgrößen durchgeführt wurden.

Im Ordner '01-daten/hooke' liegen mehrere txt-Dateien mit Messdaten zur Federausdehnung. In diesem Kapitel sollen Sie das bisher Gelernte anwenden und die folgenden Fragen beantworten.

1. Liegen ungültige Messungen vor?
2. Welche Werte können für die Federkonstanten ermittelt werden?
3. Wurden die Messungen mit der gleichen Feder (oder mit Federn mit gleicher Federkonstante) durchgeführt, wenn als Vertrauenswahrscheinlichkeit 90 % bzw. 95 % angenommen werden soll?

Im Abschnitt Kapitel 5.1 finden Sie Hinweise und im Abschnitt Kapitel 5.2 eine Musterlösung zum Einlesen der Dateien. Anschließend sollen Sie die Aufgabenstellung eigenständig bearbeiten. In Abschnitt Kapitel ?? finden Sie eine Musterlösung für die Aufbereitung der Daten. In Kapitel ?? die Musterlösung zur Bestimmung der Federkonstanten.

Hinweis: Je nach gewähltem Vorgehen können sich unterschiedliche Ergebnisse ergeben (etwa bei der Bewertung von und dem Umgang mit Extremwerten / Ausreißern).

5.1 Dateien einlesen

Für das Einlesen der Dateien können Sie das Modul `glob` verwenden (siehe [Methodenbaustein Einlesen strukturierter Datensätze](#)).

Zunächst kann der Funktion `glob.glob()` das Argument `pathname = *` übergeben werden. Der Platzhalter `*` steht für eine beliebige Zeichenfolge (außer Dateipfadelemente wie `/` oder `.`), sodass die Namen aller im angegebenen Ordner gespeicherten Dateien ausgelesen werden. Auf diese Weise kann die Anzahl der Dateien und die Dateiendung bestimmt werden, falls dies noch unbekannt ist.

```

ordnerpfad = '01-daten/hooke'

pfadliste = glob.glob(pathname = '*', root_dir = ordnerpfad, recursive = False)
print(pfadliste)
print(f"Anzahl Dateien: {len(pfadliste)}")

```

```

['team_kreativkoepfe.txt', 'team_fabi.txt', 'team_die_ahnungslosen.txt', 'team_ma.txt']
Anzahl Dateien: 4

```

Mit den Dateipfaden können die Dateien mit Hilfe einer Schleife in eine Liste eingelesen werden. Zunächst werden nur die jeweils ersten 3 Zeilen eingelesen, um einen Eindruck vom Aufbau der Dateien zu erhalten.

```

for pfad in pfadliste:
    zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, nrows = 3)
    print(pfad, "\n", zwischenspeicher, "\n", sep = '')

```

```

team_kreativkoepfe.txt
      10:33:02\t109.64 cm\t0
0   10:33:05\t109.62 cm\t0
1   10:33:08\t109.64 cm\t0
2   10:33:11\t109.62 cm\t0

```

```

team_fabi.txt
      09:17:54\t110.31 cm\t0
0   09:17:57\t110.29 cm\t0
1   09:18:03\t110.74 cm\t0
2   09:18:06\t109.95 cm\t0

```

```

team_die_ahnungslosen.txt
      11:03:23\t109.66 cm\t0
0   11:03:23\t109.62 cm\t0
1   11:03:23\t109.73 cm\t0
2   11:03:23\t109.55 cm\t0

```

```

team_ma.txt
      10:09:38\t109.26 cm\t0
0   10:09:41\t109.26 cm\t0
1   10:09:44\t109.28 cm\t0
2   10:09:47\t109.18 cm\t0

```

Die Dateien beinhalten keine Spaltenbeschriftung und verwenden den Tabulator ‘\t’ als Trennzeichen. Die erste Spalte enthält einen Zeitstempel, die zweite die gemessene Federausdehnung und die dritte (vermutlich) das angehängte Gewicht.

5.2 Aufgabe Dateien einlesen

Lesen Sie die Dateien nun ein. Prüfen Sie dabei:

- ob die Datentypen korrekt eingelesen werden und
- auf fehlende Werte.

Sie können:

- a) Jede Datei einzeln einlesen.
- b) Mit dem Modul glob die Dateien automatisch einlesen und jeweils in einem separaten Objekt speichern (*Hinweis:* Dieses Vorgehen wird im [Methodenbaustein Einlesen strukturierter Datensätze](#) gezeigt).
- c) Die Dateien mit dem Modul glob automatisch einlesen und in einem Objekt zusammenführen.

Die verschiedenen Möglichkeiten sind mit zunehmend mehr Aufwand beim Programmieren verbunden. Je mehr separate Dateien Sie auswerten möchten, desto mehr Automatisierung ist gefragt. Da bei der Auswertung von Sensordaten häufig zahlreiche Dateien ausgewertet werden müssen, wird in der Musterlösung Variante c) gezeigt.

Tipp 8: Schrittweises Vorgehen

Das Einlesen der Dateien wird voraussichtlich der aufwändigste und fehleranfälligste Arbeitsschritt sein. Entwickeln Sie Ihre Lösung Schritt für Schritt. Beginnen Sie mit der Variante a). Wenn Sie die Dateien eingelesen haben, können Sie sich durch die Weiterentwicklung zur Variante b) mit dem Modul glob vertraut machen. Darauf aufbauend können Sie mit der Variante c) die Automatisierung für beliebig viele Dateien umsetzen.

Tipp 9: Musterlösung Dateien einlesen

Der erste Versuch, die Dateien einzulesen, scheitert mit einer Fehlermeldung. Die Anweisungen werden deshalb in die Struktur zur Ausnahmebehandlung eingebettet und die verursachende Datei abgefangen. (Sollten mehrere Dateien Fehler erzeugen, müssten die Dateien in einer Liste gespeichert und später - falls möglich - mit einer Schleife weiter behandelt werden.)

```

hooke = pd.DataFrame(columns = ['Zeit', 'Abstand', 'Gewicht', 'Team']) # ein leerer DataFra
for pfad in pfadliste:
    try:
        zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, sep = '\t'

        # Dateiname als Spalte einfügen
        zwischenspeicher['Team'] = pfad[5:-4] # 'team_' und die Dateiendung abschneiden

        hooke = pd.concat([hooke, zwischenspeicher], ignore_index = True)

    except Exception as error:
        print("Pfad der Problemdatei: ", pfad, error, sep = "\n")
        pfad_problem_datei = pfad

print(hooke.info(), "\n")
print("Erfolgreich einglesen:\n", hooke['Team'].unique(), sep = '')

```

Pfad der Problemdatei:

team_ma.txt

Error tokenizing data. C error: Expected 3 fields in line 135, saw 4

```

<class 'pandas.DataFrame'>
RangeIndex: 359 entries, 0 to 358
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Zeit        355 non-null    object
1   Abstand     355 non-null    object
2   Gewicht     355 non-null    object
3   Team        359 non-null    object
dtypes: object(4)
memory usage: 11.3+ KB
None

```

Erfolgreich einglesen:

['kreativkoepfe' 'fabi' 'die_ahnungslosen']

Der Fehlermeldung zufolge besteht Zeile 135 aus 4 statt aus 3 Spalten. Die fehlerverursachende Datei wird deshalb zeilenweise durchlaufen und jede Zeile ausgegeben, die mehr als 3 Einträge hat. Zur Kontrolle werden auch die ersten 5 Zeilen ausgegeben.

```

# einen leeren DataFrame mit 3 Spalten erstellen
df = pd.DataFrame(data = [], columns = ['Zeit', 'Abstand', 'Gewicht'])

dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

index = 0
for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            print("Index =", index, ":", zwischenspeicher)
        elif index <= 5:
            print(zeile)
            index += 1
    except Exception as error:
        print(error)

dateiobjekt_problem_datei.close()

```

```
10:09:38    109.26 cm    0
```

```
10:09:41    109.26 cm    0
```

```
10:09:44    109.28 cm    0
```

```
Index = 134 : ['10:29:27', '105.49 cm', '300', '\n']
```

Jede zweite Zeile ist leer. In Zeile 134 wird ein Zeilenumbruch ‘\n’ eingelesen. Die Datei wird deshalb mit einer angepassten Schleife erneut durchlaufen. Aus der betreffenden Zeile wird der zusätzliche Zeilenumbruch ‘\n’ entfernt. Leere Zeilen werden übersprungen. Die korrekten Zeilen werden an den DataFrame hooke angefügt.

```

dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            zwischenspeicher = zwischenspeicher[:3]
        elif len(zwischenspeicher) < 3: # leere Zeilen überspringen
            continue
        # Teamnamen als 4. Spalte anfügen
        zwischenspeicher.append(pfad_problem_datei[5:-4]) # 'team_' und die Dateiendung abschneiden

        hooke.loc[len(hooke)] = pd.Series(zwischenspeicher).values

    except Exception as error:
        print(error)
        print(pd.Series(zwischenspeicher).values)

dateiobjekt_problem_datei.close()

print("Erfolgreich eingelesen:\n", hooke['Team'].unique(), "\n", sep = '')
print(hooke.info())

```

Erfolgreich eingelesen:

```
['kreativkoepfe' 'fabi' 'die_ahnungslosen' 'ma']
```

```

<class 'pandas.DataFrame'>
RangeIndex: 537 entries, 0 to 536
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Zeit        533 non-null    object
 1   Abstand     533 non-null    object
 2   Gewicht     533 non-null    object
 3   Team        537 non-null    object
dtypes: object(4)
memory usage: 16.9+ KB
None

```

Anschließend werden zum einen die Zeilen mit Nullwerten betrachtet ...

```
print(hooke.loc[hooke.apply(pd.isna).any(axis = 1), :])
```

	Zeit	Abstand	Gewicht	Team
3	NaN	NaN	NaN	kreativkoepfe
23	NaN	NaN	NaN	kreativkoepfe
28	NaN	NaN	NaN	kreativkoepfe
322	NaN	NaN	NaN	die_ahnungslosen

... und entfernt.

```
hooke.drop(np.where(hooke.apply(pd.isna).any(axis = 1))[0], inplace = True)
```

Zum anderen werden die Datentypen kontrolliert.

- Die Zeit kann als string stehen bleiben, da sie für die Auswertung nicht benötigt wird.
- Der gemessene Abstand ist mit ' cm' notiert - diese Zeichenkette wird entfernt. Anschließend sollte die Spalte als numerisch erkannt werden.
- Das Gewicht sollte numerische Werte enthalten, wird aber als Datentyp object eingelesen und muss weiter untersucht werden.
- Der Spalte Team könnte der [Pandas Datentyp category](#) zugewiesen werden, notwendig ist es aber nicht.

String ' cm' entfernen.

```
hooke.replace(' cm', '', regex = True, inplace = True)
```

	Zeit	Abstand	Gewicht	Team
0	10:33:02	109.64	0.0	kreativkoepfe
1	10:33:05	109.62	0.0	kreativkoepfe
2	10:33:08	109.64	0.0	kreativkoepfe
4	10:33:11	109.62	0.0	kreativkoepfe
5	10:33:14	109.73	0.0	kreativkoepfe
...	...	...	...	...
532	11:05:14	95.80	850\n	ma
533	11:05:17	95.61	850\n	ma
534	11:05:20	95.87	850\n	ma
535	11:05:23	95.35	850\n	ma
536	11:05:26	95.90	850\n	ma

Ob alle Elemente einer Zelle numerisch sind, kann mit der Pandas-Methode `pd.Series.str.isnumeric()` überprüft werden. Ein Blick auf die Daten zeigt die Ursache.

```
print(hooke['Gewicht'].str.isnumeric().sum())

print(hooke['Gewicht'].head())
print(hooke['Gewicht'].tail())
```

```
1
0    0.0
1    0.0
2    0.0
4    0.0
5    0.0
Name: Gewicht, dtype: object
532    850\n
533    850\n
534    850\n
535    850\n
536    850\n
Name: Gewicht, dtype: object
```

Die Zeilenumbrüche werden ebenfalls entfernt.

```
hooke.replace('\n', '', regex = True, inplace = True)
print(hooke['Gewicht'].str.isnumeric().sum())
print(hooke['Gewicht'].tail())
```

```
178
532    850
533    850
534    850
535    850
536    850
Name: Gewicht, dtype: object
```



```
print(hooke.info(), "\n")

# explizite Zuweisung
hooke['Abstand'] = hooke['Abstand'].astype('float')
hooke['Gewicht'] = hooke['Gewicht'].astype('float')

print(hooke.info())
```

```
<class 'pandas.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Zeit        533 non-null    object
1   Abstand     533 non-null    object
2   Gewicht     533 non-null    object
3   Team        533 non-null    object
dtypes: object(4)
memory usage: 20.8+ KB
None
```

```
<class 'pandas.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Zeit        533 non-null    object
1   Abstand     533 non-null    float64
2   Gewicht     533 non-null    float64
3   Team        533 non-null    object
dtypes: float64(2), object(2)
memory usage: 20.8+ KB
None
```

Das Ergebnis könnte so aussehen:

```
hooke.groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand'].describe()
```

Team	Gewicht	count
kreativkoepfe	0.0	9.0
	50.0	8.0
	100.0	10.0
	150.0	10.0
	200.0	10.0
	250.0	10.0
	300.0	10.0
	350.0	10.0
	400.0	9.0
	450.0	9.0
	500.0	8.0
	550.0	9.0
	0.0	10.0
	50.0	12.0
	101.0	9.0
fabi	152.0	10.0
	201.0	10.0
	253.0	11.0
	302.0	12.0
	353.0	11.0
	403.0	14.0
	455.0	11.0
	0.0	10.0
	50.0	8.0
	100.0	12.0
die_ahnungslosen	150.0	19.0
	200.0	18.0
	250.0	18.0
	300.0	11.0
	350.0	14.0
	400.0	11.0
	450.0	12.0
	0.0	10.0
	50.0	10.0
	100.0	10.0
	150.0	11.0
	200.0	9.0
	250.0	10.0
	300.0	10.0
	350.0	10.0
ma		

Team	Gewicht	count
	400.0	10.0
	450.0	10.0
	500.0	10.0
	550.0	10.0
	600.0	10.0
	650.0	10.0
	700.0	10.0
	750.0	9.0
	800.0	11.0
	850.0	8.0

5.3 Musterlösung Daten aufbereiten

Im nächsten Schritt werden die Daten geprüft und ggf. bereinigt. Dies umfasst die Schritte:

- auf Ausreißer prüfen (studentisierte z-Werte und grafisch) und ggf. bereinigen,
- Normierung der Abstandsmessung auf den Nullpunkt und Umrechnung der verwendeten Einheiten in SI-Einheiten und
- grafische Darstellung.

Warning 5: glob, pd.groupby und SciPy

Die Ausführung des folgenden Codes kann lokal unterschiedlich ausfallen, je nach dem, in welcher Reihenfolge die Dateien mit dem Modul glob eingelesen wurden. Im Folgenden werden aus einem groupby-Objekt Indexpositionen bestimmt, an denen eine Bedingung wahr ist. Mit diesen Indexpositionen wird dann auf den ursprünglichen DataFrame zugegriffen. Pandas sortiert die Gruppen im groupby-Objekt standardmäßig alphabetisch (`sort = True`). Wenn die mit glob eingelesenen Dateien nicht in alphabetischer Reihenfolge vorliegen, wird mit falschen Indexpositionen auf den DataFrame zugegriffen.

Dem kann durch alphabetisches Sortieren der mit glob eingelesenen Dateien bzw. des DataFrames nach den groupby-Kriterien begegnet werden oder, wie im Folgenden, durch das Argument `pd.groupby(by = Bedingung, sort = False)`. Letztere Variante ist flexibler.

Auch gibt `scipy.stats.zscore()` je nach Version eine `pd.Series` oder ein `np.array` zurück. Dementsprechend sind die Methoden aus Pandas oder NumPy zu verwenden.

1. Liegen ungültige Messungen vor?

5.3.1 Auf Ausreißer prüfen

Auf Ausreißer kann (unter anderem) mit studentisierten z-Werten und grafisch geprüft werden.

5.4 Ausreißer bestimmen

1. Anzahl der Ausreißer bestimmen und Position ausgeben.

```
# z-Werte größer gleich abs(3) finden
## Lösung für SciPy 1.14.1 (gibt pd.Series zurück)
## z_values_ge3_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))

## Lösung für SciPy 1.15 und neuer (gibt np.array zurück)
z_values_ge3_sum = hooke.groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand'].apply(scipy.stats.zscore)

print("Anzahl der studentisierten z-Werte mit Betrag 3:", z_values_ge3_sum, "\n")

# z-Werte größer gleich abs(2.5) finden
## Lösung für SciPy 1.14.1 (gibt pd.Series zurück)
## z_values_ge25_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))

## Lösung für SciPy 1.15 und neuer (gibt np.array zurück)
z_values_ge25_sum = hooke.groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand'].apply(scipy.stats.zscore)
print("Anzahl der studentisierten z-Werte mit Betrag 2.5:", z_values_ge25_sum)

# Die Zeilen mit z-Werten größer abs(2.5) ausgeben
## Lösung für SciPy 1.14.1 (gibt pd.Series zurück)
## bool_index = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x) >= 2.5)

## Lösung für SciPy 1.15 und neuer (gibt np.array zurück)
#### Schritt 1: Wahrheitswerte abgreifen
bool_index = hooke.groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand'].apply(scipy.stats.zscore)
#### Schritt 2: Rückgabe in ein eindimensionales Array überführen:
bool_index = np.hstack(bool_index)

print("\nAn diesen Indexpositionen liegen die Ausreißer:", np.nonzero(bool_index))
```

Anzahl der studentisierten z-Werte mit Betrag 3: 0

Anzahl der studentisierten z-Werte mit Betrag 2.5: 4

An diesen Indexpositionen liegen die Ausreißer: (array([156, 298, 313, 318]),)

2. Ausgabe der Zeilen aus den Messreihen

```
# Auswahl der Zeilen mit den z-Werten größer gleich 2.5
z_values_ge_25 = hooke.iloc[bool_index , :] # --> Hier muss das Problem liegen
print(z_values_ge_25, "\n")

# Kombinationen aus Gewicht & Team bestimmen
print("Kombinationen aus Gewicht & Team bestimmen")
teams = z_values_ge_25['Team'].unique()

## teams durchlaufen und jeweils die Gewichte speichern
team_gewichte = [] # leere liste
for i in range(len(teams)):
    print(teams[i])
    print(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'], "\n")
    team_gewichte.append(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'].values)

# print("Als Liste von arrays:")
# print(team_gewichte, "\n")
```

	Zeit	Abstand	Gewicht	Team
159	09:27:18	102.30	201.0	fabi
301	11:10:38	102.35	250.0	die_ahnungslosen
316	11:11:42	104.15	300.0	die_ahnungslosen
321	11:12:43	101.06	350.0	die_ahnungslosen

Kombinationen aus Gewicht & Team bestimmen

fabi

159 201.0

Name: Gewicht, dtype: float64

die_ahnungslosen

301 250.0

316 300.0

321 350.0

Name: Gewicht, dtype: float64

5.5 Gemeinsame Ausgabe der Messreihen

Diese Ausgabe dient der Veranschaulichung und Kontrolle.

```

# Messreihen auswählen
messreihen = pd.DataFrame()
for i in range(len(teams)):
    for j in range(len(team_gewichte[i])):

        print(teams[i], team_gewichte[i][j])

        messreihen = pd.concat([messreihen, hooke.loc[ (hooke['Team'] == teams[i]) & (hooke['Gewicht'] == team_gewichte[i][j]) ]])

# studentisierte z-Werte der Messreihen bilden
## Lösung für SciPy 1.14.1 (gibt pd.Series zurück)
## messreihen_z_scores = messreihen.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: (x - x.min()) / (x.max() - x.min()))

## Lösung für SciPy 1.15 und neuer (gibt np.array zurück)
## Rückgabe zeilenweise in ein eindimensionales Array überführen:
messreihen_z_scores = np.hstack(messreihen.groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand'].apply(lambda x: (x - x.min()) / (x.max() - x.min())))

# gemeinsame Ausgabe der Daten
## Lösung für SciPy 1.14.1 (gibt pd.Series zurück)
## messreihen.insert(loc = 2, column = 'z-Werte Abstand', value = messreihen_z_scores.values)

## Lösung für SciPy 1.15 und neuer (gibt np.array zurück)
messreihen.insert(loc = 2, column = 'z-Werte Abstand', value = messreihen_z_scores)
print(messreihen)

```

```

fabi 201.0
die_ahnungslosen 250.0
die_ahnungslosen 300.0
die_ahnungslosen 350.0

```

	Zeit	Abstand	z-Werte Abstand	Gewicht	Team
156	09:27:09	105.47	0.206192	201.0	fabi
157	09:27:12	105.90	0.614776	201.0	fabi
158	09:27:15	105.54	0.272706	201.0	fabi
159	09:27:18	102.30	-2.805925	201.0	fabi
160	09:27:21	105.90	0.614776	201.0	fabi
161	09:27:24	105.47	0.206192	201.0	fabi
162	09:27:27	105.44	0.177686	201.0	fabi
163	09:27:30	105.59	0.320216	201.0	fabi
164	09:27:33	105.46	0.196690	201.0	fabi
165	09:27:36	105.46	0.196690	201.0	fabi
292	11:10:11	102.99	0.376191	250.0	die_ahnungslosen
293	11:10:13	102.99	0.376191	250.0	die_ahnungslosen

294	11:10:17	102.93	0.105333	250.0	die_ahnungslosen
295	11:10:20	102.83	-0.346095	250.0	die_ahnungslosen
296	11:10:23	103.16	1.143620	250.0	die_ahnungslosen
297	11:10:26	102.80	-0.481524	250.0	die_ahnungslosen
298	11:10:29	102.90	-0.030095	250.0	die_ahnungslosen
299	11:10:32	103.33	1.911049	250.0	die_ahnungslosen
300	11:10:35	102.78	-0.571810	250.0	die_ahnungslosen
301	11:10:38	102.35	-2.512954	250.0	die_ahnungslosen
302	11:10:41	102.93	0.105333	250.0	die_ahnungslosen
303	11:10:44	102.88	-0.120381	250.0	die_ahnungslosen
304	11:10:47	102.90	-0.030095	250.0	die_ahnungslosen
305	11:10:50	102.88	-0.120381	250.0	die_ahnungslosen
306	11:10:53	102.73	-0.797524	250.0	die_ahnungslosen
307	11:10:56	102.81	-0.436381	250.0	die_ahnungslosen
308	11:10:59	102.80	-0.481524	250.0	die_ahnungslosen
309	11:11:02	103.33	1.911049	250.0	die_ahnungslosen
310	11:11:24	101.37	-0.525397	300.0	die_ahnungslosen
311	11:11:27	101.43	-0.458325	300.0	die_ahnungslosen
312	11:11:30	102.80	1.073152	300.0	die_ahnungslosen
313	11:11:33	101.72	-0.134144	300.0	die_ahnungslosen
314	11:11:36	101.36	-0.536576	300.0	die_ahnungslosen
315	11:11:39	101.46	-0.424789	300.0	die_ahnungslosen
316	11:11:42	104.15	2.582271	300.0	die_ahnungslosen
317	11:11:45	101.36	-0.536576	300.0	die_ahnungslosen
318	11:11:48	101.00	-0.939008	300.0	die_ahnungslosen
319	11:11:51	101.91	0.078251	300.0	die_ahnungslosen
320	11:11:54	101.68	-0.178859	300.0	die_ahnungslosen
321	11:12:43	101.06	2.992196	350.0	die_ahnungslosen
323	11:12:46	100.17	-0.354551	350.0	die_ahnungslosen
324	11:12:49	99.98	-1.069025	350.0	die_ahnungslosen
325	11:12:52	100.07	-0.730590	350.0	die_ahnungslosen
326	11:12:55	100.21	-0.204135	350.0	die_ahnungslosen
327	11:12:58	100.21	-0.204135	350.0	die_ahnungslosen
328	11:13:01	100.14	-0.467363	350.0	die_ahnungslosen
329	11:13:04	100.17	-0.354551	350.0	die_ahnungslosen
330	11:13:07	100.28	0.059092	350.0	die_ahnungslosen
331	11:13:10	100.55	1.074397	350.0	die_ahnungslosen
332	11:13:13	100.14	-0.467363	350.0	die_ahnungslosen
333	11:13:16	100.17	-0.354551	350.0	die_ahnungslosen
334	11:13:19	100.17	-0.354551	350.0	die_ahnungslosen
335	11:13:22	100.38	0.435131	350.0	die_ahnungslosen

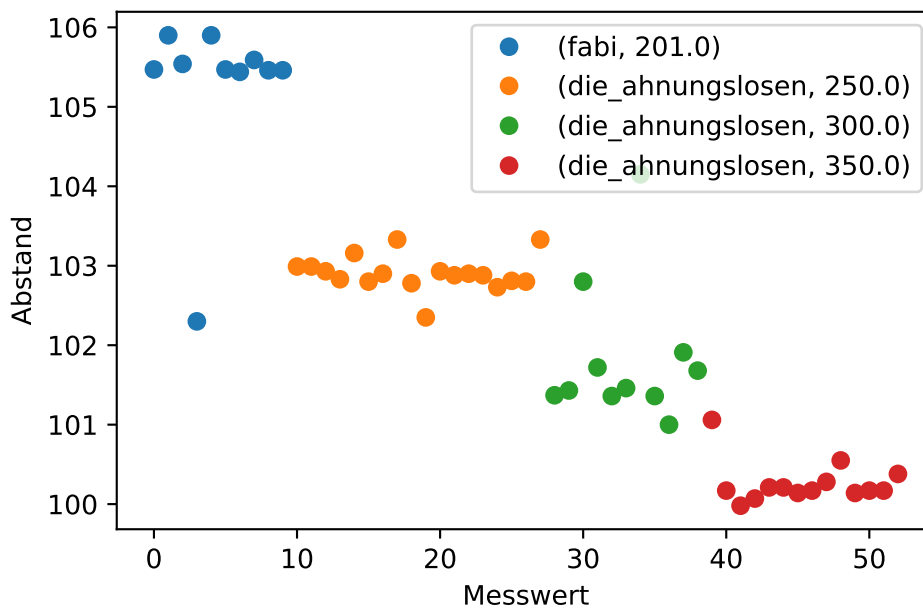
Die Werte können mit der Pandas-Methode `pd.plot()` mit wenig Aufwand dargestellt werden.

Die Methode ist jedoch nicht so flexibel, wie das Paket matplotlib. So ist das Punktdiagramm (`kind = 'scatter'`) nur für DataFrames, nicht aber für groupby-Objekte verfügbar. Dies wird durch das Setzen eines Markers und die Einstellung der Liniendicke auf 0 kompensiert.

```
messreihen.reset_index(drop = True).groupby(by = ['Team', 'Gewicht'], sort = False)['Abstand']

plt.xlabel('Messwert')
plt.ylabel('Abstand')
plt.legend()

plt.show()
```



💡 Tipp 10: Musterlösung

Abstandsmessung auf Meter und auf den Nullpunkt normieren

Abstandsmessung auf den Nullpunkt normieren. Die Spalte Abstand wird in Abstandsänderung umbenannt.

```
nullpunkte = hooke.loc[hooke['Gewicht'] == 0, :].groupby(by = 'Team', sort = False)['Abstand']  
  
print("Nullpunkte", nullpunkte, sep = '\n')  
  
teams = nullpunkte.index  
  
for i in range(len(teams)):  
  
    hooke.loc[hooke['Team'] == teams[i] , 'Abstand'] = hooke.loc[hooke['Team'] == teams[i] ,  
  
hooke.rename(columns = {'Abstand': 'Abstandsänderung'}, inplace = True)  
hooke['Abstandsänderung'] = hooke['Abstandsänderung'].div(100)
```

```
Nullpunkte  
Team  
kreativkoepfe      109.676667  
fabi                110.366000  
die_ahnungslosen   109.759000  
ma                  109.258000  
Name: Abstand, dtype: float64
```

Gewicht in wirkende Kraft umrechnen

Gewicht in g in die wirkende Kraft in N umrechnen. Die Spalte wird in den Datensatz eingefügt.

```
hooke['Kraft'] = hooke['Gewicht'].div(1000).mul(9.81)
```

Das Ergebnis könnte so aussehen. Die Spalte Abstand wurde in Abstandsänderung umbenannt.

```
hooke.groupby(by = ['Team', 'Kraft'], sort = False)['Abstandsänderung'].describe()
```

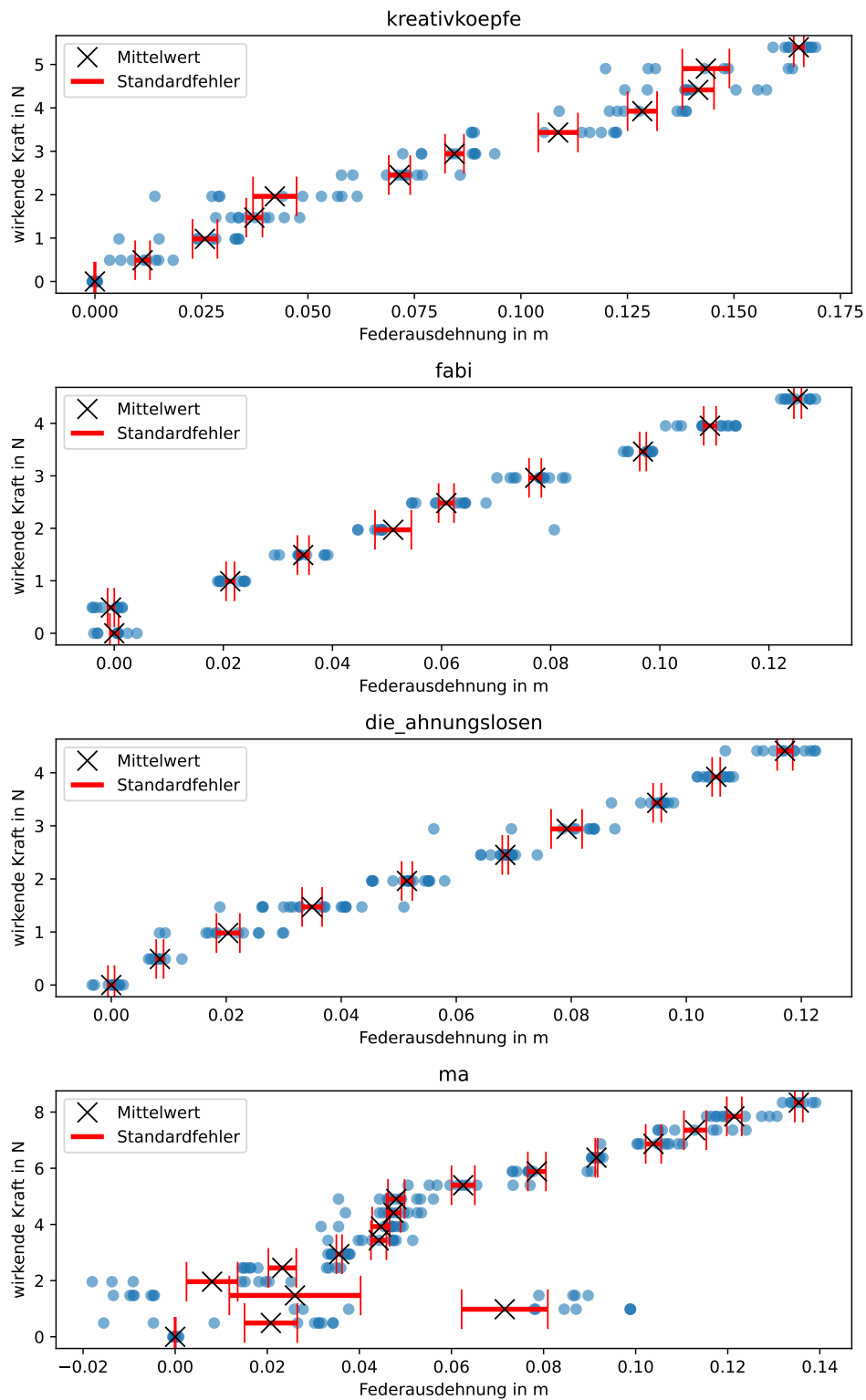
Team	Kraft	count
kreativkoepfe	0.00000	9.0
	0.49050	8.0
	0.98100	10.0
	1.47150	10.0
	1.96200	10.0
	2.45250	10.0
	2.94300	10.0
	3.43350	10.0
	3.92400	9.0
	4.41450	9.0
	4.90500	8.0
	5.39550	9.0
	0.00000	10.0
	0.49050	12.0
	0.99081	9.0
fabi	1.49112	10.0
	1.97181	10.0
	2.48193	11.0
	2.96262	12.0
	3.46293	11.0
	3.95343	14.0
	4.46355	11.0
	0.00000	10.0
	0.49050	8.0
	0.98100	12.0
	1.47150	19.0
	1.96200	18.0
	2.45250	18.0
	2.94300	11.0
	3.43350	14.0
die_ahnungslosen	3.92400	11.0
	4.41450	12.0
	0.00000	10.0
	0.49050	10.0
	0.98100	10.0
	1.47150	11.0
	1.96200	9.0
	2.45250	10.0
	2.94300	10.0
	3.43350	10.0
ma		

Team	Kraft	count
	3.92400	10.0
	4.41450	10.0
	4.90500	10.0
	5.39550	10.0
	5.88600	10.0
	6.37650	10.0
	6.86700	10.0
	7.35750	9.0
	7.84800	11.0
	8.33850	8.0

5.5.2 Grafische Darstellung

Da es nur vier Teams gibt, können die Messreihen grafisch dargestellt werden. Eine mögliche Darstellung können Sie dem ersten Reiter, die Zwischenschritte und Schlussfolgerungen den folgenden Reitern entnehmen.

5.6 Grafische Darstellung



5.7 Mittelwerte und Standardfehler berechnen

Die Ausgabe ist aus Platzgründen auf die ersten Zeilen beschränkt.

Mittelwerte der Abstandsänderung nach Team und Kraft:

Team	Kraft	
kreativkoepfe	0.0000	7.895401e-17
	0.4905	1.119167e-02
	0.9810	2.583667e-02
	1.4715	3.742667e-02
	1.9620	4.223667e-02

Name: Federausdehnung, dtype: float64

Standardfehler der Mittelwerte nach Team und Kraft

Team	Kraft	
kreativkoepfe	0.0000	0.000179
	0.4905	0.001732
	0.9810	0.002918
	1.4715	0.001897
	1.9620	0.005117

Name: Standardfehler, dtype: float64

5.8 Code

```
# Mittelwerte der Teams nach Kraft
distance_means_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']], sort = False)
distance_means_by_team_and_force.name = 'Federausdehnung'

print("Mittelwerte der Abstandsänderung nach Team und Kraft:", distance_means_by_team_and_force)

print() # leere Zeile

# Standardfehler der Teams nach Kraft
distance_stderrors_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']], sort = False)
distance_stderrors_by_team_and_force.name = 'Standardfehler'

print("Standardfehler der Mittelwerte nach Team und Kraft", distance_stderrors_by_team_and_force)

# grafische Darstellung
anzahl_teams = hooke['Team'].unique().size
```

```

plt.figure(figsize = (7.5, 12))
for i in range(anzahl_teams):

    plt.subplot(4, 1, i + 1) # plt.subplot zählt ab 1

    # Punktdiagramm
    plotting_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    plt.scatter(x = plotting_data['Abstandsänderung'], y = plotting_data['Kraft'], alpha = 0.6)

    plt.title(label = hooke['Team'].unique()[i])
    plt.xlabel("Federausdehnung in m")
    plt.ylabel("wirkende Kraft in N")

    # # Fehlerbalken
    distance_means_by_force = plotting_data.groupby(by = plotting_data['Kraft'], sort = False)
    distance_stderrors_by_force = plotting_data.groupby(by = plotting_data['Kraft'], sort = False)

    errorbar_container = plt.errorbar(x = distance_means_by_force, y = distance_means_by_force,
    linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor = 'black')

    # siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer
    plt.legend([errorbar_container.lines[0], errorbar_container.lines[2][0]],
               ['Mittelwert', 'Standardfehler'],
               loc = 'upper left')

plt.tight_layout()
plt.show()

```

5.9 Auswertung

- Die Messreihen des Teams die_ahnungslosen entsprechen dem erwarteten linearen Trend.
- Bei Team fabi scheint für das erste angehängte Gewicht (50 Gramm) ein Fehler bei der Datenerhebung vorzuliegen. Vermutlich wurde hier mit 0 Gramm gemessen.
- Die Messreihen des Teams kreativköpfe entsprechen weitgehend dem erwarteten linearen Trend.
- Die Messreihen des Teams ma scheinen wenigstens für die ersten vier angehängten Gewichten durch grobe Messfehler geprägt zu sein.

Die Messreihe des Teams ma wird wegen grober Messfehler aus dem Datensatz entfernt. Aus der Messreihe des Teams fabi wird die Messung für das Gewicht 50 Gramm entfernt.

```
hooke.drop(index = hooke.loc[hooke['Team'] == 'ma', :].index, inplace = True)
hooke.drop(index = hooke.loc[(hooke['Team'] == 'fabi') & (hooke['Gewicht'] == 50), :].index,
```

💡 Tipp 11: Vorgehen bei vielen Datensätzen

Bei einer großen Anzahl an Datensätzen kann auch die grafische Kontrolle an Grenzen stoßen. In diesem Fall empfiehlt es sich, die visuellen und kennzahlenbasierten Methoden zusammen zu nutzen, um Muster zu identifizieren und für eine große Zahl von Messungen zu überprüfen. Beispielsweise könnten nach einer visuellen Inspektion von Messreihen mit Extremwerten bzw. Ausreißern alle Messreihen daraufhin überprüft werden, ob mit zunehmenden Gewicht stets auch die mittlere Federausdehnung größer als für leichtere Gewichte ist. Abweichende Messreihen könnten dann grafisch kontrolliert werden.

5.10 Musterlösung Federkonstanten bestimmen

Im nächsten Schritt können die Federkonstanten mittels linearer Regression bestimmt werden.

2. Welche Werte können für die Federkonstanten ermittelt werden?
3. Wurden die Messungen mit der gleichen Feder durchgeführt, wenn als Vertrauenswahrscheinlichkeit 90 % bzw. 95 % angenommen werden soll?

5.11 $\alpha = 0.10$

```
kreativkoepfe Konfidenzniveau: 0.9
y = 0.3180 + 29.7643 * x
r = 0.9773 R2 = 0.9552 p = 0.0000
Standardfehler des Anstiegs: 0.6148
28.744    29.764    30.784
```

```
fabi Konfidenzniveau: 0.9
y = 0.2373 + 34.1048 * x
r = 0.9907 R2 = 0.9814 p = 0.0000
Standardfehler des Anstiegs: 0.4792
33.309    34.105    34.901
```



```

die_ahnungslosen Konfidenzniveau: 0.9
y = 0.1798 + 34.9183 * x
r = 0.9886 R2 = 0.9773 p = 0.0000
Standardfehler des Anstiegs: 0.4651
34.148    34.918    35.689

```

5.12 $\alpha = 0.05$

```

kreativkoepfe Konfidenzniveau: 0.95
y = 0.3180 + 29.7643 * x
r = 0.9773 R2 = 0.9552 p = 0.0000
Standardfehler des Anstiegs: 0.6148
28.546    29.764    30.983

```

```

fabi Konfidenzniveau: 0.95
y = 0.2373 + 34.1048 * x
r = 0.9907 R2 = 0.9814 p = 0.0000
Standardfehler des Anstiegs: 0.4792
33.154    34.105    35.056

```

```

die_ahnungslosen Konfidenzniveau: 0.95
y = 0.1798 + 34.9183 * x
r = 0.9886 R2 = 0.9773 p = 0.0000
Standardfehler des Anstiegs: 0.4651
33.998    34.918    35.838

```

5.13 Code

```

anzahl_teams = hooke['Team'].unique().size
alpha = 0.10

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)

```

```

print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

alpha = 0.05

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)

    print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

    slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
    print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
          f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
          f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

    print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

```

5.14 Auswertung

Die Punktschätzung der Federkonstante von Team fabi 34.105 liegt im 95%-Konfidenzintervall der Messung von Team die_ahnungslosen 33.998 34.918 35.838. Die Punktschätzung der Federkonstante von Team fabi 34.105 liegt **aber nicht im 90%-Konfidenzintervall** der Messung von Team die_ahnungslosen 34.148 34.918 35.689.

Unabhängig vom gewählten Vertrauensniveau liegt die Punktschätzung der Federkonstante von Team kreativkoepfe 29.764 nicht in den Konfidenzintervallen der beiden übrigen Teams.

 Warning 6: Ergebnisse

Abhängig vom gewählten Vorgehen sind andere Ergebnisse möglich, beispielsweise durch das Entfernen von als Ausreißern eingestuften Einzelwerten oder einer anderen Behandlung der Messreihe vom Team fabi für das angehängte Gewicht 50 Gramm.

6 Kalibrierung

Daten Eiskochen nur als 2-Punkt-Kalibrierung behandeln

- Referenzpunkte für 0 und 100 festlegen (mitteln) und Daten entsprechend kalibrieren
- Daten dazwischen verwerfen, weil man nicht weiß, ob der Herd überhaupt linear heizt
- praktische Kalibriermethode als Unterpunkt von Nullpunkt- und Spannfehler

Die Genauigkeit von Messungen wird durch die Korrektur systematischer Messabweichungen verbessert. In diesem Kapitel werden typische *Fehlerarten* bei Messungen behandelt:

- additive Fehler,
- multiplikative Fehler und
- Nichtlinearität.

Außerdem werden praktische Methoden der Kalibrierung vorgestellt. In diesem Abschnitt greifen wir die etablierten Begriffe auf, auch wenn die Fehler besser als systematische Messabweichungen bezeichnet werden sollten.

6.1 Kalibrieren und Justieren

Die Korrektur systematischer Messabweichungen kann auf zwei Arten geschehen: durch Kalibrierung oder Justierung des Messgeräts.

Mit dem Begriff Kalibrierung wird im engeren Sinn die Ermittlung des Zusammenhangs zwischen Messwerten bzw. ihres arithmetischen Mittelwerts und dem vereinbarten richtigen Wert der Messgröße bezeichnet.

! Wichtig 9: Kalibrierung nach DIN 1319

“Ermitteln des Zusammenhangs zwischen Meßwert [...] oder Erwartungswert [...] der Ausgangsgröße [...] und dem zugehörigen wahren [...] oder richtigen Wert [...] der als Eingangsgröße vorliegenden Meßgröße für eine betrachtete Meßeinrichtung [...]” (Deutsches Institut für Normung [1995](#), S. 22)

Im weiteren Sinn ist mit Kalibrierung auch “die Erstellung einer Korrekturstabelle [...], die Ermittlung von Kalibrierfaktoren oder einer (empirischen) Kalibrierfunktion” (Deutsches

Institut für Normung 1995, S. 22) gemeint. Dabei handelt es sich um Methoden, um mit systematischen Messabweichungen behaftete Messwerte zu korrigieren, ohne das Messgerät zu verändern.

Im Unterschied dazu verändert die Justierung das Messgerät dauerhaft.

! Wichtig 10: Justierung nach DIN 1319

“Einstellen oder Abgleichen eines Meßgeräts [...], um systematische Meßabweichungen [...] soweit zu beseitigen, wie es für die vorgesehene Anwendung erforderlich ist.” (Deutsches Institut für Normung 1995, S. 22)

6.2 Kalibriermethoden nach DIN 1319

In der DIN 1319 werden drei Kalibriermethoden genannt, die hier nur kurz behandelt werden:

1. Korrektionstabelle,
2. Ermittlung von Kalibrierfaktoren und
3. Ermittlung einer (empirischen) Kalibrierfunktion.

Welche Methode verwendet wird, hängt von dem Anwendungsbereich, der Datenverfügbarkeit sowie von der gewünschten Genauigkeit ab.

Methode	Anwendung bei ...	Vorteile	Nachteile
Korrektionstabelle	nichtlinear, schwer modellierbar	einfach zu nutzen	viele Daten erforderlich, sonst Interpolation ungenau
Kalibrierfaktoren	lineare Abweichung	leicht umsetzbar	unzureichend bei Nichtlinearität
Kalibrierfunktion	nichtlinear, modellierbar	sehr genau	höherer Aufwand

6.2.1 Korrektionstabelle

Mit der Korrektionstabelle können beliebige Korrekturen dargestellt werden, sofern ausreichend Datenpunkte verfügbar sind. In Kapitel 6.12.1 werden wir ein Beispiel für eine Korrektionstabelle kennenlernen.

6.2.2 Ermittlung von Kalibrierfaktoren

Ein Kalibrierfaktor kann angegeben werden, wenn eine konstante relative Abweichung vorliegt. Ein Beispiel aus der Praxis ist [hier auf Seite 1](#) zu finden.

6.2.3 Ermittlung einer (empirischen) Kalibrierfunktion

Eine Kalibrierfunktion erlaubt die Modellierung komplexer Zusammenhänge. Die Methoden dafür werden im [Methodenbaustein Datenfitting und Datenoptimierung](#) vorgestellt.

6.3 Additive Fehler: Der Nullpunktfehler

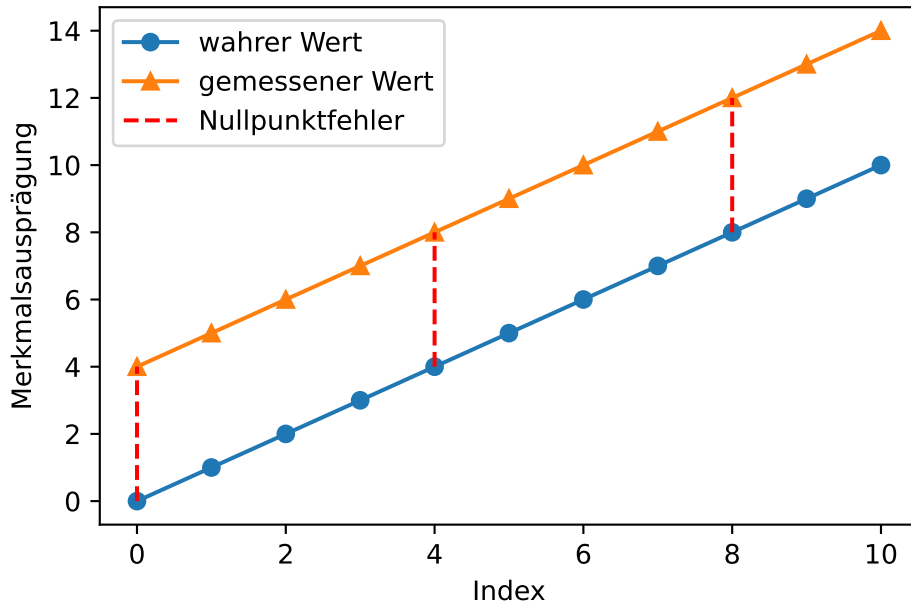
Der Nullpunktfehler beschreibt die Abweichung der Messgröße von Null, wenn der gesuchte Wert tatsächlich Null ist. Der Nullpunktfehler wird auch als Nullabgleich, Nullpunktverschiebung oder Offset-Fehler bezeichnet. Der Nullpunktfehler ist konstant und wirkt additiv auf jeden Messwert. ([ics Schneider Messtechnik](#))

```
# Daten erzeugen
messgröße = np.arange(0, 11)
messwerte = messgröße + 4

# plotten
plt.plot(messgröße, marker = 'o', label = 'wahrer Wert')
plt.plot(messwerte, marker = '^', label = 'gemessener Wert')
plt.plot([0, 0], [messgröße[0], messwerte[0]], linestyle = 'dashed', label = 'Nullpunktfehler')
plt.plot([4, 4], [messgröße[4], messwerte[4]], linestyle = 'dashed', color = 'red')
plt.plot([8, 8], [messgröße[8], messwerte[8]], linestyle = 'dashed', color = 'red')

# Achsenbeschriftung
plt.xlabel('Index')
plt.ylabel('Merkmalsausprägung')

plt.legend()
plt.show()
```



6.3.1 Nullpunktfehler quantifizieren

Der Nullpunktfehler kann leicht bestimmt werden, wenn Messwerte für den Nullpunkt vorliegen und der wahre Wert bekannt ist. (Wenn außerdem bekannt ist, dass keine weiteren systematischen Messabweichungen vorliegen, kann der Nullpunktfehler über einen beliebigen wahren Referenzwert bestimmt werden.) Für die zur grafischen Darstellung angelegten Daten ist dies der Fall.

```
print(f"Der Nullpunktfehler beträgt: {messwerte[0] - messgröße[0]}")
```

Der Nullpunktfehler beträgt: 4

Wenn keine Daten für den Nullpunkt vorliegen, kann der Nullpunktfehler mit linearer Regression aus mindestens zwei bekannten Referenzpunkten geschätzt werden. Dazu wird eine ideale Kennlinie geschätzt und ihr Schnittpunkt mit der y-Achse bestimmt. Ebenso wird für die Messwerte eine lineare Funktion geschätzt. Aus der Differenz der y-Achsen-Schnittpunkte kann der Nullpunktfehler geschätzt werden.

```

# bekannte Referenzpunkte
x1 = 4
x2 = 8
y1 = messgröße[x1]
y2 = messgröße[x2]

# ideale Kennlinie schätzen
# print(lineares_modell) gibt intercept + slope zurück
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
print("Der erste Wert ist der Schnittpunkt der idealen Kennlinie mit der y-Achse:", lm_kennl.

# lineare Funktion der Messwerte schätzen
## man benötigt mindestens 2 Punkte
lm_messwerte = poly.polyfit(x = [x1, x2], y = [messwerte[x1], messwerte[x2]], deg = 1)
print("Die aus 2 Messwerten geschätzte lineare Funktion:", lm_messwerte.round(2), sep = "\n")

## die Schätzung über alle Messwerte ist zuverlässiger
lm_messwerte2 = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)
print("Die aus allen Messwerten geschätzte lineare Funktion:", lm_messwerte2.round(2), sep =

print("\nDie Differenz zwischen dem geschätzten Nullpunkt der Messwerte und dem\ngeschätzten

```

Der erste Wert ist der Schnittpunkt der idealen Kennlinie mit der y-Achse:

```
[-0.  1.]
```

Die aus 2 Messwerten geschätzte lineare Funktion:

```
[4.  1.]
```

Die aus allen Messwerten geschätzte lineare Funktion:

```
[4.  1.]
```

Die Differenz zwischen dem geschätzten Nullpunkt der Messwerte und dem geschätzten Nullpunkt der idealen Kennlinie ist der Nullpunktfehler:

```
4.0
```

Warning 7: Regression

Eine Schätzung der Regressionsgeraden aus 2 Punkten ist im Allgemeinen weniger zuverlässig als eine Schätzung aus mehreren Messpunkten. Die Beispieldaten sind perfekt linear.

Grafisch wird das Vorgehen deutlich.


```

# lineare Funktionen berechnen
geschätzte_ideale_kennlinie = poly.polyval(x = np.arange(messwerte.size), c = lm_kennlinie)
lineare_funktion_messwerte = poly.polyval(x = np.arange(messwerte.size), c = lm_messwerte)

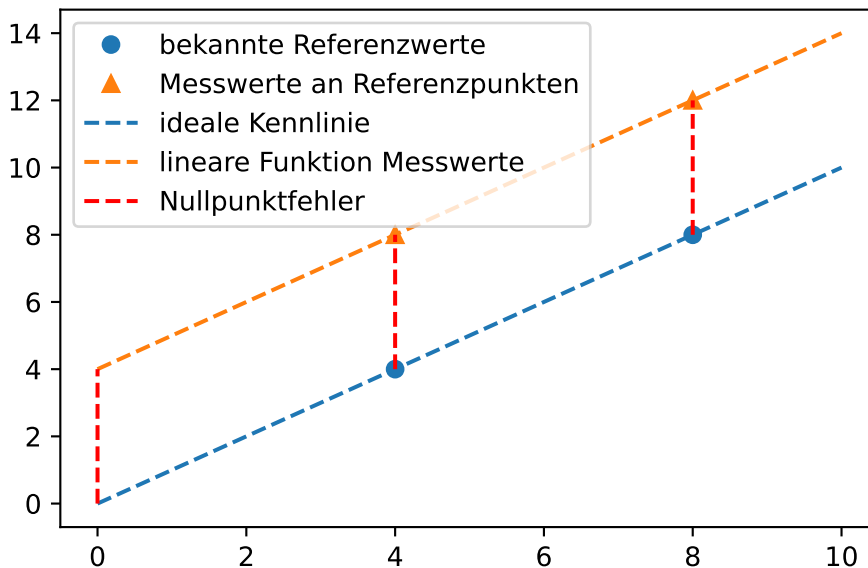
# plotten
## Punkte
plt.plot([x1, x2], [y1, y2], marker = 'o', linestyle = 'none', color = 'C0', label = 'bekannt')
plt.plot([x1, x2], [messwerte[x1], messwerte[x2]], marker = '^', linestyle = 'none', color = 'C1')

## Linien
plt.plot(np.arange(messwerte.size), geschätzte_ideale_kennlinie, marker = 'none', linestyle = 'dashed', color = 'C2')
plt.plot(np.arange(messwerte.size), lineare_funktion_messwerte, marker = 'none', linestyle = 'dashed', color = 'C3')

plt.plot([0, 0], [messgröße[0], messwerte[0]], linestyle = 'dashed', label = 'Nullpunktfehler')
plt.plot([4, 4], [messgröße[4], messwerte[4]], linestyle = 'dashed', color = 'red')
plt.plot([8, 8], [messgröße[8], messwerte[8]], linestyle = 'dashed', color = 'red')

plt.legend()
plt.show()

```



to do: gibt es weitere Methoden?

6.3.2 Nullpunktfehler korrigieren

Der Nullpunktfehler wird durch Subtraktion von den Messwerten korrigiert.

```
# Daten erzeugen
messgröße = np.arange(0, 11) + 5
messwerte = messgröße + 2.5

# bekannte Referenzpunkte
x1 = 2
x2 = 5
y1 = messgröße[x1]
y2 = messgröße[x2]

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)

# lineare Funktion der Messwerte schätzen
## die Schätzung über alle Messwerte ist zuverlässiger
lm_messwerte = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)

# Nullpunktfehler bestimmen
nullpunktfehler = lm_messwerte[0] - lm_kennlinie[0]
print(f"Der Nullpunktfehler beträgt: {nullpunktfehler.round(2)}")

# Nullpunktfehler korrigieren
korrigierte_messwerte = messwerte - nullpunktfehler
```

Der Nullpunktfehler beträgt: 2.5

6.3.3 Übung Nullpunktfehler

Die Quantifizierung des Nullpunktfehlers ist manchmal schwierig, beispielsweise weil der wahre Nullpunkt einer Größe nur schwer vermessen werden kann oder die Messdaten nicht durch eine einfache Funktion approximiert werden können. In diesem Fall muss eine möglichst gute Schätzung gefunden werden.

In einem Klimaschrank wurden 16 Thermoelemente Temperaturen von -10 °C bis 140 °C in Schritten von 10 °C ausgesetzt. Die Messgenauigkeit des verwendeten Datenloggers TCTempX16 liegt bei 0,01 °C, die Auflösung bei 0,1 °C. Für die Thermoelemente soll der Nullpunktfehler bestimmt werden.

⚠ Warning 8: Nullpunkt

0 ° C auf der Celsius-Skala ist kein sinnvoller Referenzpunkt zur Bestimmung des Nullpunktfehlers, weil die Abwesenheit von Wärme erst bei -273,15 °C gegeben ist. Da der Nullpunktfehler über alle Messwerte konstant ist, kann der Nullpunktfehler geschätzt werden, indem für jede Temperaturstufe der Nullpunktfehler separat geschätzt und die Ergebnisse anschließend gemittelt werden.

Die Messdaten liegen in der Datei '01-daten/kalibration_tc.xlsx'. Die Daten können mit dem folgenden Befehl eingelesen werden.

```
klimaschrank = pd.read_excel(io = '01-daten/kalibration_tc.xlsx', sheet_name = 'T15949 Multi')
print(klimaschrank.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2771 entries, 0 to 2770
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Datum                                2771 non-null   datetime64[us]
1   Zeit                                 2771 non-null   datetime64[us]
2   Thermoelement 1 (°C)                2771 non-null   float64
3   Thermoelement 2 (°C)                2771 non-null   float64
4   Thermoelement 3 (°C)                2771 non-null   float64
5   Thermoelement 4 (°C)                2771 non-null   float64
6   Thermoelement 5 (°C)                2771 non-null   float64
7   Thermoelement 6 (°C)                2771 non-null   float64
8   Thermoelement 7 (°C)                2771 non-null   float64
9   Thermoelement 8 (°C)                2771 non-null   float64
10  Thermoelement 9 (°C)                2771 non-null   float64
11  Thermoelement 10 (°C)               2771 non-null   float64
12  Thermoelement 11 (°C)               2771 non-null   float64
13  Thermoelement 12 (°C)               2771 non-null   float64
14  Thermoelement 13 (°C)               2771 non-null   float64
15  Thermoelement 14 (°C)               2771 non-null   float64
16  Thermoelement 15 (°C)               2771 non-null   float64
17  Thermoelement 16 (°C)               2771 non-null   float64
dtypes: datetime64[us](2), float64(16)
memory usage: 389.8 KB
None
```

Mit `klimaschrank.describe()` kann ein Überblick über die Daten gewonnen werden (hier

aus Platzgründen für die ersten 4 Spalten).

```
print(klimaschrank.iloc[:, 0:4].describe())
```

	Datum	Zeit \
count	2771	2771
mean	2026-01-13 14:46:20.314688	2026-01-13 14:46:20.314688
min	2026-01-13 10:55:17	2026-01-13 10:55:17
25%	2026-01-13 12:50:48.500000	2026-01-13 12:50:48.500000
50%	2026-01-13 14:46:21	2026-01-13 14:46:21
75%	2026-01-13 16:41:52	2026-01-13 16:41:52
max	2026-01-13 18:37:24	2026-01-13 18:37:24
std	NaN	NaN

	Thermoelement 1 (°C)	Thermoelement 2 (°C)
count	2771.000000	2771.000000
mean	59.761097	60.795669
min	-9.100000	-11.400000
25%	20.900000	20.950000
50%	58.800000	60.400000
75%	98.300000	100.200000
max	138.700000	140.500000
std	43.075393	44.310769

Die Spalten Datum und Zeit enthalten die selben Informationen. Eine der beiden Spalten kann deshalb entfernt werden.

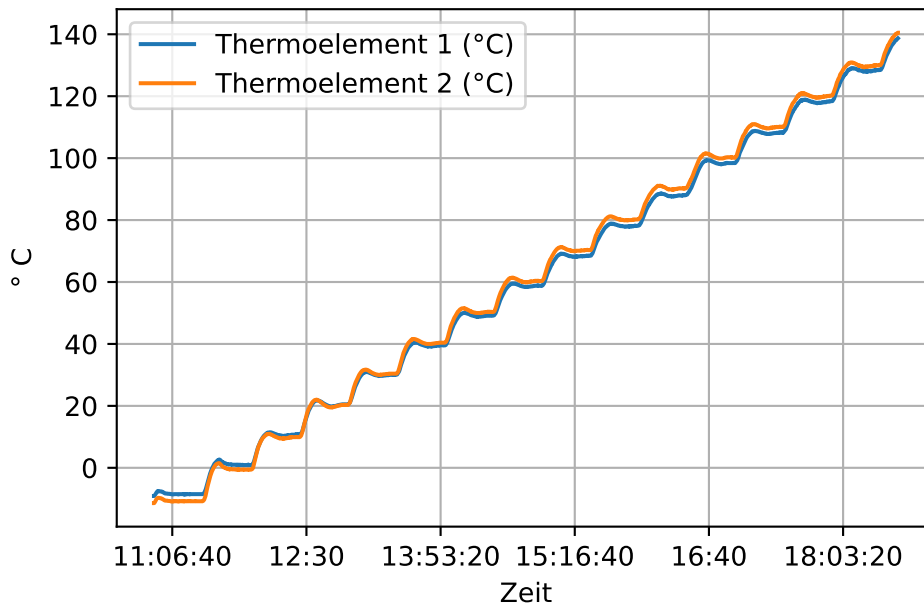
```
klimaschrank.drop(labels = 'Datum', axis = 1, inplace = True)
```

Betrachten wir den Verlauf der Daten für die ersten beiden Thermoelemente. (In der Darstellung mit `pd.plot()` ist die automatisch gewählte x-Achsenbeschriftung unansehnlich. Deshalb wird ein Datenobjekt für die Darstellung angelegt und die Spalte 'Zeit' auf die Uhrzeit reduziert.)

```
# Darstellung mit automatischer Achsenbeschriftung
# klimaschrank.iloc[:, 0:3].plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'])

plotting_data = klimaschrank.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)
plt.ylabel(ylabel = '° C')
plt.show()
```



Es ist zu erkennen, dass der Klimaschrank beim Aufheizen auf die nächste Zieltemperatur zunächst zu stark aufheizt, bevor die eingestellte Temperatur für einige Zeit konstant gehalten wird. Die Temperatur von 140 °C wird nur kurz erreicht, es liegen aber kaum Messwerte vor.

Bestimmen Sie nun den Nullpunktfehler für alle Thermoelemente.

1. Wählen Sie für jedes Thermoelement die Bereiche der Datenreihe für die Temperaturstufen -10 bis 130 °C aus.
2. Bestimmen Sie für jede Temperaturstufe die gemessene Temperatur im Bereich der Datenreihe, in der die Referenztemperatur konstant gehalten wird.
3. Berechnen Sie den Nullpunktfehler für jeden Ausschnitt der Datenreihe und mitteln Sie die Ergebnisse für jedes Thermoelement.

💡 Tipp 12: Lösungshinweise

Tipp 1: Lagemaße wie der Median, Mittelwert oder der Modus können eine einfacher zu bestimmende Schätzgröße für den Nullpunktfehler sein.

Tipp 2: Gehen Sie schrittweise vor: Versuchen Sie zunächst, eine Temperaturstufe eines Thermoelements zu isolieren und die gemessene Temperatur im konstanten Bereich der Datenreihe zu bestimmen. Schreiben Sie anschließend eine Funktion, um den Programmcode für mehrere Thermoelemente und Temperaturstufen zu wiederholen.

Tipp 3: Eine gleichermaßen für 13 Temperaturstufen und 16 Thermoelemente geeignete Lösung dürfte kaum zu finden sein. Manchmal wird ein Wert gesucht, manchmal liegt der

gesuchte Wert zwischen zwei Messwerten. Eine geringfügige Unsicherheit kann durch die Automatisierung in Kauf genommen werden.

💡 Tipp 13: Musterlösung Thermoelemente

Lösung für ein Thermoelement.

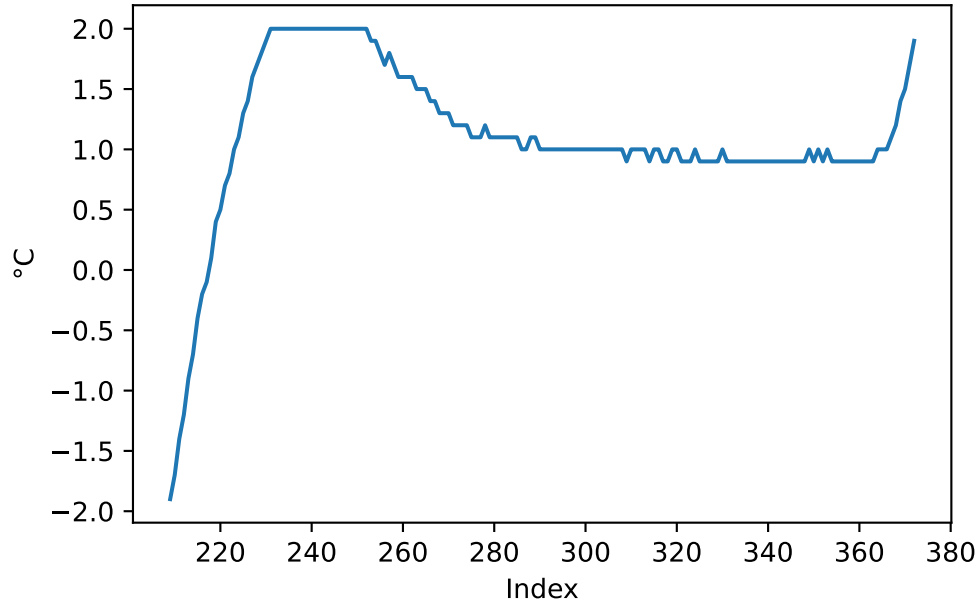
1. Bereich der Datenreihe für eine Temperatur auswählen, z. B. $0\text{ }^{\circ}\text{C}$. Dazu grenzen wir die Messwerte grob um den gesuchten Wert $0 \pm 2^{\circ}\text{C}$ ein.

```
# Thermoelement 1
## Messwerte grob um den gesuchten Wert 0 °C eingrenzen
gesuchter_wert = 0

maske1 = klimaschrank['Thermoelement 1 (°C)'].le(gesuchter_wert + 2) # le = kleiner gleich
maske2 = klimaschrank['Thermoelement 1 (°C)'].ge(gesuchter_wert - 2) # ge = größer gleich

klimaschrank.loc[maske1 * maske2, 'Thermoelement 1 (°C)'].plot()
plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '°C')

plt.show()
```



In diesem Ausschnitt kann das Überschießen der Messwerte beobachtet werden, bevor sich die Messwerte etwa bei + 1 °C einpendeln. Auch ist der Anstieg zur folgenden Temperaturstufe zu erkennen.

2. Im zweiten Schritt sollen die Daten im konstanten Temperaturbereich ausgewählt werden. Dazu soll ein möglichst einfaches Kriterium verwendet werden: der häufigste im Ausschnitt vorkommende Wert, also der Modus. Dazu wird die Methode `pd.value_counts()` verwendet, die eine absteigend sortierte Series der Häufigkeiten zurückgibt, wobei im Index die Werte gespeichert sind (siehe Beispiel). Der im Index gespeicherte häufigste Wert wird mit der Methode `pd.idxmax()` ausgelesen.

Beispiel 6.1: `pd.value_counts()`

Die Ausgabe von `pd.value_counts()`:

```
daten = pd.Series([1, 2, 3, 3, 3, 3, 3, 4, 4, 5])
daten.value_counts()
```

```
3    5
4    2
1    1
2    1
5    1
Name: count, dtype: int64
```

```

# Thermoelement 1

## Messwerte grob um den gesuchten Wert 0 °C eingrenzen
maske1 = klimaschrank['Thermoelement 1 (°C)'].le(gesuchter_wert + 2) # le = kleiner gleich
maske2 = klimaschrank['Thermoelement 1 (°C)'].ge(gesuchter_wert - 2) # ge = größer gleich

## häufigster Wert im eingegrenzten Temperaturbereich
modus_bereich = klimaschrank.loc[maske1 * maske2, 'Thermoelement 1 (°C)'].value_counts().idxmax()

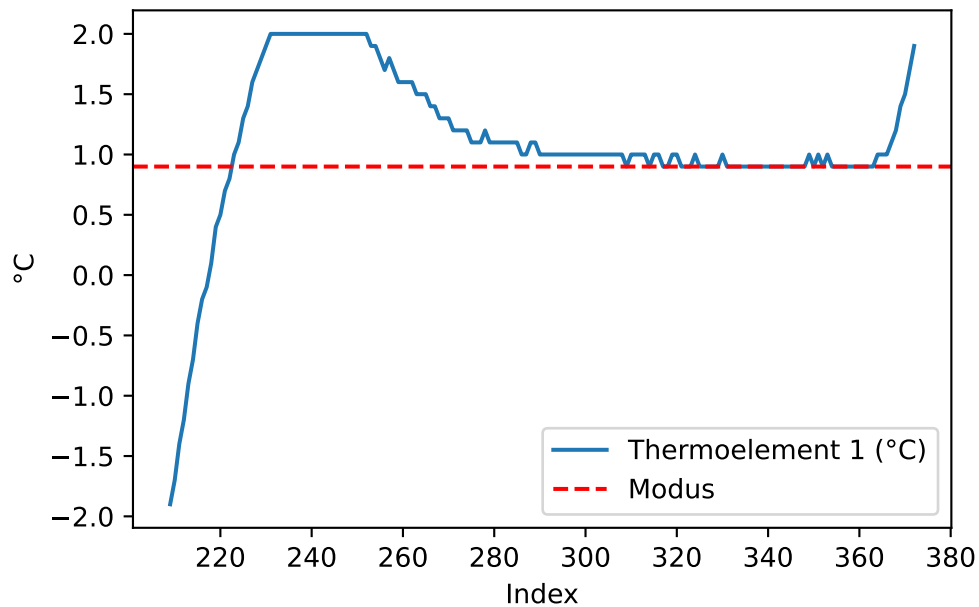
## Nullpunktfehler bestimmen
nullpunktfehler = modus_bereich - gesuchter_wert
print(f"Nullpunktfehler: {nullpunktfehler}")

## plotten
klimaschrank.loc[maske1 * maske2, 'Thermoelement 1 (°C)'].plot()
plt.axhline(y = modus_bereich, color = 'r', linestyle = '--', label = 'Modus')
plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '°C')

plt.legend()
plt.show()

```

Nullpunktfehler: 0.9



Der Nullpunktfehler für Thermoelement 1 wird somit mit 0,9 °C bestimmt.

3. Um die Berechnung für die verschiedenen Temperaturstufen zu automatisieren, schreiben wir eine Funktion.

```
# Eingabe: data = pd.Series, gesuchter_wert = array like, schwellwert, puffer = Skalar
# Verarbeitung: Elementweise werden die Werte in gesuchter_Wert ± puffer in data gesucht
# Verarbeitung: Die Differenz aus dem Modus jedes Datenausschnitts und dem gesuchten Wert v
# Ausgabe: Wenn output = True wird DataFrame der gesuchten Werte und Nullpunktfehler ausgeg

def nullpunktfehler(data, gesuchte_werte, puffer = 2, output = False):

    # Zielobjekt und Zähler anlegen
    ausgabe = pd.DataFrame({'Gesuchter Wert': pd.Series(), 'Nullpunktfehler': pd.Series()})
    i = 0

    for gesuchter_wert in gesuchte_werte:

        ## Messwerte grob um den gesuchten Wert eingrenzen
        maske1 = data.le(gesuchter_wert + puffer) # le = kleiner gleich
        maske2 = data.ge(gesuchter_wert - puffer) # ge = größer gleich

        ## Kriterium häufigster Wert im eingegrenzten Temperaturbereich
        modus_bereich = data.loc[maske1 * maske2].value_counts().idxmax()

        # Nullpunktfehler bestimmen
        nullpunktfehler = modus_bereich - gesuchter_wert

        # Werte eintragen
        ausgabe.loc[i] = pd.Series([gesuchter_wert, nullpunktfehler]).values

        # Zähler erhöhen
        i += 1

    # Optional Ausgabe
    if output: # output is True
        print(ausgabe)

    # Rückgabe der gemittelten Nullpunktfehler
    return ausgabe['Nullpunktfehler'].mean()
```

Angewendet auf Thermoelement 1 (°C):

```
nullpunktfehler(data = klimaschrank['Thermoelement 1 (°C)'],
                 gesuchte_werte = np.arange(-10, 140, 10),
                 output = True)
```

	Gesuchter Wert	Nullpunktfehler
0	-10.0	1.5
1	0.0	0.9
2	10.0	0.7
3	20.0	0.5
4	30.0	0.0
5	40.0	-0.4
6	50.0	-0.9
7	60.0	-1.2
8	70.0	-1.7
9	80.0	-2.0
10	90.0	-2.0
11	100.0	-1.6
12	110.0	-1.8
13	120.0	-1.7
14	130.0	-1.6

```
np.float64(-0.7533333333333327)
```

Anwendung auf den Datensatz

Im letzten Schritt wenden wir die Funktion spaltenweise auf den DataFrame an.

```
# gibt pd.Series zurück
nullpunktfehler_klimaschrank = klimaschrank.iloc[:, 1:].apply(nullpunktfehler, axis = 0, g

print(nullpunktfehler_klimaschrank.round(3))
```

Thermoelement 1 (°C)	-0.753
Thermoelement 2 (°C)	0.027
Thermoelement 3 (°C)	-0.000
Thermoelement 4 (°C)	0.547
Thermoelement 5 (°C)	0.513
Thermoelement 6 (°C)	-0.233
Thermoelement 7 (°C)	0.073
Thermoelement 8 (°C)	-0.767
Thermoelement 9 (°C)	-0.373
Thermoelement 10 (°C)	-0.980

```
Thermoelement 11 (°C)    -0.900
Thermoelement 12 (°C)     0.513
Thermoelement 13 (°C)   -0.760
Thermoelement 14 (°C)   -0.593
Thermoelement 15 (°C)   -0.047
Thermoelement 16 (°C)   -0.987
dtype: float64
```

6.4 Multiplikative Fehler: Der Empfindlichkeitsfehler

Als Empfindlichkeits- oder Spannf Fehler wird ein über den Wertebereich (die Spanne) der gesuchten Größe zu- oder abnehmender Fehler bezeichnet. Die relative Messabweichung δ ist dabei konstant, die absolute Messabweichung abhängig vom Messwert. ([ics Schneider Messtechnik](#))

$$\text{gemessene Werte} = \text{wahre Werte} \cdot \text{Empfindlichkeitsfehler}$$

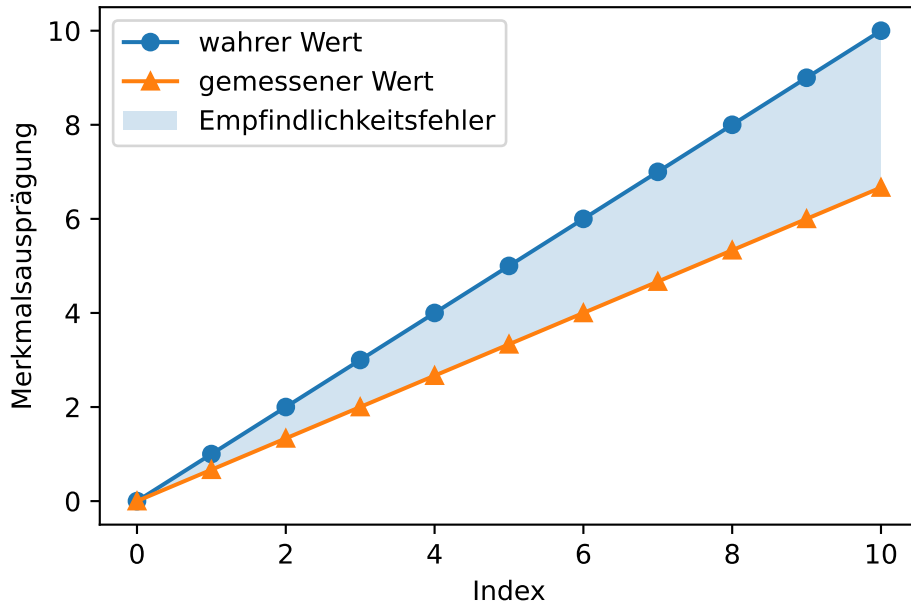
```
# Daten erzeugen
messgröße = np.arange(0, 11)
messwerte = messgröße - messgröße / 3

# plotten
plt.plot(messgröße, marker = 'o', label = 'wahrer Wert')
plt.plot(messwerte, marker = '^', label = 'gemessener Wert')

# füllen
plt.fill_between(x = np.arange(messwerte.size), y1 = messgröße, y2 = messwerte, alpha = 0.2,

# Achsenbeschriftung
plt.xlabel('Index')
plt.ylabel('Merkmalsausprägung')

plt.legend()
plt.show()
```



6.4.1 Empfindlichkeitsfehler quantifizieren

Der Empfindlichkeitsfehler kann geschätzt werden, wenn Referenzwerte bekannt sind. Dazu wird das Verhältnis der Größenänderung (des Anstiegs) der gemessenen und der wahren Werte gebildet.

$$\text{Empfindlichkeitsfehler} = \frac{\Delta_{\text{gemessene Werte}}}{\Delta_{\text{wahre Werte}}}$$

Mit den vollständigen Beispieldaten geht es leicht (in dem Beispiel wird eine Warnung wegen Division durch 0 unterdrückt):

```
print(f"Der Empfindlichkeitsfehler beträgt: {round((np.nanmean((messwerte / messgröße)) - 1) * 100, 2)} %")
```

Der Empfindlichkeitsfehler beträgt: -33.33 %

Bei nur punktuellen Daten wird eine ideale Kennlinie geschätzt, mit der der Empfindlichkeitsfehler geschätzt werden kann.

```

# bekannte Referenzwerte
x1 = 4
x2 = 8
y1 = messgröße[x1]
y2 = messgröße[x2]

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
geschätzte_ideale_kennlinie = poly.polyval(x = np.arange(messwerte.size), c = lm_kennlinie)

# lineare Regression der Messwerte
lm_messwerte = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)

# Berechnung Empfindlichkeitsfehler
spannfehler = lm_messwerte[1] / lm_kennlinie[1]

# Ausgabe
print(f"Der Anstieg der Messwerte: {round(lm_messwerte[1], 2)}")
print(f"Der Anstieg der idealen Kennlinie: {round(lm_kennlinie[1], 2)}")
print(f"Der Empfindlichkeitsfehler beträgt: {round((spannfehler - 1) * 100, 2)} %")

```

Der Anstieg der Messwerte: 0.67
 Der Anstieg der idealen Kennlinie: 1.0
 Der Empfindlichkeitsfehler beträgt: -33.33 %

Grafisch wird das Vorgehen deutlich.

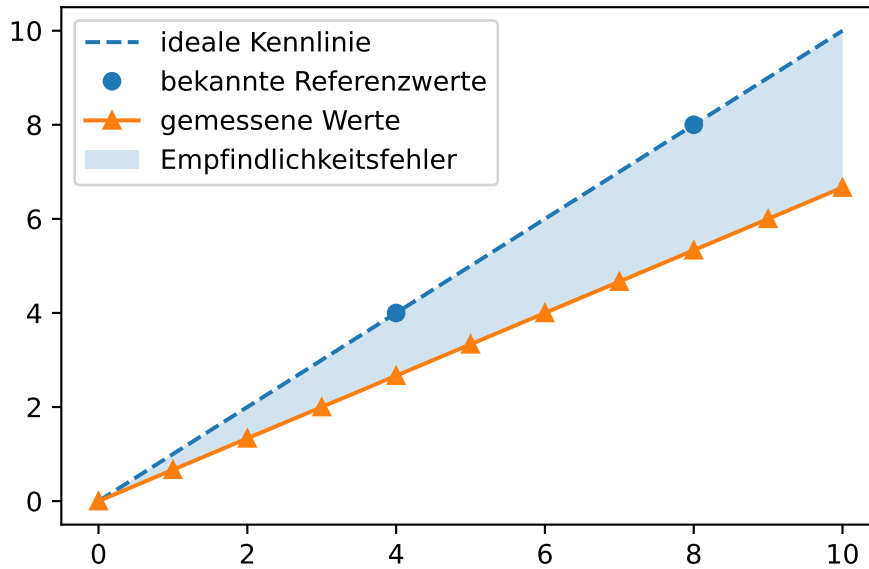
```

# Punkte und Linien
plt.plot([0, 10], [messgröße[0], messgröße[-1]], marker = 'none', linestyle = 'dashed', color = 'C0')
plt.plot([x1, x2], [y1, y2], marker = 'o', linestyle = 'none', color = 'C0', label = 'bekannte Werte')
plt.plot(messwerte, marker = '^', color = 'C1', label = 'gemessene Werte')

# füllen
plt.fill_between(x = np.arange(messwerte.size), y1 = messgröße, y2 = messwerte, alpha = 0.2)

plt.legend()
plt.show()

```



6.4.2 Empfindlichkeitsfehler korrigieren

Der Empfindlichkeitsfehler wird durch die Division der Messwerte durch den Empfindlichkeitsfehler korrigiert.

$$\text{wahre Werte} = \frac{\text{gemessene Werte}}{\text{Empfindlichkeitsfehler}}$$

```
# Daten erzeugen
messgröße = np.arange(0, 11) + 5
messwerte = messgröße * 1.07

# bekannte Referenzpunkte
x1 = 2
x2 = 5
y1 = messgröße[x1]
y2 = messgröße[x2]

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)

# lineare Funktion der Messwerte schätzen
lm_messwerte = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)
```

```
# Empfindlichkeitsfehler bestimmen
spannfehler = lm_messwerte[1] / lm_kennlinie[1]
print(f"Der Empfindlichkeitsfehler beträgt: {( (spannfehler - 1) * 100).round(2)} %")

# Empfindlichkeitsfehler korrigieren
korrigierte_messwerte = messwerte / spannfehler

# Ausgabe
print(f"Wahre Werte:\n{messgröße}")
print(f"Messwerte:\n{messwerte}")
print(f"Korrigierte Messwerte:\n{korrigierte_messwerte}")
```

```
Der Empfindlichkeitsfehler beträgt: 7.0 %
Wahre Werte:
[ 5  6  7  8  9 10 11 12 13 14 15]
Messwerte:
[ 5.35  6.42  7.49  8.56  9.63 10.7  11.77 12.84 13.91 14.98 16.05]
Korrigierte Messwerte:
[ 5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15.]
```

6.4.3 Übung Empfindlichkeitsfehler

In einem Klimaschrank wurden 16 Thermoelemente einem Temperaturanstieg von -10 °C bis 140 °C ausgesetzt. Die Messgenauigkeit des verwendeten Datenloggers TCTempX16 liegt bei 0,01 °C, die Auflösung bei 0,1 °C. Für die Thermoelemente soll der Empfindlichkeitsfehler bestimmt werden.

Die Messdaten liegen in der Datei '01-daten/kalibration_tc_lineare_rampe.xlsx'. Die Daten können mit dem folgenden Befehl eingelesen werden.

```
klimaschrank = pd.read_excel(io = '01-daten/kalibration_tc_lineare_rampe.xlsx', sheet_name = 
print(klimaschrank.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2961 entries, 0 to 2960
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Datum                 2961 non-null  datetime64[us]
1   Zeit                 2961 non-null  datetime64[us]
2   Thermoelement 1 (°C) 2961 non-null  float64
```

```

3  Thermoelement 2 (°C)  2961 non-null  float64
4  Thermoelement 3 (°C)  2961 non-null  float64
5  Thermoelement 4 (°C)  2961 non-null  float64
6  Thermoelement 5 (°C)  2961 non-null  float64
7  Thermoelement 6 (°C)  2961 non-null  float64
8  Thermoelement 7 (°C)  2961 non-null  float64
9  Thermoelement 8 (°C)  2961 non-null  float64
10 Thermoelement 9 (°C)  2961 non-null  float64
11 Thermoelement 10 (°C) 2961 non-null  float64
12 Thermoelement 11 (°C) 2961 non-null  float64
13 Thermoelement 12 (°C) 2961 non-null  float64
14 Thermoelement 13 (°C) 2961 non-null  float64
15 Thermoelement 14 (°C) 2961 non-null  float64
16 Thermoelement 15 (°C) 2961 non-null  float64
17 Thermoelement 16 (°C) 2961 non-null  float64

```

dtypes: datetime64[us](2), float64(16)

memory usage: 416.5 KB

None

Mit `klimaschrank.describe()` kann ein Überblick über die Daten gewonnen werden (hier aus Platzgründen für die ersten 4 Spalten).

```
print(klimaschrank.iloc[:, 0:4].describe())
```

	Datum	Zeit \
count	2961	2961
mean	2026-01-15 14:43:19.065856	2026-01-15 14:43:19.065856
min	2026-01-15 10:36:25	2026-01-15 10:36:25
25%	2026-01-15 12:39:52	2026-01-15 12:39:52
50%	2026-01-15 14:43:19	2026-01-15 14:43:19
75%	2026-01-15 16:46:46	2026-01-15 16:46:46
max	2026-01-15 18:50:13	2026-01-15 18:50:13
std	NaN	NaN

	Thermoelement 1 (°C)	Thermoelement 2 (°C)
count	2961.000000	2961.000000
mean	79.988011	81.124688
min	-8.500000	-10.700000
25%	41.300000	42.400000
50%	94.400000	96.200000
75%	117.200000	118.800000
max	147.100000	148.700000

std

45.318636

46.109421

Die Spalten Datum und Zeit enthalten die selben Informationen. Eine der beiden Spalten kann deshalb entfernt werden.

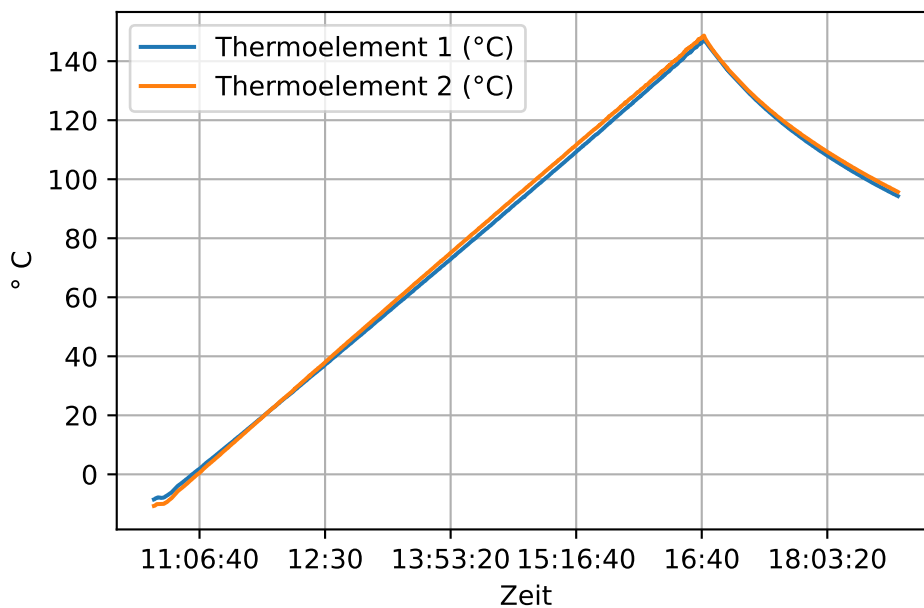
```
klimaschrnk.drop(labels = 'Datum', axis = 1, inplace = True)
```

Betrachten wir den Verlauf der Daten für die ersten beiden Thermoelemente. (In der Darstellung mit `pd.plot()` ist die automatisch gewählte x-Achsenbeschriftung unansehnlich. Deshalb wird ein Datenobjekt für die Darstellung angelegt und die Spalte 'Zeit' auf die Uhrzeit reduziert.)

```
# Darstellung mit automatischer Achsenbeschriftung
# klimaschrnk.iloc[:, 0:3].plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'])

plotting_data = klimaschrnk.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)
plt.ylabel(ylabel = '°C')
plt.show()
```



Die Messung startet kurz vor der Aufheizphase. Etwa 16:40 Uhr wurde der Klimaschrank abgeschaltet. Die Messung lief danach noch einige Zeit weiter. Es müssen also der Start- und der Endpunkt der Aufheizphase bestimmt werden.

In diesem Beispiel soll angenommen werden, dass bei der Messung kein Nullpunktfehler vorliegt. Dazu suchen wir mit der Funktion aus Tipp 13 passende Messreihen. Aus dem anfänglich konstanten Bereich -10 °C wird der Nullpunktfehler bestimmt.

```
# gibt pd.Series zurück
nullpunktfehler_klimaschrank = klimaschrank.iloc[:, 1:].apply(nullpunktfehler, axis = 0, ge

print(nullpunktfehler_klimaschrank)
```

```
Thermoelement 1 (°C)    2.0
Thermoelement 2 (°C)   -0.2
Thermoelement 3 (°C)    2.0
Thermoelement 4 (°C)    0.0
Thermoelement 5 (°C)   -0.1
Thermoelement 6 (°C)    1.8
Thermoelement 7 (°C)    0.1
Thermoelement 8 (°C)    1.7
Thermoelement 9 (°C)    1.6
Thermoelement 10 (°C)   1.7
Thermoelement 11 (°C)   1.8
Thermoelement 12 (°C)  -0.1
Thermoelement 13 (°C)   1.6
Thermoelement 14 (°C)   1.9
Thermoelement 15 (°C)   0.0
Thermoelement 16 (°C)   1.8
dtype: float64
```

Die Messreihen für Thermoelement 4 und Thermoelement 15 scheinen geeignet zu sein. Schauen wir uns die Messreihen einmal an:

6.5 Thermoelement 4

```
# Thermoelement 4
gesuchter_wert = -10

## Messwerte grob um den gesuchten Wert 0 °C eingrenzen
```

```

maske1 = klimaschrank['Thermoelement 4 (°C)'].le(gesuchter_wert + 2) # le = kleiner gleich
maske2 = klimaschrank['Thermoelement 4 (°C)'].ge(gesuchter_wert - 2) # ge = größer gleich

## häufigster Wert im eingegrenzten Temperaturbereich
modus_bereich = klimaschrank.loc[maske1 * maske2, 'Thermoelement 4 (°C)'].value_counts().idxmax()

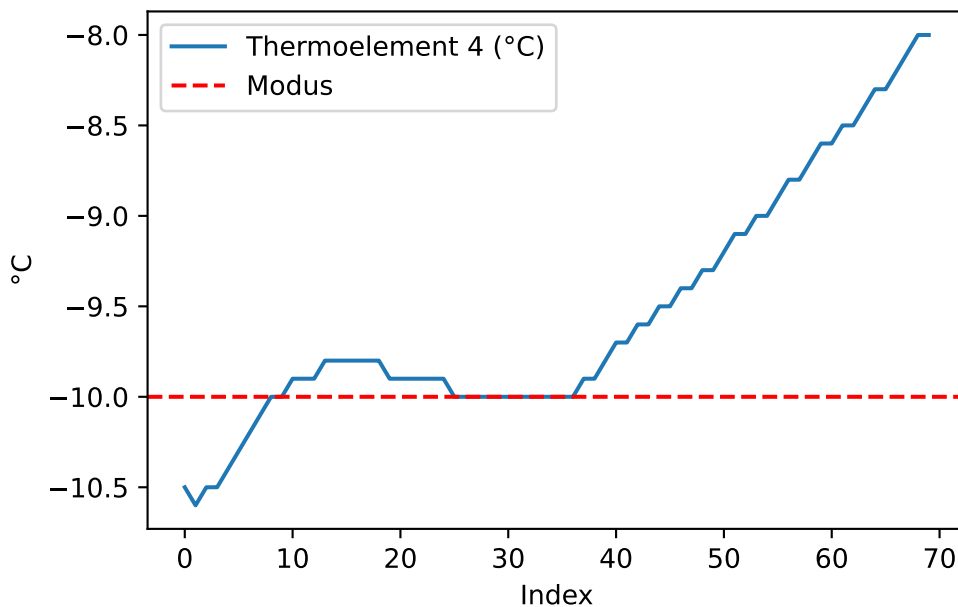
## Nullpunktfehler bestimmen
nullpunktfehler = modus_bereich - gesuchter_wert
print(f"Nullpunktfehler: {nullpunktfehler}")

## plotten
klimaschrank.loc[maske1 * maske2, 'Thermoelement 4 (°C)'].plot()
plt.axhline(y = modus_bereich, color = 'r', linestyle = '--', label = 'Modus')
plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '°C')

plt.legend()
plt.show()

```

Nullpunktfehler: 0.0



6.6 Thermoelement 15

```
# Thermoelement 15
gesuchter_wert = -10

## Messwerte grob um den gesuchten Wert 0 °C eingrenzen
maske1 = klimaschrank['Thermoelement 15 (°C)'].le(gesuchter_wert + 2) # le = kleiner gleich
maske2 = klimaschrank['Thermoelement 15 (°C)'].ge(gesuchter_wert - 2) # ge = größer gleich

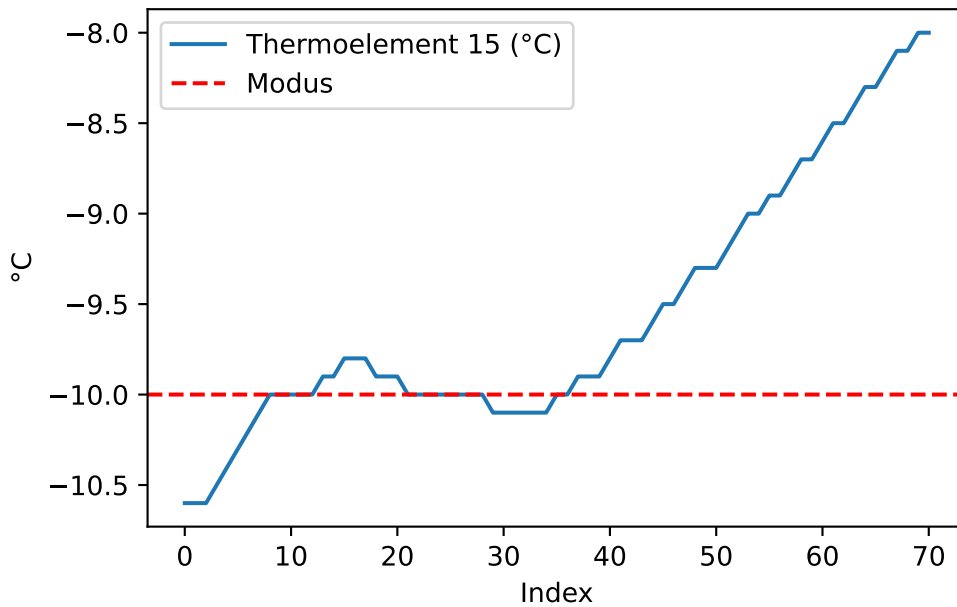
## häufigster Wert im eingegrenzten Temperaturbereich
modus_bereich = klimaschrank.loc[maske1 * maske2, 'Thermoelement 15 (°C)'].value_counts().id

## Nullpunktfehler bestimmen
nullpunktfehler = modus_bereich - gesuchter_wert
print(f"Nullpunktfehler: {nullpunktfehler}")

## plotten
klimaschrank.loc[maske1 * maske2, 'Thermoelement 15 (°C)'].plot()
plt.axhline(y = modus_bereich, color = 'r', linestyle = '--', label = 'Modus')
plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '°C')

plt.legend()
plt.show()
```

Nullpunktfehler: 0.0



Bestimmen Sie den Empfindlichkeitsfehler für Thermoelement 4 und Thermoelement 15. Das wahre Minimum der Temperatur betrage $-10\text{ }^{\circ}\text{C}$ und das wahre Maximum betrage $140\text{ }^{\circ}\text{C}$.

1. Bestimmen Sie den Startpunkt der Aufheizphase.
2. Bestimmen Sie den Endpunkt der Aufheizphase.
3. Schätzen Sie eine ideale Kennlinie von $-10\text{ }^{\circ}\text{C}$ bis $140\text{ }^{\circ}\text{C}$ für die Aufheizphase.
4. Ermitteln und korrigieren Sie den Empfindlichkeitsfehler in den Daten.

💡 Tipp 14: Musterlösung Empfindlichkeitsfehler

1. Startpunkt ermitteln, indem die Position des letzten Werts $-10\text{ }^{\circ}\text{C}$ bestimmt wird.

```
start4 = klimaschrnk.loc[::-1, 'Thermoelement 4 (°C)'].eq(-10).idxmax()
start15 = klimaschrnk.loc[::-1, 'Thermoelement 15 (°C)'].eq(-10).idxmax()
```

2. Endpunkt ermitteln, indem die Position des Maximums bestimmt wird.

```
ende4 = klimaschrnk.loc[:, 'Thermoelement 4 (°C)'].idxmax()
ende15 = klimaschrnk.loc[:, 'Thermoelement 15 (°C)'].idxmax()
```

Ausgabe.

```
print(f"Messreihe Thermoelement 4 von Index {start4} bis Index {ende4}.",
      f"Messreihe Thermoelement 15 von Index {start15} bis Index {ende15}.",
      sep = '\n')
```

Messreihe Thermoelement 4 von Index 36 bis Index 2187.
 Messreihe Thermoelement 15 von Index 36 bis Index 2188.

3. Ideale Kennlinie und lineare Funktion der Messwerte schätzen, Empfindlichkeitsfehler bestimmen.

6.7 Thermoelement 4

```
# bekannte Referenzpunkte
x1 = start4
x2 = ende4
y1 = -10
y2 = 140

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)

# lineare Funktion der Messwerte schätzen
lm_messwerte = poly.polyfit(x = np.arange(x1, x2 + 1), y = klimaschrank.loc[x1 : x2, 'Therm

# Empfindlichkeitsfehler bestimmen
spannfehler = lm_messwerte[1] / lm_kennlinie[1]
print(f"Der Empfindlichkeitsfehler beträgt: {( (lm_messwerte[1] / lm_kennlinie[1]) - 1) *

# Empfindlichkeitsfehler korrigieren
klimaschrank.loc[ : , 'Thermoelement 4 (°C)'] = klimaschrank.loc[ : , 'Thermoelement 4 (°C)
```

Der Empfindlichkeitsfehler beträgt: 6.66 %

6.8 Thermoelement 15

```

# bekannte Referenzpunkte
x1 = start15
x2 = ende15
y1 = -10
y2 = 140

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)

# lineare Funktion der Messwerte schätzen
lm_messwerte = poly.polyfit(x = np.arange(x1, x2 + 1), y = klimaschrnk.loc[x1 : x2, 'Thermoelement 15 (°C)'], deg = 1)

# Empfindlichkeitsfehler bestimmen
spannfehler = lm_messwerte[1] / lm_kennlinie[1]
print(f"Der Empfindlichkeitsfehler beträgt: {( (lm_messwerte[1] / lm_kennlinie[1]) - 1) * 100} %")

# Empfindlichkeitsfehler korrigieren
klimaschrnk.loc[:, 'Thermoelement 15 (°C)'] = klimaschrnk.loc[:, 'Thermoelement 15 (°C)'] * (1 - spannfehler)

Der Empfindlichkeitsfehler beträgt: 5.62 %

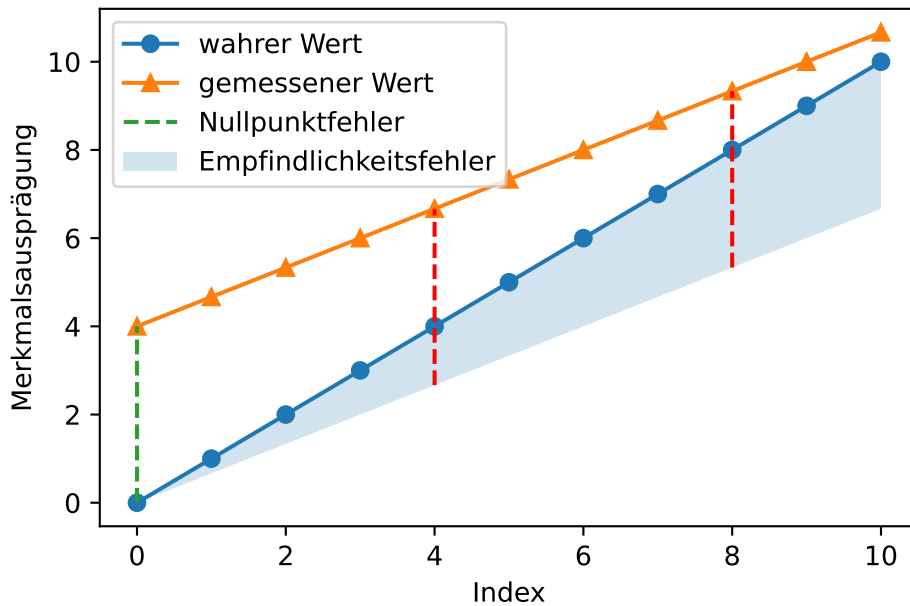
```

6.9 Nullpunkt- und Empfindlichkeitsfehler

Bezeichnung und Berechnung / Korrektur Spannfeder ändern

Absolute und multiplikative Abweichungen können gemeinsam auftreten. In diesem Beispiel wird die absolute Messabweichung über den dargestellten Wertebereich kleiner, da der Nullpunktfehler mit positivem Vorzeichen und der Empfindlichkeitsfehler mit negativen Vorzeichen auftreten.

6.10 Grafik



6.11 Code

```
# Daten erzeugen
messgröße = np.arange(0, 11)
messwerte = messgröße + 4 - messgröße / 3

# plotten
plt.plot(messgröße, marker = 'o', label = 'wahrer Wert')
plt.plot(messwerte, marker = '^', label = 'gemessener Wert')
plt.plot([0, 0], [messgröße[0], messwerte[0]], linestyle = 'dashed', label = 'Nullpunktfehler')

# füllen
hilfslinie_spannfehler = messwerte - 4
plt.fill_between(x = np.arange(messwerte.size), y1 = messgröße, y2 = hilfslinie_spannfehler,

# Nullpunktfehler abtragen
plt.plot([4, 4], [hilfslinie_spannfehler[4], messwerte[4]], linestyle = 'dashed', color = 'r')
plt.plot([8, 8], [hilfslinie_spannfehler[8], messwerte[8]], linestyle = 'dashed', color = 'r')

# Achsenbeschriftung
```



```
plt.xlabel('Index')
plt.ylabel('Merkmalsausprägung')

plt.legend()
plt.show()
```

6.11.1 Nullpunkt- und Empfindlichkeitsfehler quantifizieren

Aus mindestens zwei bekannten Referenzpunkten können mit linearer Regression der Nullpunkt- und der Empfindlichkeitsfehler geschätzt werden. In der linearen Regression wird ein bei 0 beginnender Index für die x-Werte verwendet. Um den Nullpunktfehler korrekt ermitteln zu können, wird dieser Index um den geschätzten wahren Nullpunkt verschoben.

```
# Daten erzeugen
messgröße = np.arange(0, 11) + 7
messwerte = messgröße + 24 - messgröße / 3

# bekannte Referenzpunkte
x1 = 4
x2 = 8
y1 = messgröße[x1]
y2 = messgröße[x2]

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
print(f"lm_kennlinie: {lm_kennlinie}")

# lineare Regression der Messwerte
lm_messwerte = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)
print(f"lm_messwerte: {lm_messwerte}")

# lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
lm_messwerte_plus_intercept_kennlinie = poly.polyfit(np.arange(messwerte.size) + lm_kennlinie[0], y = messwerte, deg = 1)
print(f"lm_messwerte_plus_intercept_kennlinie: {lm_messwerte_plus_intercept_kennlinie}")

# Nullpunktfehler bestimmen
nullpunktfehler = lm_messwerte_plus_intercept_kennlinie[0]
print(f"Der Nullpunktfehler beträgt: {nullpunktfehler.round(2)}")

# Empfindlichkeitsfehler bestimmen
```

```
spannfehler = lm_messwerte_plus_intercept_kennlinie[1] / lm_kennlinie[1]
print(f"Der Empfindlichkeitsfehler beträgt: {((spannfehler - 1) * 100).round(2)} %")
```

```
lm_kennlinie: [7. 1.]
lm_messwerte: [28.66666667  0.66666667]
lm_messwerte_plus_intercept_kennlinie: [24.          0.66666667]
Der Nullpunktfehler beträgt: 24.0
Der Empfindlichkeitsfehler beträgt: -33.33 %
```

6.11.2 Nullpunkt- und Empfindlichkeitsfehler korrigieren

Bei der Korrektur von Nullpunkt- und Empfindlichkeitsfehler unterscheidet sich das Vorgehen abhängig von der Reihenfolge der vorgenommenen Korrekturen.

1. Zuerst wird der Nullpunktfehler korrigiert, danach der Empfindlichkeitsfehler.

$$\text{korrigierte Messwerte} = \frac{(\text{Messwerte} - \text{Nullpunktfehler})}{\text{Empfindlichkeitsfehler}}$$

2. Zuerst wird der Empfindlichkeitsfehler korrigiert, danach der Nullpunktfehler.

$$\text{korrigierte Messwerte} = \frac{\text{Messwerte}}{\text{Empfindlichkeitsfehler}} - \frac{\text{Nullpunktfehler}}{\text{Empfindlichkeitsfehler}}$$

Ein Beispiel: **hierhier**

```
# Daten erzeugen
messgröße = np.arange(0, 11) + 7
messwerte = messgröße + 4 - messgröße / 3

# bekannte Referenzpunkte
x1 = 4
x2 = 8
y1 = messgröße[x1]
y2 = messgröße[x2]

# Ausgabe
print(f"Wahre Werte:\n{messgröße}")
print(f"Messwerte:\n{messwerte}")
print()
```

```

# ideale Kennlinie schätzen
lm_kennlinie = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
print(f"lm_kennlinie: {lm_kennlinie}")

# lineare Regression der Messwerte
lm_messwerte = poly.polyfit(x = np.arange(messwerte.size), y = messwerte, deg = 1)
print(f"lm_messwerte: {lm_messwerte}")

# lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
lm_messwerte_plus_intercept_kennlinie = poly.polyfit(np.arange(messwerte.size) + lm_kennlinie[0], y = messwerte, deg = 1)
print(f"lm_messwerte_plus_intercept_kennlinie: {lm_messwerte_plus_intercept_kennlinie}")

# Nullpunktfehler bestimmen
nullpunktfehler = lm_messwerte_plus_intercept_kennlinie[0]
print(f"Der Nullpunktfehler beträgt: {nullpunktfehler.round(2)}")

# Empfindlichkeitsfehler bestimmen
spannfehler = lm_messwerte_plus_intercept_kennlinie[1] / lm_kennlinie[1]
print(f"Der Empfindlichkeitsfehler beträgt: {((lm_messwerte_plus_intercept_kennlinie[1] / lm_kennlinie[1]) * 100).round(2)}%")

# erst Nullpunktfehler, dann Empfindlichkeitsfehler korrigieren
## Nullpunktfehler korrigieren
korrigierte_messwerte = messwerte - nullpunktfehler

## Empfindlichkeitsfehler korrigieren
korrigierte_messwerte = korrigierte_messwerte / spannfehler

print(f"erst Nullpunktfehler, dann Empfindlichkeitsfehler korrigiert:\n{korrigierte_messwerte}")

# erst Empfindlichkeitsfehler, dann Nullpunkt korrigieren
## Empfindlichkeitsfehler korrigieren
korrigierte_messwerte = messwerte / spannfehler

## Nullpunktfehler korrigieren
korrigierte_messwerte = korrigierte_messwerte - (nullpunktfehler / spannfehler)

print(f"erst Empfindlichkeitsfehler, dann Nullpunktfehler korrigiert:\n{korrigierte_messwerte}")
print()

```

Wahre Werte:

[7 8 9 10 11 12 13 14 15 16 17]

Messwerte:

```
[ 8.66666667  9.33333333 10.          10.66666667 11.33333333 12.
 12.66666667 13.33333333 14.          14.66666667 15.33333333]
```

```
lm_kennlinie: [7. 1.]
lm_messwerte: [8.66666667 0.66666667]
lm_messwerte_plus_intercept_kennlinie: [4.          0.66666667]
Der Nullpunktfehler beträgt: 4.0
Der Empfindlichkeitsfehler beträgt: -33.33 %
erst Nullpunktfehler, dann Empfindlichkeitsfehler korrigiert:
[ 7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17.]
erst Empfindlichkeitsfehler, dann Nullpunktfehler korrigiert:
[ 7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17.]
```

Simones Vorgehen führt zu korrekt kalibrierten Messwerten. Die Variable a ist der Korrekturfaktor für den Empfindlichkeitsfehler. Die Variable b ist die Nullpunktkorrektur, wenn erst der Empfindlichkeitsfehler korrigiert wird.

Das hat etwas mit der Konstruktion der Daten bzw. der Korrektur zu tun. b ist die Korrektur des Nullpunktfehlers * Empfindlichkeitsfehler. Das heißt aus der Arnoldformel kann der Nullpunktfehler als $-1 * \frac{b}{a}$ ermittelt werden.

6.11.3 Übung Nullpunkt- und Empfindlichkeitsfehler

to do: hier könnte man andere Thermoelemente aus der letzten Übung verwenden

6.12 Nichtlinearität: Linearitätsfehler

Bei einer idealen Messung hängt der gesuchte Wert linear vom gemessenen Wert ab. Viele analoge Sensoren reagieren aufgrund von Materialeigenschaften oder abhängig von der Temperatur nicht linear auf die gemessene Größe. Das heißt, die Messwerte sind nicht direkt proportional zur Messgröße. Dieser sogenannte **Linearitätsfehler** muss korrigiert werden, um genau zu messen.

Dafür gibt es zwei Möglichkeiten. Zum einen können nicht lineare Verfahren zur Parameterschätzung verwendet werden, was im [Methodenbaustein Datenfitting und Datenoptimierung](#) behandelt wird. Zum anderen kann der Linearitätsfehler korrigiert werden, um mit den leichter zu handhabenden linearen Verfahren der Parameterschätzung arbeiten zu können. Die Korrektur von Linearitätsfehlern wird in diesem Kapitel behandelt.

6.12.1 Beispiel Pt100

Das Pt100 ist ein Platin-Widerstandsthermometer (Pt = Platin) mit einem definierten Widerstandswert von 100Ω bei einer Temperatur von 0°C (daher der Name **Pt100**). Der Widerstand eines Pt100 steigt mit der Temperatur. Bei 100°C beträgt der Widerstand beispielsweise $138,51\Omega$. Der Zusammenhang zwischen der Eingangsgröße, dem elektrischen Widerstand, und der so gemessenen Temperatur ist jedoch nur näherungsweise linear.

Die Eigenschaften eines Pt100 Widerstandes sind in der Norm DIN EN IEC 60751 (Deutsches Institut für Normung [2023](#)) festgelegt. Dort wird der Zusammenhang durch zwei Polynome für den Temperaturbereich von -200°C bis 0°C und für den Temperaturbereich von 0°C bis 850°C beschrieben.

Für den Temperaturbereich -200°C bis 0°C :

$$R_T = R_0 \cdot (1 + A \cdot T + B \cdot T^2 + C \cdot (T - 100^\circ\text{C}) \cdot T^3)$$

Für den Temperaturbereich von 0°C bis $+850^\circ\text{C}$:

$$R_T = R_0 \cdot (1 + A \cdot T + B \cdot T^2)$$

Dabei gilt:

- R_T ist der Widerstand bei der Temperatur T ,
- R_0 ist der Widerstand bei 0°C ,
- A , B und C sind Konstanten, die den spezifischen Charakter des Sensors beschreiben. Dabei sind:

- $A = 3.9083 \times 10^{-3} \text{ }^\circ\text{C}^{-1}$
- $B = -5.775 \times 10^{-7} \text{ }^\circ\text{C}^{-2}$
- $C = -4.183 \times 10^{-12} \text{ }^\circ\text{C}^{-4}$

(Deutsches Institut für Normung [2023](#), S. 13)

Im Anhang der Norm befinden sich Tabellen, die die Beziehung zwischen Temperatur und gemessenem Widerstand wiedergeben. Das Format der Tabellen erlaubt es, aus einem gemessenen Widerstandswert schnell die gemessene Temperatur zu ermitteln. Dazu sind in der ersten Spalte die Temperaturen in Zehnerschritten, in den folgenden zehn Spalten die Einerstelle eingetragen. Für Temperaturen unter Null ist die Einerstelle jeweils zu subtrahieren (erkennbar am Vorzeichen $-$), für Temperaturen über Null dagegen zu addieren (erkennbar am Vorzeichen $+$). Eine zusammengefasste Darstellung finden Sie zum Beispiel [hier](#).

(Deutsches Institut für Normung [2023](#), S. 25)

$t_{90}/^{\circ}\text{C}$	Widerstand in Ohm bei der Temperatur $t_{90}/^{\circ}\text{C}$									$R_{90}/^{\circ}\text{C}$	
	0	-1	-2	-3	-4	-5	-6	-7	-8	-9	
-200	18,52										-200
-190	22,83	22,40	21,97	21,54	21,11	20,68	20,25	19,82	19,38	18,95	-190
-180	27,10	26,67	26,24	25,82	25,39	24,97	24,54	24,11	23,68	23,25	-180
-170	31,34	30,91	30,49	30,07	29,64	29,22	28,80	28,37	27,95	27,52	-170

Abbildung 6.1: Ausschnitt der Tabelle A.1

Die Notation $t_{90}/^{\circ}\text{C}$ beruht auf der Internationalen Temperaturskala ITS-90 von 1990, die Temperaturen T_{90} in Kelvin und t_{90} in Grad Celsius definiert. Die Notation zeigt also an, dass in der Tabelle Angaben in Grad Celsius stehen. (Deutsches Institut für Normung (2023), S. 10)

6.12.2 Aufgabe Pt100-Tabelle einlesen

Für die Datenanalyse ist das tabellarische Format weniger geeignet. Die [hier](#) zusammengefasste Darstellung wurde in zwei CSV-Dateien kopiert. Lesen Sie die Dateien so ein, dass der elektrische Widerstand und die Temperatur jeweils eine aufsteigende Datenreihe bilden (z. B. als Liste, NumPy-Array oder eine Pandas-Datenstruktur). Die Dateipfade lauten:

```
'skript/01-daten/pt100-table-below-zero.csv'
'skript/01-daten/pt100-table-above-zero.csv'
```

 Tipp 15: Musterlösung Pt100-Tabelle einlesen

```
dateipfad_belowzero = '01-daten/pt100-table-below-zero.csv'
dateipfad_abovezero = '01-daten/pt100-table-above-zero.csv'
```

Dateien einlesen und betrachten. Zunächst die Datei mit Temperaturen unter 0 Grad Celsius.

```
# belowzero
belowzero = pd.read_csv(filepath_or_buffer = dateipfad_belowzero, sep = ',')

print(belowzero.head(), "\n")
print(belowzero.tail())
```

```

    Temperatur in °C Unnamed: 1 Unnamed: 2 Unnamed: 3 Unnamed: 4 Unnamed: 5 \
0                NaN          0         -1         -2         -3         -
4
1          -200.0         18,52          NaN          NaN          NaN          NaN
2          -190.0         22,83         22,4         21,97         21,54         21,11
```

3	-180.0	27,1	26,67	26,24	25,82	25,39
4	-170.0	31,34	30,91	30,49	30,07	29,64

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10
0	-5	-6	-7	-8	-9
1	NaN	NaN	NaN	NaN	NaN
2	20,68	20,25	19,82	19,38	18,95
3	24,97	24,54	24,11	23,68	23,25
4	29,22	28,8	28,37	27,95	27,52

	Temperatur in °C	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5
17	-40.0	84,27	83,87	83,48	83,08	82,69
18	-30.0	88,22	87,83	87,43	87,04	86,64
19	-20.0	92,16	91,77	91,37	90,98	90,59
20	-10.0	96,09	95,69	95,3	94,91	94,52
21	0.0	100	99,61	99,22	98,83	98,44

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10
17	82,29	81,89	81,5	81,1	80,7
18	86,25	85,85	85,46	85,06	84,67
19	90,19	89,8	89,4	89,01	88,62
20	94,12	93,73	93,34	92,95	92,55
21	98,04	97,65	97,26	96,87	96,48

Es gibt 201 Werte (von -200 bis 0). Die Daten beginnen in der Zeile mit dem Index 1 (wenn die erste Zeile als header eingelesen wird). Diese enthält nur einen Eintrag in der Spalte '0'. Die folgenden Zeilen sind von rechts nach links einzulesen. Das Dezimaltrennzeichen ist das ,.

Das Einlesen der Daten von rechts nach links kann auf viele Arten bewerkstelligt werden. Eine einzeilige Lösung beginnt damit, der Methode `pd.iloc[::-1]` eine negative Schrittweite zu übergeben.

```
df = pd.DataFrame(np.array([[3, 2, 1], [6, 5, 4]]))
print(df, "\n")
print(df.iloc[::-1])
```

	0	1	2
0	3	2	1
1	6	5	4

	0	1	2
1	6	5	4

```
0  3  2  1
```

Das führt dazu, dass die *Zeilen* des DataFrame in umgekehrter Reihenfolge ausgegeben werden. Um die *Spalten* in umgekehrter Reihenfolge auszugeben, wird der DataFrame mit der Methode `pd.T` zwei mal transponiert.

```
print(df.T.iloc[::-1].T)
```

```
    2  1  0
0   1  2  3
1   4  5  6
```

Mit der NumPy-Methode `np.flatten()` kann ein Array in eine eindimensionale Struktur reduziert werden. Dafür wird mit der Methode `pd.to_numpy()` der DataFrame als NumPy-Array ausgegeben.

```
print(df.T.iloc[::-1].T.to_numpy())
print() # leere Zeile
print(df.T.iloc[::-1].T.to_numpy().flatten())
```

```
[[1 2 3]
 [4 5 6]]
```

```
[1 2 3 4 5 6]
```

Versuchen wir es mit dem Kopf der Pt100-Daten.

```
belowzero = pd.read_csv(filepath_or_buffer = dateipfad_belowzero, sep = ',', decimal = ',',
print(belowzero.head())
print() # leere Zeile
print(belowzero.head().T.iloc[::-1].T.to_numpy().flatten())
```

```

          0      -1      -2      -3      -4      -5      -6      -7      -8      -
9
-200  18.52   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
-190  22.83  22.40  21.97  21.54  21.11  20.68  20.25  19.82  19.38  18.95
-180  27.10  26.67  26.24  25.82  25.39  24.97  24.54  24.11  23.68  23.25
-170  31.34  30.91  30.49  30.07  29.64  29.22  28.80  28.37  27.95  27.52
-160  35.54  35.12  34.70  34.28  33.86  33.44  33.02  32.60  32.18  31.76

[ nan   nan   nan   nan   nan   nan   nan   nan   nan  18.52  18.95  19.38
```



```
19.82 20.25 20.68 21.11 21.54 21.97 22.4 22.83 23.25 23.68 24.11 24.54
24.97 25.39 25.82 26.24 26.67 27.1 27.52 27.95 28.37 28.8 29.22 29.64
30.07 30.49 30.91 31.34 31.76 32.18 32.6 33.02 33.44 33.86 34.28 34.7
35.12 35.54]
```

Um die fehlenden Werte zu überspringen, wandeln wir das bisherige Ergebnis wieder in eine `pd.Series()` um und verwenden die Methode `pd.Series.dropna()`.

```
pd.Series(belowzero.head().T.iloc[:, :-1].T.to_numpy().flatten()).dropna()
```

```
9      18.52
10     18.95
11     19.38
12     19.82
13     20.25
14     20.68
15     21.11
16     21.54
17     21.97
18     22.40
19     22.83
20     23.25
21     23.68
22     24.11
23     24.54
24     24.97
25     25.39
26     25.82
27     26.24
28     26.67
29     27.10
30     27.52
31     27.95
32     28.37
33     28.80
34     29.22
35     29.64
36     30.07
37     30.49
38     30.91
39     31.34
40     31.76
```

```
41    32.18
42    32.60
43    33.02
44    33.44
45    33.86
46    34.28
47    34.70
48    35.12
49    35.54
dtype: float64
```

Und jetzt für die gesamte Datei:

```
belowzero_ohm = pd.Series(belowzero.T.iloc[:, -1].T.to_numpy().flatten()).dropna()
print(belowzero_ohm.head(), belowzero_ohm.tail(), sep = '\n')
```

```
9      18.52
10     18.95
11     19.38
12     19.82
13     20.25
dtype: float64
205     98.44
206     98.83
207     99.22
208     99.61
209    100.00
dtype: float64
```

Die Temperatur können wir einfach durch ein range-Objekt erzeugen:

```
belowzero_temperatur = pd.Series(range(-200, 1)) # range stop ist exklusiv
print(belowzero_temperatur.head(), belowzero_temperatur.tail(), sep = '\n')
```

```
0    -200
1    -199
2    -198
3    -197
4    -196
dtype: int64
196     -4
197     -3
```

```
198    -2
199    -1
200     0
dtype: int64
```

Für die Temperaturen oberhalb von 0 Grad kann das Vorgehen vereinfacht werden, da die Datei gleich aufgebaut ist und die Werte aufsteigend sortiert sind.

```
abovezero = pd.read_csv(filepath_or_buffer = dateipfad_abovezero, sep = ',', decimal = ',')

abovezero_ohm = pd.Series(abovezero.to_numpy().flatten()).dropna()
print(abovezero_ohm.head(), abovezero_ohm.tail(), sep = '\n')
```

```
0    100.00
1    100.39
2    100.78
3    101.17
4    101.56
dtype: float64
846    389.31
847    389.60
848    389.90
849    390.19
850    390.48
dtype: float64
```

Beide Datensätze enthalten einen Eintrag für die Temperatur 0 Grad, sodass dieser in der zweiten Datei entfernt werden kann.

```
abovezero_ohm = abovezero_ohm[1:]
print(abovezero_ohm.head(), abovezero_ohm.tail(), sep = '\n')
```

```
1    100.39
2    100.78
3    101.17
4    101.56
5    101.95
dtype: float64
846    389.31
847    389.60
848    389.90
849    390.19
```

```
850      390.48
dtype: float64
```

Die Temperatur wird wieder durch ein range-Objekt erzeugt.

```
abovezero_temperatur = pd.Series(range(1, 851)) # range stop ist exklusiv
```

Die aus beiden Datensätzen erzeugten Series können zusammengefasst werden:

```
pt100 = pd.DataFrame({'Temperatur': pd.concat([belowzero_temperatur, abovezero_temperatur])})
print(pt100.head(), pt100.tail(), sep = '\n')

print(pt100.info())
```

```

   Temperatur  Ohm
0         -200  18.52
1         -199  18.95
2         -198  19.38
3         -197  19.82
4         -196  20.25
   Temperatur  Ohm
1046         846  389.31
1047         847  389.60
1048         848  389.90
1049         849  390.19
1050         850  390.48
<class 'pandas.DataFrame'>
RangeIndex: 1051 entries, 0 to 1050
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Temperatur  1051 non-null    int64
1   Ohm         1051 non-null    float64
dtypes: float64(1), int64(1)
memory usage: 16.6 KB
None
```

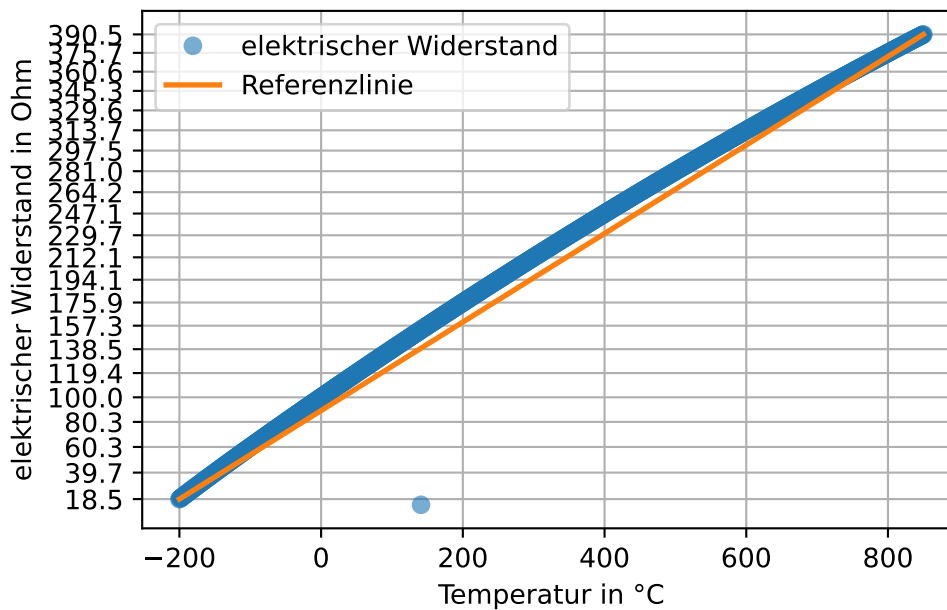
6.12.3 Nichtlinearität darstellen

Mit den Daten aus Tipp 15 kann die Nichtlinearität des Sensors dargestellt werden.

```
plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrischer Widerstand')
plt.plot([pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]], [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
         label = 'Referenzlinie', linestyle = '-', marker = None)
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()
```



Es fällt auf, dass die Daten einen Fehlwert enthalten. Die Position des Fehlwerts wird mit zwei Pandas-Methoden bestimmt. `pd.diff()` gibt die Differenz jedes Werts zu seinem Vorgänger zurück. `pd.idxmin()` gibt den Zeilenindex (genauer das label) des kleinsten Werts zurück. (Läge der Fehlwert oberhalb der Linie, würde `pd.idxmax()` verwendet werden.)

```
# := ist der sog. Walross-Operator
print(( position_fehlwert := pt100['Ohm'].diff().idxmin() ))
print(pt100.iloc[list(range(position_fehlwert - 2, position_fehlwert + 3))])
```

341

	Temperatur	Ohm
339	139	153.21
340	140	153.58
341	141	13.96
342	142	154.33
343	143	154.71

Der korrekte Wert kann aus der DIN-Norm (Deutsches Institut für Normung [2023](#), S. 26) abgelesen werden (153,96). Man könnte den Wert aber auch interpolieren (siehe Beispiel).

i Hinweis 17: Interpolation

Die einfachste Form der Interpolation ist die lineare Interpolation.

```
interpolate_me = pt100.loc[position_fehlwert - 1 : position_fehlwert + 1, 'Ohm'].copy()
interpolate_me[position_fehlwert] = np.nan
print(interpolate_me, "\n")
```

```
print("linear interpolierter Wert:", interpolate_me.interpolate(), sep = "\n")
```

```
340    153.58
341         NaN
342    154.33
Name: Ohm, dtype: float64
```

```
linear interpolierter Wert:
340    153.580
341    153.955
342    154.330
Name: Ohm, dtype: float64
```

Durch Runden wird das korrekte Ergebnis erreicht (darauf verlassen sollte man sich aber nicht).

```
print("linear interpolierter Wert:", interpolate_me.interpolate().round(2), sep = "\n")
korrekter_wert = interpolate_me.interpolate().round(2)[position_fehlwert]
```

```
linear interpolierter Wert:
340    153.58
341    153.96
342    154.33
Name: Ohm, dtype: float64
```

Eine andere Option ist die nichtlineare Interpolation (die nicht Inhalt dieses Bausteins ist).

Beispiel 6.2: Nichtlineare Interpolation

Die nichtlineare Interpolation benötigt mehr Datenpunkte. Wir versuchen die kubische Interpolation:

```
interpolate_me_again = pt100.loc[position_fehlwert - 3 : position_fehlwert + 4, 'Ohm']
interpolate_me_again[position_fehlwert] = np.nan
print("polynomial interpolierter Wert:", interpolate_me_again.interpolate(method = 'polynomial'))
```

polynomial interpolierter Wert:

```
338    152.830000
339    153.210000
340    153.580000
341    153.952213
342    154.330000
343    154.710000
344    155.080000
345    155.460000
```

Name: Ohm, dtype: float64

Das Ergebnis ist ungenauer. Warum das so ist, erfahren Sie im [Methodenbaustein Datenfitting und Datenoptimierung](#).

Der Fehlwert (dessen Position in obigem Beispiel bestimmt wird) wird korrigiert:

```
pt100.loc[position_fehlwert, 'Ohm'] = korrekter_wert
print(pt100.loc[position_fehlwert - 1 : position_fehlwert + 1, 'Ohm'])
```

```
340    153.58
341    153.96
342    154.33
```

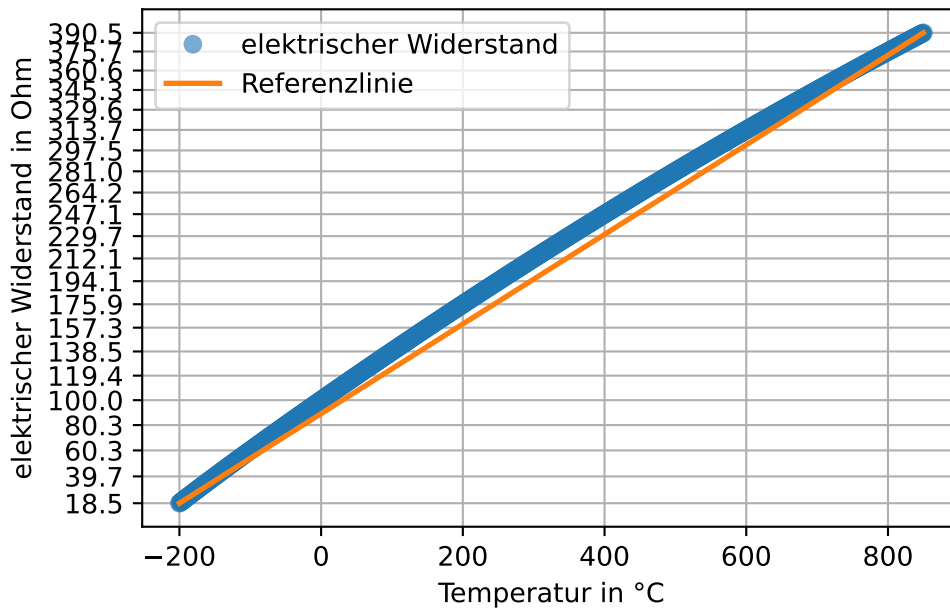
Name: Ohm, dtype: float64

Sodann kann die Nichtlinearität erneut dargestellt werden:

```
plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrischer Widerstand')
plt.plot([pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]], [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
         label = 'Nichtlinearität')
plt.yticks(pt100['Ohm'][:50]);
```

```
plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()
```



6.12.4 Nichtlinearität quantifizieren

Zwei Methoden zur Quantifizierung der Nichtlinearität sind die Festpunkt- und die Toleranzbandmethode.

Festpunktmethode

! Wichtig 11: Festpunktmethode

Die Endpunkte der realen Kennlinie werden durch eine Gerade verbunden. Der Linearitätsfehler ist das Maximum der Abweichung der Kennlinie zu dieser Geraden. (Grote und Feldhusen 2011, W2-W3 (S. 1661-1662))

Dazu bestimmen wir zunächst die Vorhersagewerte einer Geraden durch die Endpunkte.


```

lm = poly.polyfit(
    x = [pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]],
    y = [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
    deg = 1)
print(lm.round(2)) # intercept + slope

vorhersagewerte = poly.polyval(x = pt100['Temperatur'], c = lm)

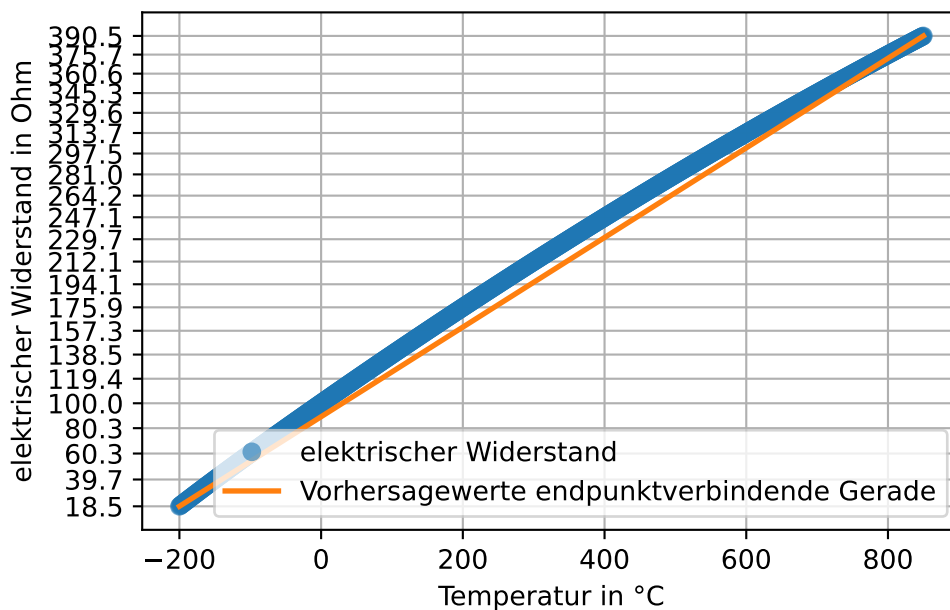
plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrischer Widerstand')
plt.plot(pt100['Temperatur'], vorhersagewerte, linewidth = 2, label = 'Vorhersagewerte endpunktverbindende Gerade')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()

```

[89.37 0.35]



Aus der Differenz des gemessenen elektrischen Widerstands und der linearen Vorhersagewerte kann der maximale Linearitätsfehler bestimmt werden.

```
linearitätsfehler = (pt100['Ohm'] - vorhersagewerte).abs().max()
print(f"Linearitätsfehler nach Festpunktmethode: {linearitätsfehler:.2f} Ohm.")
```

Linearitätsfehler nach Festpunktmethode: 16.43 Ohm.

Toleranzbandmethode

! Wichtig 12: Toleranzbandmethode

Bei der Toleranzbandmethode wird eine Gerade so durch die Messpunkte gelegt, dass die Summe der quadrierten Abweichungen der Messpunkte zu dieser Geraden (Methode der kleinsten Quadrate) oder die größte einzelne Abweichung ([Tschebyscheff-Approximation](#)) minimiert wird. Die Größe des Linearitätsfehlers ist die maximale senkrechte Entfernung der Kennlinie zu dieser Ausgleichsgeraden. (Grote und Feldhusen 2011, W3 (S. 1662))

Das Prinzip der Methode der kleinsten Quadrate haben wir bereits kennengelernt.

```
lm = poly.polyfit(
    x = pt100['Temperatur'],
    y = pt100['Ohm'],
    deg = 1)
print(lm.round(2)) # intercept + slope

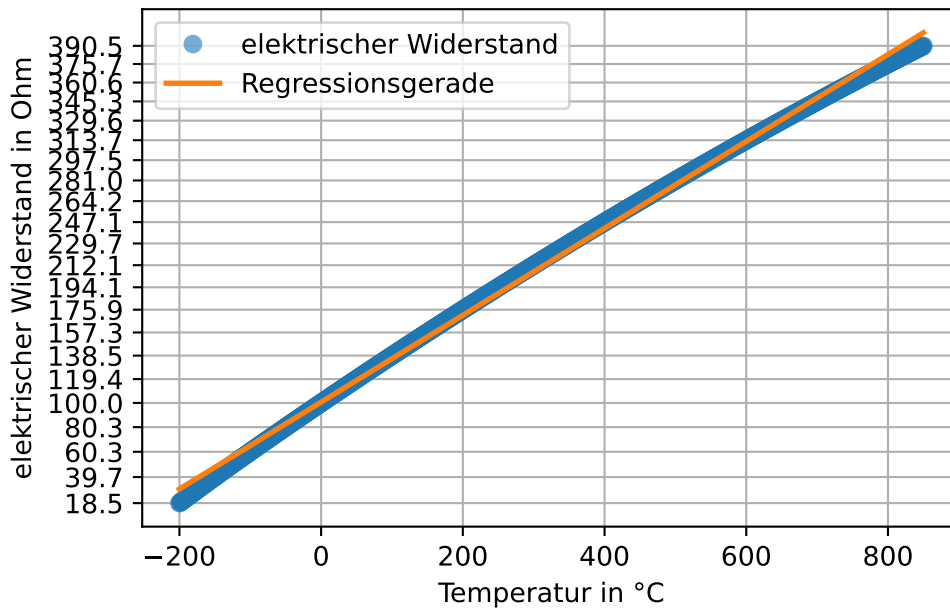
vorhersagewerte = poly.polyval(x = pt100['Temperatur'], c = lm)

plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrisch')
plt.plot(pt100['Temperatur'], vorhersagewerte, linewidth = 2, label = 'Regressionsgerade')
plt.yticks(pt100['Ohm'][:,50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()
```

[100.67 0.35]



Aus der Differenz des gemessenen elektrischen Widerstands und der linearen Vorhersagewerte kann der maximale Linearitätsfehler (= das größte Residuum) bestimmt werden.

```
linearitätsfehler = (pt100['Ohm'] - vorhersagewerte).abs().max()
print(f"Linearitätsfehler nach Toleranzbandmethode: {linearitätsfehler:.2f} Ohm.")
```

Linearitätsfehler nach Toleranzbandmethode: 11.45 Ohm.

6.13 Die Zweipunktkalibrierung

Die Parameterschätzung mit linearer Regression nach der Toleranzbandmethode führt zur besten linearen Approximation von Messwerten (da die als Fehler zu interpretierenden Residuen minimiert werden). Nicht-lineares Messverhalten oder Daten mit größeren Messungenauigkeiten behaftete Daten führen aber fast zwangsläufig einen Nullpunktfehler in die geschätzte Gerade ein.

Die Festpunktmethode umgeht dieses Problem, führt aber zu einem größeren Linearitätsfehler. Die Kalibrierung nach der Festpunktmethode kann über zwei bekannte Fixpunkte berechnet werden, **wenn ein linearer Zusammenhang in den Daten besteht.**

```

# Daten erzeugen
messgröße = np.arange(0, 11) + 7
messwerte = messgröße + 4 - messgröße / 3

# bekannte Referenzpunkte
x1 = 4
x2 = 8
y1 = messgröße[x1]
y2 = messgröße[x2]

# Ausgabe
print(f"Wahre Werte:\n{messgröße}")
print(f"Messwerte:\n{messwerte}")
print()

# Zweipunktkalibrierung
wahres_minimum = messgröße.min()
wahres_maximum = messgröße.max()

a = (wahres_maximum - wahres_minimum) / (messwerte.max() - messwerte.min())
b = wahres_minimum - a * messwerte.min()

korrigierte_messwerte = a * messwerte + b

print(f"Zweipunktkalibrierung\n",
      f"a = {a:.1f}\n",
      f"b = {b:.1f}\n",
      f"\na * messwerte + b\n{korrigierte_messwerte}",
      sep = '')

```

Wahre Werte:

[7 8 9 10 11 12 13 14 15 16 17]

Messwerte:

[8.66666667	9.33333333	10.	10.66666667	11.33333333	12.
12.66666667	13.33333333	14.	14.66666667	15.33333333]	

Zweipunktkalibrierung

a = 1.5

b = -6.0

a * messwerte + b

[7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17.]

Die Variable a ist der Korrekturfaktor für den Empfindlichkeitsfehler. Die Variable b ist die Nullpunktkorrektur, nachdem der Empfindlichkeitsfehler korrigiert wurde. Der Empfindlichkeitsfehler kann als $\frac{1}{a} - 1$ ermittelt werden. Der Nullpunktfehler kann als $-1 * \frac{b}{a}$ ermittelt werden.

```
print(f"Empfindlichkeitsfehler: {round( ((1 / a) - 1) * 100, 2)} %")
print(f"Nullpunktfehler: {round(-1 * b / a, 2)}")
```

Empfindlichkeitsfehler: -33.33 %

Nullpunktfehler: 4.0

6.14 Eis kochen



Abbildung 6.2: Versuchsaufbau Eis kochen

In einem thermodynamischen Feldlabor wurde ein induktives Heizelement benutzt, um Eiswasser in einem metallischen Flüssigkeitsbehälter mit hoher Wärmeleitfähigkeit zum Kochen zu bringen. Mit einer nichtleitenden Halterung wurden zwei Thermoelemente eines Widerstandsthermometers fixiert. Ein TCTempX16 und eine automatische Protokolleinheit zeichneten die Daten auf.

Als Übung das gekochte Eis verwenden. (man muss den Nullpunkt und 100 Grad als Referenzwerte nehmen). Das Problem ist, dass das Eiskochen auch nichtlinear ist (ab 50 °C). Beachten, man hat keinen Referenzwert für den Nullpunkt (der liegt bei -273 °C), man muss also mit linearer Regression schätzen. Vermutlich

muss man dafür nicht in Kelvin umrechnen, wegen der absoluten Verschiebung.
→ Dass man nicht verschieben braucht, haben wir ja für die Berechnung ohne Messwerte für den Nullpunkt gezeigt.

- Die Nichtlinearität ist kein Fehler des Sensorsystems, sondern ein physikalischer Effekt. Wie gehen wir damit um? → man könnte sagen, dass der Effekt über den linear geschätzten Spannf Fehler approximiert wird. Zwischen nichtlinearer Messwertekurve und dem linear geschätzten Spannf Fehler entsteht dann ein Linearitätsfehler

Das Problem ist, dass die Daten bereinigt werden müssen, ohne sie zu interpolieren oder zu glätten → mit linearem Trend und sd arbeiten oder auf den Baustein datenfitting und datenoptimierung verweisen.

Datei einlesen und Daten darstellen

Die Messreihe ist in der Datei '01-daten/Eis_Messung_1.xlsx' im Tabellenblatt 'T15949 MultiChannel - Daten' gespeichert.

Lesen Sie die Datei ein und bereiten Sie die Daten für die weitere Analyse auf.

- Welches Format haben die Daten?
- Gibt es fehlende Werte und falls ja, wie sind diese kodiert?
- Gibt es fehlerhafte Werte?
- to do: Messgenauigkeit aus den Metadaten recherchieren lassen (Als Übungsaufgabe unten)

```
eis = pd.read_excel(io = '01-daten/Eis_Messung_1.xlsx', sheet_name = 'T15949 MultiChannel - D
print(eis.head(n = 10))
```

	Gerätename:	Unnamed: 1	\
0	Gerätebeschreibung:	NaN	
1	Serien-Nummer:	NaN	
2	Geräte-ID:	NaN	
3	NaN	NaN	
4	NaN	NaN	
5	Datum	Zeit	
6	2025-09-25 16:12:00	2025-09-25 16:12:00	
7	2025-09-25 16:12:01	2025-09-25 16:12:01	
8	2025-09-25 16:12:02	2025-09-25 16:12:02	
9	2025-09-25 16:12:03	2025-09-25 16:12:03	
		TCTempX16	\

```

0 16-Kanal Thermoelement Temperatur-Datenlogger
1                                T15949
2                                MultiChannel
3                                NaN
4                                Kanal 2
5                                Thermoelement 1 (°C)
6                                1
7                                1
8                                1
9                                1

```

```

                                TCTempX16.1
0 16-Kanal Thermoelement Temperatur-Datenlogger
1                                T15949
2                                MultiChannel
3                                NaN
4                                Kanal 4
5                                Thermoelement 2 (°C)
6                                0.5
7                                0.5
8                                0.5
9                                0.5

```

Im Kopf des Tabellenblatts stehen Metadaten der Messung. Es gibt vier Spalten mit Daten: Datum und Zeit sowie die Messreihen der beiden Thermoelemente.

```

eis = pd.read_excel(io = '01-daten/Eis_Messung_1.xlsx', sheet_name = 'T15949 MultiChannel - 1')
print(eis.info())

```

```

<class 'pandas.DataFrame'>
RangeIndex: 565 entries, 0 to 564
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Datum                 565 non-null   datetime64[us]
1   Zeit                  565 non-null   datetime64[us]
2   Thermoelement 1 (°C) 565 non-null   float64
3   Thermoelement 2 (°C) 565 non-null   float64
dtypes: datetime64[us](2), float64(2)
memory usage: 17.8 KB
None

```

Die Daten wurden korrekt eingelesen. Mit der Methode `pd.describe()` wird der Wertebereich der Spalten überprüft, um numerisch kodierte fehlende oder fehlerhafte Werte zu identifizieren.

```
print(eis.describe())
```

	Datum	Zeit \
count	565	565
mean	2025-09-25 16:16:47.024778	2025-09-25 16:16:47.024778
min	2025-09-25 16:12:00	2025-09-25 16:12:00
25%	2025-09-25 16:14:25	2025-09-25 16:14:25
50%	2025-09-25 16:16:47	2025-09-25 16:16:47
75%	2025-09-25 16:19:09	2025-09-25 16:19:09
max	2025-09-25 16:21:32	2025-09-25 16:21:32
std	NaN	NaN

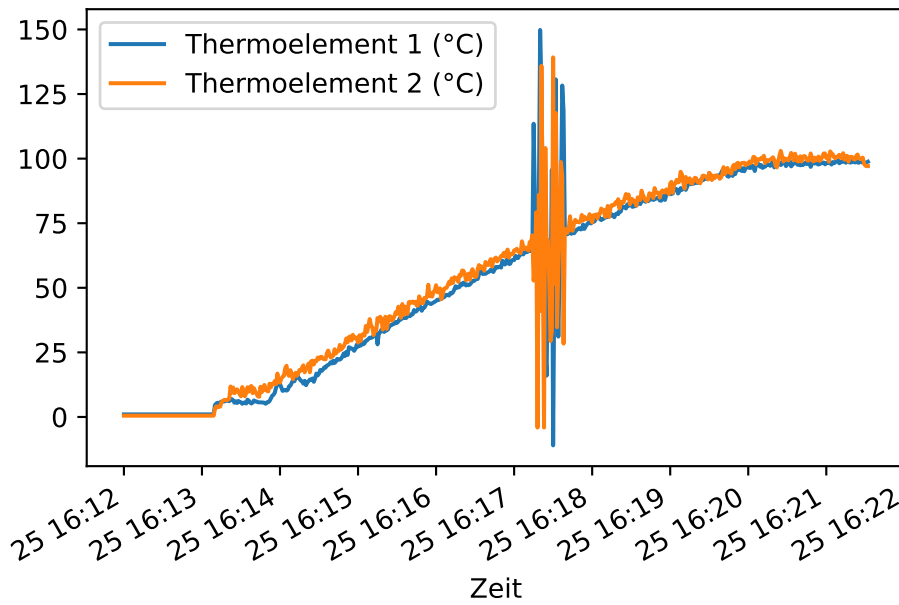
	Thermoelement 1 (°C)	Thermoelement 2 (°C)
count	565.000000	565.000000
mean	53.581239	55.451858
min	-11.000000	-4.100000
25%	14.900000	20.300000
50%	56.200000	58.300000
75%	90.300000	91.200000
max	149.800000	139.200000
std	36.886184	36.542679

Die Spalten Datum und Zeit enthalten die selben Informationen. Eine der beiden Spalten kann deshalb entfernt werden. Die Temperaturmessungen sollten näher betrachtet werden. Eishaltiges Wasser, das zum Kochen gebracht wird, sollte sich in einem Temperaturbereich von 0 bis 100 °C bewegen.

```
eis.drop(labels = 'Datum', axis = 1, inplace = True)
```

Wir stellen den Datensatz grafisch dar.

```
eis.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'])
```

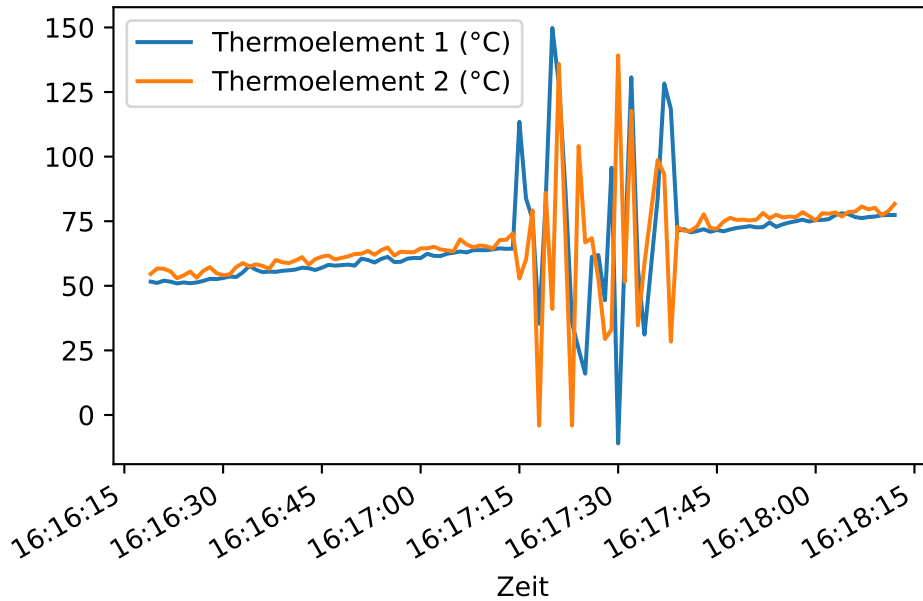



Etwa in der Mitte des Datensatzes verzeichnen beide Sensoren extreme Fehlwerte. Diese sollen entfernt werden.

Fehlwerte entfernen

Zunächst betrachten wir den Bereich genauer.

```
n_zeilen = eis.shape[0]
eis.iloc[int(n_zeilen * 0.45): int(n_zeilen * 0.65)].plot(x = 'Zeit', y = ['Thermoelement 1
```



In einem bestimmten Abschnitt scheinen keine gültigen Werte vorzuliegen und diese sollen als ungültig markiert werden. Dazu betrachten wir zunächst die typische Veränderung der Messwerte, also die Differenz jedes Werts zu seinem Vorgänger mit der Methode `pd.Series.diff()`. Ebenfalls wird die Veränderung in studentisierten z-Werten ausgedrückt.

Die z-Werte werden mit der scipy-Funktion `scipy.stats.zscore(a, ddof = 1, nan_policy = 'omit')` ermittelt. `a` steht für array-artige Daten, `ddof = 1` spezifiziert die Stichprobenstandardabweichung und das Argument `nan_policy = 'omit'` wird verwendet, um mit dem Wert `np.nan` an der ersten Stelle arbeiten zu können, der aus der Berechnung der Veränderung entsteht, da für den ersten Wert einer Reihe kein gültiger Wert berechnet werden kann.

⚠ Warning 9: np.diff() und pd.diff()

NumPy und Pandas verfügen über eine Funktion `diff`. Diese verhalten sich standardmäßig unterschiedlich. Um die Länge einer Datenreihe mit `np.diff()` zu erhalten, können die Parameter `prepend` oder `append` verwendet werden, um der Datenreihe vor Ausführung der Operation einen Wert voranzustellen oder einen Wert anzuhängen. So bleiben die Länge der Datenreihe und die Indexpositionen der Werte erhalten.

```
array = np.array([1, 3, 5, 6])
print(np.diff(array))
series = pd.Series(array)
print(series.diff())
print("\n", np.diff(array, prepend = np.nan))
```

```
[2 2 1]
0    NaN
1    2.0
2    2.0
3    1.0
dtype: float64
```

```
[nan 2. 2. 1.]
```

```
# absolute Differenzen ermitteln
diff_thermoelement_1 = eis['Thermoelement 1 (°C)'].diff()
diff_thermoelement_2 = eis['Thermoelement 2 (°C)'].diff()

# absolute z-scores ermitteln
abs_z_score_thermoelement_1 = np.abs(scipy.stats.zscore(diff_thermoelement_1, ddof = 1, nan_
abs_z_score_thermoelement_2 = np.abs(scipy.stats.zscore(diff_thermoelement_2, ddof = 1, nan_

# plotten
fig = plt.figure(figsize = (7.5, 7.5)) # sharex = True

ax = fig.add_subplot(2, 2, 1)
ax.plot(diff_thermoelement_1, color = 'C0', label = 'Thermoelement 1')
ax.set_xlabel(xlabel = 'Index')
ax.set_ylabel(ylabel = 'Temperaturdifferenz in °C')
ax.grid()

ax = fig.add_subplot(2, 2, 2)
ax.plot(abs_z_score_thermoelement_1, color = 'C0')
```

```

ax.set_xlabel(xlabel = 'Index')
ax.set_ylabel(ylabel = 'absolute z-Werte')
ax.grid()

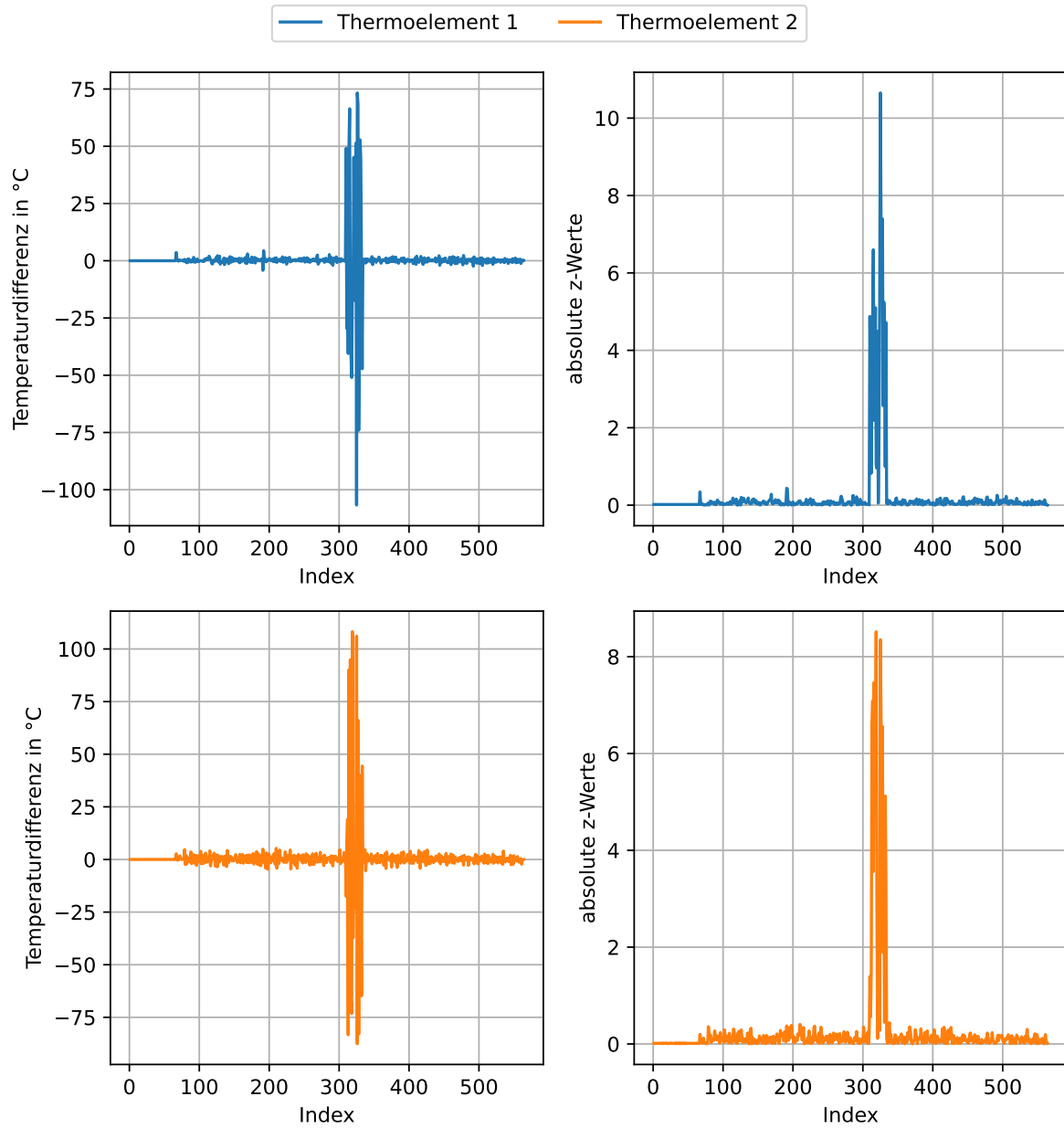
ax = fig.add_subplot(2, 2, 3)
ax.plot(diff_thermoelement_2, color = 'C1', label = 'Thermoelement 2')
ax.set_xlabel(xlabel = 'Index')
ax.set_ylabel(ylabel = 'Temperaturdifferenz in °C')
ax.grid()

ax = fig.add_subplot(2, 2, 4)
ax.plot(abs_z_score_thermoelement_2, color = 'C1')
ax.set_xlabel(xlabel = 'Index')
ax.set_ylabel(ylabel = 'absolute z-Werte')
ax.grid()

fig.legend(loc = 'outside upper center', ncols = 2, bbox_to_anchor = (0.5, 1.05))

plt.tight_layout()
plt.show()

```



Die Position der ungültigen Werte kann sowohl über eine sinnvolle Schwelle der (absoluten) Temperaturveränderung oder der absoluten z-Werte bestimmt werden. In diesem Fall verwenden wir die absolute Temperaturveränderung und wählen den Schwellwert 10. Es wird die Position des ersten und des letzten Werts, der oberhalb dieses Schwellwerts liegt, bestimmt.

```

# absolute Änderung bestimmen
diff_thermoelement_1 = eis['Thermoelement 1 (°C)'].diff().abs()
diff_thermoelement_2 = eis['Thermoelement 2 (°C)'].diff().abs()

# Schwellwert festlegen und prüfen
schwellwert = 10
bool_series_thermoelement_1 = diff_thermoelement_1 > schwellwert
bool_series_thermoelement_2 = diff_thermoelement_2 > schwellwert

# ersten und letzten Wert größer Schwellwert finden
## Thermoelement 1
### Wo steht nicht False?
### np.nonzero() gibt ein Tupel zurück
### Dieses enthält für jede Dimension ein array der Indexwerte
positionen_thermoelement_1 = np.nonzero(bool_series_thermoelement_1)

## Thermoelement 2
### Wo steht nicht False?
### np.nonzero() gibt ein Tupel zurück
### Dieses enthält für jede Dimension ein array der Indexwerte
positionen_thermoelement_2 = np.nonzero(bool_series_thermoelement_2)

# Ausgabe
## Thermoelement 1
print("Thermoelement 1")
print(51 * "=")
print("Anzahl der Temperaturänderungen mit Betrag > 10:", bool_series_thermoelement_1.sum())
print(f"Die Position des ersten Werts: {positionen_thermoelement_1[0][0]}")
print(f"Die Position des letzten Werts: {positionen_thermoelement_1[0][-1]}")

## Thermoelement 2
print("\nThermoelement 2")
print(51 * "=")
print("Anzahl der Temperaturänderungen mit Betrag > 10:", bool_series_thermoelement_2.sum())
print(f"Die Position des ersten Werts: {positionen_thermoelement_2[0][0]}")
print(f"Die Position des letzten Werts: {positionen_thermoelement_2[0][-1]}")

```

Thermoelement 1

```

=====
Anzahl der Temperaturänderungen mit Betrag > 10: 20
Die Position des ersten Werts: 310
Die Position des letzten Werts: 333

```

Thermoelement 2

```
=====
Anzahl der Temperaturänderungen mit Betrag > 10: 20
Die Position des ersten Werts: 310
Die Position des letzten Werts: 333
```

Für beide Thermoelemente kann der gleiche Bereich als ungültig markiert werden. Vor der weiteren Bearbeitung wird eine Kopie des aufgeräumten Datensatzes erstellt.

```
von = positionen_thermoelement_1[0][0]
bis = positionen_thermoelement_1[0][-1]
eis.loc[von : bis, ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)']] = np.nan

# Kopie von eis anlegen
eis_backup = eis.copy()
```

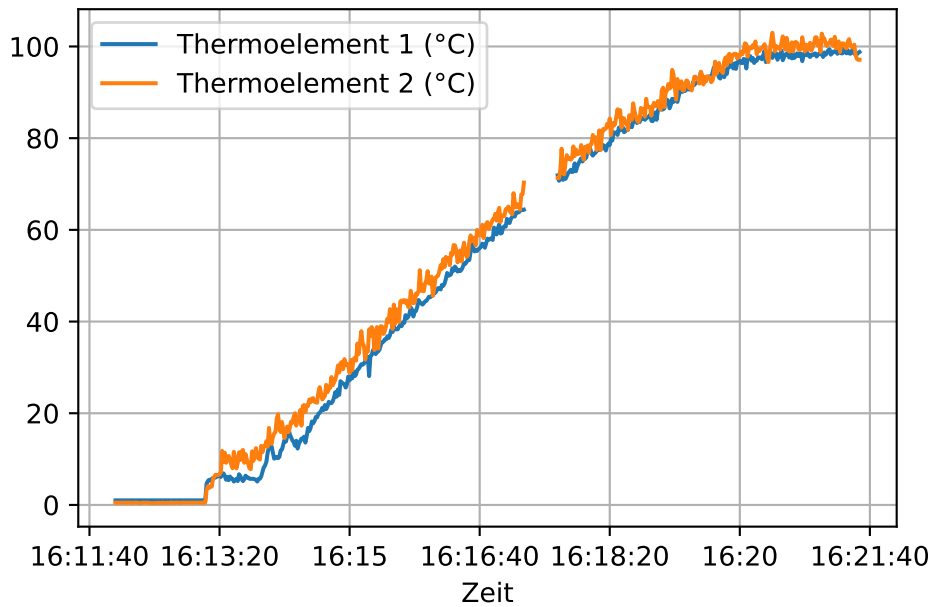
Der Datensatz wird dargestellt. (In der Darstellung mit `pd.plot()` ist die automatisch gewählte x-Achsenbeschriftung unansehnlich. Deshalb wird ein Datenobjekt für die Darstellung angelegt und die Spalte 'Zeit' auf die Uhrzeit reduziert.)

```
# Darstellung mit automatischer Achsenbeschriftung
# eis.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)

plotting_data = eis.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)

plt.show()
```

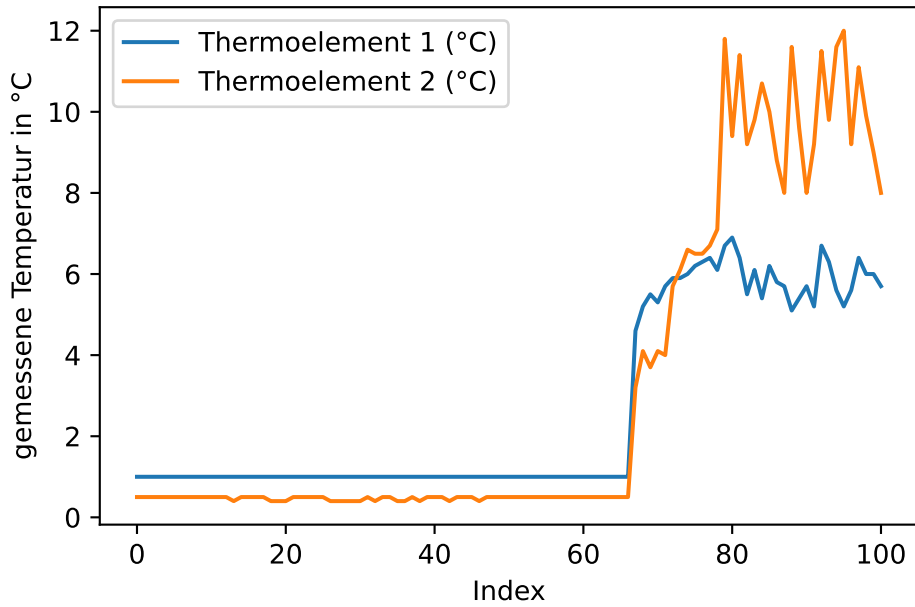


Referenzpunkte bestimmen

Aus diesen Daten können der Nullpunkt- und der Spannfehler quantifiziert werden. Als Referenzpunkte sind die Temperaturen 0 und 100 Grad Celsius bekannt: Solange ohne Energiezufuhr Eis im Wasser schwimmt, hat es 0 °C, und bei 100 °C erreicht das Wasser seinen Siedepunkt.

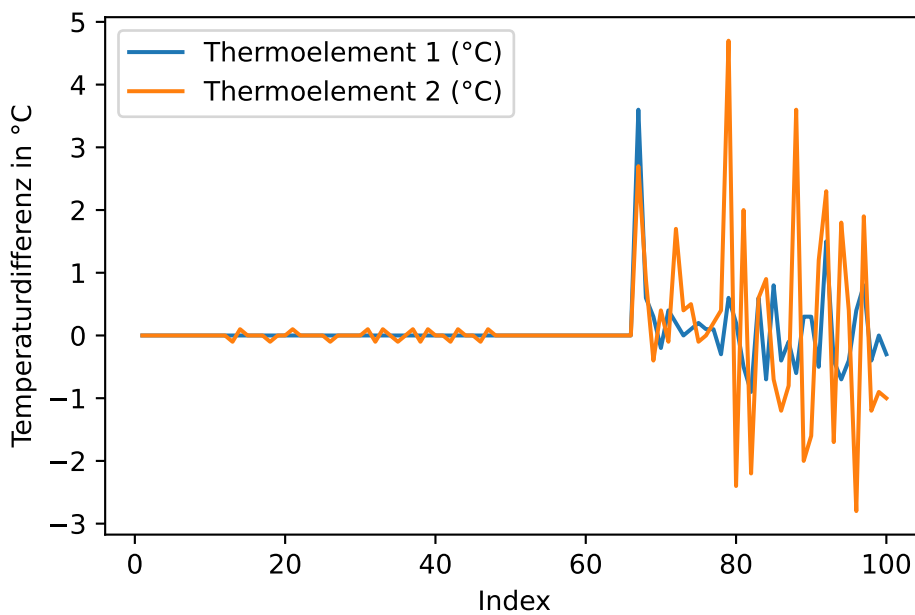
Für die wahre Temperatur 0 °C sind viele Messpunkte verfügbar. Schauen wir uns die ersten 100 Messwerte an:

```
eis.loc[0 : 100, ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)']].plot(xlabel = 'Index', y1
```

Der Zeitpunkt, ab dem die Wärmezufuhr beginnt, kann z. B. mit der Methode `pd.diff()` ermittelt werden:

```
eis.loc[0 : 100, ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)']].diff().plot(xlabel = 'Index')
```



Da das Thermoelement 1 leichte Messabweichungen aufweist, wird ein Schwellwert etwas größer als 0 gewählt. Damit wird die Position des ersten Werts bestimmt, der oberhalb des Schwellwerts liegt.

```
# absolute Änderung bestimmen
diff_thermoelement_1 = eis.loc[0 : 100, 'Thermoelement 1 (°C)'].diff().abs()
diff_thermoelement_2 = eis.loc[0 : 100, 'Thermoelement 2 (°C)'].diff().abs()

# Schwellwert festlegen und prüfen
schwellwert = 0.2
bool_series_thermoelement_1 = diff_thermoelement_1 > schwellwert
bool_series_thermoelement_2 = diff_thermoelement_2 > schwellwert

# ersten Wert größer Schwellwert finden
## Thermoelement 1
### Wo steht nicht False?
### np.nonzero() gibt ein Tupel zurück
### Dieses enthält für jede Dimension ein array der Indexwerte
positionen_thermoelement_1 = np.nonzero(bool_series_thermoelement_1)

## Thermoelement 2
### Wo steht nicht False?
### np.nonzero() gibt ein Tupel zurück
### Dieses enthält für jede Dimension ein array der Indexwerte
positionen_thermoelement_2 = np.nonzero(bool_series_thermoelement_2)

# Ausgabe
## Thermoelement 1
print("Thermoelement 1")
print(f"Die Position des ersten Werts: {positionen_thermoelement_1[0][0]}")

## Thermoelement 2
print("\nThermoelement 2")
print(f"Die Position des ersten Werts: {positionen_thermoelement_2[0][0]}")
```

```
Thermoelement 1
Die Position des ersten Werts: 67
```

```
Thermoelement 2
Die Position des ersten Werts: 67
```

Die Wärmezufuhr beginnt also bei dem Wert an Indexposition 67. Der Nullpunkt der Messreihe liegt somit an der Indexposition 67 - 1.

```
start_wärmezufuhr = 67
start = start_wärmezufuhr - 1
```

Somit kann der Nullpunktfehler nun für beide Thermoelemente aus der Differenz zwischen dem Mittelwert der gemessenen Werte bis *exklusiv* Indexposition 67 und dem wahren Wert 0 direkt berechnet werden. Diese Position haben wir in der Variable start gespeichert, die für beide Thermoelemente gilt.

⚠ Warning 10: Slicing mit pd.loc[]

Das Slicing mit pd.loc[] ist inklusiv.

```
nullpunktfehler_thermoelement_1 = eis.loc[0 : start, 'Thermoelement 1 (°C)'].mean() - 0
nullpunktfehler_thermoelement_2 = eis.loc[0 : start, 'Thermoelement 2 (°C)'].mean() - 0

print(f"Der Nullpunktfehler von Thermoelement 1 beträgt: {nullpunktfehler_thermoelement_1}")
print(f"Der Nullpunktfehler von Thermoelement 2 beträgt: {nullpunktfehler_thermoelement_2:.2f}")
```

```
Der Nullpunktfehler von Thermoelement 1 beträgt: 1.0
Der Nullpunktfehler von Thermoelement 2 beträgt: 0.48
```

Damit kann der Nullpunktfehler für beide Messreihen korrigiert werden.

```
eis['Thermoelement 1 (°C)'] -= nullpunktfehler_thermoelement_1
eis['Thermoelement 2 (°C)'] -= nullpunktfehler_thermoelement_2
```

Auch für die wahre Temperatur 100 Grad sind viele Messpunkte verfügbar. Schauen wir uns die letzten 120 Messwerte an:

hier müssen wir mal aus dem m-datenfitting und datenoptimierung entlehnen

```
anzahl = 120

eis.loc[eis.shape[0] - anzahl : , ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)']].plot(alpha=0.5)
plt.hlines(y = 100, xmin = eis.shape[0] - anzahl, xmax = eis.shape[0], linestyle = 'dashed')

# Daten glätten
window = 9
```

```

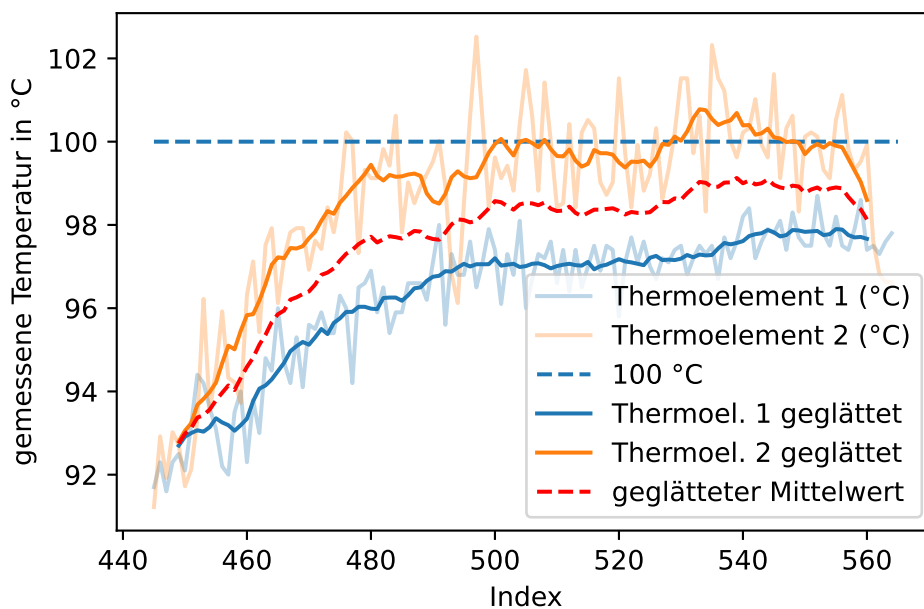
weights = np.ones(window) / window
thermoelement_1_glatt = np.convolve(eis['Thermoelement 1 (°C)'], weights, mode = 'valid')
thermoelement_2_glatt = np.convolve(eis['Thermoelement 2 (°C)'], weights, mode = 'valid')

plt.plot(eis.index[eis.shape[0] - anzahl + (window-1)//2 : -(window//2)], thermoelement_1_glatt)
plt.plot(eis.index[eis.shape[0] - anzahl + (window-1)//2 : -(window//2)], thermoelement_2_glatt)

mittelwert_der_sensoren = (thermoelement_1_glatt[eis.shape[0] - anzahl : ] + thermoelement_2_glatt[eis.shape[0] - anzahl : ]) / 2
plt.plot(eis.index[eis.shape[0] - anzahl + (window-1)//2 : -(window//2)], mittelwert_der_sensoren)

plt.legend()
plt.show()

```



Der Punkt, an dem das Wasser siedet, ist nicht so einfach zu bestimmen. Starke Messabweichungen erzeugen ein verrauschtes Bild. Um die grafische Analyse zu erleichtern wurden für beide Messreihen der gleitende Durchschnitt sowie der Mittelwert der geglätteten Messreihen eingezeichnet. Die Techniken dafür werden im [Methodenbaustein Datenfitting und Datenoptimierung](#) vermittelt.

```

print(f"Maximum der gemittelten, geglätteten Messreihen: {mittelwert_der_sensoren.max():.2f}")
print(f"An Indexposition: {np.where(mittelwert_der_sensoren == mittelwert_der_sensoren.max())}")

```

Maximum der gemittelten, geglätteten Messreihen: 99.13
An Indexposition: 535

Zu bedenken ist:

- MF: Die Temperatur, bei der Wasser beim Luftdruck in Deutschland kocht, ist tatsächlich eher 99,4°C. [Dieser Rechner](#) und 1007 hPa: 99.8
- Wasser kocht nicht sprunghaft als ganzes. Erst siedet es und dabei kocht das Wasser unten im Topf früher als oben, erkennbar an den Luftbläschen. Die Messung kann durch die Luftbläschen verfälscht werden (aber so stark? Die Messung ist im Allgemeinen extrem ungenau). Der Temperaturverlauf bis zum Erreichen der Siedetemperatur muss aber linear sein (siehe Grafik von Marc).

Behelfsweise bestimmen wir den Referenzpunkt für 100 °C, indem die Werte zwischen dem Indexwert 500 und bis zu den letzten 10 Werten des Datensatzes gemittelt werden. Anschließend wird der Index bestimmt, an dem die Messwerte diesen Wert erstmalig erreichen. (*Hinweis: Die Daten wurden bereits um ermittelten Nullpunktfehler korrigiert.*)

```
thermoelement_1_max = eis['Thermoelement 1 (°C)'].iloc[500 : -10].mean()
thermoelement_2_max = eis['Thermoelement 2 (°C)'].iloc[500 : -10].mean()

# ge = greater or equal, idxmax = find first occurrence of maximum value (0 and 1)
ende_thermoelement_1 = eis['Thermoelement 1 (°C)'].ge(thermoelement_1_max).idxmax()
ende_thermoelement_2 = eis['Thermoelement 2 (°C)'].ge(thermoelement_2_max).idxmax()

print(f"Gemitteltes Maximum Thermoelement 1: {thermoelement_1_max:.2f} °C",
      f"Erstmalig erreicht an Position: {ende_thermoelement_1 }", sep = "\n")
print(f"Gemitteltes Maximum Thermoelement 2: {thermoelement_2_max:.2f} °C",
      f"Erstmalig erreicht an Position: {ende_thermoelement_2 }", sep = "\n")
```

```
Gemitteltes Maximum Thermoelement 1: 97.32 °C
Erstmalig erreicht an Position: 491
Gemitteltes Maximum Thermoelement 2: 99.93 °C
Erstmalig erreicht an Position: 476
```

Spannfehler ermitteln

Mit den bekannten Start- und Endwerten kann der Spannfehler geschätzt werden. Da die Funktion `numpy.polynomial.polynomial.polyfit()` nicht mit fehlenden Werten wie `np.nan` umgehen kann, müssen diese herausgefiltert werden. Dies geht zum Beispiel so:

```
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 1 (°C)'])
gültige_werte = ~np.isnan(eis.loc[start:ende, 'Thermoelement 1 (°C)'])
```

Test: Festpunktmethode

kaum macht man es richtig, schon geht es... die Korrektur mit $\text{spannfehler} = \text{anstieg messwerte} - \text{anstieg ideal}$ und $\text{korrigierte werte} = \text{messwerte} / (1 + \text{spannfehler})$ passt irgendwie nicht. Aber: für Thermoelement 2 werden die Ergebnisse eher schlechter

Thermoelement 1

```
ende = ende_thermoelement_1

# ideale Kennlinie schätzen
# x muss bei Null beginnen
lm_kennlinie = poly.polyfit(
    x = [start - start, ende - start],
    y = [0, 100],
    deg = 1)
print(lm_kennlinie.round(2))

## lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
### gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 1 (°C)'])

### Regression
lm_messwerte_thermoelement_1 = poly.polyfit(
    x = [start - start, ende - start],
    y = [eis.loc[start, 'Thermoelement 1 (°C)'], eis.loc[ende, 'Thermoelement 1 (°C)']], deg = 1)

## Spannfehler bestimmen
spannfehler_thermoelement_1 = lm_messwerte_thermoelement_1[1] / lm_kennlinie[1]

## Ausgabe
print("\nThermoelement 1")
print(51 * "=")
print(f"lineare Schätzung der idealen Kennlinie:\n{lm_kennlinie}")
print(f"lineare Schätzung für Thermoelement 1:\n{lm_messwerte_thermoelement_1}")
print(f"Anzahl gültiger Werte: {len(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte])}")
print(f"Der Anstieg der Messwerte: {round(lm_messwerte_thermoelement_1[1], 2)}")
print(f"Der Anstieg der idealen Kennlinie: {round(lm_kennlinie[1], 2)}")
print(f"Der Spannfehler beträgt: {((spannfehler_thermoelement_1 - 1) * 100).round(2)} %")

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte],
         eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte],
```

```

        label = 'Thermoelement 1')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennn
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte], linie,
        label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()

```

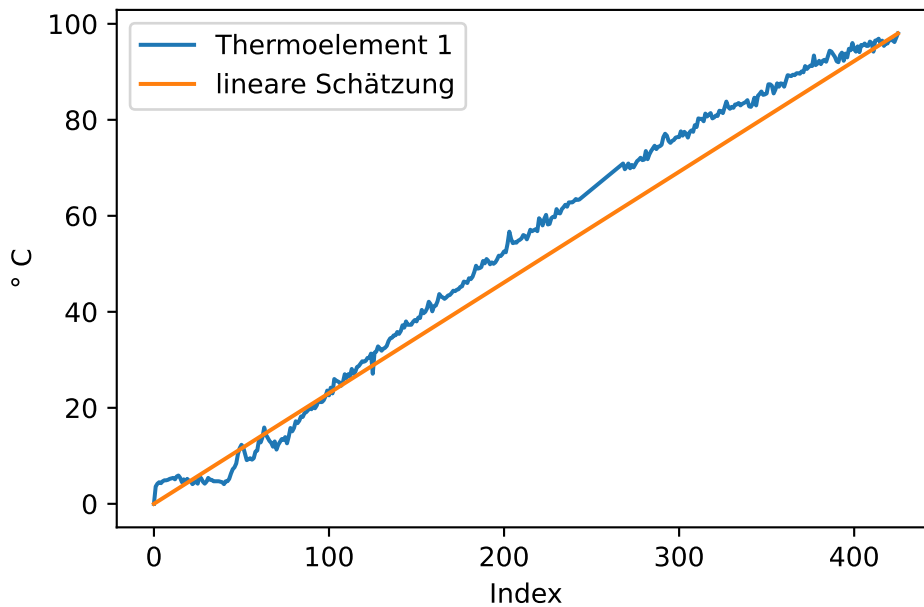
[-0. 0.24]

Thermoelement 1

```

=====
lineare Schätzung der idealen Kennlinie:
[-8.93311885e-15  2.35294118e-01]
lineare Schätzung für Thermoelement 1:
[-2.45118368e-15  2.30588235e-01]
Anzahl gültiger Werte: 402
Der Anstieg der Messwerte: 0.23
Der Anstieg der idealen Kennlinie: 0.24
Der Spannfehler beträgt: -2.0 %

```



Thermoelement 2

```
ende = ende_thermoelement_2

# ideale Kennlinie schätzen
# x muss bei Null beginnen
lm_kennlinie = poly.polyfit(
    x = [start - start, ende - start],
    y = [0, 100],
    deg = 1)
print(lm_kennlinie.round(2))

## lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
### gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 2 (°C)'])

### Regression
lm_messwerte_thermoelement_2 = poly.polyfit(
    x = [start - start, ende - start],
    y = [eis.loc[start, 'Thermoelement 1 (°C)'], eis.loc[ende, 'Thermoelement 2 (°C)']], deg = 1)

## Spannfehler bestimmen
spannfehler_thermoelement_2 = lm_messwerte_thermoelement_2[1] / lm_kennlinie[1]

## Ausgabe
print("\nThermoelement 2")
print(51 * "=")
print(f"lineare Schätzung der idealen Kennlinie:\n{lm_kennlinie}")
print(f"lineare Schätzung für Thermoelement 2:\n{lm_messwerte_thermoelement_2}")
print(f"Anzahl gültiger Werte: {len(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte])}")
print(f"Der Anstieg der Messwerte: {round(lm_messwerte_thermoelement_2[1], 2)}")
print(f"Der Anstieg der idealen Kennlinie: {round(lm_kennlinie[1], 2)}")
print(f"Der Spannfehler beträgt: {((spannfehler_thermoelement_2 - 1) * 100).round(2)} %")

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte],
         eis.loc[start:ende, 'Thermoelement 2 (°C)'][gültige_werte],
         label = 'Thermoelement 2')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
                    y = lm_kennlinie[1] * np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0])
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte], linie,
         label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
```



```
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()
```

```
[-0.    0.24]
```

Thermoelement 2

=====

lineare Schätzung der idealen Kennlinie:

```
[-8.93311885e-15  2.43902439e-01]
```

lineare Schätzung für Thermoelement 2:

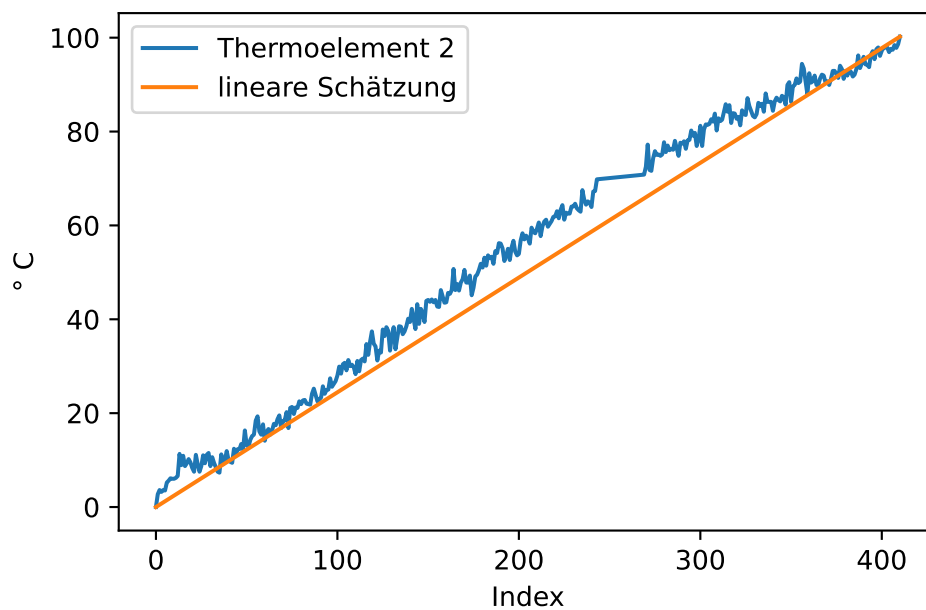
```
[1.37512113e-16  2.44444849e-01]
```

Anzahl gültiger Werte: 387

Der Anstieg der Messwerte: 0.24

Der Anstieg der idealen Kennlinie: 0.24

Der Spannfehler beträgt: 0.22 %



Somit kann der Spannfehler korrigiert werden nach `korrigierte_messwerte = messwerte / spannfehler`.

```
# Spannfehler Thermoelement 1 korrigieren
eis['Thermoelement 1 (°C)'] = eis['Thermoelement 1 (°C)'].div(spannfehler_thermoelement_1)
eis['Thermoelement 2 (°C)'] = eis['Thermoelement 2 (°C)'].div(spannfehler_thermoelement_2)

print(f"Maximum kalibriertes Thermoelement 1: {eis['Thermoelement 1 (°C)'].max()}",
      f"Maximum kalibriertes Thermoelement 2: {eis['Thermoelement 2 (°C)'].max()}",
      sep = '\n')
```

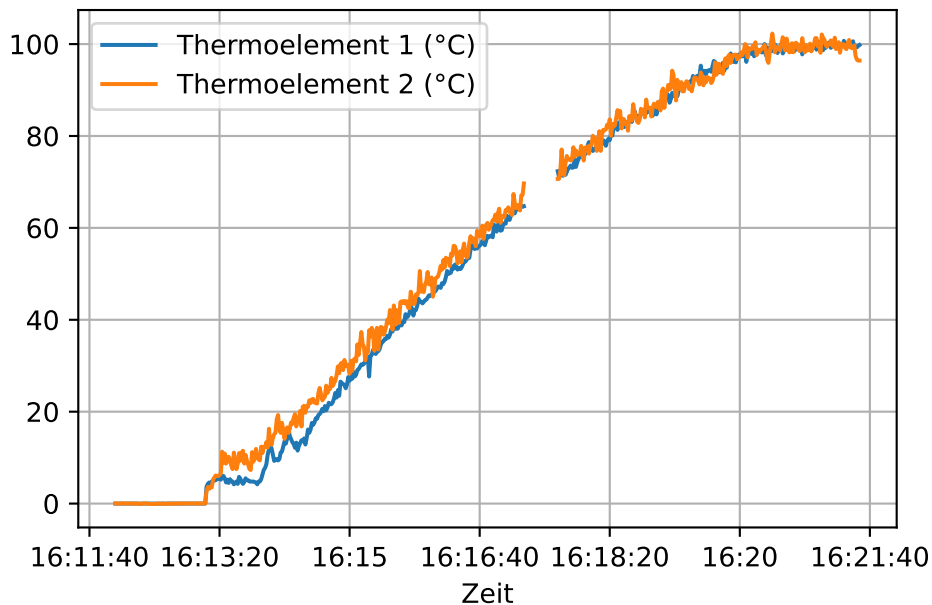
Maximum kalibriertes Thermoelement 1: 100.71428571428572

Maximum kalibriertes Thermoelement 2: 102.29489642436968

Schauen wir uns die kalibrierten Daten erneut an:

```
plotting_data = eis.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)
plt.show()
```



Falsch: Toleranzbandmethode

hierhier: bei Thermoelement 1 ist irgendwo ein Fehler drin - der Spannfehler hat das falsche Vorzeichen - die Messwerte sind ja zu klein

das müsste man mal grafisch darstellen, weil die geschätzte Gerade der Messwerte wird den tatsächlichen Endwert überschießen

Daten wiederherstellen.

```
eis = eis_backup.copy()
eis['Thermoelement 1 (°C)'] -= nullpunktfehler_thermoelement_1
eis['Thermoelement 2 (°C)'] -= nullpunktfehler_thermoelement_2
```

hier sieht man leider das Problem der linearen Regression nach Toleranzbandmethode: Es wird ein Offset-Fehler erzeugt und der Anstieg falsch geschätzt

Das Vorgehen sollte besser werden, wenn die Festpunktmethode verwendet wird. (Dafür wird der Linearitätsfehler größer)

Da die lineare Schätzung der Messwerte bei 0 beginnt, ist die Addition des y-Achsenabschnitts der idealen Kennlinie unnötig (erfolgt hier aber trotzdem).

Thermoelement 1:

```
# Thermoelement 1
## bekannte Referenzwerte
ende = ende_thermoelement_1

x1 = start
x2 = ende

y1 = 0
y2 = 100

## ideale Kennlinie schätzen
## Index muss bei 0 beginnen
lm_kennlinie = poly.polyfit(x = [x1 - start, x2 - start], y = [y1, y2], deg = 1)

## lineare Regression der Messwerte plus y-Achsenabschnittspunkt der idealen Kennlinie
### gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 1 (°C)'])

### Regression
lm_messwerte_thermoelement_1_plus_intercept_kennlinie = poly.polyfit(
```

```

x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
y = eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte], deg = 1)

## Spannfehler bestimmen
spannfehler_thermoelement_1 = lm_messwerte_thermoelement_1_plus_intercept_kennlinie[1] / lm_kennlinie[1]

## Ausgabe
print("\nThermoelement 1")
print(51 * "=")
print(f"lineare Schätzung der idealen Kennlinie:\n{lm_kennlinie}")
print(f"lineare Schätzung für Thermoelement 1 plus y-Achsenschnittpunkt der idealen Kennlinie: {lm_kennlinie[1]}")
print(f"Anzahl gültiger Werte: {len(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte])}")
print(f"Der Anstieg der Messwerte: {round(lm_messwerte_thermoelement_1_plus_intercept_kennlinie[1], 2)}")
print(f"Der Anstieg der idealen Kennlinie: {round(lm_kennlinie[1], 2)}")
print(f"Der Spannfehler beträgt: {((spannfehler_thermoelement_1 - 1) * 100).round(2)} %")

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte],
         eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte],
         label = 'Thermoelement 1')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
                    y = lm_kennlinie[1])
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte], linie,
         label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()

```

Thermoelement 1

=====

lineare Schätzung der idealen Kennlinie:

[-8.93311885e-15 2.35294118e-01]

lineare Schätzung für Thermoelement 1 plus y-Achsenschnittpunkt der idealen Kennlinie:

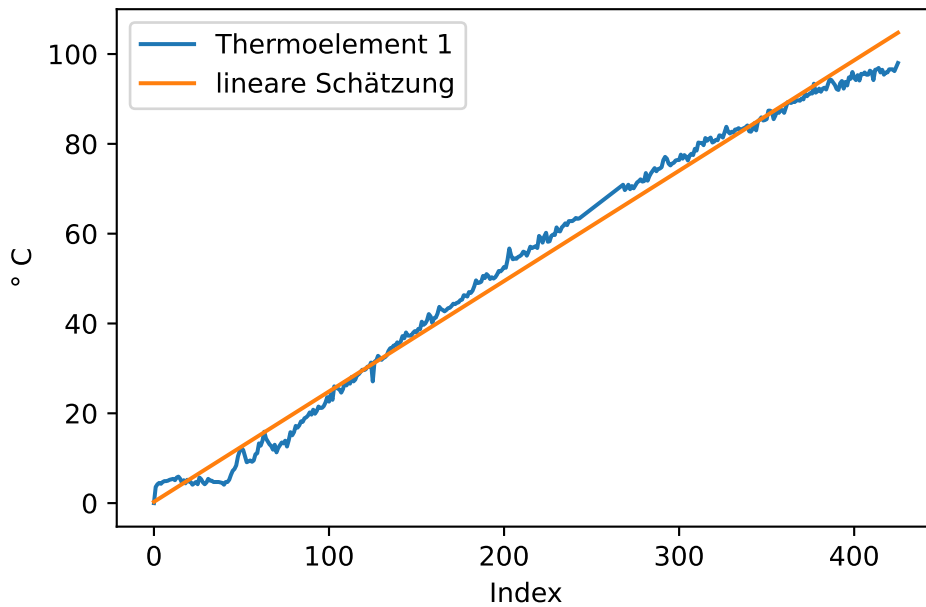
[0.28764301 0.24579133]

Anzahl gültiger Werte: 402

Der Anstieg der Messwerte: 0.25

Der Anstieg der idealen Kennlinie: 0.24

Der Spannfehler beträgt: 4.46 %



Thermoelement 2:

```
# Thermoelement 2
## bekannte Referenzwerte
ende = ende_thermoelement_2

x1 = start
x2 = ende

y1 = 0
y2 = 100

## ideale Kennlinie schätzen
## Index muss bei 0 beginnen
lm_kennlinie = poly.polyfit(x = [x1 - start, x2 - start], y = [y1, y2], deg = 1)

## lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
### gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 2 (°C)'])

### Regression
lm_messwerte_thermoelement_2_plus_intercept_kennlinie = poly.polyfit(
    x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
    y = eis.loc[start:ende, 'Thermoelement 2 (°C)'][gültige_werte], deg = 1)
```

```

## Spannfehler bestimmen
spannfehler_thermoelement_2 = lm_messwerte_thermoelement_2_plus_intercept_kennlinie[1] / lm_kennlinie[1]

## Ausgabe
print("\nThermoelement 2")
print(51 * "=")
print(f"lineare Schätzung der idealen Kennlinie:\n{lm_kennlinie}")
print(f"lineare Schätzung für Thermoelement 2 plus y-Achsenschnittpunkt der idealen Kennlinie:\n{lm_messwerte_thermoelement_2_plus_intercept_kennlinie}")
print(f"Anzahl gültiger Werte: {len(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte])}")
print(f"Der Anstieg der Messwerte: {round(lm_messwerte_thermoelement_2_plus_intercept_kennlinie[1], 2)}")
print(f"Der Anstieg der idealen Kennlinie: {round(lm_kennlinie[1], 2)}")
print(f"Der Spannfehler beträgt: {((spannfehler_thermoelement_2 - 1) * 100).round(2)} %")

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte],
         eis.loc[start:ende, 'Thermoelement 2 (°C)'][gültige_werte],
         label = 'Thermoelement 2')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0], y = lm_kennlinie[1])
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte], linie,
         label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()

```

Thermoelement 2

=====

lineare Schätzung der idealen Kennlinie:

[-8.93311885e-15 2.43902439e-01]

lineare Schätzung für Thermoelement 2 plus y-Achsenschnittpunkt der idealen Kennlinie:

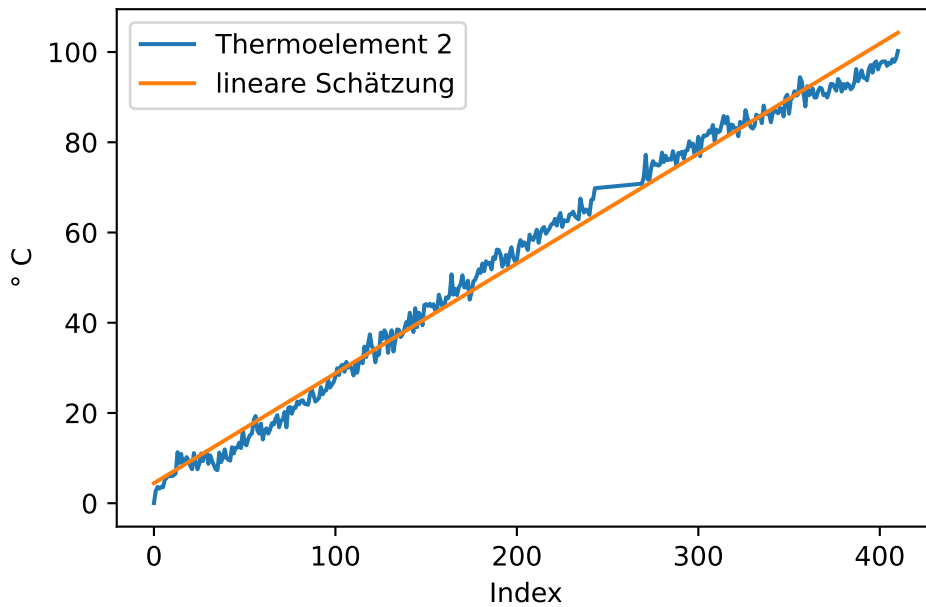
[4.43914138 0.2435162]

Anzahl gültiger Werte: 387

Der Anstieg der Messwerte: 0.24

Der Anstieg der idealen Kennlinie: 0.24

Der Spannfehler beträgt: -0.16 %



Somit kann der Spannfehler korrigiert werden nach `korrigierte_messwerte = messwerte / (1 + spannfehler)`.

```
# Spannfehler Thermoelement 1 korrigieren
eis['Thermoelement 1 (°C)'] = eis['Thermoelement 1 (°C)'].div(spannfehler_thermoelement_1)
eis['Thermoelement 2 (°C)'] = eis['Thermoelement 2 (°C)'].div(spannfehler_thermoelement_2)

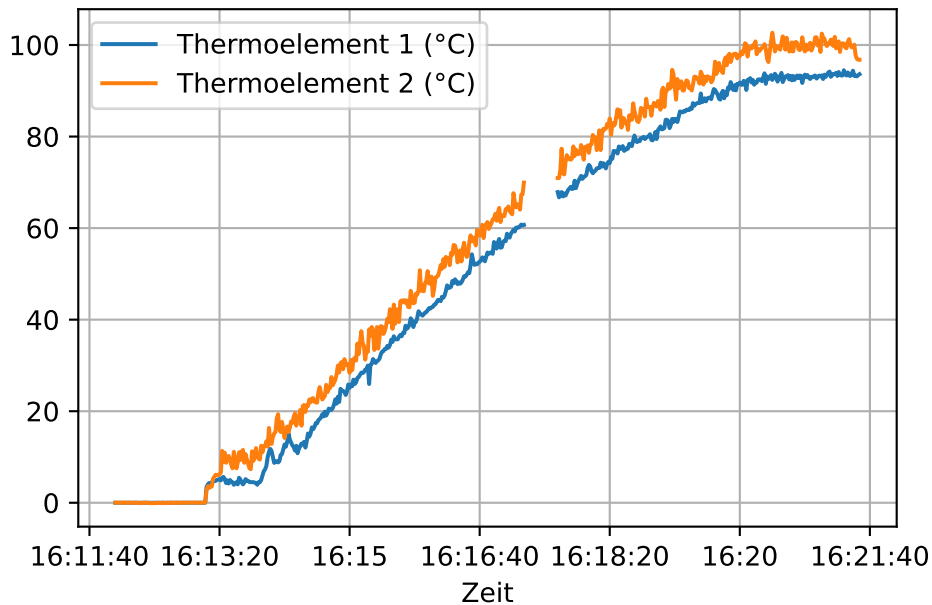
print(f"Maximum kalibriertes Thermoelement 1: {eis['Thermoelement 1 (°C)'].max()}",
      f"Maximum kalibriertes Thermoelement 2: {eis['Thermoelement 2 (°C)'].max()}",
      sep = '\n')
```

```
Maximum kalibriertes Thermoelement 1: 94.48473890114899
Maximum kalibriertes Thermoelement 2: 102.68499596723346
```

Schauen wir uns die kalibrierten Daten erneut an:

```
plotting_data = eis.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)
plt.show()
```



Vorgehen mit linearer Regression

hier sieht man leider das Problem der linearen Regression: Es wird ein Offset-Fehler erzeugt

Das Vorgehen sollte besser werden, wenn die Festpunktmethode verwendet wird. (Dafür wird der Linearitätsfehler größer)

Was hier passiert:

- hier auch den Nullpunktfehler schätzen, dazu Kopie wieder herstellen
- $x = [x1 - start, x2 - start]$ muss gesetzt werden, um den Index auf 0 zu setzen
- size durch shape[0] ersetzen, das führt bei dfs zu weniger Fehlern
- prüfen: plus y-Achsenschnittpunkt der idealen Kennlinie bei der Regression der Messwerte aus dem Abschnitt Nullpunkt- und Spannfehler quantifizieren
- poly.polyfit() kann nicht mit nan umgehen :(
- die linien erreichen zu hohe Schätzwerte - am Ende muss ja 100 °C stehen, aber es werden 105 °C für den letzten Wert geschätzt (was so wäre, wenn der Zusammenhang linear wäre)

```
## kann es sein, dass gültige Werte mit Index die falschen Werte filtert?
## für y = eis.loc passt es
## bei x = np.arange(eis.loc[start:ende, :].shape[0]) passt es auch, da nicht mit index abge
## print(gültige_werte[ : 5])
```


der x-Index ist kürzer für die Messwerte, deshalb ist der Anstieg größer, weil mit weniger x-Werten gerechnet wird. Denn der Anstieg müsste ja kleiner als bei der idealen Kennlinie und der Spannfehler negativ sein. Die ideale Kennlinie berechnet den Anstieg über 435 x-Werte, das lm der Messreihe wird für nur 411 Werte berechnet - deshalb ist der Anstieg größer. Aber müsste der letzte Wert nicht trotzdem 435 sein?

```
# TEMPORÄR
eis = eis_backup.copy()

#####
# DEBUGGING #
#####

# Thermoelement 1

## Referenzpunkte
x1 = start
x2 = ende_thermoelement_1
ende = ende_thermoelement_1

y1 = 0
y2 = 100

print("start:", x1, y1)
print("ende:", x2, y2)

## ideale Kennlinie schätzen
## x-Werte müssen bei 0 beginnen
lm_kennlinie = poly.polyfit(x = [x1 - start, x2 - start], y = [y1, y2], deg = 1)
print(f"lm_kennlinie: {lm_kennlinie}")

## lineare Regression der Messwerte
## gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 1 (°C)'])
print(f"Anzahl gültiger Werte: {len(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte])}")
print(f"So sieht das Ende der Indexreihe aus:\n{np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte]}")

## lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
lm_messwerte_thermoelement_1_plus_intercept_kennlinie = poly.polyfit(
    x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
    y = eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte], deg = 1)
print(f"lm_messwerte Thermoelement 1 plus intercept: {lm_messwerte_thermoelement_1_plus_intercept_kennlinie}")
```

```

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
         eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte],
         label = 'Thermoelement 1')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
                    eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte],
                    label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()

print(f"letzter Wert von Thermoelement 1 im Bereich: {eis.loc[start:ende, 'Thermoelement 1 (°C)'][gültige_werte][-1]}")
print(f"letzter Wert von linie: {linie[-1]}")

## Nullpunktfehler Thermoelement 1 bestimmen
nullpunktfehler = lm_messwerte_thermoelement_1_plus_intercept_kennlinie[0]
print(f"Der Nullpunktfehler beträgt: {nullpunktfehler.round(2)}")

## Spannfehler Thermoelement 1 bestimmen
spannfehler_thermoelement_1 = lm_messwerte_thermoelement_1_plus_intercept_kennlinie[1] - lm_kennlinie[1]
print(f"Der Spannfehler beträgt: {(spannfehler_thermoelement_1 * 100).round(2)} %")

print()

# Thermoelement 2
#####
# DEBUGGING #
#####
## Referenzpunkte
x1 = start
x2 = ende_thermoelement_2
ende = ende_thermoelement_2

y1 = 0
y2 = 100

print("start:", x1, y1)
print("ende:", x2, y2)

## ideale Kennlinie schätzen

```

```

## x-Werte müssen bei 0 beginnen
lm_kennlinie = poly.polyfit(x = [x1 - start, x2 - start], y = [y1, y2], deg = 1)
print(f"lm_kennlinie: {lm_kennlinie}")

# gültige Werte bestimmen
gültige_werte = np.isfinite(eis.loc[start:ende, 'Thermoelement 2 (°C)'])

## lineare Regression der Messwerte plus y-Achsenschnittpunkt der idealen Kennlinie
lm_messwerte_thermoelement_2_plus_intercept_kennlinie = poly.polyfit(
    x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
    y = eis.loc[start:ende, 'Thermoelement 2 (°C)'][gültige_werte], deg = 1)
print(f"lm_messwerte Thermoelement 2 plus intercept: {lm_messwerte_thermoelement_2_plus_intercept_kennlinie}")

## mal grafisch angucken
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
         eis.loc[start:ende, 'Thermoelement 2 (°C)'][gültige_werte],
         label = 'Thermoelement 2')
linie = poly.polyval(x = np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0],
                    lm_messwerte_thermoelement_2_plus_intercept_kennlinie)
plt.plot(np.arange(eis.loc[start:ende, :].shape[0])[gültige_werte] + lm_kennlinie[0], linie,
         label = 'lineare Schätzung')

plt.xlabel(xlabel = 'Index')
plt.ylabel(ylabel = '° C')
plt.legend()
plt.show()

print(f"letzter Wert von Thermoelement 2 im Bereich: {eis.loc[start:ende, 'Thermoelement 2 (°C)'].iloc[-1]}")
print(f"letzter Wert von linie: {linie[-1]}")

## Nullpunktfehler Thermoelement 2 bestimmen
nullpunktfehler = lm_messwerte_thermoelement_2_plus_intercept_kennlinie[0]
print(f"Der Nullpunktfehler beträgt: {nullpunktfehler.round(2)}")

## Spannfehler Thermoelement 1 bestimmen
spannfehler_thermoelement_2 = lm_messwerte_thermoelement_2_plus_intercept_kennlinie[1] - lm_kennlinie[1]
print(f"Der Spannfehler beträgt: {(spannfehler_thermoelement_2 * 100).round(2)} %")

```

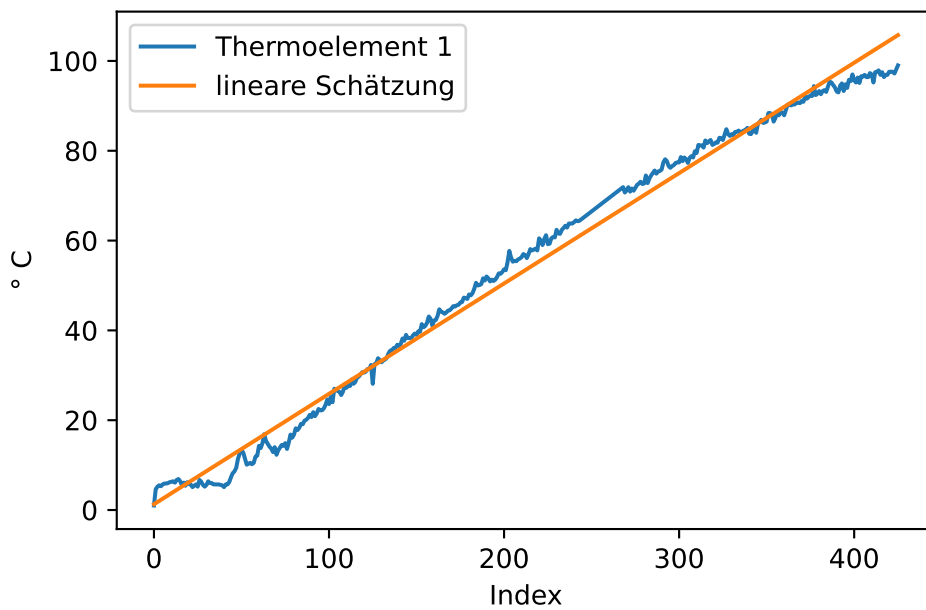
```

start: 66 0
ende: 491 100
lm_kennlinie: [-8.93311885e-15  2.35294118e-01]
Anzahl gültiger Werte: 402
So sieht das Ende der Indexreihe aus:

```

[421 422 423 424 425]

lm_messwerte Thermoelement 1 plus intercept: [1.28764301 0.24579133]



letzter Wert von Thermoelement 1 im Bereich: 99.0

letzter Wert von linie: 105.74895723801889

Der Nullpunktfehler beträgt: 1.29

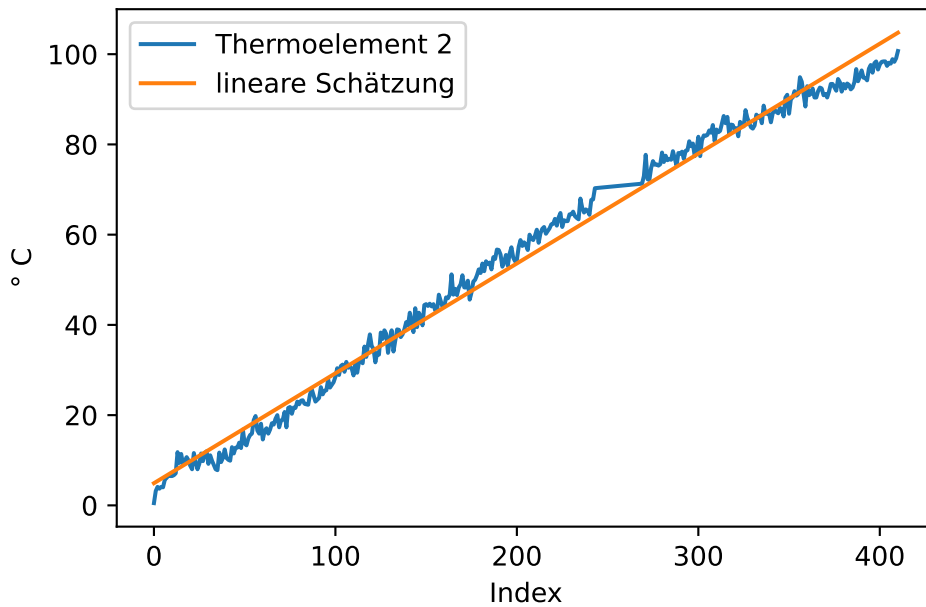
Der Spannfehler beträgt: 1.05 %

start: 66 0

ende: 476 100

lm_kennlinie: [-8.93311885e-15 2.43902439e-01]

lm_messwerte Thermoelement 2 plus intercept: [4.91675332 0.2435162]



letzter Wert von Thermoelement 2 im Bereich: 100.7
 letzter Wert von linie: 104.758397270617
 Der Nullpunktfehler beträgt: 4.92
 Der Spannfehler beträgt: -0.04 %

6.14.1 Arnoldformel

$a = (\text{wahres_maximum} - \text{wahres_minimum}) / (\text{messwerte.max}() - \text{messwerte.min}())$ $b = \text{wahres_minimum} - a * \text{messwerte.min}()$

$\text{korrigierte_messwerte} = a * \text{messwerte} + b$

```

eis = eis_backup.copy()

wahres_minimum = 0
wahres_maximum = 100

# Thermoelement 1
ende = ende_thermoelement_1

a = (wahres_maximum - wahres_minimum) / (eis.loc[start:ende, 'Thermoelement 1 (°C)'].max() -
b = wahres_minimum - a * eis.loc[start:ende, 'Thermoelement 1 (°C)'].min()
  
```

```

eis['Thermoelement 1 (°C)'] = a * eis['Thermoelement 1 (°C)'] + b

## Ausgabe
print(f"Thermoelement 1\n",
      f"a = {a:.1f}\n",
      f"b = {b:.1f}\n",
      sep = '')

# Thermoelement 2
ende = ende_thermoelement_2

a = (wahres_maximum - wahres_minimum) / (eis.loc[start:ende, 'Thermoelement 2 (°C)'].max() -
b = wahres_minimum - a * eis.loc[start:ende, 'Thermoelement 2 (°C)'].min()

eis['Thermoelement 2 (°C)'] = a * eis['Thermoelement 2 (°C)'] + b

## Ausgabe
print(f"Thermoelement 2\n",
      f"a = {a:.1f}\n",
      f"b = {b:.1f}\n",
      sep = '')

# plotten
plotting_data = eis.copy()
plotting_data['Zeit'] = plotting_data['Zeit'].dt.time

plotting_data.plot(x = 'Zeit', y = ['Thermoelement 1 (°C)', 'Thermoelement 2 (°C)'], grid = True)

plt.show()

```

```

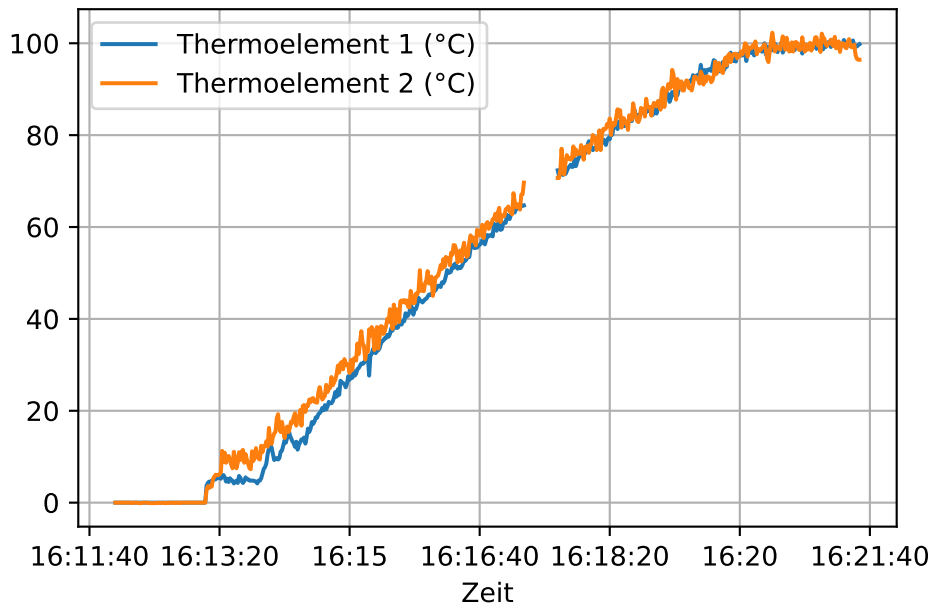
Thermoelement 1
a = 1.0
b = -1.0

```

```

Thermoelement 2
a = 1.0
b = -0.5

```



```
print(f"Maximum kalibriertes Thermoelement 1: {eis['Thermoelement 1 (°C)'].max()} ",
      f"Maximum kalibriertes Thermoelement 2: {eis['Thermoelement 2 (°C)'].max()} ",
      sep = '\n')
```

```
Maximum kalibriertes Thermoelement 1: 100.71428571428572
Maximum kalibriertes Thermoelement 2: 102.29540918163671
```

6.15 Weitere Fehlerarten

Hysteresis: Verzögerung zwischen Änderung des Wertes und Änderungsgeschwindigkeit des Messwerts
Drift: Rauschen:

Quellen

- Baitsch, Matthias (2019). *Vorlesungsskript Mathematik B (Master) Stochastik*.
- Deutsches Institut für Normung (1995). *Grundlagen der Meßtechnik - Teil 1: Grundbegriffe*. DOI: [10.31030/2713411](https://doi.org/10.31030/2713411).
- (1996). *Grundlagen der Meßtechnik - Teil 3: Auswertung von Messungen einer einzelnen Meßgröße, Meßunsicherheit*. DOI: [10.31030/7204542](https://doi.org/10.31030/7204542).
 - (2023). *Industrielle Platin-Widerstandsthermometer und Platin-Temperatursensoren (IEC 60751:2022). Deutsche Fassung EN IEC 60751:2022*. DOI: [10.31030/3405985](https://doi.org/10.31030/3405985).
- Dinter, Ralf (2011). *Fehlerrechnung für Einsteiger. Eine beispielorientierte Einführung für Studierende der TUHH*. URL: <https://www2.physnet.uni-hamburg.de/TUHH/Versuchsanleitung/Fehlerrechnung.pdf>.
- Eden, Klaus und Hermann Gebhard (2024). *Dokumentation in der Mess- und Prüftechnik. Messen - Auswerten - Darstellen Protokolle - Berichte - Präsentationen*. 3. Auflage. DOI: [10.1007/978-3-658-44779-3](https://doi.org/10.1007/978-3-658-44779-3).
- Grote, Karl-Heinrich und Jörg Feldhusen, Hrsg. (2011). *Dubbel: Taschenbuch für den Maschinenbau*. 23. Aufl. Springer Berlin, Heidelberg. ISBN: 978-3-642-17306-6. DOI: [10.1007/978-3-642-17306-6](https://doi.org/10.1007/978-3-642-17306-6). URL: <https://doi.org/10.1007/978-3-642-17306-6>.
- Hempel, Rainer (2016). *Script zur Einführung in die Grundlagen der Fehlerrechnung*. URL: <https://www.tu-braunschweig.de/index.php?eID=dumpFile&t=f&f=45992&token=b041478add5f7a3e136b69905b72bfa0e192d82b>.
- Hochschule Aalen (2016). *Anleitung zur Fehlerrechnung im Physikpraktikum*. URL: https://www.hs-aalen.de/uploads/mediapool/media/file/13430/F_Fehlerrechnung.pdf.
- Schrüfer, Elmar, Leonhard M. Reindl und Bernhard Zagar (2022). *Elektrische Messtechnik: Messung elektrischer und nichtelektrischer Größen*. 13. Aufl. München: Carl Hanser Verlag.